ОГЛАВЛЕНИЕ

Введение 3

Глава 1. **Способы задания и распознавания формальных языков** 6
§ 1.1. Алфавит и слова 6
§ 1.2. Языки и операции над языками 7
§ 1.3. Грамматики 10
§ 1.4. Классификация грамматик 14
§ 1.5. Распознаватели 16
§ 1.6. Упражнения 18

Глава 2. **Регулярные языки** 19
§ 2.1. Регулярные множества и регулярные выражения 19
§ 2.2. Уравнения и системы уравнений с регулярными коэффициентами . . . 21
§ 2.3. Алгоритм решения систем уравнений с регулярными коэффициентами 25
§ 2.4. Совпадение классов регулярных и ПЛ-языков 28
§ 2.5. Упражнения 32

Глава 3. **Конечные автоматы** 34
§ 3.1. Определения и примеры 34
§ 3.2. Редукция недетерминированных конечных автоматов к детерминированным 38
§ 3.3. Граф переходов 41
§ 3.4. Совпадение классов конечно-автоматных, регулярных и ПЛ-языков . . 42
§ 3.5. Лемма о разрастании для регулярных языков 45
§ 3.6. Упражнения 47

Глава 4. **Конечные автоматы со спонтанными переходами** 50
§ 4.1. Определения и примеры 50
§ 4.2. Редукция ε-НКА к ДКА 53
§ 4.3. Преобразование регулярного выражения в автомат 55
§ 4.4. Построение ε-НКА по ПЛ-грамматике 60

§ 4.5. Вычисление языка ε-НКА 64
§ 4.6. Задача минимизации конечного автомата 67
§ 4.7. Упражнения 78

Глава 5. **Булева алгебра регулярных языков** 80
§ 5.1. Свойства регулярных языков 80
§ 5.2. Замкнутость регулярных множеств относительно базисных операций . 87
§ 5.3. Алгоритмические проблемы регулярных языков 88
§ 5.4. Упражнения 90

Глава 6. **Контекстно-свободные языки** 92
§ 6.1. Деревья выводов в КС-грамматиках 92
§ 6.2. Проблема непустоты и устранение бесполезных символов 95
§ 6.3. Построение приведенной КС-грамматики 100
§ 6.4. Упражнения 105

Глава 7. **Нормальные формы КС-грамматик** 107
§ 7.1. Нормальная форма Хомского 107
§ 7.2. Проблема принадлежности для КС-языков 111
§ 7.3. Матричный метод перехода к нормальной форме Грейбах 114
§ 7.4. Упражнения 119

Глава 8. **Автоматы с магазинной памятью** 121
§ 8.1. Определения и примеры 121
§ 8.2. Расширенный МП-автомат 126
§ 8.3. Автомат, допускающий слово опустошением магазина 130
§ 8.4. Эквивалентность МП-автоматов и КС-грамматик 134
§ 8.5. Детерминированный МП-автомат 140
§ 8.6. Упражнения 141

Список литературы 143

Приложение A. **Алгоритмы для контекстно-свободных грамматик** 144

Приложение B. **Задание к курсовой работе** 150

Приложение C. **Варианты заданий** 152

Приложение D. **Пример выполнения заданий курсовой работы** 157

ВВЕДЕНИЕ

В настоящее время в связи с бурным развитием информационных технологий и расширения сферы их применения в теоретической информатике возникают новые и развиваются старые направления. Всё это находит своё отражение в формировании учебных дисциплин по направлению «Фундаментальная информатика и информационные технологии».

Целью изучения дисциплины «Теория конечных автоматов и формальных языков» является выработка у студентов компетенций, связанных с теоретическими понятиями, идеями, методами, моделями, алгоритмами, применяемыми при разработке и сопровождению проблемно-ориентированных языков. Эта дисциплина содержит сведения, необходимые для научно-исследовательской и практической работы в области системного программирования, использования и развития языков программирования.

К основным задачам дисциплины относится следующее:

- освоение математических основ теории формальных языков; изучение классов формальных языков и конечных способов задания потенциально бесконечных множеств; овладение методами разбора слов, принадлежащим языку, определения эквивалентности выводов, установление однозначности грамматик;
- освоение математических основ теории регулярных языков и выражений; изучение свойств регулярных выражений;
- овладение методами построения регулярных выражений для языков, заданных неформально и в виде формальных грамматик; освоение методов перехода от грамматики к регулярному выражению и наоборот;
- освоение математических основ теории детерминированных и недетерминированных конечных автоматов; изучение общей схемы автомата-распознавателя; освоение метода редукции недетерминированного автомата к недетерминированному; овладение методами построение конеч-

ного автомата по регулярной грамматике; изучение свойств праволинейных, регулярных и автоматных языков;

- освоение понятия конечного автомата со спонтанными переходами; овладение методами построения конечного автомата по праволинейной грамматике и вычисления языка конечного автомата с помощью последовательного исключения состояний; изучение понятий отношения эквивалентности на множестве конечных автоматов, недостижимых состояний автомата;
- изучение задачи минимизации детерминированного конечного автомата; освоение понятий различимых и неразличимых состояний и метода построения системы отношений неразличимости состояний конечного автомата;
- овладение методами определения регулярности языка; изучение алгоритмических проблем регулярных языков;
- освоение математических основ теории контекстно-свободных (КС) языков; изучение понятий дерева вывода в КС-грамматике; овладение методами проверки КС-грамматики на непустоту, построения по КС-грамматике стабилизационного множества, построения приведенной КС-грамматики; изучение нормальных форм Хомского и Грейбах КС-языков; освоение метода определения принадлежности цепочки языку, заданного КС-грамматикой;
- освоение математических основ теории автоматов с магазинной памятью (МП-автоматов); изучение понятия недетерминизма для МП-автоматов, расширенной формы МП-автомата (РМП-автомат), автомата, допускающего язык опустошением магазина (МПε-автомат); овладение методами перехода от МП-автомата к МПε-автомату и наоборот;
- овладение методом построения МПε-автомата по КС-грамматике; овладение методом перехода от МПε-автомата к КС-грамматике.

Исчерпывающее изложение элементов теории формальных языков и конечных автоматов имеется в ставшей редкостью замечательной монографии А.Ахо и Дж.Ульмана [1], материалы которой в современной форме представлены в [3]. Настоящий учебник содержит систематическое изложение значительной части материала курса «Теория конечных автоматов и формальных языков» в объёме, достаточном для успешного его освоения. При изложе-

нии материала мы пытались придерживаться канонов монографии А.Ахо и Дж.Ульмана [1], опуская, однако, более глубокие аспекты рассматриваемой теории, на которые не достает времени в рамках данного курса. По курсу «Теория автоматов и формальных языков» можно порекомендовать ряд хороших книг, названия некоторых содержатся в списке литературы.

Учебник состоит из введения, восьми глав, списка литературы и четырёх приложений. В главе 1 идет речь о способах задания и распознавания формальных языков; в главе 2 исследуются регулярные языки; в главах 3 и 4 изучаются разные типы конечных автоматов и доказывается совпадение классов конечно-автоматных, регулярных и праволинейных языков; в главе 5 изучается булева алгебра регулярных языков, свойства замкнутости операций над регулярными множествами и алгоритмические проблемы регулярных языков; в главах 6 и 7 исследуются контекстно-свободные грамматики и языки; в главе 8 устанавливается связь между контекстно-свободными языками и конечными автоматами с магазинной памятью. Каждая глава снабжена набором упражнений для лучшего понимания и усвоения материала. В учебнике имеется четыре приложения, которые содержат алгоритмы для контекстно-свободных грамматик, задания к курсовой работе и варианты к ним, пример выполнения одного варианта курсовой работы.

Отметим, что у читателя предполагается знакомство с некоторыми темами стандартного курса «Дискретной математики», в остальном же изложение замкнуто. Полезным, однако, является хорошее освоение материала курсов «Математическая логика» и «Теория алгоритмов».

Нумерация всех утверждений имеет вид: $\alpha.\beta.\gamma$, где α — номер главы, β — номер раздела главы, γ — номер утверждения в разделе. Формулировки теорем, лемм и утверждений заканчиваются символом $_\square$, а доказательства заканчиваются символом ■. Формулировки упражнений заканчиваются символом □, а окончания примеров помечаются символом ■.

В тексте не выделяются и не нумеруются определения, однако некоторые важные понятия, появляющиеся по ходу изложения, набраны *курсивом*.

ГЛАВА 1

СПОСОБЫ ЗАДАНИЯ И РАСПОЗНАВАНИЯ ФОРМАЛЬНЫХ ЯЗЫКОВ

1.1. Алфавит и слова

Алфавитом будем называть любое множество символов. Предполагается, что термин «символ» имеет достаточно ясный интуитивный смысл и не нуждается в дальнейшем пояснении. Алфавит, вообще говоря, не обязан быть ни конечным ни даже счетным, но везде далее мы будем предполагать его конечным. Термины «буква» и «знак» используются как синонимы термина «символ» для обозначения элемента алфавита. Термин «буква» будет для нас основным.

Если написать последовательность букв алфавита Σ, располагая их одну за другой, то получится «слово». Термины «цепочка», «строка» и даже «предложение» часто используются как синонимы термина «слово». Термин «слово» будет для нас основным.

Слово называется пустым, если оно не содержит ни одной буквы. Это слово обозначается символом ε.

Пример 1.1.1. Латинский алфавит: множество, состоящее из 26 прописных и 26 строчных латинских букв; *begin* — слово в этом алфавите. ■

Пример 1.1.2. Бинарный (или двоичный) алфавит: множество $\{0; 1\}$; 001001 — слово в этом алфавите. ■

Определим две операции над словами:

Конкатенация Если α и β — слова, то слово $\alpha\beta$ называется *конкатенацией* (*сцеплением*) α и β. Например, если α = вино, β = град, то $\alpha\beta$ = виноград. Для любого слова α всегда $\alpha\varepsilon = \varepsilon\alpha = \alpha$.

Обращение *Обращением* слова α называется слово α^R, которое отличается от α только порядком следования входящих в него букв, то есть если $a_1, a_2, \ldots,$

a_n — буквы и $\alpha = a_1 \dots a_n$, то $\alpha^R = a_n \dots a_1$. Ясно, что $\varepsilon^R = \varepsilon$. Например, если α = нос, то α^R = сон.

Пусть α, β и γ — произвольные слова в некотором алфавите Σ. Назовем α *префиксом* слова $\alpha\beta$, а β — *суффиксом* слова $\alpha\beta$. Слово β назовем подсловом слова $\alpha\beta\gamma$. Префикс и суффикс являются подсловами. Заметим, что пустое слово является префиксом, суффиксом и подсловом любого слова. Если $\alpha \neq \beta$ и α — префикс слова β, то α называется собственным префиксом слова β; аналогично определяются собственные суффиксы и подслова.

Слова вида $aa\dots a$ (n букв) будем записывать короче: a^n. Например:

$$aabbba = a^2b^3a, \qquad \varepsilon = a^0.$$

Длиной слова будем называть число букв в нем. Длину слова α будем обозначать $|\alpha|$. Например, $|aab| = 3$, $|\varepsilon| = 0$.

Упражнение 1.1.1. Выясните, сколько букв в русском алфавите. □

Упражнение 1.1.2. Найдите все префиксы, суффиксы и подслова слова *арбуз*.□

1.2. Языки и операции над языками

Языком над алфавитом Σ называется произвольное множество слов, записанных буквами из Σ. Под это определение, конечно, подходит почти любое известное понятие языка. Например, английский и русский языки, различные алгоритмические языки и т. д.

Рассмотрим простейшие примеры языков над некоторым алфавитом Σ:

- пустое множество $\varnothing$;
- множество $\{\varepsilon\}$, состоящее из одного пустого слова;
- множество $\{a\}$, где $a \in \Sigma$, состоящее из одного однобуквенного слова.

Отметим, что $\varnothing$ и $\{\varepsilon\}$ — два различных языка.

Обозначим через Σ^* множество, содержащее все слова в алфавите Σ, включая ε. Пусть $\Sigma^+ = \Sigma^* - \{\varepsilon\}$. Например, если Σ — бинарный алфавит $\{0, 1\}$, то

$$\Sigma^* = \{\varepsilon; 0; 1; 00; 01; 10; 11; 000; 001; \dots\}.$$

Каждый язык L в алфавите Σ является подмножеством множества Σ^* и содержит язык $\varnothing$. Отметим, что слово “*liFe*”, составленное из английских букв, не яв-

ляется словом английского языка, а слово «жызнь», составленное из русских букв, не является словом русского языка.

Пример 1.2.1. Рассмотрим язык L_1, содержащий все слова из нуля или более букв a. Тогда $L_1 = \{a^i \mid i \geqslant 0\} = \{a\}^*$. ■

В тех случаях, когда это не может привести к путанице, мы будем обозначать множество, состоящее из одного элемента, самим элементом. В соответствии с этим соглашением $a^* = \{a\}^*$.

Если язык L таков, что произвольное слово из L не может являться собственным префиксом (суффиксом) никакого другого слова из L, то говорят, что L обладает префиксным (суффиксным) свойством. Например, язык a^* не обладает префиксным свойством, а язык $\{a^i b \mid i \geqslant 0\}$ этим свойством обладает. В некотором смысле слова из языка, обладающего префиксным свойством, не продолжимы вправо, а слова из языка, обладающего суффиксным свойством, не продолжимы влево.

Рассмотрим некоторые операции над языками. Из того, что язык является множеством, вытекает, что операции объединения, пересечения и разности применимы к произвольным языкам. Операцию конкатенации можно применять к языкам так же, как и к словам: а именно, если L_1 — язык в алфавите Σ_1, а L_2 — язык в алфавите Σ_2, то язык L_1L_2, называемый конкатенацией (а также сцеплением или произведением) языков L_1, и L_2. определяется равенством:

$$L_1L_2 = \{xy \mid x \in L_1, y \in L_2\} \quad (\subset \{\Sigma_1 \cup \Sigma_2\}^*).$$

Итерации языка L определяются следующим образом:

1) $L^0 = \{\varepsilon\}$,
2) $L^n = LL^{n-1}$ для $n \geqslant 1$,
3) $L^* = \bigcup_{n\geqslant 0} L^n$ — полная итерация,
4) $L^+ = \bigcup_{n\geqslant 1} L^n$ — позитивная итерация.

Отметим, что

$$L^+ = LL^* = L^*L, \qquad L^* = L^+ \cup \{\varepsilon\}.$$

Пусть Σ_1 и Σ_2 — алфавиты. Рассмотрим произвольное отображение $h\colon \Sigma_1 \to \Sigma_2^*$ и расширим его до отображения Σ_1^* в Σ_2^*, полагая

$$h(\varepsilon) = \varepsilon, \quad h(xa) = h(x)(a)$$

для всех $x \in \Sigma_1^*$ и $a \in \Sigma_1$. Легко показать, что новое отображение определено корректно. Для него мы сохраним символ h. Отображение $h\colon \Sigma_1^* \to \Sigma_2^*$ называется *гомоморфизмом*.

Применяя гомоморфизм h к языку L, мы получаем новый язык $h(L)$, который представляет собой множество слов $\{h(\omega) \mid \omega \in L\}$.

Пример 1.2.2. Рассмотрим алфавиты $\Sigma_1 = \{0; 1\}$ и $\Sigma_2 = \{a; b\}$. Пусть

$$L = \{0^n 1^n \mid n \geqslant 1\}.$$

Предположим, мы хотим заменить каждое вхождение буквы 0 в словах из языка L на букву a, а каждое вхождение буквы 1 - на bb. Тогда можно определить гомоморфизм h так, что $h(0) = a, h(1) = bb$. В этом случае $h(L) = \{a^n b^{2n} \mid n \geqslant 1\}$. ■

Пусть Σ_1 и Σ_2 — алфавиты, L — язык над алфавитом Σ_2. Рассмотрим произвольный гомоморфизм $h\colon \Sigma_1^* \to \Sigma_2^*$. *Прообразом* языка L называется язык

$$h^{-1}(L) = \bigcup_{y \in L} h^{-1}(y) = \{x \mid h(x) \in L\} \ (\subset \Sigma_1^*).$$

Пример 1.2.3. Пусть $h\colon \{0; 1\}^* \to \{a; b\}^*$ — гомоморфизм, для которого

$$h(0) = h(1) = a.$$

Тогда $h^{-1}(a) = \{0; 1\},\ h^{-1}(b) = \varnothing$. Пусть $L_1 = \{b\}^*, L_2 = \{a\}^*$. Тогда $h^{-1}(L_1) = \{\varepsilon\}$, $h^{-1}(L_2) = \{0; 1\}^*$. ■

Пример 1.2.4. Пусть $h\colon \{0; 1\}^* \to \{a; b\}^*$ — гомоморфизм, для которого $h(0) = a$, $h(1) = \varepsilon$. Тогда $h^{-1}(\varepsilon) = 1^*,\ h^{-1}(a) = \{1^i 0 1^j \mid i, j \geqslant 0\}$. ■

Упражнение 1.2.1. Верно ли, что $L^+ = L^* \setminus \{\varepsilon\}$. □

Упражнение 1.2.2. Какие из следующих языков обладают префиксным (суффиксным) свойством:

- $\varnothing$;
- $\{\varepsilon\}$;
- $\{a^n b^n \mid n \geqslant 1\}$;
- L^*, если L обладает префиксным (суффиксным) свойством;
- $\{\omega \mid \omega \in \{a, b\}^*$ и число символов a в ω равно числу символов $b\}$. □

Упражнение 1.2.3. Пусть $h\colon\{0;1;2\}^* \to \{a;b\}^*$ — гомоморфизм, определённый равенствами $h(0) = a$, $h(1) = bb$ и $h(2) = \varepsilon$. Опишите языки $h(\{012\}^*)$, $h(\{0;1;2\}^*)$, $h^{-1}(\{ab\}^*)$. □

Упражнение 1.2.4. Докажите или опровергните следующие утверждения:

$$h^{-1}(h(L)) = L,$$
$$h(h^{-1}(L)) = L.$$ □

1.3. Грамматики

Грамматика — это математическая система, определяющая язык. В грамматике G, определяющей язык L, используются два конечных непересекающихся множества символов: множество нетерминальных символов, которое часто будет обозначаться буквой N, и множество терминальных символов, обозначаемое обычно Σ. Из терминальных символов образуются слова языка L, а нетерминальные символы играют вспомогательную роль. Ядром грамматики является конечное множество P правил образования, которые описывают процесс порождения слов языка.

Дадим теперь точное определение грамматики. *Грамматикой* называется четверка $G = (N, \Sigma, P, S)$, где N — конечное множество нетерминальных символов (алфавит нетерминалов), Σ — не пересекающееся с N конечное множество терминальных символов (алфавит терминалов), P — конечное подмножество множества

$$(N \cup \Sigma)^* N (N \cup \Sigma)^* \times (N \cup \Sigma)^*.$$

Пара $(\alpha, \beta) \in P$, где $\alpha \in (N \cup \Sigma)^* N (N \cup \Sigma)^*$,$\beta \in (N \cup \Sigma)^*$, называется *правилом* или *продукцией* и записывается в виде $\alpha \to \beta$. S — выделенный символ из N, называемый *начальным* или *стартовым* символом.

Пример 1.3.1. Рассмотрим грамматику $G = (\{A;S\}, \{0;1\}, P, S)$, где P состоит из продукций

$$\begin{aligned} &S \to 0A1,\\ &S \to 01A01,\\ &S \to 00A1,\\ &A \to \varepsilon. \end{aligned} \tag{1.3.1}$$

Нетерминальными символами в этой грамматике являются буквы A и S, терминальными — 0 и 1, а S — начальный символ. ■

Продукции с одинаковыми правыми частями будем иногда записывать в одну строчку через символ | : например, две первых продукции из примера 1.3.1 будем записывать так:

$$S \to 0A1 \mid 01A01. \tag{1.3.2}$$

Грамматика определяет язык рекурсивным образом. Рекурсивность проявляется в определении особого рода слов, называемых выводимыми словами грамматики $G = (N, \Sigma, P, S)$: S — выводимое слово; если $\alpha\beta\gamma$ — выводимое слово и $(\beta \to \delta) \in P$, то $\alpha\delta\gamma$ — тоже выводимое слово; никакие другие слова не являются выводимыми. Выводимое слово грамматики G, не содержащее нетерминальных символов, называется терминальным словом, порождаемым грамматикой G.

Язык $L(G)$, порождаемый грамматикой G, определяется как множество всех терминальных слов, порождаемых грамматикой G.

Пусть $G = (N, \Sigma, P, S)$ - некоторая грамматика. Если $\alpha\beta\gamma \in (N \cup \Sigma)^*$ и $(\beta \to \delta) \in P$, то будем говорить, что слово $\alpha\delta\gamma$ непосредственно выводимо из $\alpha\beta\gamma$ и записывать это так: $\alpha\beta\gamma \to_G \alpha\delta\gamma$. В тех случаях, когда из контекста ясно, о какой грамматике идет речь, нижний индекс G будем опускать. Через $\to^k$ будем обозначать k-ю степень отношения $\to$, то есть будем записывать $\alpha \to^k \beta$, если существует последовательность $k+1$ слов $\alpha_0, \alpha_1, \ldots, \alpha_k$, для которых $\alpha = \alpha_0$, $\alpha_{i-1} \to \alpha_i$ при $1 \leqslant i \leqslant k$ и $\alpha_k = \beta$. Эта последовательность слов называется выводом длины k слова β из слова α в грамматике G. Транзитивное замыкание отношения $\to$ обозначим через $\to^+$; $\varphi \to^+ \psi$ читается так: слово ψ выводимо из φ нетривиальным образом. Рефлексивное и транзитивное замыкание отношения $\to$ обозначим через $\to^*$; $\varphi \to^* \psi$ читается так: ψ выводимо из φ. Отметим, что $\alpha \to^+ \beta$ тогда и только тогда, когда $\alpha \to^i \beta$ для некоторого $i \geqslant 1$, а $\alpha \to^* \beta$ тогда и только тогда, когда $\alpha \to^i \beta$ для некоторого $i \geqslant 0$. Таким образом, $L(G) = \{\omega \mid \omega \in \Sigma^*, S \to^* \omega\}$.

Упражнение 1.3.1. Рассмотрим грамматику $G = (\{A; S\}, \{0; 1\}, P, S)$, где P состоит из продукций:

$$\begin{aligned} &S \to 0A1, \\ &0A \to 00A1, \\ &A \to \varepsilon. \end{aligned} \tag{1.3.3}$$

Доказать, что $L(G) = \{0^n 1^n \mid n \geqslant 1\}$. □

Приведем существенные для дальнейшего примеры грамматик.

Пример 1.3.2. Пусть $G = (\{Ц\}, \{0;1;\ldots;9\}, Ц \to 0 \mid 1 \mid \ldots \mid 9, Ц)$. В грамматике G единственный нетерминальный символ. Ясно, что $L(G) = \{0;1;\ldots;9\}$. ■

Пример 1.3.3. Пусть $G_0 = (\{E;T;F\},\ \{a;+;*;(;)\}, P, E)$, где P состоит из продукций:

$$\begin{aligned} &E \to E+T \mid T,\\ &T \to T*F \mid F,\\ &F \to (E) \mid a. \end{aligned} \tag{1.3.4}$$

Пример вывода в этой грамматике:

$$E \Rightarrow E+T \Rightarrow T+T \Rightarrow F+T \Rightarrow a+T \Rightarrow a+T*F \Rightarrow a+F*F \Rightarrow a+a*F \Rightarrow a+a*a.$$

Можно доказать, что язык $L(G_0)$ это множество арифметических выражений, построенных из пяти символов $+$, $*$, a, $($, $)$. ■

Пример 1.3.4. Рассмотрим грамматику $G = (\{B;C;S\}, \{a,b,c\}, P, S)$, где P состоит из продукций:

$$\begin{aligned} &S \to aSBC \mid abC,\\ &CB \to BC,\\ &bB \to bb,\\ &bC \to bc,\\ &cC \to cc. \end{aligned} \tag{1.3.5}$$

Пример вывода в этой грамматике:

$$S \Rightarrow aSBC \Rightarrow aabCBC \Rightarrow aabBCC \Rightarrow aabbCC \Rightarrow aabbcC \Rightarrow aabbcc.$$

можно доказать, что эта грамматика порождает язык $\{a^n b^n c^n \mid n \geqslant 1\}$. ■

Пример 1.3.5. Пусть $G = (\{A;B;C;D;S\}, \{a;b\}, P, S)$, где P состоит из продукций:

$$\begin{aligned} S &\to CD, & Ab &\to bA,\\ C &\to aCA \mid bCB \mid \varepsilon, & Ba &\to aB,\\ AD &\to aD, & Bb &\to bB,\\ BD &\to bD, & Aa &\to aA,\\ D &\to \varepsilon. \end{aligned}$$

Пример вывода в этой грамматике:

$$S \Rightarrow CD \Rightarrow aCAD \Rightarrow abCBAD \Rightarrow abBAD \Rightarrow abBaD \Rightarrow ababD \Rightarrow abab.$$

Покажем, что $L(G) = \{\omega\omega \mid \omega \in \{a,b\}^*\}$, то есть язык $L(G)$ состоит из слов четной длины, составленных из букв a и b, причем первая половина каждого слова совпадает со второй половиной.

Сначала докажем, что $\{\omega\omega \mid \omega \in \{a,b\}^*\} \subseteq L(G)$. Для этого надо проверить, что каждое слово вида $\omega\omega$ можно вывести из S. Непосредственно проверяется, что в G возможны следующие выводы:

1) $S \Rightarrow CD$

2) $C \Rightarrow^n c_1c_2\ldots c_nCX_nX_{n-1}\ldots X_1 \Rightarrow c_1c_2\ldots c_nX_nX_{n-1}\ldots X_1$, где для всех i $c_i = a$ тогда и только тогда, когда $X_i = A$, и $c_i = b$ тогда и только тогда, когда $X_i = B$;

3)

$$\begin{aligned} X_n\ldots X_2X_1D \Rightarrow X_n\ldots X_2c_1D \Rightarrow^{n-1} c_1X_n\ldots X_2D \Rightarrow \\ c_1X_n\ldots X_3c_2D \Rightarrow^{n-2} c_1c_2X_n\ldots X_3D \Rightarrow \ldots \Rightarrow \\ c_1c_2\ldots c_{n-1}X_nD \Rightarrow c_1c_2\ldots c_{n-1}c_nD \Rightarrow c_1c_2\ldots c_{n-1}c_n. \end{aligned}$$

В 2) из C выводится слово, составленное из букв a и b, за которым следует его зеркальное отражение, составленное соответственно из букв A и B.

В 3) нетерминалы A и B перемещаются к правому концу слова, где A становится терминалом a, а B становится терминалом b, вступая в контакт с нетерминалом D. Нетерминалы A и B могут превратиться в терминалы единственным способом - только передвинувшись к правому концу слова. При этом слово, составленное из букв A и B, будет обращено и совпадет, таким образом, со словом с буквами a и b, выведенным из C в выводе 2).

Комбинируя выводы 1), 2) и 3), получаем для $n \geqslant 0$

$$S \Rightarrow^* c_1c_2\ldots c_nc_1c_2\ldots c_n,$$

где $c_i \in \{a,b\}$ для $1 \leqslant i \leqslant n$. Итак, $\{\omega\omega \mid \omega \in \{a,b\}^*\} \subseteq L(G)$.

Теперь докажем, что $L(G) \subseteq \{\omega\omega \mid \omega \in \{a,b\}^*\}$. Для этого надо проверить, что из S выводятся только те терминальные слова, которые имеют вид $\omega\omega$. Зададим два гомоморфизма:

$$g\colon \{a;b;A;B\}^* \to \{a;b;A;B\}^*, \qquad h\colon \{a;b;A;B\}^* \to \{a;b;A;B\}^*,$$

удовлетворяющие условиям:

$$g(a) = a, \quad g(b) = b, \quad g(A) = g(B) = \varepsilon, \quad h(A) = A, \quad h(B) = B, \quad h(a) = h(b) = \varepsilon.$$

Применяя метод математической индукции по параметру m и анализируя множество продукций P, легко доказать следующее вспомогательное утверждение: если $S \Rightarrow^m \alpha$, то α представимо в виде

$$c_1 c_2 \dots c_n U \beta V,$$

где $c_i \in \{a; b\}$, $U \in \{C; \varepsilon\}$, $V \in \{D; \varepsilon\}$, а β такое слово длины n из языка $\{a; b; A; B\}^*$, что

$$g(\beta) = c_1 c_2 \dots c_i, \qquad h(\beta) = X_n X_{n-1} \dots X_{i+1},$$

где

$$X_j = \begin{cases} A, \text{ если } c_j = a; \\ B, \text{ если } c_j = b. \end{cases}$$

Из этого утверждения вытекает, что все слова из $L(G)$ имеют вид

$$c_1 c_2 \dots c_n c_1 c_2 \dots c_n,$$

где $c_i \in \{a, b\}$, поэтому $L(G) \subseteq \{\omega\omega \mid \omega \in \{a, b\}^*\}$.

Итак, равенство $L(G) = \{\omega\omega \mid \omega \in \{a, b\}^*\}$ доказано. ■

Упражнение 1.3.2. Докажите, что язык $L(G_0)$ из примера 1.3.3 совпадает со множеством всех арифметических выражений, построенных из пяти символов:

$$+, \quad *, \quad a, \quad (, \quad).$$ □

Упражнение 1.3.3. Докажите, что грамматика из примера 1.3.4 порождает язык

$$\{a^n b^n c^n \mid n \geqslant 1\}.$$ □

1.4. Классификация грамматик

В соответствии с подходом Хомского грамматики классифицируют по структуре их продукций. Грамматика $G = (N, \Sigma, P, S)$ называется:

1) *праволинейной*, если каждая продукция из P имеет вид $A \to xB$ или $A \to x$, где $A, B \in N$, $x \in \Sigma^*$;

2) *контекстно-свободной* (или *бесконтекстной*), если каждая продукция из P имеет вид $A \to \alpha$, где $A \in N$, $\alpha \in (N \cup \Sigma)^*$;

3) *контекстно-зависимой* (или *неукорачивающейся*), если каждая продукция из P имеет вид $\alpha \to \beta$, где $|\alpha| \leqslant |\beta|$.

Грамматика, не удовлетворяющая ни одному из указанных ограничений. называется грамматикой общего вида (или без ограничений). Далее мы будем пользоваться сокращениями ПЛ, КС и КЗ для терминов «праволинейный», «контекстно-свободный» и «контекстно-зависимый» соответственно.

Грамматика примера 1.3.2 — праволинейная. Такой же является и грамматика $G = (\{S\}, \{0; 1\}, \{S \to 0S \mid 1S \mid \varepsilon\}, S)$ с языком $\{0, 1\}^*$.

Примером КС-грамматики служит грамматика из примера 1.3.3. Ясно, что каждая ПЛ-грамматика является контекстно-свободной.

В примере 1.3.4 грамматика является контекстно-зависимой.

КЗ-грамматика не допускает продукций вида $A \to \varepsilon$, называемых ε-продукциями. Поэтому КС-грамматика, содержащая ε-продукции, не является КЗ-грамматикой.

Если язык L порождается грамматикой типа i, то L называется языком типа i. Таким образом, язык $L(G)$ примера 1.3.2 — праволинейный, язык $L(G_0)$ примера 1.3.3 — контекстно-свободный, а язык $L(G)$ примера 1.3.4 — контекстно-зависимый. Язык, порожденный грамматикой примера 1.3.5, — это язык без ограничений.

Если язык задан какой-то грамматикой, это еще не значит, что его нельзя породить менее мощной грамматикой. Например, КС-грамматика

$$G = (\{A; S\}, \{0; 1\}, \{S \to AS \mid \varepsilon; A \to 0 \mid 1\}, S)$$

порождает язык $\{0; 1\}^*$, который, как отмечено выше, можно получить и с помощью ПЛ-грамматики.

Определенные выше четыре типа грамматик и языков называют *иерархией Хомского*.

Упражнение 1.4.1. Постройте ПЛ-грамматику для языка, состоящего из идентификаторов, которые могут быть произвольной длины, но должны начинаться с буквы (как в Алголе). □

Упражнение 1.4.2. Постройте ПЛ-грамматику для языка, состоящего из идентификаторов, которые должны содержать от одного до шести символов и начи-

наться с буквы I, J, K, L, M или N(как идентификаторы целых переменных в Фортране). □

Упражнение 1.4.3. Постройте КС-грамматику, порождающую язык

$$(a_1 a_2 \dots a_n a_n \dots a_2 a_1 \mid a_i \in \{0,1\}, 1 \leqslant i \leqslant n\}.$$ □

1.5. Распознаватели

В 1.3 было отмечено, что грамматика — это математическая система, определяющая язык. Второй метод, обеспечивающий задание языка конечными средствами, состоит в использовании *распознавателей*. Распознаватель это очень схематизированный алгоритм, определяющий некоторое множество. В нём можно выделить три основные части: *входную ленту* ВЛ, *управляющее устройство* с конечной памятью УУ, *вспомогательную* или *рабочую память* РП.

Входная лента (ВЛ) это линейная последовательность ячеек, причем каждая ячейка содержит точно одну букву из некоторого конечного входного алфавита. Самую левую и самую правую ячейки могут занимать особые концевые маркеры; причем маркер может стоять только на правом конце ленты или маркеров может не быть совсем.

Входная головка в каждый данный момент читает одну входную ячейку. За один шаг работы распознавателя входная головка может сдвинуться на одну ячейку влево, остаться неподвижной или сдвинуться на одну ячейку вправо. Распознаватель, который никогда не передвигает свою входную головку влево, называется односторонним. Обычно предполагается, что входная головка только читает, то есть в ходе работы распознавателя буквы на входной ленте не меняются. Но можно рассматривать и такие распознаватели, входная головка которых и читает, и пишет.

Рабочей памятью (РП) распознавателя может быть любого типа хранилище информации. Предполагается, что алфавит памяти конечен, и хранящаяся в памяти информация образована только из букв этого алфавита. Предполагается также, что в любой момент времени можно конечными средствами описать содержимое и структуру памяти, хотя с течением времени объем памяти может становиться сколь угодно большим.

Ядром распознавателя является управляющее устройство с конечной памятью (УУ), под которым можно понимать программу, управляющую поведением распознавателя. УУ представляет собой конечное множество состояний

вместе с функцией, которое описывает, как меняются состояния в соответствии с текущим входным символом и текущей информацией, извлеченной из памяти. УУ определяет также, в каком направлении сдвинуть входную головку и какую информацию поместить в память.

Распознаватель работает, проделывая некоторую последовательность *шагов* (или *тактов*). В начале такта читается текущий входной символ и исследуется память. После этого с распознавателем происходит следующее:

1) входная головка сдвигается на одну ячейку влево, одну ячейку вправо или сохраняется в исходном положении;

2) в память помещается некоторая информация;

3) изменяется состояние управляющего устройства.

Поведение распознавателя удобно описывать в терминах *конфигураций* распознавателя, которые описывают состояние УУ, содержимое ВЛ вместе с положением входной головки, и, наконец, содержимое РП. Во множестве всех состояний выделяют начальное состояние и множество заключительных состояний. Конфигурация называется *начальной*, если УУ находится в начальном состоянии, входная головка читает самую левую букву на ВЛ, и РП имеет заранее установленное начальное содержимое. Конфигурация называется *заключительной*, если УУ находится в заключительном состоянии, а входная головка читает правый концевой маркер или, если маркер отсутствует, сошла с правого конца входной ленты. (Иногда требуют, чтобы РП в заключительной конфигурации тоже удовлетворяла некоторым условиям.)

Говорят, что распознаватель *допускает* входное слово ω, если, начиная с начальной конфигурации, в которой слово ω записано на входной ленте, распознаватель может проделать последовательность шагов, заканчивающуюся заключительной конфигурацией. Язык, определяемый распознавателем, это множество входных слов, которые он допускает.

Для каждого класса грамматик из иерархии Хомского существует естественный класс распознавателей, определяющий тот же класс языков. Некоторые из таких распознавателей будут изучаться далее.

1.6. Упражнения

Для каждой грамматики, встречающейся в заданиях, следует указать её тип (в иерархии Хомского). Написать грамматику, порождающую:

1) язык Σ^*, где (a) $\Sigma = \{0, 1\}$; (b) Σ — произвольный (конечный) алфавит;
2) некоторый конечный язык $L = \{\omega_i\}_{i=1}^{n}$;
3) $\{a^+b^+\}, \{a^nb^n \mid n \in \mathbb{N}\}, \{a^nb^na^m \mid m, n \in \mathbb{N}\}$; $\{a^nb^nc^n \mid n \in \mathbb{N}\}$;
4) множество правильных скобочных последовательностей («язык Дика») с одним типом скобок;
   множество правильных скобочных последовательностей («язык Дика») с двумя типами скобок;

ГЛАВА 2

РЕГУЛЯРНЫЕ ЯЗЫКИ

2.1. Регулярные множества и регулярные выражения

Пусть Σ — конечный алфавит. *Регулярное множество* над алфавитом Σ определяется рекурсивно следующим образом: множества $\varnothing$, $\{\varepsilon\}$ и $\{a\}$ для каждого a из Σ — регулярные; если P и Q — регулярные множества, то таковы же и множества $P \cup Q$, PQ, P^*; ничто другое не является регулярным множеством. Другими словами, множество регулярно над алфавитом Σ тогда и только тогда, когда оно либо $\varnothing$, либо $\{\varepsilon\}$, либо $\{a\}$ для некоторого $a \in \Sigma$, либо его можно получить из этих множеств применением конечного числа операций объединения, конкатенации и итерации.

Регулярное множество над алфавитом Σ иначе называется регулярным языком над этим алфавитом

Для обозначения регулярных множеств используют регулярные выражения. Регулярные выражения над алфавитом Σ и регулярные множества, которые они обозначают, определяются рекурсивно следующим образом: $\varnothing$ — регулярное выражение, обозначающее регулярное множество $\varnothing$; ε — регулярное выражение, обозначающее регулярное множество $\{\varepsilon\}$ если $a \in \Sigma$, то a — регулярное выражение, обозначающее регулярное множество $\{a\}$; если p и q — регулярные выражения, обозначающие регулярные множества P и Q соответственно, то $(p + q)$, (pq) и $(p)^*$ — регулярные выражения, обозначающие $P \cup Q$, PQ и p^* соответственно; ничто другое не является регулярным выражением.

Для сокращения будем далее вместо pp^* писать p^+ и устранять из регулярных выражений лишние скобки там, где это не приводит к недоразумениям. При этом будем предполагать, что наивысшим приоритетом обладает итерация, затем конкатенация и, наконец, операция +. Например, вместо $(01(1^*)) + (1(0^*)))$ будем писать $01^+ + 10^*$.

Пример 2.1.1. Приведем несколько регулярных выражений и соответствующих им множеств:

1) $(0+1)^*$ обозначает $\{0;1\}^*$;
2) $(0+1)^*0101$ обозначает множество всех слов над алфавитом $\{0;1\}$ с суффиксом 0101;
3) $(a+b+c+d)(a+b+c+d+0+1)^*$ обозначает множество всех слов над алфавитом $\{0;1;a;b;c;d\}$ с префиксами a, b, c или d;
4) $(00+11)^*((01+10)(00+11)^*(01+10)(01+11)^*)^*$обозначает множество всех слов над алфавитом $\{0;1\}$, содержащих четное число нулей и четное число единиц.

Последний пример представляется весьма поучительным и хотелось бы порекомендовать читателю поразмышлять над ним до тех пор, пока он не станет «очевидным». ■

Для каждого регулярного множества можно найти по крайней мере одно регулярное выражение, которое его обозначает. Обратно, для каждого регулярного выражения можно построить регулярное множество, обозначаемое этим выражением. Но для каждого регулярного множества существует бесконечно много обозначающих его регулярных выражений. Будем говорить, что два регулярных выражения равны $(=)$, если они обозначают одно и то же множество.

Лемма 2.1.1. *Пусть α, β, γ — регулярные выражения. Тогда*

1) $\alpha + \varnothing = \alpha$,
2) $\alpha + \alpha = \alpha$,
3) $\alpha + \beta = \beta + \alpha$,
4) $\alpha + (\beta + \gamma) = (\alpha + \beta) + \gamma$,
5) $\alpha(\beta + \gamma) = \alpha\beta + \alpha\gamma$,
6) $(\alpha + \beta)\gamma = \alpha\gamma + \beta\gamma$,
7) $\alpha\varepsilon = \varepsilon\alpha = \alpha$,
8) $\varnothing\alpha = \alpha\varnothing = \varnothing$,
9) $\alpha(\beta\gamma) = (\alpha\beta)\gamma$,
10) $\varnothing^* = \varepsilon$,
11) $(\alpha^*)^* = \alpha^*$,
12) $\alpha^* = \alpha + \alpha^*$,
13) $(\alpha + \beta)^* = (\alpha^*\beta^*)^*$,
14) $\alpha\alpha^* + \varepsilon = \alpha^*$. □

Доказательство. Каждое из равенств легко проверить, используя свойства множеств и приведенные выше определения. Рассмотрим, для примера, лишь некоторые из приведенных равенств.

Равенство 4). Пусть α, β и γ — регулярные выражения, обозначающие регулярные множества A, B и Γ соответственно. Тогда $\beta+\gamma$ обозначает $B\cup\Gamma$, а $\alpha+(\beta+\gamma)$ обозначает $A\cup(B\cup\Gamma)$; аналогично, $(\alpha+\beta)+\gamma$ обозначает $(A\cup B)\cup\Gamma$. Но $A\cup(B\cup\Gamma)=(A\cup B)\cup\Gamma$, поэтому $\alpha+(\beta+\gamma)=(\alpha+\beta)+\gamma$.

Равенство 12). Пусть α — регулярное выражение, обозначающее регулярное множество A. Ясно, что множество $A^*=\bigcup_{n\geqslant 0}A^n$ содержит A, поэтому $A^*\cup A=A^*$ и, следовательно, $\alpha^*+\alpha=\alpha^*$. ■

Упражнение 2.1.1. Приведите доказательства всех равенств из леммы 2.1.1 Постарайтесь обнаружить новые равенства и доказать их (см. [5]). □

В дальнейшем, если это не будет приводить к недоразумению, мы не будем различать регулярное выражение и обозначаемое им регулярное множество.

2.2. Уравнения и системы уравнений с регулярными коэффициентами

Рассмотрим уравнение

$$X=\alpha X+\beta, \tag{2.2.1}$$

где α и β — регулярные выражения над некоторым алфавитом Σ. Прямой подстановкой проверим, что $X=\alpha^*\beta$ — решение уравнения (2.2.1):

(?) $\alpha^*\beta=\alpha(\alpha^*\beta)+\beta$,

(?) $\alpha^*\beta=(\alpha\alpha^*)\beta+\varepsilon\beta$,

(?) $\alpha^*\beta=(\alpha\alpha^*+\varepsilon)\beta$,

(!) $\alpha^*\beta=\alpha^*\beta$.

Решение уравнения с регулярными коэффициентами может быть не единственным. Например, если в уравнении (2.2.1) потребовать, чтобы $\varepsilon\in\alpha$, то

$$X=\alpha^*(\beta\cup L)$$

будет решением этого уравнения для любого (не обязательно регулярного) языка L. Действительно,

(?) $\alpha^*(\beta\cup L)=\alpha(\alpha^*(\beta\cup L))\cup\beta$,

(?) $\alpha^*(\beta\cup L)=(\alpha\alpha^*)(\beta\cup L)\cup\beta$,

(?) $\alpha^*(\beta\cup L)=\alpha^*(\beta\cup L)\cup\beta$,

(!) $\alpha^*(\beta\cup L)=\alpha^*(\beta\cup L)$.

Таким образом, уравнение (2.2.1) при $\varepsilon \in \alpha$ имеет бесконечно много решений.

В теории формальных языков представляет интерес, главным образом, наименьшее решение, которое называют наименьшей неподвижной точкой. (В математике решение уравнения вида $x = f(x)$ принято называть неподвижной точкой отображения f, поэтому термин «неподвижная точка» используется и здесь [4].)

Теорема 2.2.1. *Пусть α и β — регулярные выражения над алфавитом Σ и $X \notin \Sigma$. Рассмотрим уравнение* (2.2.1)*. Тогда*

1) *$X = \alpha^*\beta$ — наименьшая неподвижная точка уравнения;*
2) *если $\varepsilon \notin \alpha$, то $X = \alpha^*\beta$ — единственное решение уравнения;*
3) *если $\varepsilon \in \alpha$, то для произвольного (не обязательно регулярного) языка L язык $\alpha^*(\beta \cup L)$ является решением уравнения, и наоборот, всякое решение уравнения может быть записано в таком виде.* □

Доказательство. 1) Ранее мы проверили, что $X = \alpha^*\beta$ — решение уравнения (2.2.1). Пусть Ω — произвольное решение этого уравнения; покажем, что $\alpha^*\beta \subset \Omega$. Предположим, что $\omega \in \alpha^*\beta$. Тогда $\omega = \omega_k\omega_{k-1}\ldots\omega_1 b$, где $\omega_i \in \alpha$, $b \in \beta$. Из того, что Ω- решение уравнения (2.2.1), вытекает, что

$$\Omega = \alpha\Omega + \beta$$

и, следовательно, $\beta \subset \Omega$, $\alpha\Omega \subset \Omega$. Тогда

$$b \in \Omega, \omega_1 b \in \Omega, \omega_2\omega_1 b \in \Omega, \ldots, \omega_k\omega_{k-1}\ldots\omega_1 b \in \Omega.$$

Таким образом, $\alpha^*\beta \subset \Omega$. Это означает, что $X = \alpha^*\beta$ — наименьшая неподвижная точка.

2) Пусть $\varepsilon \notin \alpha$. Предположим, что Ω — произвольное решение уравнения (2.2.1), и покажем, что $\Omega = \alpha^*\beta$. Пусть $\omega = a_1a_2\ldots a_m \in \Omega$ где $a_i \in \Sigma$. Из того, что $\Omega = \alpha\Omega + \beta$, вытекает: либо $\omega \in \beta \subset \alpha^*\beta$, либо $\omega \in \alpha\Omega$. Если $\omega \in \alpha\Omega = \alpha\alpha\Omega + \alpha\beta$, то либо $\omega \in \alpha\beta \subset \alpha^*\beta$, либо $\omega \in \alpha\alpha\Omega$. Продолжим этот процесс до тех пор, пока альтернатива $\omega \in \alpha\alpha\ldots\alpha\Omega$ окажется невозможной (в силу того, что ω имеет m букв, а $\varepsilon \notin \alpha$, это обязательно произойдет по крайней мере при $k > m$). В итоге получим, что $\omega \in \alpha^*\beta$.

3) Пусть $\varepsilon \in \alpha$. Ранее мы проверили, что множество $\alpha^*(\beta \cup L)$, где L — произвольный язык, является решением уравнения (2.2.1). Пусть теперь Ω —

какое-нибудь решение этого уравнения; покажем, что Ω можно записать в виде $\alpha^*(\beta \cup L)$, где L — некоторый язык. По условию

$$\Omega = \alpha\Omega + \beta.$$

Отсюда вытекает, что

$$\alpha\Omega \subseteq \Omega, \beta \subseteq \Omega.$$

С другой стороны, из условия $\varepsilon \in \alpha$ следует, что

$$\Omega \subseteq \alpha\Omega.$$

Получаем равенство:

$$\Omega = \alpha\Omega.$$

Его непосредственным следствием является равенство:

$$\Omega = \alpha^*\Omega.$$

Тогда в силу соотношения $\beta \subseteq \Omega$ получаем:

$$\Omega = \alpha^*(\Omega \cup \beta),$$

так как $\Omega = \Omega \cup \beta$. Итак, действительно показано, что если множество Ω — решение уравнения (2.2.1), то оно имеет такой вид:

$$\Omega = \alpha^*(L \cup \beta),$$

где L — некоторый язык. ■

Систему уравнений с регулярными коэффициентами над некоторым алфавитом Σ назовем стандартной системой со множеством неизвестных

$$\Delta = \{X_1; X_2; \ldots; X_n\},$$

если она имеет вид

$$\begin{cases} X_1 = \alpha_{10} + \alpha_{11}X_1 + \alpha_{12}X_2 + \ldots + \alpha_{1n}X_n \\ X_2 = \alpha_{20} + \alpha_{21}X_1 + \alpha_{22}X_2 + \ldots + \alpha_{2n}X_n \\ \ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots \\ X_n = \alpha_{n0} + \alpha_{n1}X_1 + \alpha_{n2}X_2 + \ldots + \alpha_{nn}X_n \end{cases} \tag{2.2.2}$$

где α_{ij} — регулярные выражения над Σ и $\Sigma \cap \Delta = \varnothing$.

Коэффициентами уравнений являются регулярные выражения α_{ij}. Если $\alpha_{ij} = \varnothing$, то в уравнении для X_i нет компонента, содержащего X_j. Аналогично, если $\alpha_{ij} = \varepsilon$, то в уравнении для X_i компонент, содержащий X_j, совпадает с X_j. Иными словами, $\varnothing$ играет роль коэффициента 0, а ε — роль коэффициента 1 в обычных линейных алгебраических уравнениях.

Пусть Q — стандартная система уравнения со множеством неизвестных Δ и с коэффициентами над алфавитом Σ. Отображение f множества Δ во множество языков над Σ называют решением системы Q, если после подстановки $f(X)$ вместо $X(\in \Delta)$ в каждое уравнение системы все уравнения становятся равенствами. Отображение $f\colon \Delta \to P(\Sigma^*)$ называют наименьшей неподвижной точкой системы Q, если, во-первых, f — решение и, во-вторых, $f(X) \subseteq g(X)$ для любого другого решения g и всех $X \in \Delta$.

Теорема 2.2.2. *Пусть Q — стандартная система уравнений, а Y_i - множество всех слов $\omega \dots \omega_m$, где $m \geqslant 1$ и $\omega_m \in \alpha_{j_m 0}$, $\omega_k \in \alpha_{j_k j_{k+1}}$ для некоторой последовательности $j_i = j_1, \dots, j_m (\in \{1; 2; \dots; n\})$. Отображение f множества $\Delta = \{X_1; X_2; \dots; X_n\}$ во множество языков над Σ определим равенством: $f(X_i) = Y_i$, где $i \in \{1; 2; \dots; n\}$. Тогда f — наименьшая неподвижная точка системы Q.* □

Доказательство. Путем непосредственной подстановки проверяется, что для всех i

$$f(X_i) = \alpha_{i0} \cup \alpha_{i1} f(X_1) \cup \dots \cup \alpha_{in} f(X_n).$$

Таким образом, f — решение.

Покажем, что f — наименьшая неподвижная точка. Пусть g - какое-нибудь решение системы и $i \in \{1; 2; \dots; n\}$. Рассмотрим произвольное слово ω из $f(X_i)$. Тогда $\omega = \omega_1 \dots \omega_m (m \geqslant 1)$, где для некоторой последовательности чисел $j_1, \dots, j_m (\in \{1; 2; \dots; n\})$ выполнены условия:

$$\omega_m \in \alpha_{j_m 0}, \quad \omega_k \in \alpha_{j_k j_{k+1}}$$

Так как g — решение, то

$$g(X_i) = \alpha_{i0} \cup \alpha_{i1} g(X_1) \cup \dots \cup \alpha_{in} g(X_n).$$

В частности,

$$\alpha_{i0} \subseteq g(X_i), \alpha_{ik} g(X_k) \subseteq g(X_i)$$

для всех i и k. Тогда

$$\begin{gathered}\omega_m \in \alpha_{j_m 0} \subseteq g(X_{j_m}),\\ \omega_{m-1}\omega_m \in \alpha_{j_{m-1}j_m} g(X_{j_m}) \subseteq g(X_{j_{m-1}}),\\ \dots\\ \omega = \omega_1\omega_2\dots\omega_m \in \alpha_{j_1 j_2} g(X_{j_2}) \subseteq g(X_{j_1}) = g(X_i).\end{gathered}$$

Таким образом, $f(X_i) \subseteq g(X_i)$. Отсюда следует, что f — наименьшая неподвижная точка системы Q. ■

В этой теореме получено описание наименьшей неподвижной точки стандартной системы с регулярными коэффициентами, но, во-первых, нет гарантий, что наименьшая неподвижная точка является регулярным множеством, и, во-вторых, неясно, как эту наименьшую неподвижную точку найти. Эти вопросы будут рассмотрены в следующем пункте.

Упражнение 2.2.1. Проверьте (любым способом), что наименьшая неподвижная точка системы

$$\begin{cases} X_1 = a_{10} + a_{11}X_1 + a_{12}X_2 \\ X_2 = a_{20} + a_{21}X_1 + a_{22}X_2 \end{cases} \tag{2.2.3}$$

имеет вид

$$\begin{cases} f(X_1) = (a_{11} + a_{12}a_{22}^*a_{21})^*(a_{10} + a_{12}a_{22}^*a_{20}), \\ f(X_2) = (a_{22} + a_{21}a_{11}^*a_{12})^*(a_{20} + a_{21}a_{11}^*a_{10}) \end{cases} \tag{2.2.4}$$

□

Упражнение 2.2.2. Сформулируйте и докажите аналог теоремы 2.2.1 для систем. □

2.3. Алгоритм решения систем уравнений с регулярными коэффициентами

Рассмотрим стандартную систему уравнений с регулярными коэффициентами вида (2.2.2).

Алгоритм 2.3.1 Решение систем уравнений с регулярными коэффициентами

Вход Стандартная система уравнений с регулярными коэффициентами над алфавитом Σ и множеством неизвестных $\Delta = \{X_1, \ldots, X_n\}$.

Выход Решение системы в виде $X_1 = \alpha_1, \ldots, X_n = \alpha_n$, где α_i — регулярные выражения над алфавитом Σ.

Метод Метод напоминает обычный метод Гаусса решения систем линейных уравнений и основан на систематическом использовании формулы $X = \alpha^*\beta$ для наименьшей неподвижной точки уравнения (2.2.1).

Шаг 1. Положим $i = 1$.

Шаг 2. Если $i = n$, перейти к шагу 4. В противном случае с помощью тождеств из леммы 2.1.1 записать уравнение для X_i в виде $X_i = \alpha X_i + \beta$, где α и β — регулярные выражения над Σ. Затем в правых частях уравнений для $X_{i+1}, \ldots, X_n$ заменить X_i регулярным выражением $\alpha^*\beta$.

Шаг 3. Увеличить i на 1 и вернуться к шагу 2.

Шаг 4. Записать уравнение для X_n в виде $X_n = \alpha X_n + \beta$ где α и β — регулярные выражения над Σ. (После выполнения шага 2 для каждого i при $l < n$ в правой части уравнения для X_i не будет неизвестных $X_1, \ldots, X_{i-1}$. В частности, на шаге 4 этим свойством будет обладать и уравнение для X_n.) Перейти к шагу 5 (при этом $i = n$).

Шаг 5. Уравнение для X_i имеет вид $X_i = \alpha X_i + \beta$, где α и β — регулярные выражения над Σ. Записать $X_i = \alpha^*\beta$ и в уравнения для $X_{i-1}, \ldots, X_1$ подставить $\alpha^*\beta$ вместо X_i.

Шаг 6. Если $i = 1$, то остановиться. В противном случае уменьшить i на 1 и вернуться к шагу 5.

Теорема 2.3.1. *Алгоритм 2.3.1 строит наименьшую неподвижную точку системы уравнений как регулярный язык.* □

Доказательству теоремы 2.3.1 предпошлём две леммы.

Лемма 2.3.1. *Пусть Q_1 и Q_2 — системы уравнений с регулярными коэффициентами до и после о д н о г о применения шага 2 алгоритма 2.3.1. Тогда эти системы имеют одну и ту же наименьшую неподвижную точку.* □

Доказательство. Отметим, что в ходе работы алгоритма для $1 \leqslant l < i$ коэффициент при A_l в уравнении для A_i оказывается равным $\varnothing$. Предположим, что на

шаге 2 рассматривается уравнение

$$A_i = \alpha_{i0} + \alpha_{ii}A_i + \alpha_{i,i+1}A_{i+1} + \ldots + \alpha_{in}A_n$$

системы Q_1.

В системах Q_1 и Q_2 уравнения для A_j при $j \leqslant i$ совпадают, а при $j > i$ эти уравнения совпадать не обязаны.

Пусть $j > i$ и

$$A_j = \alpha_{j0} + \sum_{h=i}^{n} \alpha_{jh}A_h \tag{2.3.1}$$

это уравнение для A_j в системе Q_1. Тогда в силу того, что

$$A_i = \alpha_{ii}^{*}\alpha_{i0} + \sum_{h=i+1}^{n} \alpha_{ii}^{*}\alpha_{ih}A_h,$$

уравнение для A_j в системе Q_2 имеет вид

$$A_j = (\alpha_{j0} + \alpha_{ji}\alpha_{ii}^{*}\alpha_{i0}) + \sum_{h=i}^{n} (\alpha_{jk} + \alpha_{ji}\alpha *_{ii} \alpha_{ih})A_h. \tag{2.3.2}$$

Пусть

$$\beta_{jk} = \alpha_{jh} + \alpha_{ji}\alpha *_{ii} \alpha_{jk} \quad (k \in \{0; 1 + 1; \ldots; n\}.$$

Через f_1 и f_2 обозначим наименьшие неподвижные точки систем Q_1 и Q_2 соответственно. Далее будем пользоваться тем описанием наименьших неподвижных точек систем, которое дано в теореме 2.2.2. В частности, $f_2(A_j)$ — множество всех слов, представимых в виде $\omega_1 \ldots \omega_m$, где $m \geqslant 1$ и $\omega_m \in \beta_{j_m 0}$, $\omega_k \in \beta_{j_k j_{k+1}}$ для некоторой последовательности $j_1 = j, \ldots, j_m (\in \{1; 2; \ldots; n\})$.

Сравнение формул (2.3.1) и (2.3.2) показывает, что

$$f_2(A_j) \subseteq f_1(A_j).$$

Действительно, любое слово x из $\alpha_{ji}\alpha_{ii}^{*}\alpha_{jk}$ можно представить в виде $x_1 x_2 \ldots x_r$, где $\omega_1 \in \alpha_{ji}, \omega_r \in \alpha_{jk}$ и $\omega_2, \ldots, \omega_{r-1} \in \alpha_{ii}$. Таким образом, слово ω является конкатенацией слов, каждое из которых принадлежит множеству, обозначаемому некоторым коэффициентом системы Q_1, и индексы этих коэффициентов удовлетворяют необходимым условиям.

Докажем обратное включение

$$f_2(A_j) \supseteq f_1(A_j).$$

Предположим, что $\omega \in f_1(A_j)$. Тогда $\omega = \omega_1 \ldots \omega_m$, где для некоторой последовательности ненулевых индексов $l_i = l_1, \ldots, l_m$ выполнены условия $\omega_m \in \alpha_{l_m 0}$ и $\omega_p \in \alpha_{l_p l_{p+1}}$ для $1 \leqslant p < m$. Сгруппируем слова ω_p так, чтобы имело место равенство $\omega = y_1 \ldots y_r$, где $y_p = \omega_t \ldots \omega_s$, и

1) если $l_t \leqslant i$, то $s = t + 1$,

2) если $l_t > i$, то s таково, что $l_{t+1} = \ldots = l_s = i$, $l_{s+1} \neq i$.

Отсюда следует, что в любом случае y_p — коэффициент при $A_{j_{s+1}}$ в уравнении для A_{l_t} системы Q_2, и, значит, $\omega \in f_2(A_j)$. В итоге получаем, что $f_1(A_j) = f_2(A_j)$ для всех j. ■

Лемма 2.3.2. *Пусть Q_1 и Q_2 — системы уравнений с регулярными коэффициентами до и после одного применения шага 5 алгоритма 2.3.1. Тогда эти системы имеют одну и ту же наименьшую неподвижную точку.* □

Доказательство. Аналогично доказательству леммы 2.3.1 и здесь не приводится.

Доказательство теоремы 2.3.1. После того как шаг 5 алгоритма 2.3.1 применен для всех i, мы получим новую систему, у которой все уравнения имеют вид $X_i = \alpha_i$, где α_i — регулярное выражение над алфавитом Σ. Согласно теореме 2.2.2 отображение f, для которого $f(X_i) = \alpha_i$, является наименьшей неподвижной точкой этой новой системы. В силу лемм 2.3.1 и 2.3.2 в ходе работы алгоритма наименьшая неподвижная точка не менялась. Таким образом, показано, что алгоритм 2.3.1 строит наименьшую неподвижную точку системы уравнений как регулярный язык. ■

Упражнение 2.3.1. Найдите наименьшую неподвижную точку системы (2.2.2) при условии, что $\alpha_{i0} = \varnothing$ для всех i. □

2.4. Совпадение классов регулярных и ПЛ-языков

Лемма 2.4.1. *Пусть Σ — конечный алфавит. Множества $\varnothing$, $\{\varepsilon\}$ и $\{a\}$ для всех $a \in \Sigma$ являются ПЛ-языками.* □

Доказательство. Чтобы доказать лемму, достаточно для каждого из трех множеств построить такую ПЛ-грамматику G, чтобы язык $L(G)$ совпадал с этим множеством:

1) $G_\varnothing = (\{S\}, \Sigma, \varnothing, S)$ — ПЛ-грамматика и $L(G_\varnothing) = \varnothing$;

2) $G_\varepsilon = (\{S\}, \Sigma, \{S \to \varepsilon\}, S)$ — ПЛ-грамматика и $L(G_\varepsilon) = \{\varepsilon\}$;

3) $G_a = (\{S\}, \Sigma, \{S \to a\}, S)$ — ПЛ-грамматика и $L(G_a) = \{a\}$. ■

Лемма 2.4.2. *Пусть Σ — конечный алфавит. Если L_1, L_2 — ПЛ-языки над Σ, то и языки $L_3 = L_1 \cup L_2$, $L_4 = L_1 L_2$, $L_5 = L_1^*$ тоже являются ПЛ-языками над Σ.* □

Доказательство. Пусть $G_1 = (N_1, \Sigma, P_1, S_1)$, $G_2 = (N_2, \Sigma, P_2, S_2)$ — такие ПЛ-грамматики, для которых $L(G_1) = L_1$, $L(G_2) = L_2$. Не теряя общности будем предполагать, что алфавиты N_1 и N_2 не пересекаются. Теперь, как и в доказательстве леммы 2.4.1, для каждого из трех новых языков L_i построим такую ПЛ-грамматику G_i, чтобы $L(G_i) = L_i$, $i \in \{3; 4; 5\}$.

1) Пусть $G_3 = (N_1 \cup N_2 \cup \{S_3\}, \Sigma, P_1 \cup P_2 \cup \{S_3 \to S_1 \mid S_2\}, S_3)$, где S_3 — новый нетерминальный символ, не принадлежащий ни N_1, ни N_2. Ясно, что G_3 - ПЛ-грамматика, причем $L(G_3) = L_1 \cup L_2$, так как для каждого вывода $S_3 \Rightarrow^+_{G_3} \omega$ существует либо вывод $S_1 \Rightarrow^+_{G_1} \omega$, либо вывод $S_2 \Rightarrow^+_{G_2} \omega$, и обратно. Таким образом, $L_3 = L_1 \cup L_2$ — ПЛ-язык.

2) Пусть $G_4 = (N_1 \cup N_2\}, \Sigma, P_4, S_1,)$, а множество продукций P_4 определяется так:

   (*i*) если $A, B \in N$, $x \in \Sigma^*$ и $(A \to xB) \in P_1$, то $(A \to xB) \in P_4$;

   (*ii*) если $A \in N$, $x \in \Sigma^*$ и $(A \to x) \in P_1$, то $(A \to xS_2) \in P_4$;

   (*iii*) $P_2 \subset P_4$.

   Ясно, что G_4 — ПЛ-грамматика. Если $S_1 \Rightarrow^+_{G_1} \omega$, то $S_1 \Rightarrow^+_{G_4} \omega S_2$, а если $S_2 \Rightarrow^+_{G_2} x$, то $S_2 \Rightarrow^+_{G_4} x$. Таким образом, $L(G_1) \cup L(G_2) \subseteq L(G_4)$.

   Проверим теперь противоположное вложение. Пусть $S_1 \Rightarrow^+_{G_4} \omega$. Так как в P_4 нет продукций вида $A \to x$, которые попали бы туда из P_1, то этот вывод можно записать в виде $S_1 \Rightarrow^+_{G_4} xS_2 \Rightarrow^+_{G_4} xy$, где $\omega = xy$, и все продукции, используемые в выводе $S_1 \Rightarrow^+_{G_4} xS_2$, попали в P_4 с помощью (i) и (ii). Следовательно, должны быть выводы $S_1 \Rightarrow^+_{G_1} x$ и $S_2 \Rightarrow^+_{G_2} y$. Отсюда вытекает вложение $L(G_4) \subseteq L(G_1)L(G_2)$.

   Итак, $L(G_4) = L(G_1)L(G_2)$.

3) Пусть грамматика $G_5 = (N_1 \cup \{S_5\}, \Sigma, P_5, S_5)$ такова, что $S_5 \notin N_1$, а P_5 строится следующим образом:

   (*i*) если $A, B \in N$, $x \in \Sigma^*$ и $(A \to xB) \in P_1$, то $(A \to xB) \in P_5$;

   (*ii*) если $A \in N$, $x \in \Sigma^*$ и $(A \to x) \in P_1$, то $\{A \to xS_5; A \to x\} \in P_5$;

   (*iii*) $\{S_5 \to S_1 \mid \varepsilon\} \subset P_5$.

Равенство $L(G_5) = (L(G_1))^*$ является следствием того, что вывод

$$S_5 \Rightarrow^+_{G_5} x_1 S_5 \Rightarrow^+_{G_5} x_1 x_2 S_5 \Rightarrow^+_{G_5} x_1 x_2 \dots x_{n-1} S_5 \Rightarrow^+_{G_5} x_1 x_2 \dots x_{n-1} x_n$$

реализуем тогда и только тогда, когда реализуема последовательность выводов

$$S_1 \Rightarrow^+_{G_1} x_1,\ S_1 \Rightarrow^+_{G_1} x_2,\ \dots\ , S_1 \Rightarrow^+_{G_1} x_n.$$

■

Лемма 2.4.3. *Рассмотрим ПЛ-грамматику* $G = (N, \Sigma, P, S)$*, где*

$$N = \{A_1 = S, \dots, A_n\},$$

и построим стандартную систему уравнений с регулярными коэффициентами, неизвестными которой служат нетерминалы из N*, а уравнение для* A_i *определяется равенством*

$$A_i = \alpha_{i0} + \alpha_{i1} A_1 + \dots + \alpha_{in} A_n,$$

где

1) $\alpha_{i0} = x_1 + \dots + x_k$*, если* $A_i \to x_1 \mid \dots \mid x_k$ *— все продукции с левой частью* A_i *и правой частью, состоящей только из терминалов (при* $k = 0$ *полагаем* $\alpha_{i0} = \varnothing$*);*
2) *для* $j > 0$ $\alpha_{ij} = x_1 + \dots + x_m$*, если* $A_i \to x_1 A_j \mid \dots \mid x_m A_j$ *— все продукции с левой частью* A_i *и правой частью, содержащей* A_j *(при* $m = 0$ *полагаем* $\alpha_{ij} = \varnothing$*).*

Пусть f *— наименьшая неподвижная точка построенной системы уравнений. Тогда* $L(G) = f(S)$ *— регулярный язык.* □

Доказательство. Пусть $\omega \in f(S)$. Ввиду теоремы 2.2.2 слово ω можно записать в виде: $\omega = \omega_1 \dots \omega_m$, где $m \geqslant l$ и $\omega_m \in \alpha_{j_m 0}$, $\omega_k \in \alpha_{j_k j_{k+1}}$ для некоторой последовательности $j_1 = 1, \dots, j_m (\in \{1; 2; \dots; n\})$. Заметим, что коэффициенты α_{ij} определены таким образом, что в грамматике G возможен вывод:

$$S \Rightarrow_G \omega_1 A_{j_2} \Rightarrow_G \omega_1 \omega_2 A_{j_3} \Rightarrow_G \dots \Rightarrow_G \omega_1 \omega_2 \dots \omega_{m-1} A_{j_m} \Rightarrow_G \omega_1 \omega_2 \dots \omega_{m-1} \omega_m.$$

Это означает, что $f(S) \subseteq L(G)$. С помощью похожих рассуждений проверяется обратное вложение $L(G) \subseteq f(S)$. Напомним теперь, что алгоритм 2.3.1 строит $f(S)$ как регулярный язык. Таким образом, язык $L(G) = f(S)$ — регулярный. ■

Теорема 2.4.1. *Пусть* Σ *— конечный алфавит. Для того, чтобы язык* L *над алфавитом* Σ *был регулярным, необходимо и достаточно, чтобы он был праволинейным.* □

Доказательство. Достаточность вытекает из леммы 2.4.3, проверим необходимость. Пусть L — регулярное множество над Σ. Напомним, что множество регулярно над алфавитом Σ тогда и только тогда, когда оно либо $\varnothing$, либо $\{\varepsilon\}$, либо $\{a\}$ для некоторого $a \in \Sigma$, либо его можно получить из этих множеств применением конечного числа операций объединения, конкатенации и итерации. По лемме 2.4.11 $\varnothing$, $\{\varepsilon\}$ и $\{a\}$ — ПЛ-языки. В силу леммы 2.4.2 для завершения доказательства теперь можно воспользоваться индукцией по числу шагов построения регулярного множества, где один шаг соответствует применению одного из правил, определяющих регулярное множество. ■

Пример 2.4.1. Пусть $G = (\{A; B; S\}, \{0; 1\}, P, S)$, где P состоит из продукций

$$S \to 0A \mid 1S \mid \varepsilon$$
$$A \to 0B \mid 1A$$
$$B \to 0S \mid 1B.$$

Построим язык $L(G)$. Применим конструкцию из леммы 2.4.3 и составим систему уравнений

$$\begin{cases} X_1 = 0X_2 + 1X_1 + \varepsilon \\ X_2 = 0X_3 + 1X_2 \\ X_3 = 0X_1 + 1X_3 \end{cases}$$

где $X_1 = S$, $X_2 = A$, $X_3 = B$. Для решения системы воспользуемся алгоритмом 2.3.1 и на его выходе получим:

$$\begin{cases} X_3 = (01^*01^*0 + 1)^*01^* \\ X_2 = 1^*0(01^*01^*0 + 1)^*01^* \\ X_1 = 1^*(01^*0(01^*01^*0 + 1)^*01^* + \varepsilon \end{cases}$$

Из теоремы 2.2.2 вытекает, что $L(G) = f(X_1)$. где f - наименьшая неподвижная точка системы уравнений. Таким образом,

$$L(G) = 1^*(01^*0(01^*01^*0 + 1)^*01^* + \varepsilon).$$ ■

Упражнение 2.4.1. Проверьте, что язык $L(G)$ из примера 2.4.1 состоит из множества всех таких слов над алфавитом $\{0; 1\}$, в которых число нулей кратно трем. □

2.5. Упражнения

Составление регулярных выражений

Написать регулярное выражение для

1) языка над $\{a, b, c\}$ из всех слов, содержащих хотя бы один символ a;
2) языка над $\{a, b, c\}$ из всех слов, содержащих хотя бы один символ a и хотя бы один символ b;
3) языка над $\{0, 1\}$ из всех слов, в которых третий с правого края символ равен 1;
4) языка над $\{0, 1\}$ из всех слов, в которых нет двух подряд идущих единиц;
5) языка над $\{0, 1\}$ из всех слов, в которых любая пара смежных нулей, расположена левее любой пары смежных единиц;
6) языка над $\{0, 1\}$ из всех слов с чередующимися нулями и единицами.

Системы линейных уравнений с регулярными коэффициентами

1) Решить системы уравнений:

$$\begin{cases} X_1 = aX_1 + aX_2; \\ X_2 = bX_2 + b. \end{cases} \tag{2.5.1}$$

$$\begin{cases} X_1 = 1X_1 + 0X_2 + \varnothing X_3; \\ X_2 = 1X_1 + \varnothing X_2 + 0X_3; \\ X_3 = 0X_1 + \varnothing X_2 + 1X_3. \end{cases} \tag{2.5.2}$$

$$\begin{cases} X_1 = a^*X_1 + (a + b)^*X_2; \\ X_2 = (a + b^*)X_1 + aX_2 + b^*. \end{cases} \tag{2.5.3}$$

$$\begin{cases} X_1 = (a + b)X_1 + \varnothing X_2 + a^*X_3; \\ X_2 = \varnothing X_1 + aX_2 + a^*; \\ X_3 = b^*X_1 + \varnothing X_2 + a^*X_3. \end{cases} \tag{2.5.4}$$

2) Написать регулярные выражения для языков, заданных грамматиками со следующими продукциями:

$$\begin{aligned} &S \to 1A \mid 2S; \\ &A \to 0B \mid 0S \mid 1A; \\ &B \to 1C \mid 2C; \\ &C \to \varepsilon \mid 1S \mid 2A. \end{aligned} \tag{2.5.5}$$

$$\begin{aligned} &S \to 0A \mid 1S \mid \varepsilon; \\ &A \to 0B \mid 1A; \\ &B \to 0S \mid 1B. \end{aligned} \tag{2.5.6}$$

ГЛАВА 3

КОНЕЧНЫЕ АВТОМАТЫ

3.1. Определения и примеры

Конечный автомат является одним из простейших распознавателей (см. 1.5). Он состоит из входной ленты и управляющего устройства с конечной памятью. Управляющее устройство может быть недетерминированным. Входную головку далее будем полагать односторонней, более того, фактически мы потребуем, чтобы входная головка сдвигалась вправо на каждом такте.

Недетерминированным конечным автоматом называется пятерка

$$M = (Q, \Sigma, \delta, q_0, F),$$

где Q — конечное множество состояний, Σ — конечное множество допустимых входных символов, δ — отображение множества Σ во множество $P(Q)$ всех подмножеств множества Q, q_0 $(\in Q)$ — начальное состояние управляющего устройства, F $(\subseteq Q)$ — множество заключительных состояний.

Функцию δ называют *функцией переходов*, именно она фактически определяет поведение управляющего устройства. Функция переходов по данному «текущему» состоянию и «текущему» входному символу указывает набор всех возможных следующих состояний. Нестрого говоря, автомат дублирует себя так, что в каждом из возможных следующих состояний находится как бы один из равноправных экземпляров этого устройства; при этом недетерминированный автомат допускает входное слово в том случае, если какой-нибудь из его параллельно работающих экземпляров достигает допускающего состояния.

«Недетерминизм» конечного автомата не следует смешивать со «случайностью», при которой автомат может случайно выбрать одно из следующих состояний с фиксированными вероятностями, но этот автомат всегда имеется только в одном экземпляре. Такой автомат называется «вероятностным», и мы его изучать не будем.

Детерминированный конечный автомат определим как частный случай недетерминированного. Именно, пусть $M = (Q, \Sigma, \delta, q_0, F)$ — недетерминированный конечный автомат. Автомат M назовем *детерминированным*, если множество $\delta(q, a)$ содержит не более одного состояния для любых $q \in Q$ и $a \in \Sigma$. Если $\delta(q, a)$ содержит точно одно состояние, то автомат M назовем *полностью определенным*.

Работа конечного автомата представляет собой некоторую последовательность тактов. Такт определяется текущим состоянием управляющего устройства и входным символом, считываемым в данный момент входной головкой. Сам такт состоит из изменения состояния и сдвига входной головки на одну ячейку вправо. Для того чтобы определить будущее поведение конечного автомата, нужно знать лишь текущее состоятие управляющего устройства и слово на входной ленте, состоящее из буквы под головкой и всех букв, расположенных вправо от неё. Эти два элемента информации дают мгновенное описание конечного автомата, которое называют конфигурацией. Другими словами, если $M = (Q, \Sigma, \delta, q_0, F)$ — конечный автомат, то пара $(q, \omega) \in Q \times \Sigma^*$ называется конфигурацией автомата M. Конфигурация $(q_0\omega)$ называется начальной, а конфигурация (q, ε), где $q \in F$, называется заключительной.

Состояние p называется достижимым, если существует такое слово ω, что $(q_0, \omega) \vdash^* (p, \varepsilon)$.

Такт автомата M представляется бинарным отношением $\vdash_M$ (или, короче, $\vdash$), определенным на конфигурациях: если $q' \in \delta(q, a)$, то $(q, a\omega) \vdash (q', \omega)$ для всех $\omega \in \Sigma^*$. Таким образом, если M находится в состоянии q и входная головка обозревает входной символ a, то автомат M может делать такт, за который он переходит в состояние q' сдвигает головку на одну ячейку вправо. Так как автомат M, вообще говоря, может быть недетерминированным, то могут быть и другие состояния, в которые он тоже может перейти за один такт.

Запись $C \vdash_M^0 C'$ означает, что $C = C'$, а $C_0 \vdash_M^k C_k$ (для $k \geqslant 1$ — что существуют такие конфигурации $C_i, \ldots, C_{k-1}$, что $C_i \vdash_M C_{i+1}$ $0 \leqslant i < k$. Запись $C \vdash_M^+ C'$ означает, что $C \vdash_M^k C'$ для некоторого $k \geqslant 1$, а $C \vdash_M^* C'$ - что $C \vdash_M^k C'$ для некоторого $k \geqslant 0$. Будем опускать нижний индекс M там, где это не приведет к недоразумениям.

Говорят, что автомат M допускает слово ω, если $(q_0, \omega) \vdash^* (q, \varepsilon)$ для некоторого $q \in F$. Языком, допускаемым (определяемым, распознаваемым). автоматом M, называется множество $L(M)$ всех входных слов, допускаемых автома-

том M, то есть

$$L(M) = \{\omega(\in \Sigma^*) \mid \exists q \in F\colon (q_0, \omega) \vdash^* (q, \varepsilon)\}.$$

Рассмотрим два конечных автомата, один из которых является детерминированным (пример 3.1.1). а второй — недетерминированным (пример 3.1.2).

Пример 3.1.1. Пусть $M = (\{p; q; r\}, \{0; 1\}, \delta, p, \{r\})$ — конечный автомат, где функция переходов δ задаётся таблицей 3.1.

δ		Вход	
		0	1
Состояние	p	$\{q\}$	$\{p\}$
	q	$\{r\}$	$\{p\}$
	r	$\{r\}$	$\{r\}$

Таблица 3.1. Функция перехода δ для автомата из примера 3.1.1.

Этот автомат допускает все слова из нулей и единиц, содержащие два стоящих рядом нуля. Начальное состояние p можно интерпретировать так: «два стоящих рядом нуля еще не появились, и предыдущая буква не нуль». Состояние q означает, что «два стоящих рядом нуля еще не появились, но предыдущая буква — нуль». Состояние r означает, что «Два стоящих рядом нуля уже появились». Попав в состояние r, автомат остается в этом состоянии навсегда.

Например, если на вход автомата поступило слово $\omega = 1010011$, то единственно возможной последовательностью конфигураций, начинающейся с конфигурации $(p, 1010011)$, будет следующая последовательность:

$$(p, 1010011) \vdash (p, 010011) \vdash (q, 10011) \vdash (p, 0011) \vdash (q, 011) \vdash (r, 11) \vdash (r, 1) \vdash (r, \varepsilon).$$

Таким образом, $1010011 \in L(M)$. ■

Далее в случае детерминированных конечных автоматов мы будем писать, как правило, $\delta(q, a) = p$ вместо $\delta(q, a) = \{p\}$. Если $\delta(q, a) = \varnothing$, то мы часто будем говорить, что $\delta(q, a)$ не определено.

Упражнение 3.1.1. Постройте конечный автомат $M = (Q, \Sigma, \delta, q_0, F)$ допускающий те и только те слова над алфавитом $\Sigma = \{0; 1\}$, у которых встречается три стоящих рядом нуля, но после этого подслова нет двух стоящих рядом единиц.□

Упражнение 3.1.2. Покажите, что конечный автомат из примера 3.1.1 допускает язык $(0+1)^*00(0+1)^*$. □

Пример 3.1.2. Построим недетерминированный конечный автомат

$$M = (Q, \Sigma, \delta, q_0, F),$$

допускающий те и только те слова над алфавитом $\Sigma = \{1; 2; 3\}$, у которых последняя буква уже появлялась раньше. Прежде всего введем так называемое нейтральное состояние q_0, в котором автомат не пытается ничего распознать. Введем также состояния q_1, q_2 и q_3, в каждом из которых M делает предположение о том, что последняя буква слова совпадает с индексом состояния. Пусть q_f — одно заключительное состояние. Находясь в состоянии q_0, один из экземпляров автомата «бездумно» остаётся в нем и дальше, а другой экземпляр переходит в состояние q_i, если i — очередной символ. Находясь в состоянии q_i, один из экземпляров автомата, независимо от считываемой буквы, остаётся в нем и «размышляет» о том, что букву i в слове он уже обнаружил, а именно она и может совпасть с последней буквой в слове. Другой же экземпляр, если видит букву i, сразу без всяких сомнений переходит в финальное состояние q_f. Из состояния q_f автомат никуда не переходит, так как вопрос о том, допускается ли входное слово, решается только один раз, когда автомат сочтет входную букву последней. Итак, формально автомат определяется как пятерка

$$M = (\{q_0; q_1; q_2; q_3; q_f\}, \{1; 2; 3\}, \delta, q, \{q_f\}),$$

где функция переходов δ задается таблицей 3.2.

δ		Вход		
		1	2	3
Состояние	q_0	$\{q_0; q_1\}$	$\{q_0; q_2\}$	$\{q_0; q_3\}$
	q_1	$\{q_1; q_f\}$	$\{q_1\}$	$\{q_1\}$
	q_2	$\{q_2\}$	$\{q_2; q_f\}$	$\{q_2\}$
	q_3	$\{q_3\}$	$\{q_3\}$	$\{q_3; q_f\}$
	q_f	$\varnothing$	$\varnothing$	$\varnothing$

Таблица 3.2. Функция перехода δ для автомата из примера 3.1.2.

Работа этого автомата при чтении слова $\omega = 12321$ проиллюстрирована схемой на рисунке 3.1.

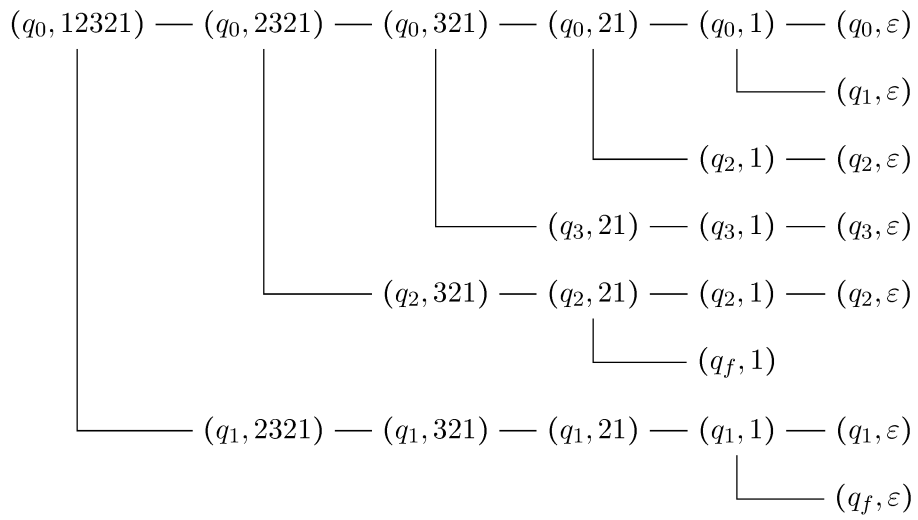


Рис. 3.1. Схема работы автомата из примера 3.1.2 на слове $\omega = 12321$.

Таким образом, возможна последовательность конфигураций:

$$(q_0, 12321) \vdash (q_1, 2321) \vdash (q_1, 321) \vdash (q_1, 21) \vdash (q_1, 1) \vdash (q_f, \varepsilon).$$

Это означает, что $12321 \in L(M)$. ■

Упражнение 3.1.3. Докажите, что конечный автомат из примера 3.1.2 допускает язык $\{\omega a \mid a \in \{1, 2, 3\}$ и a входит в $\omega\}$. Постройте регулярное выражение для этого языка. □

3.2. Редукция недетерминированных конечных автоматов к детерминированным

В этом разделе будет доказано, что класс языков, определяемых недетерминизированными конечными автоматами, совпадает с классом языков, определяемых полностью определенными детерминизированными конечными автоматами. В силу этого произвольный язык из этого класса мы будем называть далее конечно-автоматным.

Теорема 3.2.1. *Пусть $M = (Q, \Sigma, \delta, q_0, F)$ — недетерминированный конечный автомат. Тогда существует такой детерминированый конечный автомат M' что $L(M) = L(M')$.* □

Доказательство. По данному недетерминированному автомату M построим новый автомат $M' = (Q', \Sigma, \delta', q_0', F')$ следующим образом:

1) $Q' = P(Q)$, то есть состояниями автомата M' являются всевозможные подмножества множества состояний автомата M;

2) Если $S \in Q'$, $a \in \Sigma$, то определим $\delta'(S, a)$ равенством

$$\delta'(S, a) = \bigcup_{q \in S} \delta(q, a);$$

3) $q_0' = \{q_0\}$;

4) F' состоит из всех таких подмножеств S множества состояний Q, что

$$S \cap F \neq \varnothing.$$

Применяя метод математической индукции по i, нетрудно проверить, что $(S, \omega) \vdash_{M'}^{i} (S', \varepsilon)$ в том и только в том случае, когда

$$S' = \{p \mid \exists q \in S : (q, \omega) \vdash_{M}^{i} (p, \varepsilon)\}$$

Отсюда, в частности, следует, что $(\{q_0\}, \omega) \vdash_{M'}^{i} (S', \varepsilon)$ для некоторого $S' \in F'$ тогда и только тогда, когда $(q_0, \omega) \vdash_{M}^{i} (p, \varepsilon)$ для некоторого $p \in F$. Итак, $L(M') = L(M)$. ■

Пример 3.2.1. Пусть

$$M = (\{q_0; q_1; q_2; q_3; q_f\}, \{1; 2; 3\}, \delta, q, \{q_f\})$$

недетерминированный конечный автомат, определенный в примере 3.1.2. Воспользуемся конструкцией из доказательства теоремы 3.2.1 и построим полностью определенный детерминированный конечный автомат $M' = (Q', \{1; 2; 3\}, \delta', \{q_0\}, F')$ допускающий язык $L(M)$.

Так как автомат M имеет 5 состояний, то новый автомат M' может иметь 32 $(= 2^5)$ состояния, однако не все они достижимы из начального. (Напомним, что состояние p называется достижимым, если существует такое слово ω, что $(q_0, \omega) \vdash^{*} (p, \varepsilon)$.) Мы будем строить, естественно, только достижимые состояния.

Сделаем несколько полезных наблюдений.

1) Очевидно, что состояние $\{q_0\}$ достижимо.

2) Достижимыми являются состояния $\{q_0; q_a\}$ для $a \in \{1; 2; 3\}$; действительно,

$$\delta'(\{q_0\}, a) = \{q_0; q_a\}.$$

3) Достижимыми являются состояния $\{q_0; q_a; q_f\}$ для $a \in \{1; 2; 3\}$; действительно, $\delta'(\{q_0; q_a\}, a) = \{q_0; q_a; q_f\}$. Аналогично проверяется, что состояния $\{q_0; q_1; q_2\}$, $\{q_0; q_1; q_3\}$, $\{q_0; q_2; q_3\}$ — достижимы.

4) Из предыдущего факта легко выводится достижимость состояний

$$\{q_0; q_1; q_2; q_3\}, \quad \{q_0; q_1; q_2; q_f\}, \quad \{q_0; q_1; q_3; q_f\}, \quad \{q_0; q_2; q_3; q_f\},$$

а отсюда — достижимость состояния $\{q_0; q_1; q_2; q_3; q_f\}$.

Всего мы выявили 16 достижимых состояний. Анализ показывает, что подмножество D множества состояний автомата M, то есть состояние автомата M', достижимо тогда и только тогда, когда выполняется условия: 1) D содержит q_0, 2) если D содержит q_f, то оно содержит также и q_1, q_2 или q_3. Легко видеть, что состояния, отличные от выделенных, не достижимы.

Итак, Q' — множество всех достижимых состояний автомата M' — состоит из 15 элементов и вместе с функцией переходов δ' представлено в следующей таблице 3.3, где для простоты вместо состояний вида $\{q_i; q_j; \ldots; q_k\}$ помещены списки $(i; j; \ldots; k)$.

δ'	Вход		
	1	2	3
$\{0\}$	$\{0;1\}$	$\{0;2\}$	$\{0;3\}$
$\{0;1\}$	$\{0;1;f\}$	$\{0;1;2\}$	$\{0;1;3\}$
$\{0;2\}$	$\{0;1;2\}$	$\{0;2;f\}$	$\{0;2;3\}$
$\{0;3\}$	$\{0;1;3\}$	$\{0;2;3\}$	$\{0;3;f\}$
$\{0;1;2\}$	$\{0;1;2;f\}$	$\{0;1;2;f\}$	$\{0;1;2;3\}$
$\{0;1;3\}$	$\{0;1;3;f\}$	$\{0;1;2;3\}$	$\{0;1;3;f\}$
$\{0;1;f\}$	$\{0;1;f\}$	$\{0;1;2\}$	$\{0;1;3\}$
$\{0;2;3\}$	$\{0;1;2;3\}$	$\{0;2;3;f\}$	$\{0;2;3;f\}$
$\{0;2;f\}$	$\{0;1;2\}$	$\{0;2;f\}$	$\{0;2;3\}$
$\{0;3;f\}$	$\{0;1;3\}$	$\{0;2;3\}$	$\{0;3;f\}$
$\{0;1;2;f\}$	$\{0;1;2;f\}$	$\{0;1;2;f\}$	$\{0;1;2;3\}$
$\{0;1;3;f\}$	$\{0;1;3;f\}$	$\{0;1;2;3\}$	$\{0;1;3;f\}$
$\{0;2;3;f\}$	$\{0;1;2;3\}$	$\{0;2;3;f\}$	$\{0;2;3;f\}$
$\{0;1;2;3\}$	$\{0;1;2;3;f\}$	$\{0;1;2;3;f\}$	$\{0;1;2;3;f\}$
$\{0;1;2;3;f\}$	$\{0;1;2;3;f\}$	$\{0;1;2;3;f\}$	$\{0;1;2;3;f\}$

Таблица 3.3. Функция перехода δ' для автомата из примера 3.2.1

Начальный состоянием автомата M' является $\{q_0\} \in Q'$, а множество заключительных состояний F' имеет вид:

$$F' = \{ \quad \{q_0, q_1, q_f\}; \quad \{q_0, q_2, q_f\}; \quad \{q_0, q_3, q_f\};$$
$$\{q_0, q_1, q_2, q_f\}; \quad \{q_0, q_1, q_3, q_f\}; \quad \{q_0, q_2, q_3, q_f\};$$
$$\{q_0, q_1, q_2, q_3, q_f\} \quad \}.$$

■

3.3. Граф переходов

Пусть $M = (Q, \Sigma, \delta, q_0, F)$ — недетерминированный конечный автомат. Графом переходов (или диаграммой) автомата M называют неупорядоченый помеченный граф, вершины которого помечены именами состояний; в графе есть дуга (p, q), если существует такая буква $a \in \Sigma$, что $q \in \delta(p, a)$; кроме того, дуга (p, q) помечается списком, состоящим из всех таких a, что $q \in \delta(p, a)$.

При изображении графа переходов условимся начальное состояние указывать направленной в него стрелкой, помеченной словом «начало», а заключительные состояния обводить двойным кружком.

Пример 3.3.1. Рассмотрим детерминированный конечный автомат из примера 3.1.1. Его граф переходов изображен на рисунке 3.2.

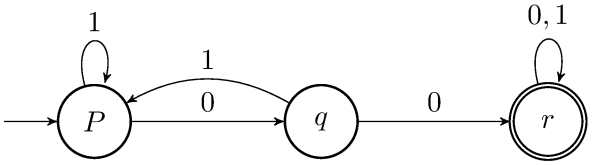


Рис. 3.2. Граф переходов автомата из примера 3.1.1

Пример 3.3.2. Рассмотрим недетерминированный конечный автомат из примера 3.1.2. Его граф переходов изображен на рисунке 3.3.

Упражнение 3.3.1. Изобразите граф переходов детерминированного конечного автомата из примера 3.2.1. □

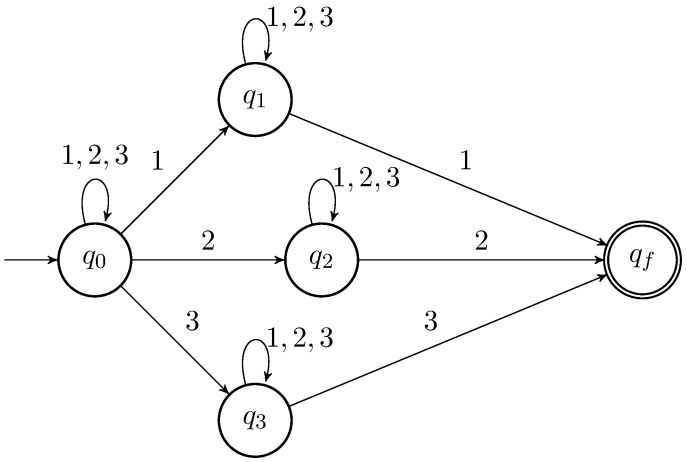


Рис. 3.3. Граф переходов автомата из примера 3.1.2.

3.4. Совпадение классов конечно-автоматных, регулярных и ПЛ-языков

Лемма 3.4.1. *Пусть* $M = (Q, \Sigma, \delta, q_0, F)$ *— детерминированный конечный автомат. Тогда существует такая ПЛ-грамматика* G*, что* $L(M) = L(G)$. □

Доказательство. Рассмотрим грамматику $G = (Q, \Sigma, P, q_0)$, где множество продукций P определяется по правилам:

1) если $q, r \in Q$, $a \in \Sigma$ и $\delta(q, a) = r$, то $(q \to ar) \in P$.
2) если $p \in F$, то $(p \to \varepsilon) \in P$.

Грамматика G определена таким образом, что каждый шаг вывода в ней имитирует такт автомата M.

Применяя метод математической индукции по параметру i, докажем следующее вспомогательное

Утверждение 3.4.1. $q \Rightarrow^{i+1} \omega$ *для* $q \in Q$ *тогда и только тогда, когда* $(q, \omega) \vdash^i (r, \varepsilon)$ *для* $r \in F$. □

Если $i = 0$, то это утверждение приобретает вид: $q \Rightarrow \varepsilon$ тогда и только тогда, когда $(q, \varepsilon) \vdash^0 (q, \varepsilon)$ для $q \in F$, и является очевидным.

Предположим теперь, что утверждение верно для $i = k$, и проверим его для $i = k + 1$. Пусть $\omega = ax$, где $|x| = k$, — слово над алфавитом Σ. Тогда вывод $q \Rightarrow^{k+1} \omega$ равносилен выводу $q \Rightarrow as \Rightarrow^k ax$ для некоторого $s \in Q$, а вывод $q \Rightarrow as$ равносилен тому, что $\delta(q, a) = s$. По предположению индукции $s \Rightarrow^k x$ тогда и

только тогда, когда $(s, x) \vdash^{k-1} (r, \varepsilon)$ для некоторого $r \in F$. Следовательно, вывод $q \Rightarrow^{k+1} \omega$ равносилен тому, что $(q, \omega) \vdash^{k} (r, \varepsilon)$ для некоторого $r \in F$.

Итак, вспомогательное утверждение верно.

Подставляя в доказанное утверждение q_0 вместо q, получаем: $q_0 \Rightarrow^{+} \omega$ тогда и только тогда, когда $(q_0, \omega) \vdash^{*} (r, \varepsilon)$ для некоторого $r \in F$. Таким образом, $L(G) = L(M)$. ■

Отметим, что в лемме 3.4.1 не просто доказывается существование нужной грамматики G, а указывается ее конструкция.

Лемма 3.4.2. *Пусть Σ — конечный алфавит. Множества $\varnothing$, $\{\varepsilon\}$ и $\{a\}$ для всех $a \in \Sigma$ являются конечно-автоматными языками.* □

Доказательство. Любой конечный автомат с пустым множеством заключительных состояний допускает $\varnothing$.

Пусть $M_\varepsilon = (\{q_0\}, \Sigma, \delta, q_0, \{q_0\})$, где $\delta(q_0, a)$ не определяется ни для каких $a \in \Sigma$. Тогда $L(M_\varepsilon) = \{\varepsilon\}$.

Пусть $M_a = (\{q_0, q_a\}, \Sigma, \delta, q_0, \{q_a\})$, где $\delta(q_0, a) = q_a$, а в остальных случаях функция δ не определена. Тогда, очевидно, $L(M_a) = \{a\}$ ■

Лемма 3.4.3. *Пусть $M_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$, $M_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$ — конечные автоматы; $L_i = L(M_i)$. Тогда множества $L_1 \cup L_2$, $L_1 L_2$ и L_1^* являются конечно-автоматными языками.* □

Доказательство. Не теряя общности, можно считать, что $Q_1 \cap Q_2 = \varnothing$, так как в противном случае состояния можно было бы переименовать. Чтобы доказать лемму, достаточно для каждого из трех множеств построить такой конечный автомат M, чтобы язык $L(M)$ совпадал с этим множеством.

1) Построим автомат $M = (Q_1 \cup Q_2 \cap \{q_0\}, \Sigma, \delta, q_0, F)$ для языка $L_1 \cup L_2$, полагая, что q_0 — новое состояние, $F = F_1 \cup F_2$ при $\varepsilon \notin L_1 \cup L_2$, и $F = F_1 \cup F_2 \cup \{q_0\}$ при $\varepsilon \in L_1 \cup L_2$ а функция переходов δ определяется так:

   (*i*) $\delta(q_0, a) = \delta_1(q_1, a) \cup \delta(q_2, a)$ для всех $a \in \Sigma$,

   (*ii*) $\delta(q, a) = \delta_1(q, a)$ для всех $q \in Q_1, a \in \Sigma$,

   (*iii*) $\delta(q, a) = \delta_2(q, a)$ для всех $q \in Q_2, a \in \Sigma$.

   Таким образом, сначала автомат M как бы решает, какой из автоматов $M1$, $M2$ ему моделировать, но так как M — недетерминированный автомат, то фактически он моделирует и тот, и другой.

Используя непосредственный анализ функции переходов δ легко проверить, что $(q_0, \omega) \vdash_M^i (q, \varepsilon)$ тогда и только тогда, когда $q \in Q_1$ и $(q_1, \omega) \vdash_{M_1}^i (q, \varepsilon)$ или $q \in Q_2$ и $(q_2, \omega) \vdash_{M_2}^i (q, \varepsilon)$.

Из этого утверждения и определения множества F вытекает, что $L_1 \cup L_2 = L(M_1) \cup L(M_2) = L(M)$ — конечно-автоматный язык.

2) Построим автомат $M = (Q_1 \cup Q_2, \Sigma, \delta, q_1, F)$ для $L_1 L_2$, полагая

$$F = \begin{cases} F_2, & \text{если } q_2 \notin F_2, \\ F_1 \cup F_2, & \text{если } q_2 \in F_2, \end{cases}$$

а функцию переходов δ определяя равенствами

(*i*) $\delta(q, a) = \delta_1(q, a)$ для всех $q \in Q_1 \backslash F_1$, $a \in \Sigma$,

(*ii*) $\delta(q, a) = \delta_1(q, a) \cup \delta_2(q, a)$ для всех $q \in F$, $a \in \Sigma$,

(*iii*) $\delta(q, a) = \delta_2(q, a)$ для всех $q \in Q_2$, $a \in \Sigma$.

Ясно, что когда M начинает работать, то он моделирует автомат M_1. Когда же M достигает заключительного состояния M_1, он может (если пожелает!) предположить, что оказался в начальном состоянии автомата M_2 и начать моделировать M_2, а может (допускается и это!) продолжать функционировать в режиме M_1.

Проверим, что $L_1 L_2 \subset L(M)$.

Пусть $x \in L$, $y \in L$. Тогда $(q_1, xy) \vdash_M^* (q, y)$ для некоторого $q \in F_1$, причем $q = q_1$ в случае $x = \varepsilon$. Пусть $y = \varepsilon$. Тогда $q_2 \in F_2$ и, следовательно, $F = F_1 \cup F_2$. Последняя конфигурация (q, ε), где $q \in F_1 \subset F$, означает, что слово $xy = x\varepsilon$ автомат M считал.

Пусть теперь $y \neq \varepsilon$. Применяя один раз (ii) и нуль или более раз (iii), получаем: $(q, y) \vdash_M^+ (r, e)$ для некоторого r из $F_2 \subset F$. Отсюда вытекает, что $xy \in L(M)$, и, следовательно, $L_1 L_2 \subset L(M)$.

Проверим, что $L(M) \subset L_1 L_2$.

Пусть $\omega \in L(M)$. Тогда $(q_1, \omega) \vdash_M^* (q, \varepsilon)$ для некоторого $q \in F$. Рассмотрим отдельно два случая: $q \in F_2$ и $q \in F_1 (\subset F)$. Если $q \in F_2$, то $\omega = xay$ для некоторого $a \in \Sigma$, удовлетворяющего условиям

$$(q_1, xay) \vdash_M^* (r, ay) \vdash_M (s, y) \vdash_M^* (q, \varepsilon),$$

где $r \in F_1$, $s \in Q_2$ и $s \in \delta(r, a)$. Следовательно, $x \in L_1$, $ay \in L_2$ и, таким образом, $\omega = xay \in L_1 L_2$. Если же $q \in F_1 (\subset F)$, то $q_2 \in F_2$, $\varepsilon \in L_2$ и, таким образом, $\omega \in L_1$, $\omega\varepsilon \in L_1 L_2$. Отсюда вытекает: $L(M) \subset L_1 L_2$.

В итоге получаем, что $L_1 L_2 = L(M)$ — конечно-автоматный язык.

3) Построим автомат $M = (Q_1 \cup \{q'\}, \Sigma, \delta, q', F \cup \{q'\})$ для языка L_1^*, полагая, что q' — новое состояние, не принадлежащее Q_1, а функцию переходов δ определяя равенствами

   (*i*) $\delta(q', a) = \delta_1(q_1, a)$ для всех $a \in \Sigma$,

   (*ii*) $\delta(q, a) = \delta_1(q, a)$, для всех $q \in Q \backslash F_1$, $a \in \Sigma$,

   (*iii*) $\delta(q, a) = \delta_1(q, a) \cup \delta_1(q_1, a)$, для всех $q \in F_1$, $a \in \Sigma$.

   Ясно, что когда M начинает работать, то он моделирует автомат M_1. Когда же M достигает заключительного состояния M_1, он может предположить, что вновь оказался в начальном состоянии автомата M_1 и снова начать моделировать M_1, а может и продолжать функционировать в режиме M_1. Доказательство того, что $L(M) = L_1^*$, аналогично доказательству из 2) и мы его не приводим. Отметим лишь, что $\varepsilon \in L(M)$, так как q — заключительное состояние автомата M. ■

Теорема 3.4.1. *Пусть Σ — конечный алфавит, L — язык над этим алфавитом. Следующие утверждения эквивалентны:*

1) *L — регулярный язык;*
2) *L — праволинейный язык;*
3) *L — конечно-автоматный язык.* □

Доказательство. Эквивалентность утверждений 1) и 2) доказана в теореме 2.4.1. То, что утверждение 2) является следствием утверждения 3) доказано в лемме 3.4.1. Из лемм 3.4.2 и 3.4.3 вытекает, что утверждение 3) — следствие утверждения 1). Таким образом, теорема доказана. ■

3.5. Лемма о разрастании для регулярных языков

Лемма о разрастании (или накачке) позволяет решать конкретную задачу: опровергать принадлежность заданного языка классу регулярных языков.

Зачем может понадобиться такое опровержение? Если считать класс регулярных языков в определенном смысле (а именно, с точки зрения иерархии Хомского) самым простым классом языков, то «непринадлежность» языка этому классу может говорить о «сложности» этого языка (в том же специальном смысле). Стоит обратить внимание на указание относительности понятий «простой» и «сложный»: язык, который с обыденной точки зрения может казаться «простым», вполне может не принадлежать классу регулярных языков.

Теорема 3.5.1 («Лемма о накачке» или «Лемма о разрастании»). *Пусть L — регулярный язык. Тогда существует такая константа $n \in \mathbb{N}$, что для любого слова $w \in L$, такого что $|w| \geqslant n$, существует разбиение $w = xyz$ слова w со следующими свойствами:*

1) $y \neq \varepsilon$;
2) $|xy| \leqslant n$;
3) $\{xy^k z \mid k \geqslant 0\} \subset L$. □

Доказательство. Так как L регулярный язык, то существует детерминированный конечный автомат $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$ с $|Q| = N$ состояниями, распознающий язык L. Пусть $w \in L$ и $|w| = N + 1$. Подадим на вход автомату $\mathcal{A}$ слово w. Очевидно, существует состояние $q \in Q$, в котором автомат окажется дважды, читая это слово (принцип Дирихле / принцип голубятни). Разобьём слово w на три части $w = xyz$, так что:

$$(q_0, xyz) \vdash^* (q, yz) \overset{\triangle}{\vdash^*} (q, z) \vdash^* (q_F, \varepsilon),$$

где $q_F \in F$. Покажем, что для любого целого $k \geqslant 0$ автомат распознает слово $xy^k z$. Действительно, последовательность переходов при чтении цепочек x и z остаётся такой же, как для слова w. Часть y^k читается k-кратным повтором последовательности переходов $\triangle$. Таким образом, $\{xy^k z \mid k \geqslant 0\} \subset L$ и выполнено условие 3).

Части x и y слова w удовлетворяют условиям 1)–2) по построению. Полагая $n = N + 1$, получаем выполненными все требования теоремы. ■

Заметим, что лемма о накачке формулирует необходимое свойство регулярного языка. То есть, если язык не обладает этим свойством, то он точно не является регулярным. На этом основано применение леммы на практике: вначале доказывается, что для рассматриваемого языка нарушается условие лем-

мы, на основе этого делается вывод о нерегулярности. Такое доказательство обычно проводится от противного.

В случае, когда нарушение свойства, описываемого леммой о накачке, доказать не удается, ничего определенного о классе языка без дополнительных рассуждений сказать нельзя.

Пример 3.5.1. Рассмотрим язык $L = \{0^m 1^m \mid m \in \mathbb{N}\}$. Предположим, что L регулярный, а значит, существует константа n, о которой идет речь в лемме. Рассмотрим слово $0^n 1^n$ ($\in L$). Пусть $xyz = 0^n 1^n$ — разбиение, о котором говорится в лемме. Заметим, что в силу условия $|xy| \leqslant n$ из леммы подстрока y целиком состоит из нулей и при этом не пуста. Очевидно, что для любого $k > 1$ слово $xy^k z$ уже не будет принадлежать языку L, так как в этом слове нулей станет больше, чем единиц. Это свидетельствует о нарушении условия леммы о накачке, а значит, о том, что язык L не является регулярным. ■

Упражнение 3.5.1 (контрпример к достаточности леммы о накачке). Необходимо понимать, что условие леммы о накачке не является достаточным для регулярности языка. То есть, если некоторый язык ему удовлетворяет, из этого не следует регулярность языка. Рассмотрим язык

$$L = \{a^i b^j c^k \mid i, j, k \in \mathbb{N}_0 \wedge (i = 1 \Rightarrow j = k)\}.$$

Покажите, что

1) L нерегулярен (с помощью свойства замкнутости из раздела 5.2),
2) L удовлетворяет условию леммы о накачке. □

3.6. Упражнения

Построение недетерминированных автоматов

Построить автомат, распознающий

1) язык над $\{0, 1\}$ из слов, заканчивающихся на 01;
2) язык, представляющий собой десятичную запись чисел, делящихся на 4;
3) язык над $\{a, b\}$, заданный регулярным выражением $(ab + aba)^*$;
4) язык над $\{0, 1, \ldots, 9\}$ из слов, в которых последняя цифра встречается ещё где-то в них;

5) язык над $\{0, 1, \ldots, 9\}$ из слов, в которых последняя цифра больше нигде в них не встречается;

6) язык над $\{0, 1\}$ из слов, в которых содержится два 0, разделённых символами, количество которых кратно 4 (нуль символов также считать кратными четырём).

Детерминизация конечных автоматов

Провести детерминизацию следующих НКА:

1) распознающего язык над $\{0, 1\}$ из слов, заканчивающихся на 01;

2) распознающего язык, заданный регулярным выражением $(ab + aba)^*$;

3)

δ	Вход	
	0	1
$\to p$	$\{p, q\}$	$\{p\}$
q	$\{r\}$	$\{r\}$
r	$\{s\}$	$\varnothing$
$\boxed{s}$	$\{s\}$	$\{s\}$

4)

δ	Вход	
	0	1
$\to p$	$\{q, s\}$	$\{q\}$
$\boxed{q}$	$\{r\}$	$\{q, r\}$
r	$\{s\}$	$\{p\}$
$\boxed{s}$	$\varnothing$	$\{p\}$

5)

δ	Вход	
	0	1
$\to p$	$\{p, q\}$	$\{p\}$
q	$\{r, s\}$	$\{t\}$
r	$\{p, r\}$	$\{t\}$
$\boxed{s}$	$\varnothing$	$\varnothing$
$\boxed{t}$	$\varnothing$	$\varnothing$

Доказательство нерегулярности языков

С помощью леммы о накачке доказать, что следующие языки нерегулярны:

1) $\{w \in \{0, 1\}^* \mid |w|_0 = |w|_1\}$;
2) $\{ww \mid w \in \{0, 1\}^*\}$;
3) $\{0^n 1^m \mid n \leqslant m\}$;
4) $\{1^p \mid p$ — простое целое число$\}$;
5) $\{0^n 10^n \mid n \in \mathbb{N}\}$;
6) $\{1^{n^2} \mid n \in \mathbb{N}\}$;
7) $\{1^{n!} \mid n \in \mathbb{N}\}$.

Запись вида $|w|_z$ (см. язык 1) означает количество вхождений символа z в строку w.

ГЛАВА 4

КОНЕЧНЫЕ АВТОМАТЫ СО СПОНТАННЫМИ ПЕРЕХОДАМИ

4.1. Определения и примеры

Конечный автомат со спонтанными переходами (или ε-переходами) — это одно из обобщений конечных автоматов. У конечного автомата появляется новое свойство — возможность совершать переходы по ε (пустой цепочке), т.е. спонтанно, не получая на вход никакого символа. Эта новая возможность, как и недетерминизм, не расширяет класс языков, допустимых конечными автоматами, но дает некоторое дополнительное «удобство программирования».

Определение. Пусть $M = (Q, \Sigma, \delta, q_0, F)$ — недетерминированный конечный автомат. Автомат M назовем *ε-НКА*, если

$$\delta: Q \times (\Sigma \cup \{\varepsilon\}) \to P(Q)$$

Пример 4.1.1. Рассмотрим ε-НКА, распознающий ключевые слова *web* и *day* в последовательности символов $\{a..z\}$. Граф переходов этого автомата представлен на рисунке 4.1.

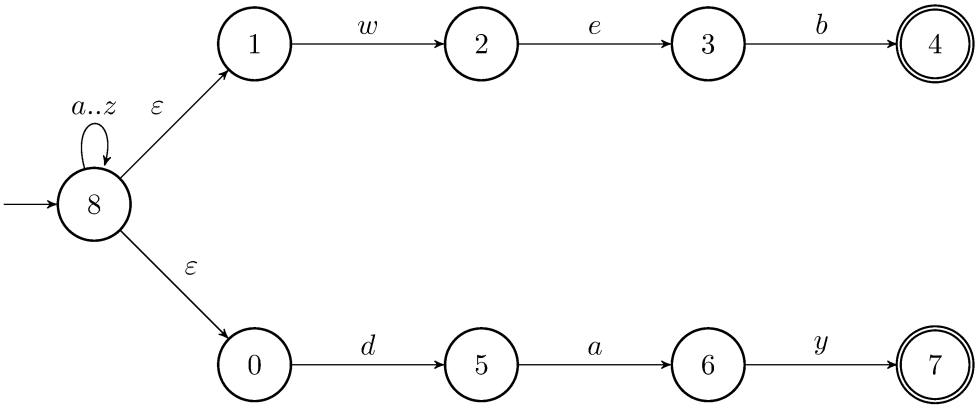


Рис. 4.1. Использование спонтанных переходов для распознавания ключевых слов

Для каждого ключевого слова строится полная последовательность состояний, как если бы это было единственное слово, которое автомат должен распознать. Затем добавляется новое начальное состояние (состояние 8 на рисунке 4.1) с ε-переходами в начальные состояния автоматов для каждого из ключевых слов. ■

Пример 4.1.2. Рассмотрим ε-НКА (рисунок 4.2), допускающий запись десятичных чисел из следующих элементов :

1) Необязательный знак + или −.
2) Цепочка цифр.
3) Разделяющая десятичная точка.
4) Еще одна цепочка цифр. Эта цепочка, как и цепочка (2), может быть пустой, но хотя бы одна из них должна быть непустой. ■

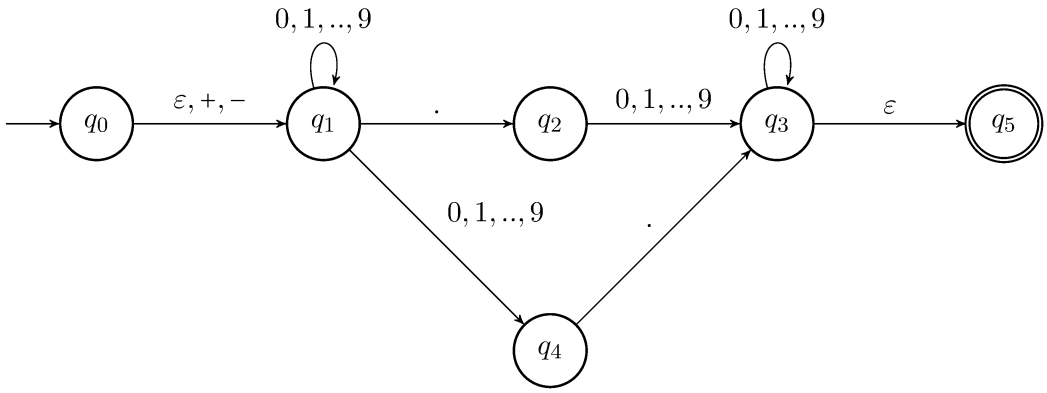


Рис. 4.2. ε-НКА, допускающий десятичные числа

Поскольку переход из состояния q_0 в q_1 может произойти по любому из символов $+, -, \varepsilon$, то состояние q_1 моделирует ситуацию, когда прочитан знак числа (если есть), он не прочитана ни одна из цифр, ни десятичная точка. Состояние q_2 соответствует ситуации, когда только что прочитана десятичная точка, а цифры целой части числа либо уже прочитаны, либо нет. В состоянии q_4 уже наверняка прочитана хотя бы одна цифра, но еще не прочитана десятичная точка. Состояние q_3 определяет ситуацию, когда прочитаны десятичная точка и хотя бы одна цифра слева или справа от нее. Автомат может оставаться в состоянии q_3, продолжая читать цифры, а может и «предположить», что цепочка цифр закончена, и спонтанно перейти в допускающее состояние q_5. δ-функция переходов автомата представлена в таблице 4.1.

δ		Вход			
		ε	{+;-}	{.}	{0,1,..,9}
Состояние	$\to q_0$	$\{q_1\}$	$\{q_1\}$	$\varnothing$	$\varnothing$
	q_1	$\varnothing$	$\varnothing$	$\{q_2\}$	$\{q_1; q_4\}$
	q_2	$\varnothing$	$\varnothing$	$\varnothing$	$\{q_3\}$
	q_3	$\{q_3\}$	$\varnothing$	$\varnothing$	$\{q_3\}$
	q_4	$\varnothing$	$\varnothing$	$\{q_3\}$	$\varnothing$
	$\boxed{q_5}$	$\varnothing$	$\varnothing$	$\varnothing$	$\varnothing$

Таблица 4.1. Функция перехода δ для автомата, распознающего десятичные числа

Пусть $M = (Q, \Sigma, \delta, q_0, F)$ — ε-НКА. Определим *ε-замыкание* состояния $q \in \Sigma$ рекурсивно следующим образом:

1) $E(q) \subseteq P(q)$.

2) $q \in E(q)$.

3) Если $p \in E(q)$ и $r \in \delta(p, \varepsilon)$, то $r \in E(q)$.

Пример 4.1.3. Построим ε-замыкание для состояния q из ε-НКА, заданного графом переходов (рисунок 4.3).

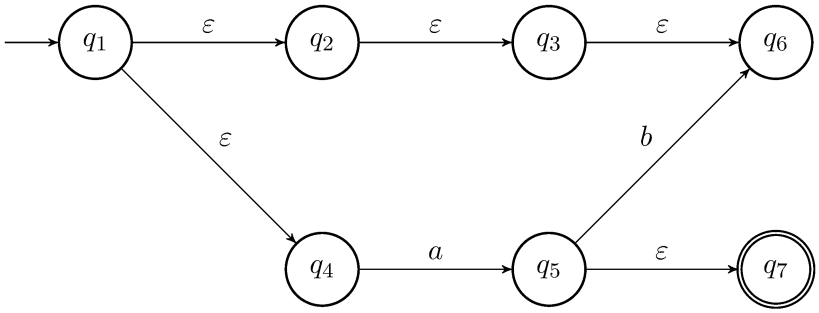


Рис. 4.3. ε-НКА из примера 4.1.3

$$E^0(q_1) = \{q_1\}; E^1(q_1) = \{q_1; q_2; q_4\}; E^2(q_1) = \{q_1; q_2; q_4; q_3\}; E^3(q_1) = \{q_1; q_2; q_4; q_3; q_6\}.$$

Таким образом, $E(q_1) = \{q_1; q_2; q_4; q_3; q_6\}$. ■

Пусть $M = (Q, \Sigma, \delta, q_0, F)$ — ε-НКА и $S \subseteq Q$ — произвольное подмножество множества Q. Назовем множестов S *ε-замкнутым*, если $S = \{q \mid$ если $p \in \delta(q, \varepsilon)$, то $p \in S\}$. Отметим, что $\varnothing$ — ε-замкнутое множество.

4.2. Редукция ε-НКА к ДКА

Для всякого ε-НКА E можно найти ДКА D, допускающий тот же язык, что и E.

Лемма 4.2.1. *Пусть $M_E = (Q_E, \Sigma, \delta_E, q_E, F_E)$ — ε-НКА. Тогда найдется такой ДКА M_D, что $L(M_E) = L(M_D)$.* □

Доказательство. По данному ε-НКА построим новый ДКА $M_D = (Q_D, \Sigma, \delta_D, q_D, F_D)$ следующим образом:

1) $Q_d = P(Q_E)$. Причем, достижимыми состояниями автомата M_D будут только такие $S \in Q_D$, что S является ε-замкнутым множеством.
2) $q_D = E(q_E)$.
3) $F_D = \{S \in Q_D : S \cap F_E \neq \varnothing\}$.
4) Функция переходов δ_D определим следующим образом:
   если $S \in Q_D$ и $a \in \Sigma$, то $\delta_d(S, a)$ строится по правилам:
   a) пусть $\{q_1; q_2; \ldots; q_n\}$ — множество состояний S;
   b) $\{r_1; r_2; \ldots; r_m\} = \bigcup_{i=1}^{n} \delta_E(q_i, a)$;
   c) $\delta_D(S, a) = \bigcup_{j=1}^{m} E(r_j)$.

Доказательство корректности конструкций леммы основывается на стягивании вершин автомата по ε-переходам. ■

Пример 4.2.1. Пусть $M_E = (\{p; q; r\}, \{a; b; c\}, \delta_E, p, \{r\})$ — ε-НКА. δ-функция переходов автомата задана таблицей 4.2.

δ		Вход			
		ε	{a}	{b}	{c}
	$\to p$	$\varnothing$	$\{p\}$	$\{q\}$	$\{r\}$
	q	$\{p\}$	$\{q\}$	$\{r\}$	$\varnothing$
Состояние	$\boxed{r}$	$\{q\}$	$\{r\}$	$\varnothing$	$\{p\}$

Таблица 4.2. Функция перехода δ для ε-НКА из примера 4.2.1

Граф переходов автомата представлен на рисунке 4.4.

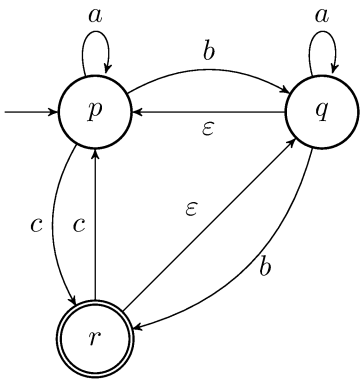


Рис. 4.4. Граф переходов ε-НКА из примера 4.2.1

Пользуясь конструкциями из леммы 4.2.1, по данному автомату построим ДКА $M_D = (Q_D, \Sigma, \delta_D, q_D, F_D)$. Начальным состоянием ДКА является ε-замыкание начального состояния исходного автомата.

$$q_D = E(p).$$

Построим ε-замыкание:

$$E^0(p) = \{p\}; E^1(p) = \{p\}.$$

Таким образом $q_D = \{p\}$.

Аналогично начальному состоянию, построим ε-замыкания для остальных состояний исходного автомата. δ-функция переходов автомата M_D представлена в таблице 4.3.

δ		Вход		
		{a}	{b}	{c}
Состояние	$\to \{p\}$	$\{p\}$	$\{q;p\}$	$\{r;q;p\}$
	$\{q;p\}$	$\{q;p\}$	$\{r;q;p\}$	$\{r;q;p\}$
	$\boxed{\{r;q;p\}}$	$\{r;q;p\}$	$\{r;q;p\}$	$\{r;q;p\}$

Таблица 4.3. Функция перехода δ для ДКА из примера 4.2.1

Граф переходов автомата M_D представлен на рисунке 4.5.

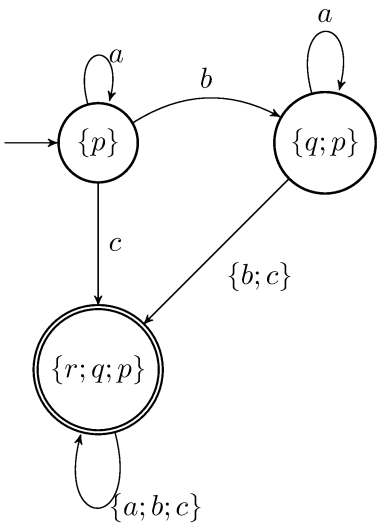


Рис. 4.5. Граф переходов ДКА из примера 4.2.1

Отметим, что множество конечных состояний F_D содержит единственное состояние $\{r; q; p\}$, поскольку состояние $\{r\}$ было конечным состоянием исходного автомата. ■

4.3. Преобразование регулярного выражения в автомат

Для любого регулярного языка L, заданного регулярным выражением R, может быть построен ε-НКА, распознающий этот язык. Эта возможность заложена в определение регулярного языка и задающего его регулярного выражения (см. раздел 2.1). Регулярными являются элементарные множества над алфавитом Σ : пустое множество (ему соответствует регулярное выражение $\varnothing$), множество из одной пустой цепочке (регулярное выражение ε) и множество из одного однобуквенного слова из любой буквы a множества Σ (регулярное выражение a). Также из определения следует, что результаты объединение, конкатенация и итерация регулярных языков являются регулярными языками. Далее мы построим автоматы для элементарных языков и опишем процесс построения более сложных автоматов, допускающих объединение, конкатенацию или итерацию языков, распознаваемых более простыми автоматами.

Для связывания простых автоматов в сложные конструкции удобно использовать ε-переходы. Условимся, что каждый автомат любой степени сложности будет иметь ровно один вход и один выход. Начальное состояние у автомата всегда единственное. Одно конечное состояние можем получить, если

ввести новое конечное состояние и связать его ε-переходами со старыми конечными состояниями.

Таким образом все конструируемые на основе регулярных выражений автоматы будут представлять собой ε-НКА с одним допускающим состоянием.

Теорема 4.3.1. *Любой язык, определяемый регулярным выражением, можно задать некоторым конечным автоматом.* □

Доказательство. Предположим, что $L = L(R)$ для регулярного выражения R. Покажем, что $L = L(E)$ для некоторого ε-НКА E, обладающего следующими свойствами:

1) Автомат имеет ровно одно допускающее состояние.
2) У автомата нет дуг, ведущих в начальное состояние.
3) У автомата нет дуг, выходящих из допускающего состояния.

Для доказательства теоремы применим структурную индукцию по выражению R, следуя рекурсивному определению регулярных выражений из раздела 2.1.

В доказательстве леммы 3.4.2 построены конечные автоматы, распознающие элементарные языки. На рисунке 4.6 приведены схемы автоматов, распознающих пустой язык (*a*), язык из одного символа eps (*b*) и язык из одного однобуквенного слова (*c*).

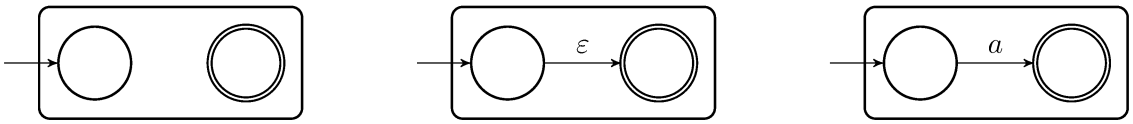


Рис. 4.6. Схемы автоматов, распознающих элементарные языки

Все эти автоматы удовлетворяют условиям (1), (2), (3) индуктивной гипотезы.

Предположим, что утверждение теоремы истинно для непосредственных подвыражений данного регулярного выражения, т.е. языки этих подвыражений являются также языками ε-НКА с единственным допускающим состоянием. Возможны четыре случая.

1) Данное выражение имеет вид $R + S$ для некоторых подвыражений R и S. Тогда ему соответствует автомат M_U, представленный на рисунке 4.7.

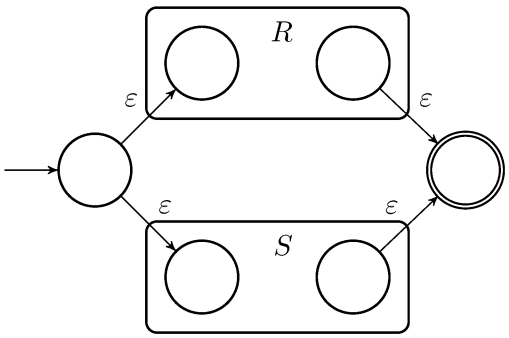


Рис. 4.7. Схема автомата, распознающего объединение двух языков

В этот автомат добавлено новое начальное состояние, из которого можно перейти в начальное состояние автомата для выражения R или S и продолжать работу, моделируя выбранный автомат. Попав в допускающее состояние автомата для R или S (распознав цепочку из языка $L(R)$ или $L(S)$ соответственно), новый автомат может по одному из ε-путей перейти в свое допускающее состояние. Таким образом, $L(M_U) = L(R) \cup L(S)$.

2) Выражение имеет вид RS для некоторых подвыражений R и S. Автомат M_C для распознавания конкатенации представлен на рисунке 4.8.

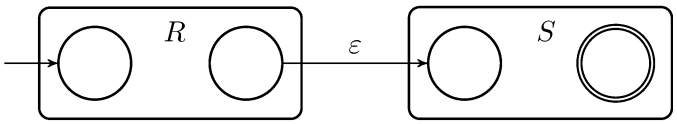


Рис. 4.8. Схема автомата, распознающего конкатенацию двух языков

Начальное состояние первого автомата становится начальным для всего автомата M_C, представляющего конкатенацию, а допускающим для него будет допускающее состояние второго автомата. Вначале автомат M_C моделирует поведение автомата для R (распознает цепочку из языка $L(R)$), потом из допускающего состояния первого автомата он переходит в начальное состояние автомата для S и моделирует его поведение (распознает цепочку из языка $L(S)$). Таким образом, $L(M_C) = L(R)L(S)$.

3) Выражение имеет вид R^* для некоторого подвыражения R. Рассмотрим автомат M_I, представленный на рисунке 4.9.

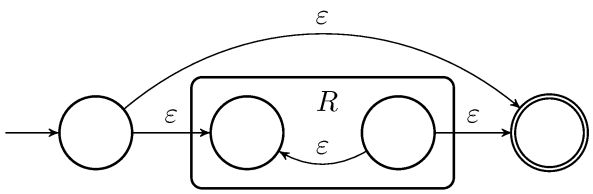


Рис. 4.9. Схема автомата, распознающего итерацию языка

Возможные случаи распознавания:

a) из начального состояния автомат M_I сразу переходит в допускающее состояние по символу ε. В этом случае допускается цепочка ε, которая принадлежит $L(R^*)$ независимо от выражения R;

b) из начального состояния автомат M_I переходит в начальное состояние автомата для R, моделирует поведение этого автомата и попадает в его допускающее состояние, из которого может начать заново моделировать автомат для R или перейти в свое допускающее состояние. Такое поведение автомата дает возможность распознавать цепочки, принадлежащие языкам $L(R)$, $L(R)L(R)$, $L(R)L(R)L(R)$ ... (R^*), за исключением, возможно, цепочки ε. Но возможность ее распознавания показана в предыдущем пункте. Таким образом, $L(M_I) = L(R^*)$.

4) Выражение имеет вид (R) для некоторого подвыражения R. Автомат для R может быть автоматом и для (R), поскольку скобки не влияют на язык, задаваемый выражением.

Построенные автоматы удовлетворяют всем трем условиям индуктивной гипотезы: одно допускающее состояние, отсутствие дуг, ведущих в начальное состояние, и дуг, выходящих из допускающего состояния. ■

Пример 4.3.1. Преобразуем регулярное выражение $(0+1)^*1(0+1)$ в ε-НКА. Вначале построим автомат для выражения $0+1$. Для этого используем два автомата, построенные по схеме на рисунке 4.6 (*c*): один автомат с меткой 0 на дуге, другой — с меткой 1. Эти автоматы соединим с помощью конструкции объединения по схеме 4.7. Полученный результат представлен на рисунке 4.10.

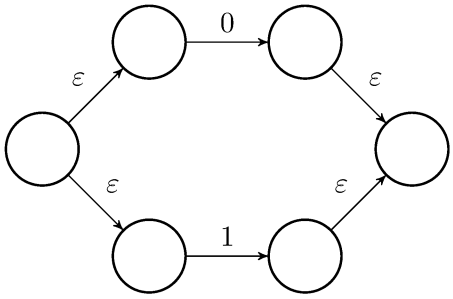


Рис. 4.10. Автомат для регулярного выражения $0 + 1$

После этого применим к полученному автомату конструкцию итерации по схеме 4.9. Получим автомат, представленный на рисунке 4.11.

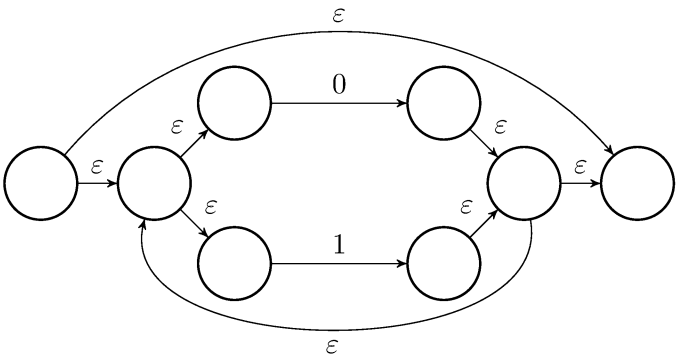


Рис. 4.11. Автомат для регулярного выражения $(0 + 1)^*$

Осталось к полученному автомату применить конструкции конкатенации по схеме 4.8. Сначала автомат, представленный на рисунке 4.11, соединяется с автоматом, допускающим только цепочку 1 (нужно еще раз применить конструкцию по схеме 4.6 (*c*) с меткой 1 на дуге). Последним автоматом в конкатенации будет еще один автомат для выражения $0 + 1$, построенный по тем же принципам, что и автомат на рисунке 4.11. Полный автомат для выражения $(0 + 1)^*1(0 + 1)$ представлен на рисунке 4.12.

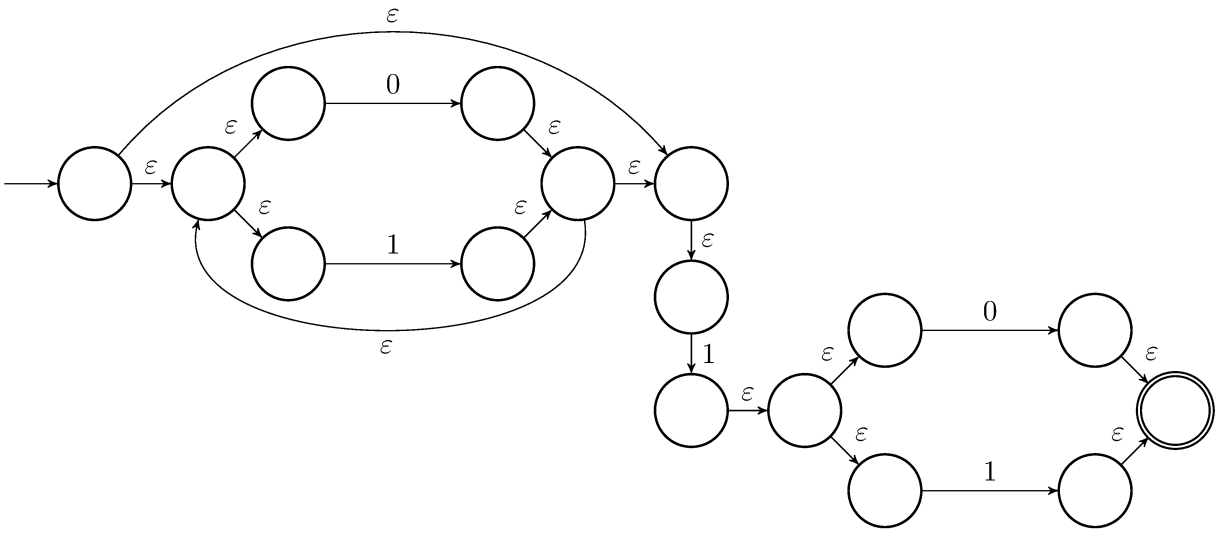


Рис. 4.12. Автомат для регулярного выражения $(0+1)^*1(0+1)$

4.4. Построение ε-НКА по ПЛ-грамматике

В разделе 1.4 приведена классификация Хомского формальных грамматик. По этой классификации ПЛ-грамматикой являются такая грамматика, все правила которой имеют вид:

$$\begin{matrix} A \to xB \\ A \to x \end{matrix},$$

где $A, B \in N$, $x \in \Sigma^*$.

Из леммы 3.4.1 известно, что для любого конечного автомата можно построить ПЛ-грамматику, порождающую тот же язык, что распознает исходный автомат. При этом получается, строго говоря, не ПЛ-грамматика, а автоматная грамматика, все правила которой имеют вид:

$$\begin{matrix} A \to xB \\ A \to x \end{matrix},$$

где $A, B \in N, x \in \Sigma$.

Фактически, автоматная грамматика - это ПЛ-грамматика, в правилах которой могут встречаться только одиночные терминальные буквы. Автомат работает следующим образом: по текущему состоянию и текущему входному символу автомат переходит в следующее состояние. В формальной грамматике нетерминальные символы соответствуют состояниям, а терминальные символы - входным символом автомата. За один раз автомат может прочитать только

один входной символ, который в принципе может состоять из нескольких букв, но автомат считает этот символ неделимым. С точки зрения же ПЛ-грамматики последовательность терминальных букв в правилах является частью выводимого слова.

Лемма 4.4.1. *Пусть задана ПЛ-грамматика* $G = (\Sigma, N, \mathcal{P}, S \in N)$. *Тогда* $\exists$ *такой* ε*-НКА* M, *что* $L(M) = L(G)$. □

Доказательство. Построим $M = (Q, \Sigma, \delta, q_0, F)$ следующим образом:

1) $Q = N \cup \{f\}$, где $f \notin N$.
2) $F = \{f\}$.
3) $q_0 = S$.
4) Для определения δ-функции введем вспомогательную $\hat{\delta}$-функцию по аналогии с конструкциями из леммы 3.4.1:
   a) Если $(A \to \alpha B) \in \mathcal{P}$, где $A, B \in N, \alpha \in \Sigma^*$, то $\hat{\delta}(A, \alpha) = B$.
   b) Если $(A \to \alpha) \in \mathcal{P}$, где $A \in N, \alpha \in \Sigma^*$, то $\hat{\delta}(A, \alpha) = f$.

   $\hat{\delta}$-функция в качестве второго входного параметра принимает цепочку (возможно, пустую) символов входного алфавита, то есть

   $$\hat{\delta}: Q \times (\Sigma^*) \to P(Q).$$

   Поскольку автомат за один такт может прочитать не более одного символа входного алфавита, необходимо перейти от $\hat{\delta}$-функции к δ-функции автомата M:

   $$\delta: Q \times (\Sigma \cup \{\varepsilon\}) \to P(Q).$$

   Пусть $\omega \in L(M)$, тогда $(q_0, \omega) \vdash_M^* (f, \varepsilon)$. Представим $\omega = \alpha_1\alpha_2...\alpha_n$. Тогда

   $$(q_0, \alpha_1\alpha_2...\alpha_n) \vdash_M (q_1, \alpha_2...\alpha_n) \vdash_M^* (q_{n-1}, \alpha_n) \vdash_M (f, \varepsilon).$$

   Дополним множество $\mathcal{P}$ правил грамматики G правилом $S \to \alpha_1 q_1$. Этому правилу будет соответствовать $\delta(q_0, \alpha_1) = \{q_1\}$. Повторяя эти действия для всех α_i, получим:

   $$S \Rightarrow^* \alpha_1\alpha_2...\alpha_{n-1}q_{n-1} \Rightarrow \alpha_1...\alpha_n = \omega.$$

Если $\hat{\delta}(q_1, \omega = \alpha_1\alpha_2...\alpha_k) = \{q_2\}$, то

$$\delta(q_1, \alpha_1) = \{q_1^{(1)}\};$$
$$\delta(q_1^{(1)}, \alpha_2) = \{q_1^{(2)}\};$$
$$...$$
$$\delta(q_1^{(k-2)}, \alpha_{k-2}) = \{q_1^{(k-1)}\};$$
$$\delta(q_1^{(k-1)}, \alpha_k) = \{q_2\}.$$

Таким образом, каждый такт автомата M соответствует одному правилу грамматики G, и $L(M) = L(G)$. ■

Пример 4.4.1. Рассмотрим грамматику $G = (\{S; T\}, \{0; 1\}, P, S)$, где множество P состоит из следующих продукций:

$$S \to 01T \mid 1S \mid \varepsilon;$$
$$T \to 11S \mid 000T \mid 01 \mid \varepsilon.$$

Построим ε-НКА M, применяя конструкции из леммы 4.4.1.

$$Q = \{S; T\} \cup \{f\};$$
$$\Sigma = \{0; 1\};$$
$$q_0 = S;$$
$$F = \{f\}.$$

По правилам грамматики G определим $\hat{\delta}$-функцию:

$$\hat{\delta}(S, 01) = T;$$
$$\hat{\delta}(S, 1) = S;$$
$$\hat{\delta}(S, \varepsilon) = f;$$
$$\hat{\delta}(T, 11) = S;$$
$$\hat{\delta}(T, 000) = T;$$
$$\hat{\delta}(T, 01) = f;$$
$$\hat{\delta}(T, \varepsilon) = f.$$

Граф переходов $\hat{\delta}$-функции представлен на рисунке 4.13.

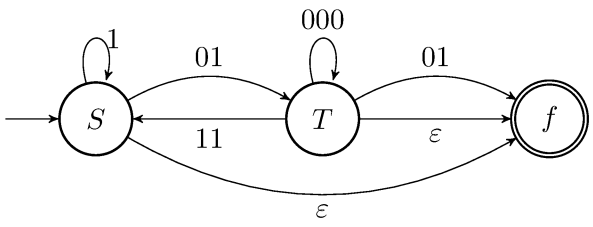


Рис. 4.13. Граф переходов $\hat{\delta}$-функции из примера 4.4.1

Для построения δ-функции автомата M определим множество «промежуточных» состояний, изменив множество P правил грамматики G следующим образом:

$$
\begin{aligned}
&S \to 0S_1 \mid 1S \mid \varepsilon;\\
&S_1 \to 1T;\\
&T \to 1T_1 \mid 0T_2 \mid 0T_3 \mid 0T_4 \mid \varepsilon;\\
&T_1 \to 1S;\\
&T_2 \to 0T_3;\\
&T_3 \to 0T;\\
&T_4 \to 1.
\end{aligned}
$$

По модифицированным правилам грамматики G определим δ-функцию:

$$
\begin{aligned}
&\delta(S, 0) = S_1;\\
&\delta(S_1, 1) = T;\\
&\delta(S, 1) = S;\\
&\delta(S, \varepsilon) = f;\\
&\delta(T, 1) = T_1;\\
&\delta(T, 0) = T_2;\\
&\delta(T, 0) = T_3;\\
&\delta(T, 0) = T_4;\\
&\delta(T, \varepsilon) = f;\\
&\delta(T_1, 1) = S;\\
&\delta(T_2, 0) = T_3;\\
&\delta(T_3, 0) = T;\\
&\delta(T_4, 1) = f.
\end{aligned}
$$

Множество состояний Q дополняется состояниями, полученными при расширении правил грамматики G:

$$Q = \{S; T; F\} \cup \{S_1; T_1; T_2; T_3; T_4\}$$ ■

Граф искомого автомата $M = (Q, \Sigma, q_0, \delta, F)$ представлен на рисунке 4.14.

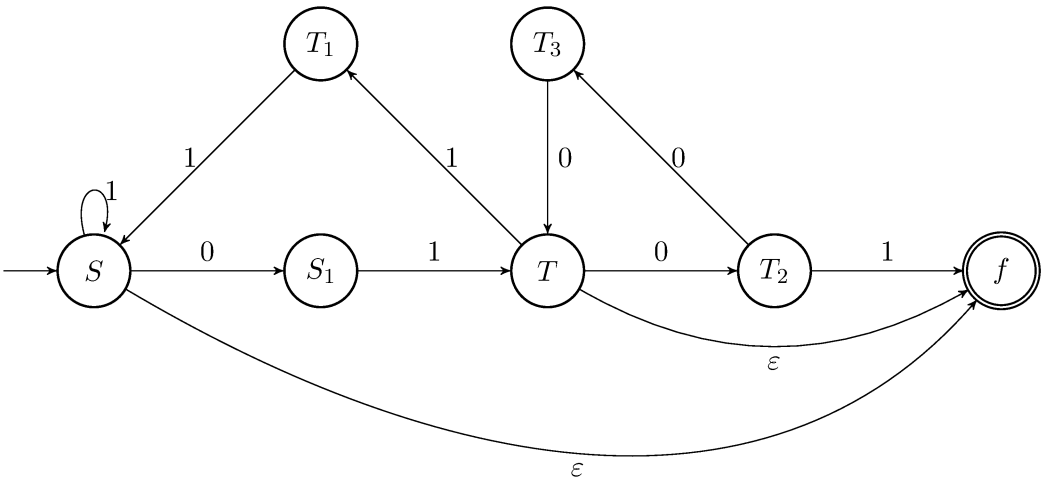


Рис. 4.14. Граф переходов ε-НКА из примера 4.4.1

4.5. Вычисление языка ε-НКА

По автомату, допускающему некоторый язык, можно построить регулярное выражение, задающее этот язык. Для этого нужно построить выражения, описывающие множества цепочек, которыми помечены определенные пути на графе переходов автомата. Эти пути могут проходить только через ограниченное подмножество состояний. При индуктивном определении таких выражений нужно начинать с самых простых выражений, описывающих пути, которые не проходят ни через одно состояние (т.е. являются отдельными вершинами или дугами). После этого индуктивно строятся выражения, которые позволяют этим путям проходить через постепенно расширяющиеся множества состояний. В конце этой процедуры будут получены пути, которые могут проходить через любые состояния, т.е. генерируются выражения, представляющие все возможные пути. Подробно эти идеи излагаются в [3] (раздел 3.2.1). В данной работе будет рассмотрен менее трудоёмкий способ вычисления регулярного выражения по конечному автомату.

Метод последовательного исключения состояний

Метод вычисления регулярного выражения, рассматриваемый в данном разделе, предполагает исключение состояний конечного автомата. Если исключить некоторое состояние r, то все пути автомата, проходящие через это со-

стояние, исчезают. Чтобы язык, допускаемый автоматом, не изменился, необходимо написать на дуге, ведущей непосредственно из некоторого состояния q в состояние p, метки всех тех путей, которые вели из состояния q в состояние p, проходя через состояние r. Поскольку теперь метка такой дуги будет содержать цепочки, а не отдельные символы, и таких цепочек может быть даже бесконечно много, то нельзя использовать список этих цепочек в качестве метки. Необходимо использовать конечный способ представления всех подобных цепочек, т.е., использовать регулярные выражения.

Таким образом, мы можем рассматривать автоматы, у которых метками являются регулярные выражения. Язык такого автомата представляет собой объединение по всем путям, ведущим от начального к заключительному состоянию, языков, образуемых с помощью конкатенации языков регулярных выражений, расположенных вдоль этих путей.

Опишем процедуру исключения состояния. Пусть $M = (Q, \Sigma, \delta, q_0, F)$ — конечный автомат. Рассмотрим подграф графа переходов автомата M (см. рис. 4.15). Введем операцию $DEL(p, r, q)$, где r — вершина, подлежащая удалению, следующим образом:

1) Если из вершины r ведет дуга с меткой γ в эту же вершину (петля), то после удаления вершины r этой дуге будет соответствовать метка γ^* (соответствие операции итерация).

2) Если $\exists$ путь из вершины p в вершину q, и этот путь проходит по дугам, помеченным α, α_1,..., β, то после удаления вершины r дуга (p, q) будет помечена меткой $\alpha\alpha_1, \ldots, \beta$, составленной из меток всех дуг, образующих этот путь (соответствие операции конкатенация).

3) Если $\exists$ другие пути, ведущие из вершины p в вершину q, то их метки объединяются (соответствие операции объединение).

Результат применения операции $DEL(p, r, q)$ к подграфу из рисунка 4.15 представлен на рисунке 4.16.

Опишем стратегию построения регулярного выражения по конечному автомату.

1) Если начальное состояние совпадает с допускающим, нужно ввести новое допускающее состояние и добавить ε-переход из начального состояния в добавленное. После этого начальное состояние перестает быть допускающим.

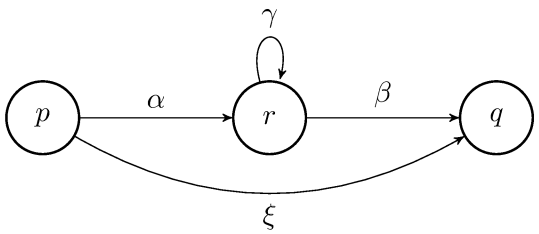


Рис. 4.15. Подграф графа конечного автомата M до удаления вершины r

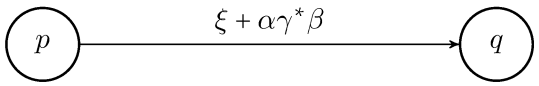


Рис. 4.16. Подграф графа конечного автомата M после удаления вершины r

2) Если автомат содержит более одного допускающего состояния, то нужно ввести новое допускающее состояние и добавить ε-переходы из старых допускающих состояний в добавленное состояние. Новое состояние становится единственным допускающим состоянием автомата.

3) С помощью алгоритма 4.5.1 исключить все состояния, кроме начального (q_0) и конечного (f).

4) После шагов 1-3 должен остаться автомат с двумя состояниями, подобный автомату на рисунке 4.17. Регулярное выражение по этому автомату может быть выписано разными способами.

   Рассмотрим выражение $(R+SU^*T)^*SU^*$ как один из возможных вариантов записи регулярного выражения по данному автомату. В автомате есть возможность переходить из начального состояния в него же любое количество раз, проходя по путям, метки которых принадлежат либо $L(R)$, либо $L(SU^*T)$. Выражение SU^*T представляет пути, которые ведут в допускающее состояние по пути с меткой из языка $L(S)$, затем, возможно, несколько раз проходят через допускающее состояние, используя пути с метками из $L(U)$, и наконец возвращаются в начальное состояние, следуя по пути с меткой из $L(T)$. Отсюда нужно перейти в допускающее состояние, уже никогда не возвращаясь в начальное, вдоль пути с меткой из $L(S)$. Находясь в допускающем состоянии, можно произвольное количество раз вернуться в него по пути с меткой из $L(U)$.

5) Искомое выражение представляет собой регулярное выражение, являющееся меткой дуги из начального состояния в конечное сокращенного автомата.

Алгоритм 4.5.1 Алгоритм исключения вершины из графа конечного автомата

Вход $\Gamma = (V, E, \phi)$ — граф автомата M, где ϕ — функция разметки;
$r \in V$ — вершина, которая исключается из графа автомата.

Выход $\Gamma' = (V', E', \phi)$ — граф автомата M без вершины r.

Метод

Шаг 1. Для $p \in V$, $p \neq r$ и $(p, r) \in E$ выполнить:
Для $q \in V$, $q \neq r$ и $(r, q) \in E$ выполнить:
$\phi(p, q) = \phi(p, q) \cup \phi(p, r)(\phi(r, r))^* \phi(r, q)$

Шаг 2. Исключить из графа Γ вершину q и инцидентные ей дуги.

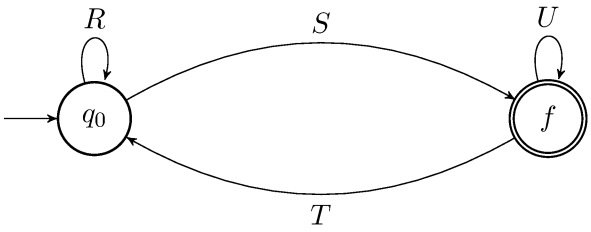


Рис. 4.17. Обобщенный автомат с двумя состояниями

4.6. Задача минимизации конечного автомата

Конечный автомат распознаёт регулярный язык, для которого может существовать несколько вариантов записи в виде регулярных выражений. В разделе 4.3 показано, что любое регулярное выражение можно преобразовать в конечный автомат, распознающий тот же язык. Таким образом один и тот же язык может быть распознан различными конечными автоматами, причём эти автоматы будут эквивалентны между собой.

В решении практических задач удобнее пользоваться конечным автоматом с меньшим числом состояний, поэтому возникает задача выбора конечного автомата с меньшим числом состояний из множества эквивалентных ему автоматов. Далее будет показано, что для любого конечного автомата можно построить эквивалентный ему автомат с меньшим числом состояний или установить минимальность исходного автомата.

Отношение эквивалентности на множестве конечных автоматов

Напомним, что отношение эквивалентности на множестве X — это бинарное отношение, для которого выполняются следующие условия:

1) Рефлексивность: $a \sim a$ для $\forall a \in X$.
2) Симметричность: $a \sim b$, то $b \sim a$ для $\forall a, b \in X$.
3) Транзитивность: $a \sim b$ и $b \sim c$, то $a \sim c$ для $\forall a, b, c \in X$.

Лемма 4.6.1. *Конечные автоматы (ε-НКА, НКА, ДКА) M_1 и M_2 над одним и тем же конечным алфавитом Σ называются эквивалентными, если $L(M_1) = L(M_2)$.*□

Доказательство. Покажем, что для введённого отношения эквивалентности на конечных автоматах выполняются свойства рефлексивности, симметричности и транзитивности. Пусть M_Σ — множество всех конечных автоматов (ε-НКА, НКА, ДКА) над входным алфавитом Σ.

1) Рефлексивность: $\forall M \in M_\Sigma \; M \sim M \Leftrightarrow L(M) = L(M)$.
2) Симметричность: $\forall M_1, M_2 \in M_\Sigma$ если $M_1 \sim M_2 \Leftrightarrow M_2 \sim M_1$, то $L(M_1) = L(M_2) \Leftrightarrow L(M_2) = L(M_1)$.
3) Транзитивность: $\forall M_1, M_2, M_3 \in M_\Sigma \; M_1 \sim M_2$ и $M_2 \sim M_3 \Rightarrow M_1 \sim M_3$, т. к. $L(M_1) = L(M_2)$ и $L(M_2) = L(M_3) \Rightarrow L(M_1) = L(M_3)$. ■

Итак, два конечных автомата (ε-НКА, НКА, ДКА) считаются эквивалентными между собой, если они допускают один и тот же язык. Нам известно, что автомат любого типа (ε-НКА, НКА) может быть сведён к ДКА. Тогда сформулируем задачу минимизации конечного автомата следующим образом:
Задача: Для произвольного ДКА найти эквивалентный ему ДКА с минимальным числом состояний. Минимальные ДКА всегда одинаковые. Если даны два минимальных ДКА для одного и того же языка, то всегда можно переименовать состояния так, что данные ДКА будут одинаковыми. Для того, чтобы построить минимальный ДКА, необходимо выделить состояния, которые можно удалить, не нарушая при этом эквивалентность исходного и модифицированного ДКА.

Недостижимые состояния конечного автомата

Безболезненно из конечного автомата можно удалить только те состояния, которые не участвуют в распознавании языка.

Алгоритм 4.6.1 Устранение недостижимых состояний конечного автомата

Вход Конечный автомат $M = (Q, \Sigma, \delta, q_0, F)$.

Выход Конечный автомат $M' = (Q', \Sigma, \delta', q_0, F')$ — КА без недостижимых состояний, такой что $L(M) = L(M')$.

Метод Рекурсивное построение расширяющейся последовательности подмножеств специального вида специального вида множества Q.

Шаг 1. Положить $Q_{D_0} = \{q_0\}$, $i = 1$.

Шаг 2. Положить $Q_{D_i} = \{p \mid \forall q \in Q_{D_{i-1}} \exists \delta(q,t) = \{p\}, t \in \Sigma\} \cup Q_{D_{i-1}}$.

Шаг 3. Если $Q_{D_i} \neq Q_{D_{i-1}}$, то положить $i = i + 1$ перейти к шагу 2, в противном случае положить $Q_D = Q_{D_i}$.

Шаг 4. Вернуть КА $M' = (Q', \Sigma, \delta', q_0, F')$, где $Q' = Q_D \cap Q$, $F' = Q_D \cap F$, $\delta' = \delta \setminus \{\delta(q,t) = p \mid q \in (Q \setminus Q_D), t \in \Sigma\}$.

Определение: Состояние $q \in Q$ конечного автомата $M = (Q, \Sigma, \delta, q_0, F)$ называется недостижимым, если $\nexists \omega \in \Sigma^* \mid (q_0, \omega) \vdash_M^* (q, \varepsilon)$.

Из определения следует, что нельзя подобрать никакую цепочку, по которой можно было бы пройти от стартового состояния до недостижимого, следовательно это состояние не используется при распознавании любых цепочек языка и может быть удалено вместе с соответствующими переходами.

Пример 4.6.1. Рассмотрим детерминированный конечный автомат $M = (Q, \Sigma, \delta, q_0, F)$, заданный графом переходов на рисунке 4.18. По алгоритму 4.6.1 построим множество достижимых состояний Q_D:

$Q_{D_0} = \{q_0\}$

Из вершины q_0 можно перейти в вершины q_1, q_2. Таким образом

$Q_{D_1} = \{q_1, q_2\} \cup \{q_0\}$

Из вершин q_1, q_2 можно попасть в вершины q_3, q_4 соответственно. Таким образом

$Q_{D_2} = \{q_3, q_4\} \cup \{q_0, q_1, q_2\}$

Из вершины q_3 можно попасть в вершины q_2, q_4, а из q_4 — в вершины q_1, q_3. Тогда $Q_{D_3} = \{q_2, q_4, q_1, q_3\} \cup \{q_0, q_1, q_2, q_3, q_4\}$, следовательно $Q_{D_2} = Q_{D_3}$ и искомое множество достижимых состояний $Q_D = \{q_0, q_1, q_2, q_3, q_4\}$. Конечный автомат без недостижимых состояний, эквивалентный исходному, представлен на рисунке 4.19. ■

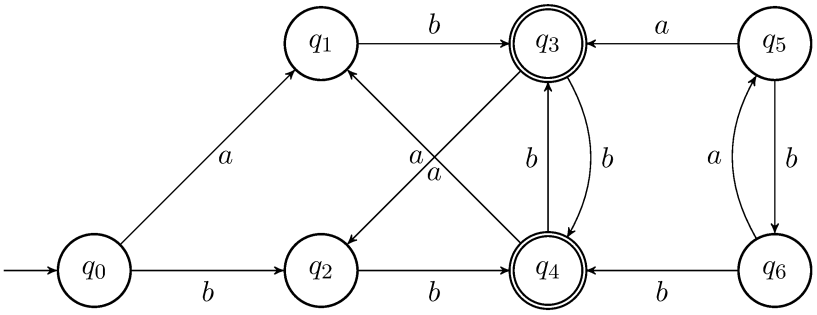


Рис. 4.18. Граф переходов ДКА с недостижимыми состояниями

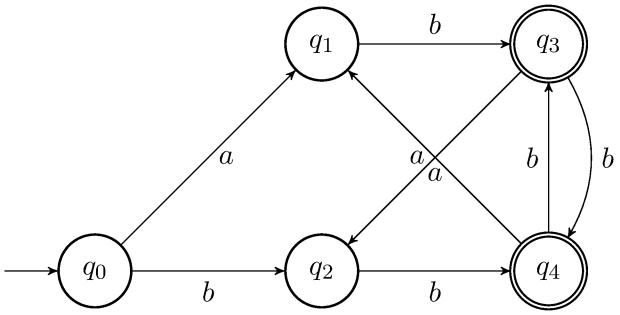


Рис. 4.19. Граф переходов ДКА без недостижимых состояний

Неразличимые состояния конечного автомата

Определение: Пусть $M = (Q, \Sigma, \delta, q_0, F)$ — детерминированный конечный автомат. Два состояния $p, q \in Q$ называются *неразличимыми*, если для $\forall \omega \in \Sigma^*$ выполняется

$$(p, \omega) \vdash_M^* (f_1, \varepsilon), (q, \omega) \vdash_M^* (f_2, \varepsilon)$$

, при этом либо $f_1, f_2 \in F$, либо $f_1, f_2 \notin F$. f_1, f_2 могут быть двумя разными или одним состоянием.

Иными словами, состояния невозможно различить, если просто проверить, допускает ли данный КА входную цепочку, начиная работу в одном (произвольно) из этих состояний.

Определение: Пусть $M = (Q, \Sigma, \delta, q_0, F)$ — детерминированный конечный автомат. Два состояния $p, q \in Q$ называются *различимыми*, если $\exists \omega \in \Sigma^*$ такая, что

$$(p, \omega) \vdash_M^* (f_1, \varepsilon), (q, \omega) \vdash_M^* (f_2, \varepsilon)$$

, при этом либо $f_1 \in F$ и $f_2 \notin F$, либо наоборот. Очевидно, что $f_1 \neq f_2$ в обязательном порядке.

Слово ω в этом случае называется различающим словом состояний p, q. Любая пара состояний конечного автомата либо различима, либо нет.

Рассмотрим лемму об отношении неразличимости.

Лемма 4.6.2. *Пусть $M = (Q, \Sigma, \delta, q_0, F)$ — ДКА. Неразличимость является отношением эквивалентности на множестве Q и разбивает Q в объединение непересекающихся классов эквивалентности, каждый из которых состоит из попарно непересекающихся состояний, но при этом любая пара состояний из разных классов эквивалентности будет различимой.* □

Доказательство. Покажем, что для введённого отношения неразличимости на множестве состояний конечного автомата выполняются свойства рефлексивности, симметричности и транзитивности.

1) Рефлексивность: $\forall p \in Q\ p \sim p \Leftrightarrow \forall \omega \in \Sigma^*\ (p, \omega) \vdash^* (f_1, \varepsilon), (p, \omega) \vdash^* (f_1, \varepsilon)$, т. к. $f_1 = f_1$, то (p, p) - неразличимы, следовательно $p \sim p$.
2) Симметричность: $\forall p, q \in Q$ если $p \sim q$, то $q \sim p$:
   $\forall \omega \in \Sigma^* (p, \omega) \vdash^* (f_1, \varepsilon), (q, \omega) \vdash^* (f_2, \varepsilon)$. Так как $p \sim q$, то либо $f_1, f_2 \in F$, либо $f_1, f_2 \notin F$.
3) Транзитивность: $\forall p, q, r$ если $p \sim q, q \sim r$, то $p \sim r$.
   $\forall \omega \in \Sigma^* (p, \omega) \vdash^* (f_1, \varepsilon), (q, \omega) \vdash^* (f_2, \varepsilon)$. Так как $p \sim q$, то либо $f_1, f_2 \in F$, либо $f_1, f_2 \notin F$.
   $(q, \omega) \vdash^* (f_2, \varepsilon), (r, \omega) \vdash^* (f_3, \varepsilon)$. Так как $q \sim r$, то либо $f_2, f_3 \in F$, либо $f_2, f_3 \notin F$.
   $(p, \omega) \vdash^* (f_1, \varepsilon), (q, \omega) \vdash^* (f_2, \varepsilon), (r, \omega) \vdash^* (f_3, \varepsilon)$ и либо $f_1, f_2, f_3 \in F$, либо $f_1, f_2, f_3 \notin F$. ■

Определение: Пусть $M = (Q, \Sigma, \delta, q_0, F)$ — ДКА. q_1, q_2 — различимые состояния M.

Будем говорить, что q_1, q_2 — k-неразличимы $q_1 \sim^k q_2$, если не существует такой цепочки ω, различающей q_1, q_2, длина которой меньше или равна k.

Состояния q_1, q_2 неразличимы ($q_1 \sim q_2$), если они k-неразличимы при любом $k \geqslant 0$.

Лемма 4.6.3. *Пусть $M = (Q, \Sigma, \delta, q_0, F)$ — ДКА с n состояниями. Состояния q_1, q_2 неразличимы $\Leftrightarrow$ они $(n-2)$ неразличимы.* □

Доказательство. Необходимость условия тривиальна. Для того, чтобы была возможность различения состояний, в конечном автомате должны быть минимум одно обычное и одно конечное состояние. Количество остальных состояний не превышает $n - 2$.

Достаточность тривиальна в тех случаях, когда множество F содержит 0 или n элементов. Рассмотрим случай, когда число элементов множества F отличается от 0 или n.

Покажем, что

$$\sim \subseteq \sim^{n-2} \subseteq \sim^{n-3} \subseteq \ldots \subseteq \sim^2 \subseteq \sim^1 \subseteq \sim^0$$

Заметим, что для любых состояний q_1, q_2 выполняются следующие свойства:

1) $q_1 \sim^0 q_2 \Leftrightarrow q_1, q_2 \in F$ или $q_1, q_2 \notin F$.

2) $q_1 \sim^k q_2 \Leftrightarrow q_1 \sim^{k-1} q_2$ и $\forall x \in \Sigma \delta(q_1, x) \sim^{k-1} \delta(q_2, x)$.

Отношение $\sim^0$ самое грубое. Оно разбивает множество Q на два класса: F и $Q \setminus F$. Если $\sim^{k+1} \neq \sim^k$, то отношение $\sim^{k+1}$ содержит по крайней мере на один класс эквивалентности больше, чем $\sim^k$, то есть оно тоньше. Поскольку каждое множество из F и $Q \setminus F$ содержит не более $n-1$ элементов, можно получить не более $n-2$ последовательных утончений отношения $\sim^0$. Если $\sim^{k+1} = \sim^k$ для некоторого k, то в силу свойства 2 $\sim^{k+1} = \sim^{k+2} = \ldots$. Таким образом, $\sim$ — это первое из отношений $\sim^k$, для которых $\sim^{k+1} = \sim^k$. ■

Вывод: если два состояния можно различить, то их можно различить с помощью входной цепочки, длина которой меньше числа состояний конечного автомата. Таким образом процесс различения любой пары состояний конечен.

Построение минимального конечного автомата

Для любого конечного автомата можно найти эквивалентный ему минимальный конечный автомат. Для этого нужно убрать из исходного автомата недостижимые и неразличимые состояния. Поскольку неразличимые состояния не участвуют в распознавании цепочек, а пара неразличимых состояний не влияет на результат распознавания, то удаление этих состояний не приведёт к изменению распознаваемого языка.

Определение: Полностью определённый детерминированный конечный автомат называется каноническим (приведённым), если он не содержит недостижимых состояний и любая пара состояний этого автомата различима.

Теорема 4.6.1. *Автомат M', который строится алгоритмом 4.6.2, содержит наименьшее число состояний среди всех эквивалентных ему конечных автоматов.* □

Алгоритм 4.6.2 Построение канонического автомата (Минимизация ДКА)

Вход Полностью определённый ДКА $M = (Q, \Sigma, \delta, q_0, F)$.

Выход Приведённый ДКА $M' = (Q', \Sigma, \delta', q_0, F')$, такой что $L(M') = L(M)$.

Метод Поиск и удаление недостижимых и неразличимых состояний.

Шаг 1. Применить к конечному автомату M алгоритм поиска недостижимых состояний и построить конечный автомат M_1 без недостижимых состояний, такой что $L(M_1) = L(M)$.

Шаг 2. Строить отношения эквивалентности $\sim^0, \sim^1, \ldots$ по описанию в лемме 4.6.3 до тех пор, пока это будет возможно, т.е. $\sim^{k+1}=\sim^k$. Взять в качестве $\sim$ отношение $\sim^k$.

Шаг 3. Построить множество Q' как множество классов эквивалентности отношения $\sim$. Через $[p]$ будем обозначать класс эквивалентности отношения $\sim$, содержащий состояние p.

Шаг 4. Построить $\delta'([p], a) = [q]$, если $delta(p, a) = q$.

Шаг 5. Обозначить q'_0 как q_0.

Шаг 6. Обозначить F' как $\{[q], q \in F\}$.

Шаг 7. Вернуть $M' = (Q', \Sigma, \delta', q_0, F')$.

Доказательство. Пусть $M' = (Q', \Sigma, \delta', q_0, F')$ — приведённый конечный автомат. Предположим, что существует такой КА $M_m = (Q_m, \Sigma, \delta_m, q_{0_m}, F_m)$, что $|Q_m| < |Q'|$ и $L(M_m) = L(M')$.

В силу шага 1 алгоритма все состояния автомата M' достижимы. Так как M_m имеет меньше состояний, то найдутся цепочки ω, x, переводящие состояние q_0 в разные состояния, а q_{0_m} — в одно и то же: $(q_{0_m}, \omega) \vdash^*_{M_m} (q, \varepsilon)$ и $(q_{0_m}, x) \vdash^*_{M_m} (q, \varepsilon)$. Следовательно, ω, x переводят автомат M' в различимые состояния, например в p, r. Следовательно, существует такая цепочка y, что точно одна из цепочек $\omega y, xy$ принадлежит $L(M')$. Но $\omega y, xy$ должны переводить M_m в одно и то же состояние s, для которого $(q, y) \vdash^*_{M_m} (s, \varepsilon)$. Таким образом, точно одна из цепочек $\omega y, xy$ не может принадлежать $L(M_m)$, а это противоречит предположению о том, что $L(M_m) = L(M')$. ■

Пример 4.6.2. Пусть $M = (\{A; B; C; D; E; F\}, \{a; b\}, \delta, A, \{A; F\})$ — детерминированный полностью определённый конечный автомат, где функция переходов δ задаётся таблицей 4.4.

δ		Вход	
		a	b
Состояние	$\boxed{\to A}$	$\{F\}$	$\{B\}$
	B	$\{E\}$	$\{D\}$
	C	$\{C\}$	$\{F\}$
	D	$\{D\}$	$\{A\}$
	E	$\{B\}$	$\{C\}$
	$\boxed{F}$	$\{F\}$	$\{E\}$

Таблица 4.4. Функция перехода δ для автомата из примера 4.6.2.

Применим к автомату M алгоритм 4.6.2 минимизации ДКА и построим приведённый автомат M', эквивалентный исходному.

Вначале убедимся, что все состояния автомата M достижимы. Для этого проследим переходы из начального автомата во все остальные: из начального состояния A достижимы состояния F, B. Из пары F, B можно перейти в состояния F, E, D. Из состояний E, D возможны переходы в состояния B, C, D, A. Таким образом все состояния автомата M достижимы.

Начнём разбиение множества Q на классы неразличимости:

- *Неразличимость цепочками длины* $0 (\sim^0)$. На этом этапе множество состояний разбивается на два класса финальных и обычных состояний. Чтобы отличить финальное состояние от нефинального, цепочка не требуется. В результате получим два непересекающихся подмножества множества Q: $[A; F], [B; C; D; E]$.
- *Неразличимость цепочками длины* $1 (\sim^1)$. Здесь будем работать с каждым классом, выделенным на предыдущем шаге. Неразличимость цепочками длины 1 означает, что переход по одной букве из проверяемой пары состояний приводит в одинаковые по типу состояния. Проверим пару состояний $[A; F]$. Из состояния A по букве a попадаем в финальное состояние F, по букве b — в обычное состояние B. Из состояния F по букве a попадаем в финальное состояние F, по букве b — в обычное состояние E. Таким образом переход по одной букве (любой из алфавита Σ не различает состояния, множество неразличимый состояний стабилизировалось. Мы нашли первую группу неразличимых состояний $[A; F]$. Проверим пары состояний из множества $[B; C; D; E]$. Для установления неразличимо-

сти цепочками длины 1 достаточно посмотреть на правую часть таблицы переходов. Если в паре строк для проверяемых состояний справа финальные и нефинальные состояния расположены одинаково, то эти состояния находятся в классе неразличимости $\sim^1$. В данном примере будут выделены следующие классы неразличимости цепочками длины 1: $[B; E]$ и $[C; D]$.

- *Неразличимость цепочками длины* $0(\sim^2)$. На этом шаге нам осталось проверить, различимы ли пары $[B; E]$ и $[C; D]$ цепочками терминалов длины 2. Для этого из каждой пары состояний нужно сделать два перехода по всем комбинациям цепочек длины 2 и сравнить пары состояний, в которых автомат остановился.

Проверим пару состояний $[B; E]$:

	$\sim^1$		$\sim^2$
B	$\{E, D\}$	E	$\{B, C\}$
E	$\{B, C\}$	D	$\{D, \mathbf{A}\}$
		B	$\{E, D\}$
		C	$\{C, \mathbf{F}\}$

Жирным в таблице выделены финальные состояния. Заметим, что расположение финальных и обычных состояний в таблице одинаково, следовательно рассматриваемая пара состояний неразличима цепочками длины 2.

Проверим пару состояний $[C; D]$:

	$\sim^1$		$\sim^2$
C	$\{C, \mathbf{F}\}$	C	$\{C, \mathbf{F}\}$
D	$\{D, \mathbf{A}\}$	F	$\{\mathbf{F}, E\}$
		D	$\{D, \mathbf{A}\}$
		A	$\{\mathbf{F}, B\}$

В получившейся таблице расположение финальных и нефинальных состояний одинаково, следовательно пара $[C; D]$ неразличима цепочками длины 2.

Поскольку на этом шаге нам не удалось уточнить предыдущие классы неразличимости (множества неразличимых состояний стабилизировались), процесс выделения классов неразличимости можно завершить.

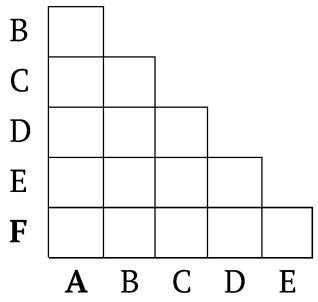
В результате выделены три класса неразличимости: $X = [A; F]$, $Y = [B; E]$ и $Z = [C; D]$. Минимальный автомат $M' = (\{X; Y; Z\}, \{a; b\}, \delta', X, \{X\})$ задаётся таблицей переходов 4.5.

δ		Вход	
		a	b
Состояние	$\to X$	$\{X\}$	$\{Y\}$
	Y	$\{Y\}$	$\{Z\}$
	$\boxed{Z}$	$\{Z\}$	$\{X\}$

Таблица 4.5. Функция перехода δ для минимального автомата из примера 4.6.2.

В примере 4.6.2 показан принцип разбиения множества состояний конечного автомата на классы неразличимости. Для простых случаев вести «дневник» переходов достаточно просто, но если нужно установить неразличимость цепочками длины больше 3, то запись переходов будет очень громоздкой. В [3] изложена методика поиска неразличимых состояний путём заполнения таблицы. В данном случае громоздкий «дневник» переходов представляется в компактной форме таблицы неразличимости.

Пример 4.6.3. Еще раз рассмотрим автомат из примера 4.6.2. Пусть $M = (\{A;B;C;D;E;F\},\{a;b\},\delta,A,\{A;F\})$ — детерминированный полностью определённый конечный автомат, где функция переходов δ задаётся таблицей 4.4. В примере 4.6.2 установлено, что этот автомат не содержит недостижимых состояний. Подготовим для множества Q таблицу неразличимости следующим образом:



Принцип заполнения таблицы следующий: в ячейку ставим X, если пара состояний, соответствующая этой ячейке, различима. На первом шаге расставим X в ячейках, соответсвующих парам финальных и обычных состояний (установим неразличимость состояний цепочками длины 0):

B	X				
C	X				
D	X				
E	X				
F		X	X	X	X
	A	B	C	D	E

Далее по таблице переходов определяем пары неразличимых состояний цепочками длины 1. Для этого нам достаточно сравнить строки справа и посмотреть расположение финальных и нефинальных состояний. Получим следующую таблицу неразличимости:

B	X				
C	X	X			
D	X	X			
E	X		X	X	
F		X	X	X	X
	A	B	C	D	E

Незаполненными остались ячейки на пересечении пар состояний $[F;A]$, $[E;B]$, $[D;C]$. Рассмотрим пару $[F;A]$. По таблице переходов из состояния F можно попасть в состояния F, E по одной букве, а из состояния A — в состояния F, B. В результате получается, что буква a не различает состояния $[F;A]$, поскольку переводит автомат в одно и то же состояние F. Буква b переводит автомат из состояний F, A в состояния B, E соответственно. В таблице неразличимости на пересечении этих состояний пока нет отметки о неразличимости. Можем условно поставить в ячейку на пересечении состояний $[F;A]$ знак вопроса и перейти к исследованию пары состояний $[E;B]$. Для этой пары состояний буква a переводит автомат в те же состояния (перекрёстно), а буква b переводит автомат в состояния D, C. Для этой пары в таблице нет отметки о неразличимости, поэтому ставим знак вопроса и переходим к анализу пары состояний $[D;C]$. Буква a переводит автомат в ту же пару состояний C, D, а b — в пару состояний F, A, для которой мы уже поставили в таблицу знак вопроса. В итоге круг замкнулся, и мы выделили пары неразличимых состояний:

B	X				
C	X	X			
D	X	X	O		
E	X	O	X	X	
F	O	X	X	X	X
	A	B	C	D	E

Далее можно переобозначить выделенные классы неразличимости: $X = [A; F]$, $Y = [B; E]$ и $Z = [C; D]$. Минимальный автомат M', эквивалентный исходному автомату M, уже построен в примере 4.6.2. ■

4.7. Упражнения

Построение ε-НКА по регулярному выражению

Построить ε-НКА по следующим регулярным выражениям:

1) $R = (a + b + c)(bab) * (a + b)$;
2) $R = 1(10 + 10)^*1(0 + 1)^*$;
3) $R = (a^*b^*)^* + (ab + b)^*$.

Для каждого полученного ε-НКА построить соответствующий ему ДКА.

Построение ε-НКА по ПЛ-грамматике

Построить ε-НКА по грамматикам со стартовым символом S и продукциями:

(1)
$$\begin{aligned} S &\to 10A \mid 101A, \\ A &\to 0A \mid 1B, \\ B &\to 1 \mid 0; \end{aligned}$$

(2)
$$\begin{aligned} S &\to abaA \mid abB, \\ A &\to a \mid aaB \mid baC, \\ B &\to b \mid abaC, \\ C &\to aaB \mid a; \end{aligned}$$

(3)
$$\begin{aligned} S &\to xA \mid B, \\ A &\to yxyA \mid \varepsilon \mid xB, \\ B &\to b \mid \varepsilon. \end{aligned}$$

Методом исключения состояний определить язык для каждого полученного автомата.

Минимизация конечного автомата

Минимизировать конечные автоматы, заданные таблицами переходов:

1)

δ	Вход		
	a	b	ε
$\to A$	$\{B;C\}$	$\{C\}$	$\{D\}$
B	$\{C\}$	$\{D\}$	
C	$\{B;C\}$	$\{D\}$	
$\boxed{D}$	$\varnothing$	$\varnothing$	

2)

δ	Вход	
	a	b
$\to q_1$	q_2	q_4
q_2	q_2	q_3
$\boxed{q_3}$	q_4	q_5
q_4	q_4	q_5
$\boxed{q_5}$	q_2	q_3

Методом исключения состояний определить язык полученных автоматов.

ГЛАВА 5

БУЛЕВА АЛГЕБРА РЕГУЛЯРНЫХ ЯЗЫКОВ

5.1. Свойства регулярных языков

Регулярные (а значит, и автоматные, и ПЛ-) языки обладают большим количеством интересных свойств, многие из которых легко доказываются на основе соответствия между упомянутыми классами языков.

Рассмотрим свойства замкнутости регулярных языков относительно теоретико-множественных операций, а также операций, свойственных для цепочек над заданным алфавитом (конкатенация, итерация, обращение, гомоморфизм). Можно сказать, что регулярные множества представляют собой подалгебру алгебры всех подмножеств Σ^*.

Пусть Σ — конечный алфавит, Σ^* — полный язык над алфавитом Σ. Выделим регулярные подмножества $L_1, L_2 \subseteq \Sigma^*$. Если L_1, L_2 — регулярные языки над алфавитом Σ, то регулярными являются следующие языки:

1) $L_1 \cup L_2$
2) $L_1 L_2$
3) L_1^*
4) $L_1^R = \{\omega^R \mid \omega \in L_1\}$
5) $\bar{L}_1 = \Sigma^* \setminus L_1$
6) $L_1 \cap L_2$
7) $L_1 \setminus L_2$
8) $H(L_1)$
9) $H^{-1}(L_1)$

Язык регулярен, если можно построить конечный автомат, распознающий этот язык. Пункты $1, 2, 3$ подробно рассматривались и доказывались в разделе 3.4. Построим конечные автоматы для языков из пунктов $4, 5, 6, 7$.

Рассмотрим язык $L_1^R = \{\omega^R \mid \omega \in L_1\}$ (пункт 4). Поскольку язык L_1 — регулярный, для него существует конечный автомат. Пусть $M_1 = (Q, \Sigma, \delta, q_0, F)$ — конечный автомат, у которого $L(M_1) = L_1$. Построим конечный автомат $M^R = (Q_R, \Sigma, \delta^R, F^R)$ следующим образом:

1) Если $\mid F \mid > 1$, то ввести новое финальное состояние q_f. Из каждого старого финального состояния $p \in F$ ввести ε-переход в новое финальное состояние. Положить $F = \{q_f\}$.
2) Обратить все стрелки диаграммы переходов автомата M.
3) Положить $q_R = q_f$, $F^R = \{q_0\}$.

Построенный таким образом автомат M^R начинает распознавать цепочку с последней буквы и входит в допускающее состояние после прочтения первой буквы.

Проверим, что результат обращения регулярного языка в алфавите Σ также является регулярным языком:

1) $\{\varnothing\}^R = \{\varnothing\}$, $\{\varepsilon\}^R = \{\varepsilon\}$, $\{a\}^R = \{a\} \forall a \in \Sigma$.
2) $(L_1 \cup L_2)^R = (L_1^R \cup L_2^R)$.
3) $(L_1 L_2)^R = (L_1^R L_2^R)$.
4) $(L_1^*)^R = \{\varepsilon\}^R \cup (L_1)^R \cup (L_2)^R \ldots (L_1^R)^*$.

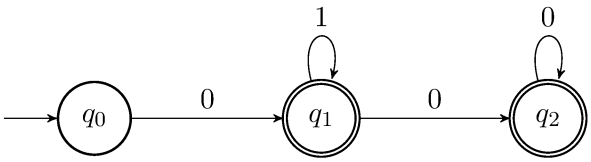


Рис. 5.1. ДКА из примера 5.1.1, распознающий язык $L = 01^*0^*$

Пример 5.1.1. Пусть $L = 01^*0^*$. Диаграмма автомата, распознающего язык L, приведена на рисунке 5.1. В автомате два допускающих состояния q_1, q_2. Сведём эти состояния в одно новое допускающее состояние q_f, добавив ε-переходы. Диаграмма конечного автомата, допускающего язык $L^R = 0^*1^*0$ представлена на рисунке 5.2. ■

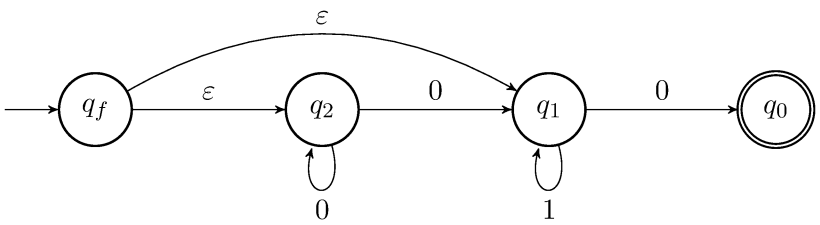


Рис. 5.2. ДКА из примера 5.1.1, распознающий язык $L^R = 0^*1^*0$

Рассмотрим язык $\bar{L}_1$ (пункт 5). Поскольку язык L_1 — регулярный, для него существует конечный автомат. Пусть $M_1 = (Q, \Sigma, \delta, q_0, F)$ — полностью определённый ДКА, у которого $L(M_1) = L_1$. Рассмотрим конечный автомат $\bar{M}_1 = (Q, \Sigma, \delta, q_0, Q \setminus F)$. Автомат $\bar{M}_1$ определяется такой же δ-функцией, что и M_1, то есть их поведения одинаковы. Возьмём произвольную цепочку $\omega \in L_1$. Для неё справедливо следующее:

$$(q_0, \omega) \vdash^*_{M_1} (q_f, \varepsilon), q_f \in F$$

В то же время в автомате $\bar{M}_1$ для этой цепочки выполняется следующее:

$$(q_0, \omega) \vdash^*_{\bar{M}_1} (q_f, \varepsilon), q_f \in (Q \setminus F)$$

Отсюда следует, что если произвольная цепочка допускается в M_1, то в автомате $\bar{M}_1$ эта цепочка приводит к недопускающему состоянию. Таким образом $L(\bar{M}_1) = \bar{L}_1$ — регулярный язык.

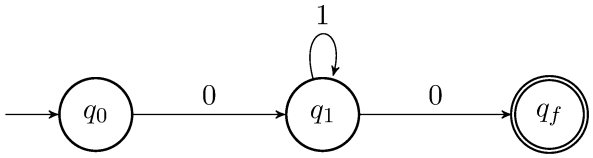


Рис. 5.3. ДКА из примера 5.1.2, распознающий язык $L = 01^*0$

Пример 5.1.2. Пусть $L = 01^*0$. Диаграмма автомата, распознающего язык L, приведена на рисунке 5.3. Представленный ДКА не полностью определён. Идея инвертирования конечного автомата заключается в том, чтобы все недопускающие состояния стали допускающими и наоборот, а в неполностью определенных ДКА дан не весь набор состояний, в который автомат может перейти. Иными словами, в полностью определённых конечных автоматах заложено описание как распознаваемого языка, так и дополнения к нему, а в неполностью определённых — только описание распознаваемого языка.

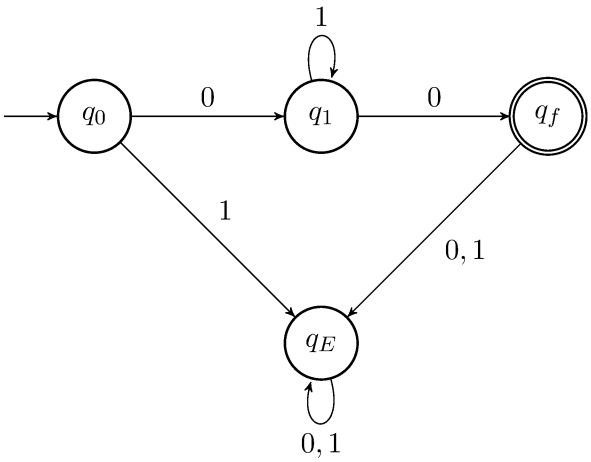


Рис. 5.4. Полностью определённый ДКА из примера 5.1.2, распознающий язык $L = 01^*0$

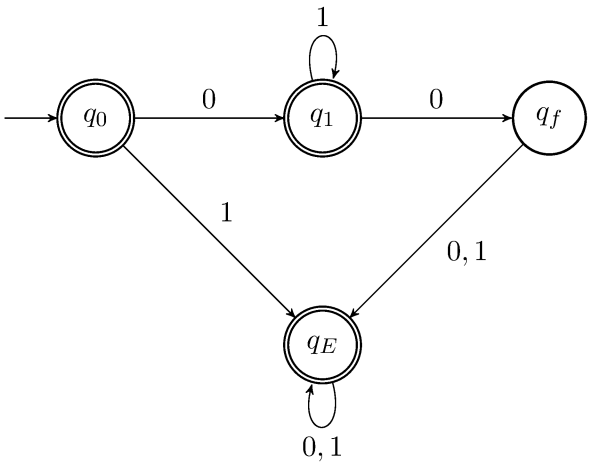


Рис. 5.5. ДКА из примера 5.1.2, распознающий язык $\bar{L} = \varepsilon+1(0+1)^*+01^*(\varepsilon+0(0+1)(0+1)^*)$

Доопределим ДКА на рисунке 5.3 следующим образом: введём новое недопускающее состояние q_E и добавим недостающие переходы в это состояние. Получившийся полностью определённый ДКА представлен на рисунке 5.4. Теперь инвертируем состояния и получим ДКА (рисунок 5.5), распознающий язык $\bar{L} = \varepsilon + 1(0 + 1)^* + 01^*(\varepsilon + 0(0 + 1)(0 + 1)^*)$ ■

Упражнение 5.1.1. Задача построения конечного автомата, распознающего дополнение языка, была сведена к взятию дополнения множества заключительных состояний детерминированного автомата исходного языка. На самом деле, этот факт требует отдельного доказательства, которое читателю предлагается построить самостоятельно. Неожиданным является то, что для недетер-

минированного автомата исходного языка такая конструкция в общем случае не работает. Приведите контрпример, иллюстрирующий эту проблему. □

Рассмотрим язык $L = L_1 \cap L_2$ (пункт 6). Поскольку языки L_1, L_2 — регулярны, для них существуют конечные автоматы. Пусть $M_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ — ДКА, у которого $L(M_1) = L_1$. $M_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$ — ДКА, у которого $L(M_2) = L_2$.

Построим конечный автомат $M = (Q_1 \times Q_2, \Sigma, \delta, (q_1, q_2), F_1 \times F_2)$, у которого

$$\delta : \forall a \in \Sigma, \forall p \in Q_1, \forall q \in Q_2 \delta((p, q), a) = (\delta_1(p, a), \delta_2(q, a)).$$

Покажем, что построенный автомат распознаёт $L_1 \cap L_2$.

Произвольная цепочка $\omega \in L(M) \Leftrightarrow ((q_1, q_2), \omega) \vdash_M^* ((q_{f_1}, q_{f_2}), \varepsilon), q_{f_1} \in F_1, q_{f_2} \in F_2$. Вместе с этим

$$(q_1, \omega) \vdash_{M_1}^* (q_{f_1}, \varepsilon), q_{f_1} \in F_1$$

$$(q_2, \omega) \vdash_{M_2}^* (q_{f_2}, \varepsilon), q_{f_2} \in F_2$$

Таким образом, цепочка допускается автоматом M, если она одновременно допускается и автоматом M_1, и автоматом M_2.

Пример 5.1.3. Пусть $L_1 = a^*bc$ и $L_2 = b^*ca^*$. $M_1 = (\{q_0; q_1; q_f\}, \{a; b; c\}, \delta_1, q_0, \{q_f\})$ и $M_2 = (\{p_0; p_f\}, \{a; b; c\}, \delta_2, p_0, \{p_f\})$.

δ_1	Вход		
	a	b	c
$\to q_0$	$\{q_0\}$	$\{q_1\}$	$\varnothing$
q_1	$\varnothing$	$\varnothing$	$\{q_f\}$
$\boxed{q_f}$	$\varnothing$	$\varnothing$	$\varnothing$

δ_2	Вход		
	a	b	c
$\to p_0$	$\varnothing$	$\{p_0\}$	$\{p_f\}$
$\boxed{p_f}$	$\{p_f\}$	$\varnothing$	$\varnothing$

Построим новую таблицу функции переходов для автомата, распознающего пересечение исходных языков:

δ	Вход		
	a	b	c
$\to (q_0, p_0)$	$\{(q_0, \varnothing)\}$	$\{(q_1, p_0)\}$	$\{(\varnothing, p_1)\}$
(q_1, p_0)	$\{(\varnothing, \varnothing)\}$	$\{(\varnothing, p_0)\}$	$\{(q_f, p_f)\}$
$\boxed{(q_f, p_f)}$	$\{(\varnothing, p_1)\}$	$(\varnothing, \varnothing)$	$(\varnothing, \varnothing)$

Таким образом

$$M = (\{(q_0, p_0); (q_1, p_0); (q_f, p_f)\}, \Sigma, \delta, (q_0, p_0), \{(q_f, p_f)\}.$$

Язык $L(M) = \{bc\}$. ■

Рассмотрим язык $L = L_1 \setminus L_2$ (пункт 7). Поскольку L_1 и L_2 — множества (регулярные), то теоретико-множественная операция разности может быть выражена через другие операции следующим образом: $L = L_1 \setminus L_2 = L_1 \cap \bar{L}_2$. В пунктах $5, 6$ показано, как строить автоматы для операций дополнения и пересечения, так что построение автомата для разности двух регулярных выражений заключается в построении автомата, распознающего дополнение к языку L_2, и построении автомата, распознающего пересечение между L_1 и дополнением к L_2.

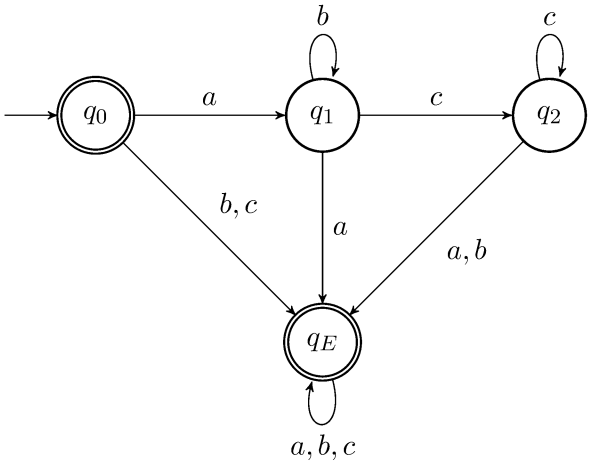


Рис. 5.6. ДКА из примера 5.1.4, распознающий язык $\bar{L}_2$

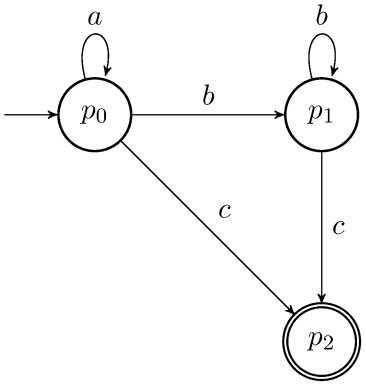


Рис. 5.7. ДКА из примера 5.1.4, распознающий язык L_1

Пример 5.1.4. Пусть $L_1 = a^*b^*c$, $L_2 = ab^*c^*$. Построим конечный автомат $M_{\bar{L}_2}$, такой что $L(M_{\bar{L}_2}) = \bar{L}_2$. Для этого необходимо построить конечный автомат, распознающий язык L_2, доопределить его, если это необходимо, и инвертировать

состояния. Диаграмма автомата, распознающего язык $\bar{L}_2$, приведена на рисунке 5.6.

Построим автомат, распознающий язык L_1. Поскольку он будет использоваться для построения автомата, распознающего пересечение языков, этот автомат доопределять не будем. Диаграмма автомата, распознающего язык L_2, приведена на рисунке 5.7.

Для построения автомата, распознающего $L_1 \cap \bar{L}_2$, воспользуемся таблицами δ-функций переходов автоматов для языка L_1 и $\bar{L}_2$:

δ_{L_1}	Вход		
	a	b	c
$\to p_0$	$\{p_0\}$	$\{p_1\}$	$\{p_2\}$
p_1	$\varnothing$	$\{p_1\}$	$\{p_2\}$
$\boxed{p_2}$	$\varnothing$	$\varnothing$	$\varnothing$

$\delta_{\bar{L}_2}$	Вход		
	a	b	c
$\to q_0$	q_1	$\{q_E\}$	$\{q_E\}$
q_1	q_E	$\{q_1\}$	$\{q_2\}$
q_2	q_E	$\{q_E\}$	$\{q_2\}$
$\boxed{p_E}$	$\{p_E\}$	$\{p_E\}$	$\{p_E\}$

На основе построенных таблиц сформируем новую таблицу функции переходов для автомата, распознающего язык $L_1 \cap \bar{L}_2$:

$\delta_{L_1 \cap \bar{L}_2}$	Вход		
	a	b	c
$\to A = (q_0, p_0)$	$\{(q_1, p_1)\}$	$\{(q_E, p_2)\}$	$\{(q_E, p_3)\}$
$B = (q_1, p_1)$	$\{(q_E, p_1)\}$	$\{(q_1, p_2)\}$	$\{(q_2, p_3)\}$
$C = (q_E, p_2)$	$\{(q_E, \varnothing)\}$	$\{(q_E, p_2)\}$	$\{(q_E, p_3)\}$
$\boxed{D = (q_E, p_3)}$	$\{(q_E, \varnothing)\}$	$\{(q_E, \varnothing)\}$	$\{(q_E, \varnothing)\}$
$E = (q_E, p_1)$	$\{(q_E, p_1)\}$	$\{(q_E, p_2)\}$	$\{(q_E, p_3)\}$
$F = (q_1, p_2)$	$\{(q_E, \varnothing)\}$	$\{(q_1, p_2)\}$	$\{(q_2, p_3)\}$
$G = (q_2, p_3)$	$\{(q_E, \varnothing)\}$	$\{(q_E, \varnothing)\}$	$\{(q_E, \varnothing)\}$

Диаграмма автомата M, распознающего $L_1 \cap \bar{L}_2$, представлена на рисунке 5.8.

Вычислив язык автомата M методом исключения состояний, можно убедиться, что $L(M) = c + bc + aaa^*(c + bc) + (ab + bb + aaa^*bb)(bb)^*bc$. ■

Упражнение 5.1.2. Пусть L_1, L_2 — регулярные языки над алфавитом Σ. Покажите, что языки $H(L_1)$ и $H^{-1}(L_1)$ являются регулярными. □

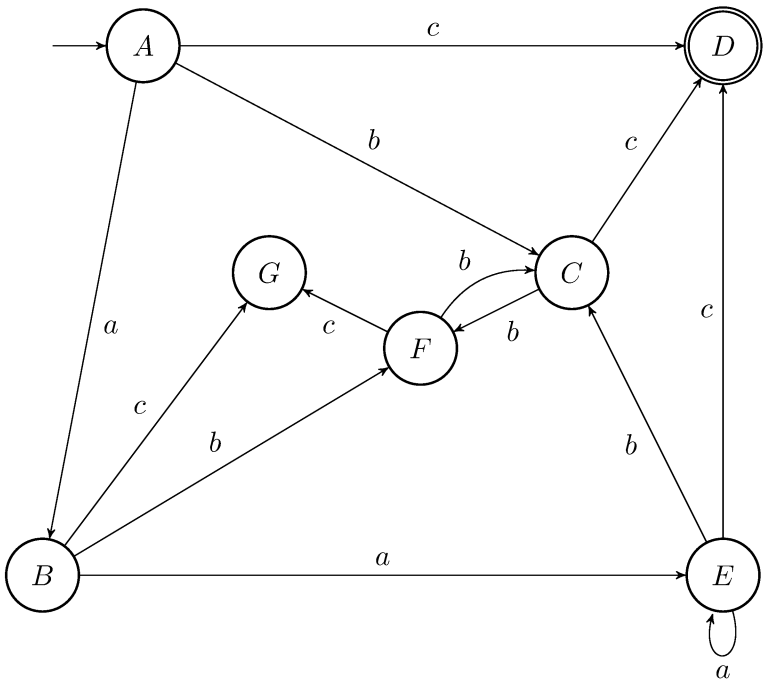


Рис. 5.8. ДКА из примера 5.1.4, распознающий язык $L_1 \cap \bar{L}_2$

5.2. Замкнутость регулярных множеств относительно базисных операций

Рассматривая свойства регулярных языков, мы получили инструкции построения конечных автоматов, распознающих языки — результаты операций над регулярными множествами. Регулярные множества обладают еще одним важным свойством, которое позволяет определять произвольные операции над языками и строить конечные автоматы для их распознавания.

Напомним, что множество A называется замкнутым относительно n-местной операции Ψ, если $\Psi(a_1, a_2, \ldots a_n) \in A \forall a_i \in A, 1 \leqslant i \leqslant n$. Так, например, множество натуральных чисел замкнуто относительно операции сложения.

Определение: Класс множеств называется булевой алгеброй множеств, если он замкнут относительно дополнения, объединения и пересечения.

Теорема 5.2.1. *Пусть Σ — произвольный (необязательно конечный) алфавит. Класс регулярных множеств, содержащихся в Σ^*, является булевой алгеброй множеств.* □

Доказательство. Замкнутость относительно объединения уже доказывалась ранее. Замкнутость относительно пересечения следует из теоретико-множественного закона $\overline{\overline{A \cap B}} = \overline{\bar{A} \cup \bar{B}}$.

Докажем замкнутость относительно дополнения. Пусть $M = (Q, \Delta, \delta, q_0, F)$ — конечный автомат, у которого $\Delta \subseteq \Sigma$, Δ — конечный алфавит. Легко показать, что каждое регулярное множество $L \subseteq \Sigma^*$ допускается некоторым таким автоматом. Тогда конечный автомат $M' = (Q, \Delta, \delta, q_0, Q \setminus F)$ допускает $\Delta^* \setminus L(M)$. При этом автомат M должен быть полностью определён.

Дополнение $\overline{L(M)}$ относительно Σ^* можно представить в виде

$$\overline{L(M)} = L(M') \cup \Sigma^*(\Sigma \setminus \Delta)\Sigma^*$$

Так как множество $\Sigma^*(\Sigma \setminus \Delta)\Sigma^*$ регулярно, то регулярность множества $L(M)$ следует из замкнутости регулярных множеств относительно объединения. ■

Из свойства замкнутости регулярных множеств относительно базисных операций дополнения, объединения и пересечения следует, в частности, что произвольные конечные комбинации этих операций также являются регулярными множествами, и через базисные операции можно вывести любые другие операции над регулярными множествами.

5.3. Алгоритмические проблемы регулярных языков

Регулярный язык может быть задан конечным образом с помощью трех типов описаний: праволинейной грамматики, регулярного выражения или конечного автомата. Имея одно описание, можно перейти к любому другому с помощью известных алгоритмов.

Для конечных описаний регулярных множеств естественным образом возникают алгоритмические проблемы. Рассмотрим три из них.

1) *Проблема принадлежности:* Дано конечное описание языка и цепочка w. Принадлежит ли цепочка w этому языку?
2) *Проблема пустоты:* Дано конечное описание языка. Пуст ли этот язык?
3) *Проблема эквивалентности:* Даны два конечных описания языка одного типа. Задают ли эти описания один и тот же язык?

Поскольку для каждого типа описания можно построить эквивалентное ему описание другого типа, то достаточно рассмотреть решение этих проблем только для одного типа описания. Будем рассматривать решения обозначенных алгоритмических проблем для конечных автоматов.

Проблема принадлежности для конечных автоматов решается с помощью следующего алгоритма. На вход алгоритм получает конечный автомат $M = (Q, \Sigma, \delta, q_0, F)$ и цепочку $w \in \Sigma^*$. На выходе алгоритм возвращает ответ «ДА», если $w \in L(M)$, в противном случае возвращается ответ «НЕТ». Принцип работы алгоритма следующий:
Пусть $w = a_1 a_2 \dots a_n$.
Найти последовательно состояния $q_1 = \delta(q_0, a_1), q_2 = \delta(q_1, a_2), \dots, q_n = \delta(q_{n-1}, a_n)$. Если $q_n \in F$, вернуть «ДА». В противном случае вернуть «НЕТ». Иными словами, задача алгоритма — определить достижимость любого финального состояния из начального по заданной цепочке.

Шаги алгоритма:

1) Представить $w = a_1 a_2 \dots a_n$. Если $n = 0$, перейти к Шагу 5.
2) Положить $i = 0$.
3) $q_{i+1} = \delta(q_i, a_{i+1})$.
4) Если $i \neq n$, $i = i + 1$ и перейти к Шагу 2.
5) Если $q_i \in F$, вернуть «ДА», иначе вернуть «НЕТ».

Проблема пустоты языка конечного автомата $M = (Q, \Sigma, \delta, q_0, F)$ решается путём определения множества состояний, достижимых из q_0. Если хотя бы одно финальное состояние принадлежит этому множеству, то алгоритм возвращает ответ «ДА», иначе — «НЕТ».

Шаги алгоритма:

1) Положить $Q_0 = q_0, i = 1$.
2) $Q_i = p \in Q \vdash \forall a \in \Sigma \delta(q, a) = p$, $q \in Q_{i-1}$,.
3) Если $Q_i = Q_{i-1}$, $i = i + 1$ и перейти на Шаг 2.
4) Если $Q_i \cap F \neq \varnothing$, вернуть «ДА», иначе вернуть «НЕТ».

Проблема эквивалентности для конечных автоматов может быть решена двумя способами. Первый способ — это проверка симметрической разности двух исходных языков на непустоту.

Пусть $M_1 = (Q_1, \Sigma_1, \delta_1, q_1, F_1)$ и $M_2 = (Q_2, \Sigma_2, \delta_2, q_2, F_2)$ конечные автоматы, такие что $L_1 = L(M_1)$ и $L_2 = L(M_2)$. Симметрическая разность двух языков может быть построена через теоретико-логические операции:

$$L_1 \triangle L_2 = (L_1 \setminus L_2) \cup (L_2 \setminus L_1) = (L_1 \cap \bar{L}_2) \cup (L_2 \cap \bar{L}_1).$$

В разделе 5.1 показано, как построить конечные автоматы для результатов операций над регулярными языками. Далее с помощью алгоритма, решающего проблему пустоты языка, можно проверить, пуста ли полученная симметрическая разность. Если множество симметрической разности двух языков пусто, значит языки совпадают между собой. Следовательно можно сделать вывод, что конечные автоматы, распознающие эти языки, эквивалентны.

Второй способ установления эквивалентности двух конечных автоматов основан на поиске неразличимых состояний в объединённом множестве состояний исходных автоматов. Изложим схему алгоритма установления эквивалентности двух автоматов.

Пусть $M_1 = (Q_1, \Sigma_1, \delta_1, q_1, F_1)$ и $M_2 = (Q_2, \Sigma_2, \delta_2, q_2, F_2)$ конечные автоматы, такие что $L_1 = L(M_1)$ и $L_2 = L(M_2)$. Построим конечный автомат $M_U = (Q_1 \cup Q_2, \Sigma_1 \cup \Sigma_2, q_1, F_1 \cup F_2)$. Поскольку автомат M_U конструируется только для анализа, выбор начального состояния не имеет значения. Далее на множестве $Q_1 \cup Q_2$ выделяются классы неразличимости состояний. Если начальные состояния исходных автоматов находятся в оном классе эквивалентности, то алгоритм возвращает ответ «ДА», иначе — «НЕТ».

5.4. Упражнения

Свойства регулярных языков

Построить регулярные выражения для множеств $L_1 \setminus L_2$, L_1^R, $L_1 \cup L_2$, $L_1 \triangle L_2$, если языки L_1 и L_2 заданы следующими регулярными выражениями:

1) $L_1 = a^*b(a+b)^*$ и $L_2 = a^*b(a+b)^*$
2) $L_1 = (00+11)^*10(1+0)^*$ и $L_2 = (00)^*(11)^*0^*1^*$

Доказательство нерегулярности языков

Пользуясь свойством замкнутости класса регулярных языков, выяснить, какие из следующих языков регулярны:

1) $\{0^n1^m \mid n \neq m\}$ (подсказка: воспользуйтесь нерегулярностью $\{0^n1^n\}$);
2) $\{w \in \{0,1\}^* \mid |w|_0 = |w|_1\}$;
3) язык из слов $w \in \{0,\ldots,9\}^*$, которые являются десятичной записью чисел, делящихся на 2 или на 3, без лишних лидирующих нулей;

4) $\{0^n 1^m 2^{n-m} \mid n \geqslant m\}$;

5) $\{a^n b a^m b a^{n+m} \mid n, m \in \mathbb{N}\}$.

ГЛАВА 6

КОНТЕКСТНО-СВОБОДНЫЕ ЯЗЫКИ

6.1. Деревья выводов в КС-грамматиках

В грамматике может быть несколько выводов, эквивалентных в том смысле, что во всех них применяются одни и те же правила в одних и тех же местах, но в различном порядке, в случае КС-грамматик можно ввести удобное графическое представление класса эквивалентных выводов, называемое деревом вывода.

Далее мы будем использовать некоторые стандартные понятия теории графов: (ориентированное!) дерево, поддерево, корень, доминирование, помеченное дерево, упорядоченное дерево и т. п. Предполагается, что читателю эти понятия известны.

Сечением дерева D назовем такое множество C вершин дерева D, что выполняются следующие свойства:

1) никакие две вершины из C не лежат на одном пути из корня в дереве D;
2) ни одну вершину дерева D нельзя добавить к C, не нарушив первого свойства.

Множество вершин дерева D, состоящее из одного корня, является сечением; листья тоже образуют сечение; остальные сечения можно расположить как бы между этими крайними сечениями.

Помеченное упорядоченное дерево D называется *деревом вывода* в КС-грамматике $G(A) = (N, \Sigma, P, A)$, если выполнены следующие условия.

1) Корень дерева D помечен A.
2) Если корень дерева имеет единственного потомка, помеченного ε, то этот потомок образует дерево, состоящее из единственное вершины, и $A \to \varepsilon$ — продукция из множества P.

3) Если $D_1, \ldots, D_k$ — поддеревья, над которыми доминируют прямые потомки корня дерева, и корень дерева D_i помечен X_i, то $A \to X_1 \ldots X_k$ — продукция из множества P. Если при этом X_i — нетерминал, то дерево D должно быть деревом вывода в грамматике $G(X_i) = (N, \Sigma, P, X_i)$, а если X_i — терминал, то дерево D_i состоит из единственной вершины, помеченной X_i.

Далее будем рассматривать естественное упорядочение листьев упорядоченного дерева — «слева направо». Кроной дерева вывода назовем слово, которое получится, если выписать слева направо метки листьев. Определим крону ω сечения C дерева D как слово, которое получается конкатенацией (в порядке слева направо) меток вершин, образующих сечение C.

Лемма 6.1.1. *Пусть $S = \alpha_0 \Rightarrow \alpha_1 \Rightarrow \ldots \Rightarrow \alpha_n$ — вывод слова α_n в КС-грамматике $G = (N, \Sigma, P, S)$. Тогда в G можно построить дерево вывода D, для которого α_n — крона, а $\alpha_0, \alpha_1, \ldots, \alpha_{n-1}$ — набор крон некоторых сечений.* □

Доказательство. Построим такую последовательность деревьев выводов D_i, где $0 \leqslant i \leqslant n$, что — крона дерева D_i.

Пусть D_0 — дерево, состоящее из единственной вершины, помеченной начальным нетерминалом S.

Предположим теперь, что дерево D_i с нужными свойствами уже построено, и по нему построим дерево D_{i+1}. Допустим, что $\alpha_i = \beta_i A \gamma_i$ и после применения продукции $A \to X_1 X_2 \ldots X_k$ получается слово $\alpha_{i+1} = \beta_i X_1 X_2 \ldots X_k \gamma_i$. Дерево D_{i+1} построим при помощи D_i добавлением к листу, помеченному выделенным вхождением A (он является $(|\beta_i|+1)$-м символом кроны дерева D_i), k прямых потомков, которые помечаются $X_1, X_2, \ldots, X_k$. Ясно, что α_{i+1} — крона дерева D_{i+1}.

Итак, $D_n = D$ — искомое дерево вывода, а $\alpha_0, \alpha_1, \ldots, \alpha_{n-1}$ — набор крон сечений этого дерева. ■

Лемма 6.1.2. *Пусть $G = (N, \Sigma, P, S)$ — КС-грамматика, а D — дерево вывода в G с кроной α. Тогда $\alpha \in L(G)$.* □

Доказательство. Пусть $C_0, C_1, C_2, \ldots, C_n$ — такая последовательность сечений дерева D, что выполняются следующие условия:

1) C содержит только корень дерева D;
2) C_{i+1} для $0 \leqslant i < n$ получается из C_i заменой одной нетерминальной вершины ее прямыми потомками;

3) C_n — множество листьев дерева D.

Ясно, что хотя бы одна такая последовательность существует. Если α_i — крона сечения C_i, то $S = \alpha_0 \Rightarrow \alpha_1 \Rightarrow \ldots \Rightarrow \alpha_n = a$ — вывод слова α из S в G. Поэтому $\alpha \in L(G)$. ■

Непосредственным следствием лемм 6.1.1 и 6.1.2 является

Теорема 6.1.1. *Пусть $G = (N, \Sigma, P, S)$ — КС-грамматика. $S \Rightarrow^* \alpha$ тогда и только тогда, когда в G существует дерево вывода с кроной α.* □

По одному дереву вывода с кроной α можно построить разные выводы в грамматике, для которых $S \Rightarrow^* \alpha$; среди всех таких выводов два вызывают особый интерес. Именно, если в доказательстве леммы 6.1.2 сечение C_{i+1} получается из C_i заменой самой левой нетерминальной вершины в C_i ее прямыми потомками, то соответствующий вывод $S = \alpha_0, \alpha_1, \ldots, \alpha_n$ называется левым выводом слова α_n из α_0 в грамматике G. Правый вывод определяется аналогично. Заметим, что и левый, и правый выводы определяются по дереву однозначно.

Если $S = \alpha_0, \alpha_1, \ldots, \alpha_n = \omega$ — левый вывод терминального слова ω и α_i $(0 \leqslant i < n)$ имеет вид $x_i A_i \beta_i$, где $x_i \in \Sigma^*$, $A_i \in N$, $\beta_i \in (N \cup \Sigma)^*$, то каждое следующее слово α_{i+1} левого вывода получается из предыдущего слова α_i заменой самого левого нетерминала A_i правой частью некоторой продукции. В правом выводе заменяется самый правый нетерминал.

Упражнение 6.1.1. Рассмотрим грамматику G из примера 1.3.3: $G_0 = (\{E; T; F\},$
$\{a; +; *; (;)\}, P, E)$, где P состоит из продукций

$$E \to E + T \mid T,$$
$$T \to T * F \mid F,$$
$$F \to (E) \mid a.$$

Постройте такое дерево вывода в этой грамматике, кроной которого является слово $a + a$. Укажите левый и правый выводы. □

Слово α будем называть левовыводимым в грамматике G и писать $S \Rightarrow_l^* \alpha$, если существует левый вывод $S = \alpha_0, \alpha_1, \ldots, \alpha_n = \alpha$. Аналогично, слово α будем называть правовыводимым и писать $S \Rightarrow_r^* \alpha$, если существует правый вывод $S = \alpha_0, \alpha_1, \ldots, \alpha_n = \alpha$. Один шаг левого вывода обозначается через $\Rightarrow_l$, а шаг правого вывода — через $\Rightarrow_r$.

Если дан вывод $S \Rightarrow^* \alpha$ в КС-грамматике G, то не всегда можно найти единственное дерево вывода с кроной α. Причина этого заключается в том, что есть КС-грамматики, у которых может быть несколько различных деревьев выводов с одной и той же кроной.

Упражнение 6.1.2. Пусть $G = (\{S\}, \{a, b\}, \{S \to aSbS \mid bSaS \mid \varepsilon\}, S)$. Постройте разные деревья выводов в этой грамматике, у которых кроной является слово $abab$. □

КС-грамматику G называют *неоднозначной*, если существует хотя бы одно слово $\omega \in L(G)$, которое является кроной двух или более различных деревьев выводов в G. В противном случае КС-грамматика G называется однозначной. Неоднозначность КС-грамматики можно связать с существованием различных левых и (или) правых выводов.

Упражнение 6.1.3. Рассмотрим КС-грамматику G. Пусть $\omega \in L(G)$, докажите, что следующие утверждения эквивалентны:

1) ω — крона двух различных деревьев выводов в G;
2) ω имеет два различных левых вывода в G;
3) ω имеет два различных правых вывода в G. □

6.2. Проблема непустоты и устранение бесполезных символов

Грамматика может, вообще говоря, содержать бесполезные символы и продукции. Например, в грамматике $G = (\{S, A\}, \{a, b\}, P, S)$, где $P = \{S \to a; A \to b\}$, нетерминал A и терминал b не могут появиться ни в каком выводимом слове. Таким образом, и эти символы, и продукцию $(A \to b)$ можно устранить из грамматики G, не изменив языка $L(G)$.

Дадим точное определение. Назовем символ $X \in N \cup \Sigma$ бесполезным в КС-грамматике $G = (N, \Sigma, P, S)$, если в ней невозможен вывод вида $S \Rightarrow^* \omega X y \Rightarrow^* \omega x y$, где $\omega, x, y \in \Sigma^*$.

Ясно, что если язык $L(G)$ пуст, то все символы бесполезны. Построим алгоритм (см. алгоритм 6.2.1 ниже), выясняющий, может ли нетерминал порождать какие-нибудь терминальные слова. Далее будет доказано, что этот алгоритм годится для проверки на непустоту произвольных КС-языков. Таким образом, проблема непустоты для КС-языков разрешима.

Алгоритм 6.2.1 Проверка на непустоту

Вход КС-грамматика $G = (N, \Sigma, P, S)$.

Выход «ДА», если $L(G) \neq \varnothing$, «НЕТ», если $L(G) = \varnothing$.

Метод Рекурсивное построение расширяющейся последовательности специального вида множества N.

Шаг 1. Положить $N_0 = \varnothing$, $i = 1$.

Шаг 2. Положить $N_i = \{A \mid A \to \alpha \in P, \alpha \in (N_{i-1} \cup \Sigma)^*\} \cup N_{i-1}$.

Шаг 3. Если $N_i \neq N_{i-1}$, то положить $i = i + 1$ и перейти к шагу 2, в противном случае положить $N_\Sigma = N_l$.

Шаг 4. Если $S \in N_\Sigma$, выдать на печать «ДА», в противном случае — «НЕТ».

Так как $N_\Sigma \subseteq N$, то алгоритм 6.2.1 должен остановиться самое большее после $|N| + 1$ повторений шага 2.

Алгоритм 6.2.1 строит множество «стабилизации» N_Σ, соответствующее алфавиту Σ. Аналог этого множества можно определить и для произвольного подмножества Ω множества $\Sigma \cup \{\varepsilon\}$:

$$N_\Omega = \{A \mid A \in N, A \Rightarrow_G^* \alpha, \alpha \in \Omega^*\}.$$

Множества такого типа встречаются в различных алгоритмах, поэтому выделим ту часть алгоритма 6.2.1, которая позволяет построить N_Ω (см. алгоритм 6.2.2).

Алгоритм 6.2.2 Построение множества N_Ω

Вход КС-грамматика $G = (M, \Sigma, P, S)$, Ω — подмножество множества $\Sigma \cup \{\varepsilon\}$.

Выход Множество N_Ω.

Метод Рекурсивное построение расширяющейся последовательности подмножеств специального вида множества N.

Шаг 1. Положить $N_0 = \varnothing$, $i = 1$.

Шаг 2. Положить $N_i = \{A \mid A \to \alpha \in P, \alpha \in (N_{i-1} \cup \Omega)^*\} \cup N_{i-1}$.

Шаг 3. Если $N_i \neq N_{i-1}$, то положить $i = i + 1$ и перейти к шагу 2, в противном случае положить $N_\Omega = N_i$..

Рассмотрим КС-грамматику $G = (N, \Sigma, P, S)$ и займемся теперь обоснованием алгоритма 6.2.1.

Лемма 6.2.1. *Пусть* $i \in \{0; 1; 2; \ldots\}$. *Если* $A \in N_i$, *то* $A \Rightarrow_G^* \omega$ *для некоторого слова* $\omega = \Sigma^*$. □

Доказательство. Применим метод математической индукции по i.

Случай $i = 0$ не нуждается в доказательстве, так как $N_0 = \varnothing$.

Предположим, что утверждение верно для $i = k$, и докажем его для $i = k+1$. Рассмотрим $A \in N_{k+1}$. Если A принадлежит также и N_k,то $A \Rightarrow_G^* \omega$ для некоторого слова $\omega \in \Sigma^*$ в силу индуктивного предположения. Если же $A \in N_{k+1} \setminus N_k$, то существует такая продукция $A \to X_1 \ldots X_m$, в которой $X \in \Sigma \cup N_k$. Тогда для каждого X_j, можно найти такое слово ω_j, что $X_j \Rightarrow_G^* \omega_j$: если $X_j \in \Sigma$, то $\omega_j = X_j$, в противном случае существование ω_j следует из индуктивного предположения. Итак, $A \Rightarrow X_1 \ldots X_m \Rightarrow_G^* \omega_1 X_2 \ldots X_m \Rightarrow_G^* \ldots \Rightarrow_G^* \omega_1 \ldots \omega_m$. (Подчеркнем, что случай $m = 0$ на первом шаге, то есть продукция $A \to \varepsilon$, не составляет исключения.)

Таким образом, лемма верна для произвольного i. ■

Лемма 6.2.2. *Пусть* $n \in \{1; 2; \ldots\}$. *Если* $A \Rightarrow_G^n \omega$ *для некоторого слова* $\omega \in \Sigma^*$, *то существует такое* $i \in \{0; 1; 2; \ldots\}$, *что* $A \in N_i$. □

Доказательство. Применим метод математической индукции по n.

В случае $n = 1$, очевидно, $i = 1$.

Допустим, что утверждение верно для $n = k$, и докажем его для $n = k + 1$. Пусть $A \Rightarrow_G^{k+1} \omega$. Тогда $A \Rightarrow X_1 \ldots X_m \Rightarrow_G^k \omega$, где слово $\omega = \omega_1 \ldots \omega_m$ таково, что $X_j \Rightarrow_G^{n_j} \omega$ для каждого j и $n_j \leqslant k$ (в дереве вывода $A \Rightarrow_G^{k+1} \omega$ слово ω_j является кроной поддерева с корнем X_j). В силу индуктивного предположения, если $X_j \in N$, то $X_j \in N_{i_j}$ для некоторого i_j, а если $X_j \in \Sigma$, то определим $i_j = 0$. Пусть $i = 1 + max(i_1, \ldots, i_k)$. Тогда $A \in N_i$.

Итак, лемма верна для произвольного n. ■

Теорема 6.2.1. *Алгоритм 6.2.1 говорит «Да» тогда и только тогда, когда* $S \Rightarrow_G^* \omega$ *для некоторого слова* ω *из* Σ^*. □

Доказательство. Согласно алгоритму 6.2.1 «Да» выводится тогда и только тогда, когда $S \in N_\Sigma$. Для завершения доказательства теоремы теперь достаточно воспользоваться леммами 6.2.1 и 6.2.2 при $A = S$, так как $N_\Sigma = \bigcup\limits_i N_i$. ■

Символ $X \in N \cup \Sigma$ назовем *недостижимым* в КС-грамматике $G = (N, \Sigma, P, S)$, если X не появляется ни в одной выводимой цепочке. Иначе говоря, символ $X \in N \cup \Sigma$ достижим, если для него существуют такие слова α, β из $(N \cup \Sigma)^*$, что

$S \Rightarrow^*_G \alpha X \beta$. Недостижимые символы являются примерами бесполезных символов; их можно устранить из КС-грамматики с помощью алгоритма 6.2.3.

Алгоритм 6.2.3 Устранение недостижимых символов

Вход КС-грамматика $G = (N, \Sigma, P, S)$.

Выход КС-грамматика $G' = (N, \Sigma, P, S)$, у которой $L(G') = L(G)$ и нет недостижимых символов.

Метод Рекурсивное построение расширяющейся последовательности подмножеств специального вида множества $N \cup \Sigma$.

Шаг 1. Положить $V_0 = \{S\}$, $i = 1$.

Шаг 2. Положить $V_i = \{X \mid (A \to \alpha X \beta) \in P, A \in V_{i-1}\} \cup V_{i-1}$.

Шаг 3. Если $V_i \neq V_{i-1}$, то положить $i = i + 1$ перейти к шагу 2, в противном случае положить $V^S = V_i$.

Шаг 4. Построить грамматику $G' = (N, \Sigma, P, S)$, где $N' = V^S \cap N$, $\Sigma' = V^S \cap \Sigma$, а в P' включены те и только те продукции из P. которые содержат только символы из V^S.

Так как $V^S \subseteq N \cup \Sigma$, то алгоритм 6.2.3 должен остановиться самое большее после $|N| + |\Sigma| + 1$ повторений шага 2. Алгоритмы 6.2.1 и 6.2.3 очень похожи; более того, обосновываются они тоже сходным образом.

Упражнение 6.2.1. Используя метод математической индукции по i, докажите, что существует вывод $S \Rightarrow^i_{G'} \alpha X \beta$ тогда и только тогда, когда $X \in V_i$ □

Упражнение 6.2.2. Докажите, что алгоритм 6.2.3 по КС-грамматике $G = (N, \Sigma, P, S)$ строит такую КС-грамматику $G' = (N', \Sigma', P', S)$, у которой $L(G') = L(G)$ и для всех $X \in N' \cup \Sigma'$ существуют такие слова α, β из $(N' \cup \Sigma')^*$, что $S \Rightarrow^*_{G'} \alpha X \beta$. (Другими словами, алгоритм 6.2.3 строит новую КС-грамматику $G' = (N', \Sigma', P', S)$ без недостижимых символов, для которой $L(G') = L(G)$.) □

Дадим пояснения к алгоритму 6.2.4 устранения бесполезных символов. На шаге 1 из G устраняются все нетерминалы, которые не могут порождать терминальных слов. Затем на шаге 2 устраняются все недостижимые символы. Каждый символ X результирующей грамматики должен появиться хотя бы в одном выводе вида $S \Rightarrow^* \omega X y \Rightarrow^* \omega x y$. В примере 6.2.1 показано, что если сначала применить алгоритм 6.2.3, а потом алгоритм 6.2.2, то в результате может получиться грамматика, содержащая бесполезные символы.

Алгоритм 6.2.4 Устранение бесполезных символов

Вход КС-грамматика $G = (N, \Sigma, P, S)$, у которой $L(G) \neq \varnothing$.

Выход КС-грамматика $G' = (N', \Sigma', P', S)$. У которой $L(G') = L(G)$ и в $N' \cup \Sigma'$ нет бесполезных символов.

Метод Последовательное применение алгоритма 6.2.2 для $\Omega = \Sigma$ и алгоритма 6.2.3.

Шаг 1. К грамматике $G = (N, \Sigma, P, S)$ применить алгоритм 6.2.2 и найти множество N_Σ; построить грамматику $G_1 = (N \cap N_\Sigma, \Sigma, P_1, S)$, где в P_1 включены те и только те продукции из P, которые содержат только символы из $N \cap N_\Sigma$.

Шаг 2. К грамматике $G_1 = (N \cap N_\Sigma, \Sigma, P_1, S)$ применить алгоритм 6.2.3 и построить грамматику $G' = (N', \Sigma', P', S)$.

Теорема 6.2.2. *Грамматика G', которую строит алгоритм 6.2.4, является КС-грамматикой без бесполезных символов, и $L(G) = L(G')$.* □

Доказательство. При переходе от грамматики G к грамматике G_1 исключаются только те нетерминалы и продукции, которые не участвуют в выводах терминальных слов, поэтому $L(G) = L(G_1)$. Равенство $L(G_1) = L(G')$ обсуждалось выше (см. упражнение 6.2.2). Следовательно, $L(G) = L(G')$.

Покажем теперь, что в G' нет бесполезных символов. Предположим, что $A \in N'$ — бесполезный символ. Тогда по определению бесполезности символа могут представиться два случая:

1) Вывод $S \Rightarrow^*_{G'} \alpha A \beta$ ни для каких α и β невозможен. Но в этом случае символ A должен был быть устраненным на шаге 2 алгоритма 6.2.4, что приводит к противоречию.

2) Вывод $S \Rightarrow^*_{G'} \alpha A \beta$ для некоторых α и β возможен, но вывода $A \Rightarrow^*_{G'} \omega$ для $\omega \in \Sigma'^*$ не существует. Ясно, что если символ A «проскочил» шаг 1, то на шаге 2 в рассматриваемой ситуации его уже не устранить. Кроме того, если в этом случае $A \Rightarrow^*_G \gamma B \delta$, то и символ B нельзя устранить на шаге 2. Итак, предположим, что символ A не устранен на первом шаге, то есть $A \Rightarrow^*_G \omega$ для $\omega \in \Sigma^*$. Если возможен вывод $A \Rightarrow^*_G \gamma B \delta \Rightarrow^*_G \omega$, то в силу сказанного выше символ B нельзя устранить на шаге 2, это означает, что $A \Rightarrow^*_{G'} \omega$. Полученное противоречие показывает, что в действительности A устраняется на шаге 1.

Доказательство того, что ни один терминал в G' не может быть бесполезным, проводится аналогично. ■

Пример 6.2.1. Рассмотрим грамматику $G = (\{S; A; B\}, \{a : b\}, P, S)$, где P состоит из продукций

$$S \to a \mid A; \quad A \to AB; \quad B \to b.$$

Применим к G алгоритм 6.2.4. На шаге 1 этого алгоритма получим: $N_\varepsilon\{S; B\}$ и $G = (\{S; B\}, \{a; b\}, \{S \to a; B \to b\}, S)$. На втором шаге, применив алгоритм 6.2.3, получим: $V_2 = V_1 = \{S, a\}$. Итак, $G' = (\{S\}, \{a\}, \{S \to a\}, S)$.

Теперь в алгоритме 6.2.4 поменяем местами шаги 1 и 2. После применения к G алгоритма 6.2.3 грамматика не изменится в силу того, что все символы достижимы. Последующее применение алгоритма 6.2.2 дает $N_\varepsilon = \{S; B\}$. Следовательно, результирующей будет грамматика G , отличная от G' . ■

6.3. Построение приведенной КС-грамматики

В пункте 6.2 было показано, как, не меняя языка, устранить из КС-грамматики все бесполезные символы. Целью этого пункта является устранение из КС-грамматики «нехороших» продукций.

Назовем КС-грамматику $G = (N, \Sigma, P, S)$ *неукорачивающейся грамматикой*, если либо P вовсе не содержит ε-продукции типа $A \to \varepsilon$, либо в P есть точно одна ε-продукция $S \to \varepsilon$ и S при этом не встречается в правых частях остальных продукций.

Алгоритм 6.3.1 Преобразование КС-грамматики в неукорачивающуюся грамматику

Вход КС-грамматика $G = (N, \Sigma, P, S)$.

Выход Неукорачивающаяся КС-грамматика $G' = (N', \Sigma, P', S')$, у которой $L(G') = L(G)$.

Метод «Устранение перегородок».

Шаг 1. Применить алгоритм 6.2.2 для $\Omega = \{\varepsilon\}$ и построить множество

$$N_\varepsilon = \{A \mid A \in N, A \Rightarrow_G^* \varepsilon\}.$$

Шаг 2. Если

$$(A \to \alpha_0 B_1 \alpha_1 B_2 \alpha_2 \dots B_k \alpha_k) \in P,$$

где $k \geqslant 0$, $B_i \in N_\varepsilon$ и ни один символ в словах α_j не принадлежит N_ε, то включить в P' все продукции вида $A \to \alpha_0 X_1 \alpha_1 X 2 \dots \alpha_{k-1} X_k \alpha_k$, где X_i — либо B_i, либо ε, но не включать продукцию $A \to \varepsilon$ (это могло бы произойти в случае, если все α_i, равны ε).

Шаг 3. Если $S \in N_\varepsilon$, то ввести новый нетерминал S' и дополнительно включить в P' продукции $(S' \to S|\varepsilon)$, в противном случае положить $N' = N, S' = S$.

Шаг 4. Положить $G' = (N', \Sigma, P', S')$.

Теорема 6.3.1. *Грамматика G', которую строит алгоритм 6.3.1 по КС-грамматике G, является неукорачивающейся КС-грамматикой, и $L(G) = L(G')$.* □

Доказательство. Тот факт, что грамматика G' является неукорачивающейся, вытекает из простого анализа алгоритма.

Применяя метод математической индукции по длине слова ω, можно доказать следующее вспомогательное утверждение: для произвольного слова ω из $\Sigma^* \setminus \{\varepsilon\}$ и буквы A из N вывод $A \Rightarrow_{G'}^*$ возможен тогда и только тогда, когда $A \Rightarrow_G^* \omega$. Применим это утверждение для $S = A$ и $\omega \in \Sigma^* \setminus \{\varepsilon\}$ получим: $\omega \in L(G)$ тогда и только тогда, когда $\omega \in L(G')$. Заметим теперь, что $\varepsilon \in L(G)$ тогда и только тогда, когда $\varepsilon \in L(G')$. Таким образом, $L(G) = L(G')$. ■

Упражнение 6.3.1. Докажите, что если на вход алгоритма 6.3.1 подается КС-грамматика без бесполезных символов, то и на выходе алгоритма получается такая же грамматика. □

Пример 6.3.1. Рассмотрим контекстно-свободную грамматику

$$G = (\{S; A; B\}, \{0; 1\}, P, S),$$

где P состоит из продукций

$$S \to 0A \mid 1B \mid \varepsilon,$$
$$A \to AB \mid 0 \mid \varepsilon,$$
$$B \to 0 \mid A.$$

Применим к грамматике G алгоритм 6.3.1 и получим неукорачивающуюся КС-грамматику $G' = (\{S'; S; A; B\}, \{0; 1\}, P, S)$, где P' состоит из продукций

$$S' \to S \mid \varepsilon,$$
$$S \to 0A \mid 0 \mid 1B \mid 1,$$
$$A \to AB \mid A \mid B \mid 0,$$
$$B \to 0 \mid A.$$ ■

Другое полезное преобразование грамматик — устранение продукций вида $A \to B$, где A и B — нетерминалы: такие продукции далее будем называть *цепными*.

Алгоритм 6.3.2 Устранение цепных продукций

Вход Неукорачивающаяся КС-грамматика $G = (N, \Sigma, P, S)$.

Выход Неукорачивающаяся КС-грамматика $G' = (N', \Sigma', P', S')$ без цепных продукций, у которой $L(G') = L(G)$.

Метод Для каждого A из N строится подмножество $N^A = \{B \mid A \Rightarrow^* B)\}$ множества N, и на основе этого конструируется новая грамматика.

Шаг 1. Для каждого A из N построить $N^A = \{B \mid A \Rightarrow^* B\}$ следующим образом.

1) Положить $N_0 = \{A\}$ и $i = 1$.
2) Положить $N_i| = \{C \mid (B \to C) \in P$ и $B \in N_{i-1}\} \cup N_{i-1}$.
3) Если $N_i \neq N_{i-1}$, то положить $i = i + 1$ и повторить шаг 1.2, в противном случае положить $N^A = N_i$.

Шаг 2. Построить P' так: если продукция $B \to \alpha$ принадлежит P и не является цепной, то включить в P' продукцию $(A \to \alpha)$ для всех таких A, что $B \in N^A$.

Шаг 3. Положить $G' = (N, \Sigma, P', S)$.

Теорема 6.3.2. *Грамматика G', которую строит алгоритм 6.3.2 по неукорачивающейся КС-грамматике G, является неукорачивающейся КС-грамматикой без цепных продукций и $L(G) = L(G')$.* □

Доказательство. Тот факт, что грамматика G' является неукорачивающейся и не имеет цепных продукций, вытекает из простого анализа алгоритма.

Покажем, что $L(G') \subseteq L(G)$. Пусть $\omega \in L(G')$. Тогда в грамматике G' существует вывод $S \Rightarrow^1 \alpha_0 \Rightarrow^1 \alpha_1 \Rightarrow^1 \ldots \Rightarrow^1 \alpha_n = \omega$. Если при переходе от некоторого α_i к α_{i+1} применяется продукция $A \to \beta$ из P', то тогда существует такой символ $B \in N$ (не исключено, что $B = A$), что $A \Rightarrow_G^* B \Rightarrow_G \beta$. Таким образом, $A \Rightarrow_G^* \beta$ и, следовательно, $\alpha_i \Rightarrow_G^* \alpha_{i+1}$. Отсюда следует, что $S \Rightarrow_G^* \omega \in L(G)$, так что $L(G') \subseteq L(G)$.

Теперь покажем, что $L(G) \subseteq L(G')$; причем далее мы будем пользоваться понятиями, которые обсуждались в конце раздела 6.1. Пусть $\omega \in L(G)$ и $S = \alpha_0 \Rightarrow_l \alpha_1 \Rightarrow_l \ldots \Rightarrow_l \alpha_n = \omega$ — левый вывод слова ω в грамматике G. Рассмотрим подпоследовательность $i_1, i_2, \ldots, i_k$ последовательности $1, 2, \ldots, n$, состоящую в точности из тех номеров j, для которых на шаге $\alpha j_{j-1} \Rightarrow_l \alpha_j$ вывода $S = \alpha_0 \Rightarrow_l \alpha_1 \Rightarrow_l \ldots \Rightarrow_l \alpha_n = \omega$ применяется нецепная продукция. В частности, $i_k = n$, так как вывод терминального слова не может оканчиваться цепной продукцией. Так как мы рассматриваем левый вывод, то последовательные применения нескольких исключительно цепных продукций заменяют нетерминальный символ, занимающий одну и ту же позицию в левовыводимых словах. Используя конструкцию P, отсюда получаем:

$$S \Rightarrow_{G'} \alpha_{i_1} \Rightarrow_{G'} \alpha_{i_2} \Rightarrow_{G'} \ldots \Rightarrow_{G'} \alpha_{i_k} = \omega.$$

Таким образом, $\omega \in L(G')$ и, следовательно, $L(G) \subseteq L(G')$.

В итоге получаем: $L(G') = L(G)$. ■

С точки зрения анализа бесполезности символов алгоритм 6.3.2 является «плохим».

Пример 6.3.2. Рассмотрим неукорачивающуюся КС-грамматику $G = (\{S; A\}, \{a\}, \{S \to A; A \to a\}, S)$, у которой нет бесполезных символов. Применяя к ней алгоритм 6.3.2, получаем грамматику $G = (\{S; A\}, \{a\}, \{S \to a; A \to a\}, S)$ с бесполезным символом A. ■

Пример 6.3.3. Рассмотрим неукорачивающуюся КС-грамматику

$$G' = (\{W; S; A; B\}, \{0; 1\}, P', W).$$

где P' состоит из продукций

$$W \to S \mid \varepsilon, \qquad S \to 0A \mid 0 \mid 1B \mid 1,$$
$$A \to AB \mid A \mid B \mid 0, \qquad B \to 0.$$

Применим к грамматике G алгоритм 6.3.2 и получим неукорачивающуюся КС-грамматику $G' = (\{W; S; A; B\}, \{0; 1\}, P', W)$, где P' состоит из продукций

$$W \to \varepsilon \mid 0A \mid 0 \mid 1B \mid 1, \qquad S \to 0A \mid 0 \mid 1B \mid 1,$$
$$A \to AB \mid 0, \qquad B \to 0,$$

среди которых нет цепных.

Легко обнаружить, что построенная грамматика G' имеет бесполезные символы. Применим к грамматике G' алгоритм 6.2.4и получим неукорачивающуюся КС-грамматику $G = (\{W; S; A; B\}, \{0; 1\}, P', W)$, где P' состоит из продукций

$$W \to \varepsilon \mid 0A \mid 0 \mid 1B \mid 1, \qquad A \to 0, \qquad B \to 0.$$

У нее нет ни цепных продукций, ни бесполезных символов. ■

Упражнение 6.3.2. Докажите, что если на вход алгоритма 6.2.4 подается неукорачивающаяся КС-грамматика без цепных продукций, то на выходе алгоритма получится неукорачивающаяся КС-грамматика без бесполезных символов и цепных продукций.

КС-грамматика $G = (H, \Sigma, P, S)$ называется грамматикой без циклов, если в ней нет выводов $A \Rightarrow^* A$ для $A \in N$. □

Упражнение 6.3.3. Докажите, что если неукорачивающаяся КС-грамматика $G = (N, \Sigma, P, S)$ не имеет цепных продукций, то в ней нет циклов. Существуют ли КС-грамматики с циклами, но без цепных продукций? □

КС-грамматика G называется *приведенной*, если она не имеет бесполезных символов, циклов и является неукорачивающейся.

Алгоритм 6.3.3 Преобразование произвольной КС-грамматики в приведенную

Вход КС-грамматика G.

Выход Приведенная КС-грамматика G', для которой $L(G') = L(G)$.

Метод Применение алгоритмов 6.2.4, 6.3.1, 6.3.2.

Шаг 1. Применить алгоритм 6.2.4 и по КС-грамматике G построить КС-грамматику без бесполезных символов G_1, для которой $L(G_1) = L(G)$.

Шаг 2. Применить алгоритм 6.3.1 и по КС-грамматике G_1 построить неукорачивающую КС-грамматику без бесполезных символов G_2, для которой $L(G_2) = L(G_1)$.

Шаг 3. Применить алгоритм 6.3.2 и по КС-грамматике G_2 построить неукорачивающую КС-грамматику без цепных продукций G_3, для которой $L(G_3) = L(G_2)$.

Шаг 4. Применить алгоритм 6.2.4 и по КС-грамматике G_3 построить искомую КС-грамматику G, для которой $L(G_3) = L(G')$.

Теорема 6.3.3. *Грамматика G', которую строит алгоритм 6.3.3 по произвольной КС-грамматике G, является приведенной КС-грамматикой и $L(G) = L(G')$.*□

Доказательство. Для доказательства этой теоремы достаточно последовательно воспользоваться теоремой 6.2.2, теоремой 6.3.1, упражнением 6.3.1, теоремой 6.3.2, теоремой 6.2.2 и упражнением 6.3.2. ■

6.4. Упражнения

Построение КС-грамматик

Построить КС-грамматики для следующих языков:

- $\{a^n b^n c^m d^m \mid n, m \in \mathbb{N}\}$;
- $\{a^i b^j c^j d^i \mid i, j \in \mathbb{N}\}$;
- $\{a^n b^n c^m d^m \mid n, m \in \mathbb{N}\} \cup \{a^i b^j c^j d^i \mid i, j \in \mathbb{N}\}$.

Неоднозначность в КС-грамматиках

1) Найдите, если это возможно, грамматику без неоднозначности для каждого языка из предыдущего упражнения.

2) Докажите или опровергните контрпримером, что если L_1 и L_2 это КС-языки, не обладающие неоднозначностью, то язык $L_1 \cup L_2$ тоже не обладает этим свойством.

Алгоритмы для КС-грамматик

Удалить бесполезные символы в грамматиках с продукциями:

(1)
$$\begin{aligned} &S \to 0 \mid A, \\ &A \to AB, \\ &B \to 1; \end{aligned}$$

(2)
$$\begin{aligned} &S \to AB \mid CA, \\ &A \to a, \\ &B \to BC \mid AB, \\ &C \to aB \mid \varepsilon. \end{aligned}$$

Преобразовать следующую грамматику к неукорачивающейся:

$$\begin{aligned} &S \to AB, \\ &A \to aAA \mid \varepsilon, \\ &B \to bBB \mid \varepsilon; \end{aligned}$$

Удалить цепные продукции из грамматики с продукциями:

$$\begin{aligned} &E \to T \mid E + T, \\ &T \to F \mid T * F, \\ &F \to I \mid (E), \\ &I \to a \mid b \mid Ia \mid Ib \mid I0 \mid I1; \end{aligned}$$

ГЛАВА 7

НОРМАЛЬНЫЕ ФОРМЫ КС-ГРАММАТИК

В предыдущем параграфе было показано, как, не меняя языка, устранить из КС-грамматики все бесполезные символы, циклы и сделать ее неукорачивающейся. В этом параграфе процесс упрощения формы записи грамматик для КС-языков будет продолжен. Мы рассмотрим две нормальные формы КС-грамматик, каждая из которых используется для решения определенной практической задачи. Эти задачи будут рассмотрены, соответственно, в конце этой и следующей главы.

7.1. Нормальная форма Хомского

Говорят, что КС-грамматика $G = (N, \Sigma, P, S)$ представлена в *нормальной форме Хомского*, если продукции из P имеют один из следующих видов:

1) $A \to BC$, где $A, B, C \in N$,

2) $A \to a$, где $A \in N$, $a \in \Sigma$,

3) $S \to \varepsilon$, если $\varepsilon \in L(G)$,

причем S не встречается в правых частях продукций.

Алгоритм 7.1.1 Преобразование КС-грамматики к нормальной форме Хомского

Вход КС-грамматика $G = (N, \Sigma, P, S)$.

Выход КС-грамматика G' в нормальной форме Хомского, у которой $L(G') = L(G)$.

Метод Предварительное приведение исходной грамматики и последовательное конструирование продукций новой грамматики.

Шаг 1. Ввести новый стартовый нетерминал S_1 и добавить к P одну продукцию $S_1 \to S$. Применить к модифицированной грамматике алгоритм 6.3.3 и построить тем самым приведенную КС-грамматику $G_1 = (N_1, \Sigma, P_1, S_1)$, в продукциях которой S_1 не встречается в правых частях. Перейти на следующий шаг, где начинается построение искомой грамматики $G' = (N', \Sigma, P', S_1)$.

Шаг 2. Включить в P' все продукции из P_1, вида $A \to a$, где $A \in N_1$, $a \in \Sigma$.

Шаг 3. Включить в P' все продукции из P_1 вида $A \to BC$, где $A, B, C \in N_1$.

Шаг 4. Включить в P' продукцию $S_1 \to \varepsilon$, если она была в P_1.

Шаг 5. Для каждой продукции из P вида $A \to X_1 \dots X_k$ включить в P' продукции

$$\begin{gathered} A \to X'_1\langle X_2 \dots X_k\rangle, \\ \langle X_2 \dots X_k\rangle \to X'_2\langle X_3 \dots X_k\rangle, \\ \dots \\ \langle X_{k-2}X_{k-1}X_k\rangle \to X'_{k-2}\langle X_{k-1}X_k\rangle, \\ \langle X_{k-1}X_k\rangle \to X'_{k-1}X'_k, \end{gathered}$$

где $X'_i = X_i$, если $X_i \in N$, и X'_i — новый нетерминал, если $X_i \in \Sigma$, а $\langle X_i \dots X_k\rangle$ — новый нетерминал в любом случае.

Шаг 6. Для каждой продукции из P вида $A \to X_1X_2$, где хотя бы один из символов X_1 и X_2 принадлежит Σ, включить в P' продукцию $A \to X'_1X'_2$, где X'_i определяется так же, как на шаге 5.

Шаг 7. Для каждого нетерминала вида a', введенного на шагах 5 и 6, включить в P' продукцию $a' \to a$.

Шаг 8. Положить $G' = (N', \Sigma, P', S_1)$.

Для обоснования алгоритма преобразования КС-грамматики к нормальной форме Хомского (см. алгоритм 7.1.1) нам понадобится вспомогательное утверждение, которое позволит без изменения языка удалять из грамматики

продукции вида $A \to \alpha$. Продукции вида $A \to \alpha$ будем называть *нетерминальными A-продукциями*.

Лемма 7.1.1. *Пусть $G = (N, \Sigma, P, S)$ — КС-грамматика и P содержит продукцию $A \to \alpha B \beta$, где $B \in N$, $\alpha, \beta \in (N \cup \Sigma)^*$. Рассмотрим все нетерминальные B-продукции этой грамматики*

$$B \to \gamma_1 \mid \gamma_2 \mid \ldots \mid \gamma_k|.$$

Пусть $G' = (N, \Sigma, P', S)$, где

$$P' = (P - \{A \to \alpha B \beta\}) \cup \{A \to \alpha\gamma_1\beta \mid \alpha\gamma_2\beta \mid \ldots \mid \alpha\gamma_k\beta\}.$$

Тогда $L(G) = L(G')$. □

Упражнение 7.1.1. Доказать лемму 7.1.1. □

Теорема 7.1.1. *Пусть G — произвольная КС-грамматика. Алгоритм 7.1.1 строит по G такую КС-грамматику G' в нормальной форме Хомского, что $L(G) = L(G')$.* □

Доказательство. Применим к КС-грамматике G алгоритм 7.1.1. В силу теоремы 6.3.3 на шаге 1 этого алгоритма строится приведенная КС-грамматика $G_1 = (N_1, \Sigma, P_1, S_1)$, в продукциях которой S_1 не встречается в правых частях и для которой $L(G) = L(G_1)$. Шаги 2-8 алгоритма 7.1.1 позволяют построить по G_1 грамматику G', которая, очевидно, имеет нормальную форму Хомского. Чтобы показать, что $L(G_1) = L(G')$, достаточно применить лемму 7.1.1 к каждой продукции грамматики G', в правую часть которой входит a', а затем применить эту лемму к продукциям с нетерминалами вида $\langle X_i \ldots X_j \rangle$. ■

Иногда вместо нормальной формы Хомского рассматривает другую, слегка измененную, каноническую форму. Будем говорить, что КС-грамматика $G = (N, \Sigma, P, S)$ представлена в *модифицированной нормальной форме Хомского*, если продукции из P имеют один из следующих видов:

1) $A \to BC$, где $A, B, C \in N$;
2) $A \to a$, где $A \in N$, $a \in \Sigma$;
3) $S \to \varepsilon$, если $\varepsilon \in L(G)$, причем в этом случае S не встречается в правых частях продукций.

Чтобы преобразовать произвольную КС-грамматику к модифицированной нормальной форме Хомского достаточно слегка подправить алгоритм 7.1.1.

Алгоритм 7.1.2 Преобразование КС-грамматики к модифицированной нормальной форме Хомского

Вход КС-грамматика $G = (N, \Sigma, P, S)$.

Выход КС-грамматика G' в модифицированной нормальной форме Хомского, у которой $L(G') = L(G)$.

Метод Модификация одного шага алгоритма 7.1.1.

Шаг 1. Применить к грамматике $G = (N, \Sigma, P, S)$ алгоритм 6.3.3 и построить тем самым приведенную КС-грамматику $G_1 = (N_1, \Sigma, P_1, S_1)$. Перейти на следующий шаг, где начинается построение искомой грамматики $G' = (N', \Sigma, P', S_1)$.

[*Шаги 2–8*] повторяют соответствующие шаги алгоритма 7.1.1.

Упражнение 7.1.2. Сформулировать и доказать аналог теоремы 7.1.1 для обоснования алгоритма 7.1.2. □

Пример 7.1.1. Преобразуем КС-грамматику

$$G = (\{A; B; S\}, \{a; b; c; d\}, P, S)$$

с продукциями

$$\begin{aligned} S &\to \alpha AB \mid BA, \\ A &\to BBB \mid a, \\ B &\to AS \mid b, \end{aligned}$$

к нормальной форме Хомского. Для этого применим к грамматике G алгоритм 7.1.1. В соответствии с первым шагом алгоритма введем новый стартовый нетерминал S_1 и добавим к P одну продукцию $S_1 \to S$. Применим теперь к полученной грамматике алгоритм 6.3.3 и построим приведенную КС-грамматику

$$G_1 = (\{A; B; S; S_1\}, \{a; b; c; d\}, P_1, S_1)$$

с продукциями

$$S_1 \to aAB \mid BA,$$
$$S \to aAB \mid BA,$$
$$A \to BBB \mid a,$$
$$B \to AS \mid b$$

(в продукциях стартовый нетерминал S_1 в правых частях не встречается). Начнём преобразовывать грамматику G_1 к грамматике G' в нормальной форме Хомского. Из P_1 в P' продукции $S_1 \to BA$, $S \to BA$, $A \to a$, $B \to AS$ и $B \to b$ переносим без изменения. Заменяем продукцию $S_1 \to aAB$ продукциями $S \to a'\langle AB\rangle$ и $\langle AB\rangle \to AB$, $S \to aAB$ — продукциями $S \to a'\langle AB\rangle$ и $\langle AB\rangle \to AB$, а $A \to BBB$ — продукциями $A \to B\langle BB\rangle$ и $\langle BB\rangle \to BB$. Наконец, добавляем к P' продукцию $a' \to a$. В результате получаем грамматику

$$G' = (\{A; B; S; S_1; \langle AB\rangle; \langle BB\rangle; a'\}, \{a, b\}, P', S_1),$$

с продукциями

$$S_1 \to a'\langle AB\rangle \mid BA, \qquad S \to a'\langle AB\rangle \mid BA,$$
$$A \to B\langle BB\rangle \mid a, \qquad B \to AS \mid b,$$
$$\langle AB\rangle \to AB, \qquad \langle BB\rangle \to BB,$$
$$a' \to a.$$

Эта грамматика имеет нормальную форму Хомского. По теореме 7.1.1

$$L(G) = L(G'),$$

и, следовательно, G'— искомая грамматика. ■

Упражнение 7.1.3. Преобразовать КС-грамматику из примера 7.1.1 к модифицированной нормальной форме Хомского. □

7.2. Проблема принадлежности для КС-языков

Нормальные формы, как правило, вводятся для удобства формулировки алгоритмов, решающих конкретные задачи. Нормальная форма Хомского позволяет, в частности, сформулировать алгоритм решения проблемы принадлежности для КС-языков, а именно, по данному слову и данному КС-языку, представленному в виде КС-грамматики его порождающей, определить, принадлежит ли это слово языку.

Алгоритм 7.2.1 Алгоритм Кока—Янгера—Касами («CYK-алгоритм»)

Вход КС-грамматика в нормальной форме Хомского $G = (N, \Sigma, P, S)$, слово $w \in \Sigma^*$ длины n.

Выход Истина, если $w \in L(G)$, и ложь в противном случае.

Метод Последовательное определение нетерминалов, выводящих всевозможные подстроки w всё большей длины.

Шаг 1. Для всех $k \in [1, n]_{\mathbb{N}}$ положить $N_{kk} = \{A \in N \mid A \to w_k \in P\}$. Таким образом учтены все подстроки w длины единица.

Шаг 2. Если $n = 1$, завершить алгоритм и вернуть результат проверки $S \in N_{11}$. Иначе положить длину рассматриваемых подстрок $s = 2$.

Шаг 3. Положить $i = 1$.

Шаг 4. Положить $j = i + s - 1$. Положить

$$N_{ij} = \{A \in N \mid A \to BC \in P;\ \exists k \in [i, j-1]_{\mathbb{N}} : B \in N_{ik},\ C \in N_{k+1\,j}\}.$$

Шаг 5. Увеличить i на единицу. Если $i < n - s$, перейти к шагу 4.

Шаг 6. Если $s = n$, завершить алгоритм и вернуть результат проверки $S \in N_{1n}$. Иначе увеличить s на единицу и перейти к шагу 3.

Для описания алгоритма 7.2.1 используется одно важное обозначение. Пусть задана КС-грамматика $G = (N, \Sigma, P, S)$ и слово $w \in \Sigma^*$. Для всех $1 \leqslant i \leqslant j \leqslant n$ определим множество

$$N_{ij}^w = \{A \in N \mid A \Rightarrow_G^* w_i \dots w_j\}.$$

То есть N_{ij}^w это множество нетерминалов, которые порождают подстроку $w_i \dots w_j$ строки w в грамматике G. Когда слово w фиксировано и ясно из контекста, будем опускать верхний индекс в обозначении N_{ij}^w.

Упражнение 7.2.1 (применение CYK-алгоритма к синтаксическому анализу). Постройте модификацию CYK-алгоритма, которая позволяет в случае $w \in L(G)$ давать на выходе вывод слова w в грамматике G. □

Замечание (о структуре CYK-алгоритма). Заметим, что вычисление множеств N_{ij} в алгоритме 7.2.1 происходит таким образом, что на более поздних итерациях (для больших значений $j - i$) используются результаты более ранних. Такой подход к проектированию алгоритмов, когда решение задачи опре-

деляется через решение нескольких таких же задач, но меньшего размера, называется *динамическим программированием*. Отличие от более простого подхода *разделяй и властвуй*, обычно основанного на прямой рекурсии, состоит в том, что подзадачи могут перекрываться. Более точно, результат решения одной подзадачи может использоваться многократно. В такой ситуации простая рекурсивная реализация будет заведомо неэффективной. Вместо этого предлагается решать все подзадачи подряд, начиная с самых маленьких, а их результаты записывать для дальнейшего использования при решении более крупных подзадач. □

Как это часто бывает в динамическом программировании, CYK-алгоритм удобно проводить с помощью заполнения некоторой таблицы (см. таблицу 7.1).

N_{15}				
N_{14}	N_{25}			
N_{13}	N_{24}	N_{35}		
N_{12}	N_{23}	N_{34}	N_{45}	
N_{11}	N_{22}	N_{33}	N_{44}	N_{55}

Таблица 7.1. Пример таблицы CYK-алгоритма для $n = 5$

Таблица заполняется снизу вверх и слева направо. При этом для подсчёта очередного множества будут использоваться лишь клетки, расположенные ниже текущей.

Рассмотрим в качестве примера, какие пары множеств нетерминалов N_{ij} потребуется просматривать, чтобы построить N_{25}. В соответствии с описанием алгоритма (шаг 4, формула для N_{ij}) это пары N_{24} и N_{55}, N_{23} и N_{45}, N_{22} и N_{35}. Они отмечены в таблице 7.2 разными типами скобок (фигурными, квадратными и круглыми соответственно), чтобы подчеркнуть геометрическую последовательность работы алгоритма. Аналогичная треугольная форма будет появляться при расчете и всех других множеств, кроме N_{ii}, которые строятся непосредственно по грамматике.

N_{15}				
N_{14}	$\boxed{N_{25}}$			
N_{13}	$\{N_{24}\}$	(N_{35})		
N_{12}	$[N_{23}]$	N_{34}	$[N_{45}]$	
N_{11}	(N_{22})	N_{33}	N_{44}	$\{N_{55}\}$

Таблица 7.2. Используемые для расчета N_{25} множества

Замечание (о сложности CYK-алгоритма). Отметим, что с точки зрения теории синтаксического анализа CYK-алгоритм проводит *восходящий (bottom—up) анализ*, то есть восстанавливает дерево вывода, начиная с кроны, а не с корня. Нетрудно видеть, что сложность CYK-алгоритма может быть оценена как $O(n^3 \cdot |P|)$, это ограничивает применение алгоритма на практике. В теории компиляторов и приложениях чаще рассматривается подкласс КС-грамматик, *детерминированные КС-грамматики* (по-другому, $LL(k)$- и $LR(k)$-грамматики), для которых существуют линейные алгоритмы разбора (сложность $O(n)$). □

7.3. Матричный метод перехода к нормальной форме Грейбах

КС-грамматика $G = (N, \Sigma, P, S)$ называется грамматикой в нормальной форме Грейбах, если она является неукорачивающейся и каждая продукция из P, отличная от $S \to \varepsilon$, имеет вид $A \to a\alpha$, где $a \in \Sigma$ и $\alpha \in N^*$. В [1] описан алгоритм преобразования произвольной КС-грамматики к нормальной форме Грейбах, основанный на предварительном устранении левой рекурсии.

Рассмотрим КС-грамматику $G = (N, \Sigma, P, S)$, в которой нет цепных правил и ε-продукций (даже вида $S \to \varepsilon$), и схематично опишем принадлежащий Розенкранцу матричный метод преобразования такой грамматики к нормальной форме Грейбах. Этот метод использует технику, напоминающую технику работы с регулярными выражениями из главы 2.

Дадим необходимые определения. Пусть Δ и Σ — два непересекающихся алфавита; алфавит Δ будем называть нетерминальным, а алфавит Σ — терминальным. Системой определяющих уравнений над конечными алфавитами Σ и Δ назовем систему уравнений вида

$$A = \alpha_1 + \ldots + \alpha_k, \tag{7.3.1}$$

где $A \in \Delta$ и $\alpha_i \in (\Delta \cup \Sigma)^*$; если $k = 0$, то уравнение имеет вид $A = \varnothing$. Предполагается, что для каждого $A \in \Delta$ в системе есть только одно уравнение с левой частью A. Решением системы определяющих уравнений назовем такое отображение f множества Δ в $P(\Sigma^*)$, что если вместо каждого $A \in \Delta$ подставить $f(A)$ в уравнения системы, то эти уравнения превратятся в равенства. Решение f назовем наименьшей неподвижной точкой, если $f(A) \subseteq g(A)$ для любого решения g и любого $A \in \Delta$. Стандартные системы линейных уравнений с регулярными коэффициентами из главы 2 входят в класс систем определяющих уравнений.

Выделим в Δ символ S. Системе определяющих уравнений над конечными алфавитами Δ и Σ с выделенным символом $S \in \Delta$ можно естественным образом сопоставить КС-грамматику, в которой Δ — нетерминальный алфавит, Σ — терминальный алфавит, S — начальный символ, а продукции определяются по каждому уравнению (7.3.1) исходной системы следующим образом:

$$A \to \alpha_1 \mid \alpha_2 \mid \ldots \mid \alpha_k. \tag{7.3.2}$$

Ясно, что разным системам сопоставляются разные грамматики и приведенная конструкция осуществляет взаимно однозначное соответствие между множеством всех систем определяющих уравнений над конечными алфавитами и множеством КС-грамматик.

Приведем без доказательства несколько результатов о системах определяющих уравнений, обобщающих результаты о стандартных системах линейных уравнений с регулярными коэффициентами. Отметим прежде всего, что при сделанных предположениях наименьшая неподвижная точка системы определяющих уравнений над алфавитами Δ и Σ существует, единственна и имеет вид

$$f(A) = \{\omega \mid A \Rightarrow_G^* \omega,\ \omega \in \Sigma^*\},$$

где G — соответствующая системе КС-грамматика.

Мы будем пользоваться далее матричным представлением систем определяющих уравнений. Именно, пусть алфавит Δ состоит из нетерминалов

$$A_1, A_2, \ldots, A_n$$

и

1) $\mathbb{A}$ — вектор-строка $[A_1; A_2; \ldots; A_n]$;
2) $\mathbb{R}$ — квадратная матрица порядка n, элементами которой служат регулярные выражения над $N \cup \Sigma$: стоящий в i-й строке и j-м столбце элемент

матрицы R определяется равенством

$$R_{ij} = \alpha_1 + \alpha_2 + \ldots + \alpha_k,$$

где $A_i\alpha_1, A_i\alpha_2, \ldots, A_i\alpha_k$ — все члены уравнения для A_j, первой буквой которых является A_i;

3) $\mathbb{B}$ — вектор-строка, состоящая из n регулярных выражений над $N \cup \Sigma$: стоящий на j-м месте элемент B_j определяется как сумма членов уравнения для A_j которые начинаются буквой из Σ.

Таким образом, B_j и R_{ij} — такие выражения, что уравнение для A_j можно записать в виде

$$A_j = A_1R_{1j} + A_2R_{2j} + \ldots + A_iR_{ij} + \ldots + A_nR_{nj} + B_j.$$

Сложение и умножение векторов и матриц определим как обычно, при этом в качестве «умножения» элементов матриц будем рассматривать конкатенацию, а в качестве «сложения» операцию объединения. Систему определяющих уравнений теперь будем представлять при помощи матриц:

$$\mathbb{A} = \mathbb{A}\mathbb{R} + \mathbb{B}. \tag{7.3.3}$$

Пример 7.3.1. Рассмотрим грамматику $G = (\{A; B\}, \{b; a; c; d\}, P, A)$ с продукциями

$$A \to AaB \mid BB \mid b,$$
$$B \to aA \mid BAa \mid Bd \mid c.$$

Построим систему определяющих уравнений для этой грамматики:

$$\begin{cases} A = AaB + BB + b \\ B = aA + BAa + Bd + c. \end{cases}$$

При помощи матриц эту систему можно переписать так:

$$[A; B] = [A; B]\begin{bmatrix} aB & \varnothing \\ B & Aa + d \end{bmatrix} + [b; aA + C]. \tag{7.3.4}$$

■

Для системы (7.3.3) можно найти такую равносильную ей систему определяющих уравнений, что все правые части продукций из КС-грамматики, которая соответствует новой системе, начинаются терминальными буквами.

Теорема 7.3.1. *Пусть* (7.3.3) *— записанная в матричном виде система определяющих уравнений над алфавитами* Δ *и* Σ, Q *— квадратная матрица порядка* $b = |\Delta|$ *с* n^2 *различными новыми элементами* $q_{i,j}$. $\widetilde{Q}$ *— множество всех элементов* $q_{i,j}$, $\widetilde{\Delta} = \Delta \cup \widetilde{Q}$. *Тогда система определяющих уравнений*

$$\begin{cases} \mathbb{A} = \mathbb{B}Q + \mathbb{B}, \\ Q = \mathbb{R}Q + \mathbb{R} \end{cases} \tag{7.3.5}$$

над алфавитами $\widetilde{\Delta}$ *и* Σ *имеет наименьшую неподвижную точку, которая совпадает на* Δ *с наименьшей неподвижной точкой системы* (7.3.3). □

Приведем алгоритм преобразования КС-грамматики к нормальной форме Грейбах, основанный на конструкции из этой теоремы: алгоритм 7.3.1.

Алгоритм 7.3.1 Преобразование к нормальной форме Грейбах

Вход Приведенная КС-грамматика $G = (N, \Sigma, P, S)$, не содержащая продукции $S \to \varepsilon$.

Выход КС-грамматика $G' = (N', \Sigma, P', S)$ в нормальной форме Грейбах.

Шаг 1. По грамматике G построить систему определяющих уравнений $\Delta = \Delta R + B$ над алфавитами N и Σ.

Шаг 2. Ввести Q — квадратную матрицу порядка $n = |N|$, состоящую из новых нетерминальных букв q_{ij}. Построить по Q и исходной системе (7.3.3) новую систему (7.3.5) и соответствующую новой системе КС-грамматику G_1. (Так как в B каждая компонента, отличная от $\varnothing$, начинается терминалом, то для $A \in N$ все нетерминальные A-продукции грамматики G_1 будут начинаться терминалами; наличие в B ненулевых компонент вытекает из приведенности грамматики G.)

Шаг 3. (Так как G — приведенная грамматика, то ε не встречается среди элементов матрицы R; поэтому для каждого элемента $q \in Q$ все нетерминальные q-продукции грамматики G_1 начинаются символами из $N \cup \Sigma$.) Для каждого нетерминала A из N в тех правых частях нетерминальных q-продукций грамматики G_1, которые начинаются буквой A, заменить этот нетерминал правыми частями всех A-продукций. В результате получится грамматика, у которой правые части всех продукций начинаются терминалом.

Шаг 4. Если в правой части продукции терминал a встречается не на первом месте, заменить его новым нетерминалом a' и добавить продукцию $a' \to a$. В итоге получим искомую КС-грамматику G'.■

Теорема 7.3.2. *Алгоритм 7.3.1 строит грамматику G' в нормальной форме Грейбах и $L(G) = L(G')$.* □

Пример 7.3.2. Рассмотрим КС-грамматику $G = (\{A; B\}, \{b; a; c\}, P, A)$ из примера 7.3.1, где реализован первый шаг алгоритма. На втором шаге введем нетерминальную матрицу

$$\begin{bmatrix} W & X \\ Y & Z \end{bmatrix}$$

и построим по этой матрице и системе 7.3.5 новую систему

$$\begin{cases} [A, B] = [b, aA + c] \begin{bmatrix} W & X \\ Y & Z \end{bmatrix} + [b, aA + c], \\ \begin{bmatrix} W & X \\ Y & Z \end{bmatrix} = \begin{bmatrix} ab & \varnothing \\ B & Aa + d \end{bmatrix} \begin{bmatrix} W & X \\ Y & Z \end{bmatrix} + \begin{bmatrix} aB & \varnothing \\ B & Aa + d \end{bmatrix}. \end{cases} \tag{7.3.6}$$

Системе 7.3.6 соответствует КС-грамматика с продукциями

$$\begin{aligned} A &\to bW \mid aAY \mid cY \mid b, & B &\to bX \mid aAZ \mid cZ \mid aA \mid c, \\ W &\to aBW \mid aB, & X &\to aBX, \\ Y &\to BW \mid AaY \mid dY \mid B, & Z &\to BX \mid AaZ \mid dZ \mid Aa \mid d. \end{aligned}$$

Заметим, что X — бесполезный символ. Следовательно, полученные продукции можно упростить:

$$\begin{aligned} A &\to bW \mid aAY \mid cY \mid b, & B &\to aAZ \mid cZ \mid aA \mid c, \\ W &\to aBW \mid aB, & Y &\to BW \mid AaY \mid dY \mid B, \\ Z &\to AaZ \mid dZ \mid Aa \mid d. \end{aligned}$$

Итак, в результате второго шага алгоритма построена грамматика

$$G_1 = (\{A; B; W; Y; Z\}, \{a; b; c; d\}, P_1, A),$$

где P_1 — описанное выше множество.

На третьем шаге алгоритма в пяти «нехороших» продукциях

$$\begin{aligned} Y &\to BW \mid AaY \mid B, \\ Z &\to AaZ \mid Aa \end{aligned}$$

нетерминалы A и B надо заменить правыми частями соответствующих нетерминальных A- и B-продукций. Например, из продукции $Y \to BW$ получаем:

$$Y \to aAZW \mid cZW \mid aAW \mid cW,$$

а из продукции $Z \to AaZ$:

$$Z \to bWaZ \mid aAYaZ \mid cYaZ \mid baZ.$$

В результате третьего шага алгоритма вместо пяти «нехороших» продукций мы построили двадцать новых:

$$\begin{aligned}
&Y \to aAZW \mid cZW \mid aAW \mid cW \mid bWaY \mid aAYaY \mid cYaY \mid baY \mid aAZ \mid cZ \mid aA \mid c,\\
&Z \to bWaZ \mid aAYaZ \mid cYaZ \mid baZ \mid bWa \mid aAYa \mid cYa \mid ba.
\end{aligned}$$

Теперь правые части всех продукций начинаются терминалами.

На четвертом шаге алгоритма в тех продукциях, у которых в правой части терминал x встречается не на первом месте, надо заменить его новым нетерминалом x': после этого надо добавить новую продукцию $x' \to x$. В итоге получаем КС-грамматику $G' = (\{A; B; W; Y; Z; a'\}, \{a; b; c; d\}, P', A)$ с множеством продукций P':

$$\begin{aligned}
A &\to bW \mid aAY \mid cY \mid b,\\
B &\to aAZ \mid cZ \mid aA \mid c,\\
W &\to aBW \mid aB,\\
Y &\to aAZW \mid cZW \mid aAW \mid cW \mid bWa'Y \mid cYa'Y \mid ba'Y \mid aAZ \mid cZ \mid aA \mid c \mid dY,\\
Z &\to bWa'Z \mid aAYa'Z \mid cYa'Z \mid ba'Z \mid bWa' \mid aAYa' \mid cYa' \mid ba' \mid dZ \mid d,\\
a' &\to a.
\end{aligned}$$

Эта грамматика имеет нормальную форму Грейбах. С другой стороны, по теореме 7.3.2 $L(G) = L(G')$. ■

7.4. Упражнения

Получение нормальных форм

Привести к нормальной форме Хомского, а также, к нормальной форме Грейбах, грамматики с продукциями:

(1) $S \to ASB \mid \varepsilon,$
$A \to aAS \mid a,$
$B \to SbS \mid A \mid bb;$

(2) $S \to 0A0 \mid 1B1 \mid BB,$
$A \to C,$
$B \to S \mid A,$
$C \to S \mid \varepsilon;$

(3) $S \to AAA \mid B,$
$A \to aA \mid B,$
$B \to \varepsilon;$

(4) $S \to aAa \mid bBb \mid \varepsilon,$
$A \to C \mid a,$
$B \to C \mid b,$
$C \to CDE \mid \varepsilon,$
$D \to A \mid B \mid ab.$

CYK-алгоритм

Используя CYK-алгоритм,

1) для грамматики G с продукциями:

$$S \to AB, \qquad A \to BB \mid a, \qquad B \to AB \mid b$$

определить, принадлежат ли $L(G)$ строки: (а) $aabbb$, (б) $babab$, (в) b^7;

2) для грамматики G с продукциями:

$$S \to AB \mid BC, \qquad A \to BA \mid a, \qquad B \to CC \mid b, \qquad C \to AB \mid a$$

определить, принадлежат ли $L(G)$ строки: (а) $ababa$, (б) $baaab$, (в) $aabab$.

ГЛАВА 8

АВТОМАТЫ С МАГАЗИННОЙ ПАМЯТЬЮ

8.1. Определения и примеры

Перед тем, как изучать этот параграф, было бы хорошо еще раз просмотреть содержание пункта 3.1.

Автомат с магазинной памятью — это односторонний недетерминированный распознаватель (см. 1.5), в потенциально бесконечной памяти которого элементы информации хранятся и используются так же, как патроны в магазине автоматического оружия, то есть в каждые момент доступен только верхний элемент магазина. Таким образом магазинная память — это синоним стека, где каждый элемент «последним пришёл — первым вышел». Можно представлять себе магазин в виде слова, причем верхним символом магазина будем считать самую левую букву.

Автомат с магазинной памятью (сокращенно МП-автомат) — это семерка

$$P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F),$$

где

1) Q — конечное множество состояний;
2) Σ — конечный входной алфавит;
3) Γ — конечный алфавит магазинных символов;
4) функция переходов δ — отображение множества $Q \times (\Sigma \cup \{\varepsilon\}) \times \Gamma$ во множество $P(Q \times \Gamma^*)$ конечных подмножеств множества $Q \times \Gamma^*$;
5) q_0 $(\in Q)$ — начальное состояние;
6) Z_0 $(\in \Gamma)$ — начальный символ магазина;
7) F $(\subseteq Q)$ — множество заключительных (финальных) состояний.

Конфигурацией МП-автомата P называется тройка (q, ω, α) из $Q \times \Sigma^* \times \Gamma^*$, где q — текущее состояние управляющего устройства; ω — непрочтённая часть входного слова (первая буква слова ω находится под входной головкой; при этом, если $\omega = \varepsilon$, то считается, что все входное слово прочитано); α — содержимое магазина (самый левый символ слова α отождествляется с верхним символом магазина; при этом, если $a = \varepsilon$, то магазин считается пустым).

Такт работы МП-автомата P будем представлять бинарным отношением $\vdash_P$ (или, короче, $\vdash$), определенным на конфигурациях. Именно, если $(q', \gamma) \in \delta(q, a, Z)$, то будем писать $(q, a\omega, Z\alpha)\} \vdash (q', \omega, \gamma\alpha)$, где $q, q' \in Q$, $a \in \Sigma \cup \{\varepsilon\}$, $\omega \in \Sigma^*$, $Z \in \Gamma$ и $\alpha, \gamma \in \Gamma^*$. Если $a \neq \varepsilon$, то формула $(q, a\omega, Z\alpha) \vdash (q', \omega, \gamma\alpha)$ говорит о том, что МП-автомат P сначала находился в состоянии q, имел a в качестве текущей входной буквы и Z в качестве верхнего символа магазина; после чего он перешел в новое состояние q, сдвинул входную головку на одну ячейку вправо и заменил верхний символ магазина словом γ, составленным из магазинных букв. Если же $a = \varepsilon$ (такой такт называется ε-тактом), то текущая входная буква не принимается во внимание и входная головка не двигается, однако состояние управляющего устройства и содержимое памяти могут измениться.

Подчеркнём, что ε-такт может происходить и тогда, когда все входное слово прочитано; при пустом же магазине следующий такт невозможен.

Обычным образом вводятся обозначения $\vdash_P^i$ для $i \geqslant 0$, $\vdash_P^*$ и $\vdash_P^+$ (далее значок P мы часто будем в этих обозначениях пропускать).

Начальной конфигурацией МП-автомата P называется конфигурация вида $\{q_0, \omega, \Sigma_0\}$; в этом случае управляющее устройство находится в начальном состоянии, входная лента содержит некоторое слово из Σ^*, а в магазине есть только начальный символ Z_0.

Заключительная конфигурация — это конфигурация вида (q, ε, α), где $q \in F$ и $\alpha \in \Gamma^*$. Говорят, что слово ω допускается МП-автоматом P, если $(q_0, \omega, Z_0) \vdash^* (q, \varepsilon, \alpha)$ для некоторых $q \in F$ и $\alpha \in \Gamma^*$. Языком, определяемым (или допускаемым) автоматом P называют множество всех слов, допускаемых автоматом P; этот язык обозначается $L(P)$.

Как и в случае обычных автоматов, недетерминированность удобно интерпретировать как наличие нескольких параллельно работающих экземпляров исходного МП-автомата.

Пример 8.1.1. Для того, чтобы задать язык

$$L = \{0^n 1^n \mid n \geqslant 0\},$$

рассмотрим МП-автомат

$$P = (\{q_0; q_1; q_2\}, \{0; 1\}, \{Z; 0\}, \delta, q_0, Z, \{q_0\}),$$

где

$$\delta(q_0, 0, Z) = \{(q_1, 0Z)\}, \quad \delta(q_1, 0, 0) = \{(q_1, 00)\}, \quad \delta(q_1, 1, 0) = \{(q_2, \varepsilon)\},$$

а для остальных элементов из $Q \times (\Sigma \cup \{\varepsilon\}) \times \Gamma$ функция переходов не определена.

Работа автомата P состоит в том, что он копирует в магазин состоящий из нулей префикс входного слова, а затем устраняет из магазина по одному нулю на каждую единицу, которую он обнаруживает на входе.

Например, для входного слова 0011 автомат P проделает такую последовательность тактов:

$$(q_0, 0011, Z) \vdash (g_1, 011, 0Z) \vdash (q_1, 11, 00Z) \vdash (q_2, 1, 0Z) \vdash (q_2, \varepsilon, Z) \vdash (q_0, \varepsilon, \varepsilon).$$

Докажем, что $L \subseteq L(P)$. Прежде всего заметим, что осуществимы следующие пять последовательностей тактов:

$$(q_0, 0, Z) \vdash (q_1, \varepsilon, 0Z),$$
$$(q_1, 0^i, 0Z) \vdash^i (q_1, \varepsilon, 0^{i+1}Z),$$
$$(q_1, 1, 0^{i+1}Z) \vdash (q_2, \varepsilon, 0^i Z),$$
$$(q_2, 1^i, 0^i Z) \vdash^i (q_2, \varepsilon, Z),$$
$$(q_2, \varepsilon, Z) \vdash (q_0, \varepsilon, \varepsilon).$$

Отсюда вытекает, что

$$(q_0, \varepsilon, Z) \vdash^0 (q_0, \varepsilon, Z)$$

и для $n \geqslant 1$ возможна последовательность

$$(q_0, 0^n 1^n, Z) \vdash^{2n+1} (q_0, \varepsilon, \varepsilon).$$

Таким образом, $L \subseteq L(P)$.

Докажем, что $L \supseteq L(P)$. Анализ функции переходов показывает, что если P допускает непустое слово, то он обязан последовательно пройти через состояния q_0, q_1, q_2, q_0. Если для $i \geqslant 1$

$$(q_0, \omega, Z) \vdash^i (q_1, \varepsilon, \alpha),$$

то $\omega = 0^i$, $\alpha = 0^i Z$. Аналогично, если для $i \geqslant 1$

$$(q_2, \omega, \alpha) \vdash^i (q_2, \varepsilon, \beta),$$

то $\omega = 1^i$, $\alpha = 0^i\beta$. Такт

$$(q_1, \omega, \alpha) \vdash (q_2, \varepsilon, \beta)$$

возможен тогда и только тогда, когда $\omega = 1$, $\alpha = 0\beta$, а последовательность тактов

$$(q_2, \omega, Z) \vdash^* (q_0, \varepsilon, \varepsilon)$$

возможна только и только тогда, когда $\omega = \varepsilon$. Таким образом, если

$$(q_0, \omega, Z) \vdash^i (q_0, \varepsilon, \alpha)$$

для некоторого $i \geqslant 0$, то либо $\omega = \varepsilon$ и $i = 0$, либо $\omega = 0^n1^n$, $i = 2n + 1$ и $\alpha = \varepsilon$. Следовательно, $L \supseteq L(P)$. ■

Пример 8.1.2. Построим МП-автомат, допускающий язык

$$L = \{\omega\omega^R \mid \omega \in \{a, b\}^+\}.$$

Пусть $P = (\{q_0; q_1; q_2\}, \{a; b\}, \{Z; a; b\}, \delta, q_0, Z, \{q_2\})$, где

1) $\delta(q_0, a, Z) = \{(q_0, aZ)\}$,
2) $\delta(q_0, b, Z) = \{(q_0, bZ)\}$,
3) $\delta(q_0, a, a) = \{(q_,aa), (q_1, \varepsilon)\}$,
4) $\delta(q_0, a, b) = \{(q_0, a, b)\}$,
5) $\delta(q_0, b, a) = \{(q_0, ba)\}$,
6) $\delta(q_0, b, b) = \{(q_0, bb), (q_1, \varepsilon)\}$,
7) $\delta(q_1, a, a) = \{(q_1, \varepsilon)\}$,
8) $\delta(q_1, b, b) = \{(q_1, \varepsilon)\}$,
9) $\delta(q_1, \varepsilon, Z) = \{(q_2, \varepsilon)\}$,

а для остальных элементов из $Q \times (\Sigma \cup \{\varepsilon\}) \times \Gamma$ функция переходов не определена.

Опишем работу МП-автомата P. Сначала P копирует в магазине какую-то часть входного слова по правилам (1), (2), (4), (5) и первыми альтернативами правил (3) и (6). Однако в том случае, когда текущая входная буква совпадет с верхним символом магазина, автомат может перейти (если пожелает!) в состояние q_1 и начать сравнивать слово в магазине с оставшейся частью входного слова. Эта возможность гарантируется вторыми альтернативами правил (3) и (6), а по правилам (7) и (8) происходит сравнение символов. Если в ходе сравнения обнаруживается несовпадение очередных букв, то соответствующий экземпляр МП-автомата «умирает». Однако в силу недетерминированности разные экземпляры P могут проделывать все возможные для него такты, и если

реализация какой-нибудь последовательности тактов приводит к тому, что Z снова оказывается верхним (и единственным!) символом магазина, то по правилу (8) P стирает Z и попадает в состояние q_2. МП-автомат P должен допустить слово тогда и только тогда, когда все сравнения обнаружили совпадение букв.

Рассмотрим работу МП-автомата P в случае, когда $aabbaa$ — входное слово. Ясно, что среди прочих возможны «смертельные» последовательности тактов, например:

$$(q_0, aabbaa, Z) \vdash (q_0, abbaa, aZ) \vdash (q_1, bbaa, Z)$$

или

$$(q_0, aabbaa, Z) \vdash (q_0, abbaa, aZ) \vdash (q_0, bbaa, aaZ) \vdash (q_0, baa, baaZ) \vdash$$
$$(q_0, aa, bbaaZ) \vdash (q_0, a, abbaaZ) \vdash (q_0, \varepsilon, aabbaaZ).$$

С другой стороны, возможна последовательность, оканчивающаяся заключительной конфигурацией:

$$(q_0, aabbaa, Z) \vdash (q_0, abbaa, aZ) \vdash (q_0, bbaa, aaZ) \vdash (q_0, baa, baaZ) \vdash$$
$$(q_1, aa, aaZ) \vdash (q_1, a, aZ) \vdash (q_1, \varepsilon, Z) \vdash (q_2, \varepsilon, \varepsilon).$$

Это означает, что МП-автомат P допускает входное слово $aabbaa$.

Докажем, что $L \subseteq L(P)$. Пусть $\omega = c_1 c_2 \dots c_{n-1} c_n c_n c_{n-1} \dots c_1$, где $C_i \in \{a, b\}$ для $1 \leqslant i \leqslant n$. Возможна следующая последовательность тактов:

$$(q_0, \omega, Z) \vdash^n (q_0, c_n, c_{n-1} \dots c_1, c_n c_{n-1} \dots c_1 Z) \vdash$$
$$(q_1, c_{n-1} \dots c_1, c_{n-1} \dots c_1, c_{n-1} \dots c_1 Z) \vdash^{n-1} (q_1, \varepsilon, Z) \vdash (q_2, \varepsilon, \varepsilon).$$

Таким образом, $L \subseteq L(P)$.

Доказательство вложения $L \supseteq L(P)$ не приводим. ■

Упражнение 8.1.1. Завершить доказательство равенства $L = L(P)$ в примере 8.1.2, то есть доказать, что если $(q_0, \omega, Z) \vdash^* (q_2, \varepsilon, \alpha)$, где $\alpha \in \Gamma^*$, то $\omega = xx^R$ для некоторого $x \in (a + b)^+$ и $\alpha = \varepsilon$. □

Упражнение 8.1.2. Для произвольного МП-автомата

$$P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$$

доказать, что если $(q, \omega, A) \vdash^n (q', \varepsilon, \varepsilon)$, то $(q, \omega, A\alpha) \vdash^n (q', \varepsilon, \alpha)$ для всех $A \in \Gamma$ и $\alpha \in \Gamma^*$. Этот факт можно было бы сформулировать так: «То, что происходит с верхним символом магазина, не зависит от того, что находится в магазине под ним». □

8.2. Расширенный МП-автомат

Напомним, что МП-автомат мог на каждом такте заменять лишь один верхний символ магазина. Теперь определение МП-автомата будет слегка изменено, именно, автомату будет позволено заменять за один такт какой-нибудь магазинный префикс.

Расширенным МП-автоматом (РМП-автоматом) назовем семерку

$$P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F),$$

где δ — отображение конечного подмножества множества $Q \times (\Sigma \cup \{\varepsilon\}) \times \Gamma^*$ во множество конечных подмножеств множества $Q \times \Gamma^*$, а все другие символы имеют такой же смысл, как и в 8.1.

Ясно, что каждый обычный МП-автомат является РМП-автоматом. Конфигурация определяется как и прежде. Мы будем писать

$$(q, a\omega, \alpha\gamma) \vdash (q', \omega, \beta\gamma),$$

если $(q', \beta) \in \delta(q, a, \alpha)$, где $q \in Q$, $a \in \Sigma \cup \{\varepsilon\}$, $\alpha, \beta, \gamma \in \Gamma^*$. Языком $L(P)$, определяемым РМП-автоматом P, называется множество всех таких слов ω, что $(q_0, \omega, Z_0) \vdash^* (q, \varepsilon, \alpha)$ для некоторых $q \in Q$ и $\alpha \in \Gamma^*$.

Отметим, что в отличие от МП-автомата РМП-автомат обладает способностью продолжать работу и тогда, когда магазин пуст.

Пример 8.2.1. Построим РМП-автомат P, распознающий язык $L = \{\omega\omega^R \mid \omega \in \{a, b\}^*\}$. Пусть $P = (\{q; p\}, \{a; b\}, \{a; b; S; Z\}, \delta, q, Z, \{p\})$, где

1) $\delta(q, a, \varepsilon) = \{(q, a)\}$,
2) $\delta(q, b, \varepsilon) = \{(q, b)\}$,
3) $\delta(q, \varepsilon, \varepsilon) = \{(q, S)\}$,
4) $(q, \varepsilon, aSa) = \{(q, S)\}$,
5) $(q, \varepsilon, bSb) = \{(q, S)\}$,
6) $(q, \varepsilon, SZ) = \{(p, \varepsilon)\}$,

а для остальных элементов из $Q \times (\Sigma \cup \{\varepsilon\}) * \Gamma$ функция переходов не определена.

Сначала автомат P записывает в магазине некоторый префикс входного слова (правила (1),(2)). Далее автомат может предположить, что середина слова достигнута и записать верхним символом магазина маркер S (правило (3)).

Можно проверить, что если автомат неправильно угадает середину слова, то рано или поздно он обязательно «умрет», но если же автомат в нужном слове угадает середину правильно, то он имеет возможность выжить и достигнуть заключительного состояния. После угадывания середины слова автомат P помещает в магазин очередную входную букву и заменяет в магазине aSa или bSb на S (правила (1),(4) или (2),(5)). Автомат P работает до тех пор, пока не исчерпается все входное слово. Если после этого в магазине останется слово SZ, то P сотрет его и тем самым будет получена заключительная конфигурация.

Рассмотрим работу РМП-автомата P в случае, когда $aabbaa$ — входное слово. Разумеется, как это обычно бывает, среди прочих возможны «смертельные» последовательности тактов, например:

$$\begin{aligned}(q, aabbaa, Z) \vdash (q, abbaa, aZ) \vdash (q, abbaa, SaZ) \vdash (q, bbaa, aSaZ) \vdash \\ (q, bbaa, SZ) \vdash (q, baa, bSZ) \vdash (q, baa, SbSZ) \vdash (q, aa, bSbSZ) \vdash \\ (q, aa, SSZ) \vdash (q, a, aSSZ) \vdash (q, a, SaSSZ) \vdash (q, a, SSZ) \vdash (q, \varepsilon, aSSZ)\end{aligned}$$

или

$$\begin{aligned}(q, aabbaa, Z) \vdash (q, abbaa, aZ) \vdash (q, bbaa, aaZ) \vdash (q, bbaa, SaaZ) \vdash \\ (q, aa, bbSaaZ) \vdash (q, a, abbSaaZ) \vdash (q, \varepsilon, aabbSaaZ).\end{aligned}$$

С другой стороны, возможна последовательность, оканчивающаяся заключительной конфигурацией:

$$\begin{aligned}(q, aabbaa, Z) \vdash (q, abbaa, aZ) \vdash (q, bbaa, aaZ) \vdash (q, baa, baaZ) \\ \vdash (q, baa, SbaaZ) \vdash (q, aa, bSbaaZ) \vdash (q, aa, SaaZ) \vdash (q, a, aSaaZ) \\ \vdash (q, a, SaZ) \vdash (q, \varepsilon, aSaZ) \vdash (q, \varepsilon, SZ) \vdash (q, \varepsilon, \varepsilon).\end{aligned}$$

Это означает, что РМП-автомат P допускает входное слово $aabbaa$. ■

Упражнение 8.2.1. Доказать, что построенный в примере 8.2.1 РМП-автомат P, действительно распознает язык $L = \{\omega\omega^R \mid \omega \in \{a, b\}^*\}$. □

Теорема 8.2.1. *Класс языков, определяемых МП-автоматами, совпадает с классом языков, определяемых РМП-автоматами.* □

Доказательство. Пусть $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$ — произвольный РМП-автомат. Для доказательства теоремы достаточно построить такой МП-автомат P_1, что

$L(P_1) = L(P)$. Ниже будет предъявлена конструкция автомата и приведена схема доказательства равенства.

Введем обозначение:

$$m = \max\{|\alpha| : \delta(q, a, \alpha) \neq \varnothing, \alpha \in \Gamma^*, q \in Q, a \in \Sigma \cup \{\varepsilon\}\}.$$

Построим МП-автомат P_1, который будет моделировать автомат P, храня верхние m символов его магазина в «буфере» длины m, занимающим часть памяти управляющего устройства автомата P_1. При этом автомат P_1 сможет сообщить в начале каждого такта, каковы m верхних символов магазина автомата P. Если в некотором такте P заменяет слово из k верхних символов магазина словом из l символов, то P_1 заменит k первых символов в буфере этим словом длины l. Если $l < k$, то P_1 сделает $k - i$ вспомогательных ε-тактов, в течение которых $k - i$ символов перейдут из верхней части магазина в буфер управляющего устройства; после этого буфер окажется заполненным, и P_1 будет готов моделировать очередной такт автомата P. Если $i > k$, то символы передаются из буфера в магазин. В качестве состояний МП-автомата P_1 будут рассматриваться упорядоченные пары $[q, \alpha]$, где q — состояние из множества Q, $\alpha (\in \Gamma_1^*)$ — буфер, $0 \leqslant |\alpha| \leqslant m$.

Итак, рассмотрим МП-автомат $P_1 = (Q_1, \Sigma, \Gamma_1, \delta_1, q_1, Z_1, F_1)$, где

$\Gamma_1 = \Gamma \cup \{Z_1\}$,

$Q_1 = \{[q, \alpha] \mid q \in Q, \alpha \in \Gamma_1^*, 0 \leqslant \alpha \leqslant m\}$,

$q_1 = [q_0, Z_0 Z_1^{m-1}]$,

$F_1 = \{[q, \alpha] \mid q \in F, \alpha \in \Gamma_1^*\}$,

а функция переходов δ_1 определяется по правилам:

1) если $(r, Y_1 \ldots Y_l) \in \delta(q, a, X_1 \ldots X_k)$, то для всех $Z \in \Gamma_1$ и всех таких $\alpha \in \Gamma_1^*$, у которых $|\alpha| = m - k$ положим

   1.1) при $l \geqslant k$ для всех $\beta \in \Gamma_1^*$, у которых $\beta\gamma = Y_1 \ldots Y_l\alpha$ и $|\beta| = m$:

   $$([r, \beta], \gamma Z) \in \delta_1([q, X_1 \ldots X_k\alpha], a, Z),$$

   1.2) при $l < k$:

   $$([r, Y_1 \ldots Y_l\alpha Z], \varepsilon) \in \delta_1([q, X_1 \ldots X_k\alpha], a, Z);$$

2) $\delta_1([q, \alpha], \varepsilon, Z) = \{([q, \alpha Z], \varepsilon)\}$ для всех $q \in Q$, $\Sigma \in \Gamma_1$ и всех таких $\alpha \in \Gamma_1^*$, у которых $|\alpha| < m$.

По этим правилам осуществляется заполнение буфера управляющего устройства, который содержит m символов. Отметим, что в начальный момент буфер содержит символ Z_0 наверху и $(m-1)$ символов Z_1 ниже. Символ Z_1 используется как специальный маркер, отмечающий «дно» магазина.

Анализ конструкции автомата P_1 позволяет показать, что такт

$$(q, a\omega, X_1 \dots X_k X_{k+1} \dots X_n) \vdash_P (r, \omega, Y_1 \dots Y_l X_{k+1} \dots X_n)$$

происходит тогда и только тогда, когда

$$([q, \alpha], a\omega, \beta) \vdash_{P_1}^{+} ([r, \alpha'], \omega, \beta'),$$

где

$$\alpha\beta = X_1 \dots X_n Z_1^m, \alpha'\beta' = Y_1 \dots Y_l X_{k+1} \dots X_n Z_1^m, |\alpha| = |\alpha'| = m,$$

и между двумя конфигурациями $([q, \alpha], a\omega, \beta)$ и $([r, \alpha'], \omega, \beta')$ МП-автомата P_1 нет ни одной, в которой состояние имело бы вторую компоненту (буфер) длины m.

Таким образом, для некоторых $q \in F$ и $\alpha \in \Gamma^*$ такт

$$(q_0, \omega, \Sigma_0) \vdash_P^* (q, \varepsilon, \alpha)$$

происходит тогда и только тогда, когда

$$([q_0, Z_0 Z_1^{m-1}], \omega, Z_1) \vdash_{P_1}^* ([q, \beta], \varepsilon, \gamma),$$

где $|\beta| = m$ и $\beta\gamma = \alpha\Sigma_1^m$. Отсюда вытекает, что $L(P_1) = L(P)$. ■

Теорема 8.2.1 предоставляет и обосновывает все необходимые конструкции для перехода от РМП к МП-автомату. Сформулируем на основе этой теоремы следующий алгоритм.

Алгоритм 8.2.1 Построение МП-автомата по РМП-автомату

Вход РМП-автомат $P = (Q, \Sigma, \Gamma, q_0, Z_0, F)$.

Выход МП-автомат $P' = (Q_1, \Sigma, \Gamma_1, q_1, Z_1, F_1)$, такой что $L(P') = L(P)$.

Метод Конструирование элементов автомата P' по правилам теоремы 8.2.1.

Шаг 1. Положить $m = max\{|\alpha| \mid \alpha \in \Gamma^*, \delta(q, a, \alpha) \neq \varnothing, q \in Q, a \in \Sigma \cup \{\varepsilon\}\}$.

Шаг 2. Положить $\Gamma_1 = \Gamma \cup \{Z_1\}$.

Шаг 3. Положить $Q_1 = \{[q, \alpha] \mid q \in Q \cup \{q_{new}\}, \alpha \in \Gamma_1^*, 0 \leqslant |\alpha| \leqslant m\}$.

Шаг 4. Определить δ_1 следующим образом:

1) если $\delta(q, a, X_1 \dots X_k) \ni (r, Y_1 \dots Y_l)$ и $l \geqslant k$, то для всех $Z \in \Gamma_1$ и $\alpha \in \Gamma_1^*, |\alpha| = m - k$,

$$\delta_1([q, X_1 \dots X_k\alpha], a, Z) \ni ([r, \beta], \gamma Z),$$

где $\beta\gamma = Y_1 \dots Y_l\alpha$ и $|\beta| = m$;

2) если $\delta(q, a, X_1 \dots X_k) \ni (r, Y_1 \dots Y_l)$ и $l \leqslant k$, то для всех $Z \in \Gamma_1$ и $\alpha \in \Gamma_1^*, |\alpha| = m - k$,

$$\delta_1([q, X_1 \dots X_k\alpha], a, Z) \ni ([r, Y_1 \dots \alpha Z], \varepsilon);$$

3) для всех $q \in Q, Z \in \Gamma_1$ и $\alpha \in \Gamma_1^*, |\alpha| \leqslant m$,

$$\delta_1(q_1, \varepsilon, Z_1) = \{([q, \alpha Z], \varepsilon)\};$$

4) $\delta_1(q_1, \varepsilon, Z_1) = \{([q_0, Z_0 Z_1^{m-1}], Z_1 Z_1)\}$.

Шаг 5. Положить $q_1 = [q_{new}, Z_1^m]$.

Шаг 6. Положить $F_1 = \{[q, \alpha] \mid q \in F, \alpha \in \Gamma_1^*\}$.

Шаг 7. Вернуть МП-автомат $P' = (Q_1, \Sigma, \Gamma_1, \delta_1, q_1, Z_1, F_1)$ в качестве результата.

8.3. Автомат, допускающий слово опустошением магазина

Напомним, что согласно данным ранее определениям, слово ω из Σ^* допускается РМП-автоматом $P = (G, \Sigma, \Gamma, \delta, q_0, Z_0, F)$ тогда и только тогда, когда

$$(q_0, \omega, Z_0) \vdash^* (q, \varepsilon, \alpha)$$

для некоторых $q \in F$ и $\alpha \in \Gamma^*$, а языком $L(P)$, определяемым автоматом P, называют множество всех слов, допускаемых автоматом P. Теперь изменим условие допускаемости слова.

Пусть $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, P)$ — РМП-автомат. Будем говорить, что автомат P допускает слово $\omega \in \Sigma^*$ *опустошением магазина*, если $(q_0, \omega, Z_0) \vdash^+ (q, \varepsilon, \varepsilon)$ для некоторого $q \in Q$. Пусть $L_\varepsilon(P)$ — множество всех слов, допускаемых автоматом P опустошением магазина. Далее, если нас в автомате (МП или РМП) будет интересовать только язык $L_\varepsilon(P)$, то такие автоматы будем называть МПε-автомат и РМПε-автомат соответственно.

Если два РМП-автомата. P и R, отличаются друг от друга только множествами заключительных состояний, то $L_\varepsilon(P) = L'_\varepsilon(R)$, хотя, разумеется, языки $L(P)$ и $L(R)$ не обязаны совпадать. Поэтому при изучении языков, допускаемых РМП-автоматами опустошением магазина, обычно рассматривают только такие автоматы, у которых $P = \varnothing$.

Выясним, как связано новое условие допускаемости слов с прежним.

Теорема 8.3.1. *Пусть $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$ — РМП-автомат. Тогда можно построить такой МПε-автомат P', что $L_\varepsilon(P') = L(P)$.* □

Доказательство. Ввиду теоремы 8.2.1 будем, не теряя общности, полагать, что P — МП-автомат.

Предположим, что автомат P' моделирует действия автомата P, и прикинем, каким требованиям он обязан в этом случае удовлетворять. Введем специальное состояние q_ε, которое позволяет опустошать магазин; всякий раз, когда P переходит в заключительное состояние, P' должен решать, продолжать ли моделирование P или перейти в состояние q_ε. Второй важный момент, который надо учесть, состоит в том, что для некоторого входного слова ω автомат P может сделать последовательность тактов, приводящую к опустошению магазина, но управляющее устройство окажется при этом не в заключительном состоянии; тогда для того, чтобы помешать P' допустить в этом случае слово ω, надо добавить к P' специальный маркер Z', отмечающий «дно» магазина, который автомат P' может устранить только в состоянии q_ε. Этот же символ, Z', будет начальным символом магазина.

Итак, пусть

$$P' = (Q \cup \{q_\varepsilon; q'\}, \Sigma, \Gamma \cup \{Z'\}, \delta', q', Z', \varnothing),$$

где δ' определяется так:

1) $\delta'(q', \varepsilon, Z') = \{(q_0, Z_0, Z')\}$;

2) $\forall q \in Q$, $\forall a \in \Sigma \cup \{\varepsilon\}$, $\forall Z \in \Gamma$: $(r, \gamma) \in \delta(q, a, Z) \Rightarrow (r, \gamma) \in \delta'(q, a, Z)$;

3) $\forall q \in F$, $\forall Z \in \Gamma \cup \{Z'\}$: $(q_\varepsilon, \varepsilon) \in \delta'(q, \varepsilon, Z)$;

4) $\forall Z \in \Gamma \cup \{Z'\}$: $\delta'(q_\varepsilon, \varepsilon, Z) = \{(q_\varepsilon, \varepsilon)\}$.

Отметим, что на первом такте автомат P' записывает в магазин $Z_0 Z'$ и переходит в начальное состояние q_0 автомата P, а Z' начинает играть роль маркера, отмечающего «дно» магазина.

Анализ функций переходов δ и δ' показывает, что для некоторых натуральных r и n, произвольного q из F и произвольных слов $Y_1, \ldots, Y_{r-1}, Y_r = Z'$ из Γ^* последовательность тактов автомата P'

$$(q', \omega, Z') \vdash_{P'} (q_0, \omega, Z_0, Z') \vdash_{P'}^{n} (q, \varepsilon, Y_1 \ldots Y_r) \vdash_{P'} (q_\varepsilon, \varepsilon, Y_2 \ldots Y_r) \vdash_{P'}^{r-1} (q_\varepsilon, \varepsilon, \varepsilon)$$

осуществима тогда и только тогда, когда

$$(q_0, \omega, Z_0) \vdash_{P}^{n} (q, \varepsilon, Y_1 \ldots Y_{r-1}).$$

Следовательно, $L_\varepsilon(P') = L(P)$. ■

Сформулируем основной результат теоремы 8.3.1 в виде следующего алгоритма.

Алгоритм 8.3.1 Построение МПε-автомата по МП-автомату

Вход МП-автомат $P = (Q, \Sigma, \Gamma, q_0, Z_0, F)$.

Выход МП-автомат $P' = (Q_1, \Sigma, \Gamma_1, q_1, Z_1, \varnothing)$, такой что $L_\varepsilon(P') = L(P)$.

Метод Конструирование элементов автомата P' по правилам теоремы 8.3.1.

Шаг 1. Положить $Q_1 = Q \cup \{q_\varepsilon, q_1\}$.

Шаг 2. Положить $\Gamma_1 = \Gamma \cup \{Z_1\}$.

Шаг 3. Определить δ_1 следующим образом:

1) $\delta_1(q_1, \varepsilon, Z_1) = \{(q_0, Z_0 Z_1)\}$;

2) если $\delta(q, a, Z) \ni (r, \gamma)$, то $\delta_1(q, a, Z) \ni (r, \gamma)$ для всех $q \in Q, a \in \Sigma \cup \{\varepsilon\}$ и $Z \in \Gamma$;

3) $\delta_1(q, \varepsilon, Z) \ni (q_\varepsilon, \varepsilon)$ для всех $a \in F$ и $Z \in \Gamma_1$;

4) $\delta_1(q_\varepsilon, \varepsilon, Z) = \{(q_\varepsilon, \varepsilon)\}$ для всех $Z \in \Gamma_1$.

Шаг 4. Вернуть МП-автомат $P' = (Q_1, \Sigma, \Gamma_1, \delta_1, q_1, Z_1, \varnothing)$ в качестве результата.

Заметим, что алгоритм 8.3.1 «заворачивает» исходный МП-автомат в автоматную обёртку, цель которой — очистить магазин после перехода исходного автомата в завершающую конфигурацию.

Справедливо обращение теоремы 8.3.1.

Теорема 8.3.2. *Пусть* $P = (Q, \Sigma, \Gamma, \delta, q_0, \Sigma_0, \varnothing)$ *— РМПε-автомат. Тогда можно построить такой МП-автомат* P'*, что* $L(P') = L_\varepsilon(P)$. □

Доказательство. Ввиду теоремы 8.2.1 будем, не теряя общности, полагать, что P — МП-автомат.

Как и при доказательстве теоремы 8.3.1 предположим, что автомат P' моделирует действия автомата P, и выясним, каким требованиям он обязан в этом случае удовлетворять. Введем заключительное состояние q_f и маркер Z' , отмечающий «дно» магазина нового автомата (Z' будет, кроме того, начальным магазинным символом). В тот момент, когда автомат P' может прочесть Z' , он будет переходить в новое заключительное состояние q_f.

Итак, пусть

$$P' = (Q \cup \{q_f; q'\}, \Sigma, \Gamma \cup \{Z'\}, \delta', q', Z', \{q_f\}),$$

где δ' определяется так:

1) $\delta'(q', \varepsilon, Z') = \{(q_0, Z_0, Z')\}$;

2) $\forall q \in Q$, $\forall a \in \Sigma \cup \{\varepsilon\}$, $\forall\ Z \in \Gamma$: $(r, \gamma) \in \delta(q, a, Z) \Rightarrow (r, \gamma) \in \delta'(q, a, Z)$;

3) $\forall q \in Q$: $\delta'(q, \varepsilon, Z') = \{(q_f, \varepsilon)\}$.

Доказательство равенства $L(P') = L_\varepsilon(P)$ не приводим. ■

Упражнение 8.3.1. Доказать равенство $L(P') = L_\varepsilon(P)$ из теоремы 8.3.1. □

Сформулируем основной результат теоремы 8.3.2 в виде следующего алгоритма.

Алгоритм 8.3.2 Построение МП-автомата по МПε-автомату

Вход МПε-автомат $P = (Q, \Sigma, \Gamma, q_0, Z_0, \varnothing)$.

Выход МП-автомат $P' = (Q_1, \Sigma, \Gamma_1, q_1, Z_1, F_1)$, такой что $L_(P') = L\varepsilon(P)$.

Метод Конструирование элементов автомата P' по правилам теоремы 8.3.2.

Шаг 1. Положить $Q_1 = Q \cup \{q_f, q_1\}$.

Шаг 2. Положить $\Gamma_1 = \Gamma \cup \{Z_1\}$.

Шаг 3. Положить $F_1 = \{q_f\}$.

Шаг 4. Определить δ_1 следующим образом:

1) $\delta_1(q_1, \varepsilon, Z_1) = \{(q_0, Z_0 Z_1)\}$;
2) если $\delta(q, a, Z) \ni (r, \gamma)$, то $\delta_1(q, a, Z) \ni (r, \gamma)$ для всех $q \in Q, a \in \Sigma \cup \{\varepsilon\}$ и $Z \in \Gamma$;
3) $\delta_1(q, \varepsilon, Z_1) = \{(q_f, \varepsilon)\}$ для всех $q \in Q$.

Шаг 5. Вернуть МП-автомат $P' = (Q_1, \Sigma, \Gamma_1, \delta_1, q_1, Z_1, F_1)$ в качестве результата.

Как и в случае алгоритма 8.3.1, алгоритм 8.3.2 создаёт над исходным МП-автоматом обёртку, цель которой — перевести результирующий автомат в финальное состояние, когда исходный автомат допустил цепочку опустошением магазина.

8.4. Эквивалентность МП-автоматов и КС-грамматик

Сформулируем теперь один из фундаментальных результатов теории КС-языков, показывающий, что языки, определяемые МП-автоматами, — это в точности КС-языки.

Теорема 8.4.1. *Пусть Σ — конечный алфавит, L — язык над этим алфавитом. Тогда следующие условия эквивалентны:*

1) *$L = L(G)$ для некоторой КС-грамматики G;*
2) *$L = L(P_1)$ для некоторого МП-автомата P_1;*
3) *$L = L(P_2)$ для некоторого РМП-автомата P_2;*
4) *$L = L_\varepsilon(P_3)$ для некоторого МП-автомата P_3.* □

Доказательство. Эквивалентность утверждений 2) и 3) доказана в теореме 8.2.1. Эквивалентность утверждении 3) и 4) вытекает из теорем 8.3.1 и 8.3.2. Ниже будут доказаны еще две теоремы — 8.4.2 и 8.4.3. В силу теоремы 8.4.2 утверждение 4) — следствие утверждения 1), а в силу теоремы 8.4.3 утверждение 1) — следствие утверждения 4). Это завершает доказательство теоремы 8.4.1. ■

Теорема 8.4.2. *Пусть $G = (N, \Sigma, P, S)$ — КС-грамматика. Тогда можно построить такой МП-автомат R, что $L_\varepsilon(R) = L(G)$.* □

Доказательство. Построим МП-автомат R так, чтобы он моделировал все левые выводы в G (см. пункт 6.1). Именно, пусть $R = (\{q\}, \Sigma, N \cup \Sigma, \delta, q, S, \varnothing)$, и функция переходов δ полностью определяется правилами:

(*i*) если $A \to \alpha \in P$, то $(q, \alpha) \in \delta(q, \varepsilon, A)$, где, напомним, $\alpha \in (N \cup \Sigma)^*$;

(*ii*) $\delta(q, a, a) = \{(q, \varepsilon)\}$ для всех $a \in \Sigma$.

Первое правило моделирует продукции грамматики G, а второе — позволяет эти продукции применять.

Перед тем, как проверить равенство $L_\varepsilon(R) = L(G)$, докажем методом математической индукции два вспомогательных утверждения.

A) Для произвольного натурального числа m из возможности вывода $A \Rightarrow^m \omega (\in \Sigma^*)$ вытекает, что $(q, \omega, A) \vdash^+ (q, \varepsilon, \varepsilon)$.

Пусть $A \Rightarrow^* \omega$. Если $m = 1$, $\omega = a_1 \dots a_k$, где $k \geqslant 1$, то из i) и ii) получаем:

$$(q, a_1 \dots a_k, A) \vdash (q, a_1 \dots a_k, a_1 \dots a_k) \vdash^k (q, \varepsilon, \varepsilon).$$

Если же $m = 1$ и $\omega = \varepsilon$, то, как легко видеть, $(q, \varepsilon, A) \vdash (q, \varepsilon, \varepsilon)$.

Теперь предположим, что утверждение верно для $m \leqslant j$ и докажем его для $m = j + 1$. Итак, пусть $A \Rightarrow^{j+1} \omega$. Первый шаг этого вывода должен иметь вид $A \Rightarrow X_1 X_2 \dots X_k$, где $X_i \Rightarrow^* x_i$, $x_i \in \Sigma$ и $x_1 x_2 \dots x_k = \omega_i$. Тогда в силу правила i) $(q, \omega, A) \vdash (q, \omega, X_1 X_2 \dots X_k)$. Если $X_i \in N$, то по предположению индукции $(q, x_i, X_i) \vdash^* (q, \varepsilon, \varepsilon)$. Если же $X_i = x_i \in \Sigma$, то $(q, x_i, X_i) \vdash (q, \varepsilon, \varepsilon)$. Объединяя эти последовательности тактов, получаем: $(q, \omega, A) \vdash^+ (q, \varepsilon, \varepsilon)$.

Таким образом, утверждение A) верно.

B) Для произвольного натурального числа n из существования последовательности тактов $(q, \omega, A) \vdash^n (q, \varepsilon, \varepsilon)$ вытекает, что $A \Rightarrow^+ \omega$.

Пусть $n = 1$. Тогда, как нетрудно убедится, $\omega = \varepsilon$ и $(A \to \varepsilon) \in P$. Таким образом, $A \Rightarrow^+ \omega$.

Теперь предположим, что утверждение верно для $n \leqslant j$ и докажем его для $n = j + 1$. Первый такт, сделанный МП-автоматом R, должен иметь вид $(q, \omega, A) \vdash (q, \omega, X_1, X_2 \dots X_k)$, где $(q, x_i, X_i) \vdash^+ (q, \varepsilon, \varepsilon)$, $x_i \in \Sigma$ и $x_1 x_2 \dots x_k = \omega_i$ (см. упражнение 8.1.2). Тогда $(A \to X_1 \dots X_k) \in P$, а по предположению индукции $X_i \Rightarrow^+ x_i$ для $X_i \in N$ и $X_i \Rightarrow^0 x_1$ при $X_i \in \Sigma$. Таким образом, искомый вывод $A \Rightarrow^+ \omega$ строится так:

$$A \Rightarrow X_i \dots X_k \Rightarrow^* x_1 X_2 \dots X_k \Rightarrow^* x_1 x_2 \dots x_{k-1} X_k \Rightarrow^* x_1 x_2 \dots x_{k-1} x_k = \omega.$$

Итак, утверждение B) верно.

Из утверждений A) и B) вытекает, что $S \Rightarrow^+ \omega$ тогда и только тогда, когда $(q, \omega, S) \vdash^+ (q, \varepsilon, \varepsilon)$. Следовательно, $L_\varepsilon(R) = L(G)$. ■

Сформулируем результаты теоремы 8.4.2 в виде следующего алгоритма.

Алгоритм 8.4.1 Построение МПε-автомата по КС-грамматике.

Вход КС-грамматика $G = (N, \Sigma, P, S)$.

Выход МПε-автомат $P = (Q, \Sigma, \Gamma, q, Z, \varnothing)$, такой что $L\varepsilon(P) = L(G)$.

Метод Конструирование элементов автомата P по правилам теоремы 8.4.2.

Шаг 1. Положить $Q = \{q\}$.

Шаг 2. Положить $\Gamma = N \cup \Sigma$.

Шаг 3. Определить δ следующим образом:

1) если $A \to \alpha \in P$, то $\delta(q, \varepsilon, A) \ni \{(q, \alpha)\}$, для всех $A \in N$ и $\alpha \in (N + \Sigma)^*$;

2) $\delta(q, a, a) = \{(q, \varepsilon)\}$ для всех $a \in \Sigma$.

Шаг 4. Вернуть МП-автомат $P = (Q, \Sigma, \Gamma, \delta, q, S, \varnothing)$ в качестве результата.

Пример 8.4.1. Рассмотрим КС-грамматику $G = (\{S\}, \{a, b\}, P, S)$, заданную множеством $P = \{ S \to aSbb \mid \varepsilon\}$. После применения алгоритма 8.4.1 получим автомат $P = (\{q\}, \{a, b\}, \{a, b, S\}, \delta, q, S, \varnothing)$, где функция переходов δ определяется следующим образом:

$$\delta(q, \varepsilon, S) = \{(q, aSbb), (q, \varepsilon)\};$$
$$\delta(q, a, a) = \{(q, \varepsilon)\};$$
$$\delta(q, b, b) = \{(q, \varepsilon)\}.$$

■

Из теоремы 8.3.2 следует, что если у нас есть МПε-автомат, то по нему всегда можно получить МП-автомат, распознающий тот же язык. Без обосновывающей теоремы приведём алгоритм, который строит РМП-автомат, моделирующий все правые выводы в заданной КС-грамматике.

Алгоритм 8.4.2 Построение РМП-автомата по КС-грамматике.

Вход КС-грамматика $G = (N, \Sigma, P, S)$.

Выход РМП-автомат $P = (Q, \Sigma, \Gamma, q, \$, F)$, такой что $L(P) = L(G)$.

Метод

Шаг 1. Положить $Q = \{q, q_f\}$.

Шаг 2. Положить $\Gamma = N \cup \Sigma \cup \{\$\}$.

Шаг 3. Положить $F = \{q_f\}$.

Шаг 4. Определить δ следующим образом:

1) $\delta(q, a, \varepsilon) = \{(q, a)\}$ для всех $a \in \Sigma$;
2) если $A \to \alpha \in P$, то $\delta(q, \varepsilon, \alpha^R) \ni \{(q, A)\}$, для всех $A \in N$ и $\alpha \in (N + \Sigma)^*$;
3) $\delta(q, \varepsilon, S\$) = \{(q_f, \varepsilon)\}$.

Шаг 5. Вернуть МП-автомат $P = (Q, \Sigma, \Gamma, \delta, q, \$, F)$ в качестве результата.

Пример 8.4.2. Рассмотрим КС-грамматику $G = (\{S\}, \{a, b\}, P, S)$, заданную множеством $P = \{ S \to aSbb \mid \varepsilon\}$. После применения алгоритма 8.4.2 получим автомат $P = (\{q, q_f\}, \{a, b\}, \{a, b, S, \$\}, \delta, q, \$, \{q_f\})$, где функция переходов δ определяется следующим образом:

$$\begin{aligned}
\delta(q, a, \varepsilon) &= \{(q, a)\};\\
\delta(q, b, \varepsilon) &= \{(q, b)\};\\
\delta(q, \varepsilon, \varepsilon) &= \{(q, S)\};\\
\delta(q, \varepsilon, aSbb) &= \{(q, S)\};\\
\delta(q, \varepsilon, S\$) &= \{(q_f, \varepsilon)\}.
\end{aligned}$$

■

Покажем теперь, что язык, определяемый МП-автоматом, контекстно-свободен.

Теорема 8.4.3. *Пусть* $R = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$ *— МП-автомат. Можно построить такую КС-грамматику* G*, что* $L(G) = L_\varepsilon(R)$. □

Доказательство. Начнем строить грамматику $G = (N, \Sigma, P, S)$. Нетерминальные символы будем записывать в виде $[qZr]$, где $q, r \in Q$ и $Z \in \Gamma$, то есть определим множество нетерминалов равенством

$$N = \{[qZr] \mid q, r \in Q, Z \in \Gamma\} \cup \{S\}.$$

Множество продукций зададим следующими условиями:

1) если $(r, X_1 \dots X_k) \in \delta(q, a, Z)$, где $k \geqslant 1$, то для каждой последовательности $s_1, s_2, \dots, s_k$ состояний из Q отнесем к P все продукции вида

$$[qZs_k] \to a[rX_1s_1][s_1X_2s_2] \dots [s_{k-1}X_ks_k];$$

2) если $(r, \varepsilon) \in \delta(q, a, Z)$, то отнесем к P продукцию $[qZr] \to a$;

3) для каждого $q \in Q$ отнесем к P продукцию $S \to [q_0\Sigma_0q]$.

Индукцией по числу продукций и числу тактов доказывается следующее вспомогательное утверждению: для любых $q, r \in Q$ и $Z \in \Gamma$ $[qZr] \Rightarrow^+ \omega$ тогда и только тогда, когда $(q, \omega, Z) \vdash^+ (r, \varepsilon, \varepsilon)$. Из этого утверждения следует, что $S \Rightarrow [q_0Z_0q] \Rightarrow^+ \omega$ тогда и только тогда, когда $(q_0, \omega, Z_0) \vdash^+ (q, \varepsilon, \varepsilon)$ для $q \in Q$. Таким образом. $L_\varepsilon(R) = L(G)$. ■

На базе теоремы 8.4.3 сформулируем следующий алгоритм.

Алгоритм 8.4.3 Построение КС-грамматики по МП-автомату.

Вход МП-автомат $R = (Q, \Sigma, \Gamma, q, S, F)$.

Выход КС-грамматика $G = (N, \Sigma, P, S)$, такая что $L(G) = L_\varepsilon(R)$.

Метод Конструирование элементов грамматики G по правилам теоремы 8.4.2.

Шаг 1. Положить $N = \{[qZr] \mid q, r \in Q, Z \in \Gamma\} \cup \{S\}$.

Шаг 2. Положить $P = \varnothing$.

Шаг 3. Если $\delta(q, a, Z) \ni (r, X_1 \dots X_k)$, где $k \geqslant 1, \quad q, r \in Q, \quad a \in \Sigma, \quad Z, X_1 \dots X_k \in \Gamma$, то включить в P все правила вида

$$[qZs_k] \to a[rX_1s_1][s_1X_2s_2] \dots [s_{k-1}X_ks_k]$$

для каждой последовательности $s_1, s_2, \dots, s_k$ состояний из Q.

Шаг 4. Если $\delta(q, a, Z) \ni (r, \varepsilon)$, где $q, r \in Q, \quad a \in \Sigma, \quad Z \in \Gamma$, то включить в P правило $[qZr] \to a$.

Шаг 5. Включить в P правила $S \to [q_0Z_0q]$ для каждого $q \in Q$.

Шаг 6. Вернуть КС-грамматику $G = (N, \Sigma, P, S)$ в качестве результата.

Пример 8.4.3. Рассмотрим МП-автомат $R = (\{q_0, q_1, q_2\}, \{a, b\}, \{a, b, Z_0\}, q_0, Z_0, \{q_0\})$, функция переходов δ которого определяется следующим образом:

$$\delta(q_0, a, Z_0) = \{(q_1, aaZ_0)\};$$
$$\delta(q_1, a, a) = \{(q_1, aaa)\};$$
$$\delta(q_1, b, a) = \{(q_2, \varepsilon)\};$$
$$\delta(q_2, b, a) = \{(q_2, \varepsilon)\};$$
$$\delta(q_2, \varepsilon, Z_0) = \{(q_0, \varepsilon)\}.$$

После применения алгоритма 8.4.3 получим следующие элементы результирующей грамматики $G = (N, \{a, b\}, P, S)$.

Множество нетерминалов $N = \{[qZr] \mid q, r \in \{q_0, q_1, q_2\}, Z \in \{a, Z_0\} \cup \{S\}\}$. Множество продукций P без бесполезных символов:

$$
\begin{aligned}
S &\to [q_0 Z_0 q_0];\\
[q_0 Z_0 q_0] &\to a[q_1 a q_2][q_2 a q_2][q_2 Z_0 q_0];\\
[q_1 a q_2] &\to a[q_1 a q_2][q_2 a q_2][q_2 a q_2];\\
[q_1 a q_2] &\to b;\\
[q_2 a q_2] &\to b;\\
[q_2 Z_0 q_0] &\to \varepsilon.
\end{aligned}
$$

После переименования множество продукций P имеет следующий вид:

$$
\begin{aligned}
&S \to aABC;\\
&A \to aABB \mid b;\\
&B \to b;\\
&C \to \varepsilon.
\end{aligned}
$$

■

Упростим правила грамматики и получим $P = \{S \to aAb; A \to aAbb \mid b\}$.

В результате искомая грамматика имеет вид $G = (\{S, A\}, \{a, b\}, P, S)$, у которой $L(G) = \{a^n b^{2n} \mid n \geqslant 1\}$.

8.5. Детерминированный МП-автомат

Выше отмечалось, что для каждой КС-грамматики G можно построить МП-автомат, распознающий $L(G)$ (теорема 8.4.2). Однако построенный автомат был недетерминированный, а в приложениях более удобны детерминированные МП-автоматы, которые в каждой конфигурации могут сделать не более одного очередного такта.

Дадим точное определение: МП-автомат $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$ называется *детерминированным* (сокращенно ДМП-автоматом), если для каждых $q \in Q$ и $\Sigma \in \Gamma$ либо $\delta(q, a, Z)$ содержит не более одного элемента для каждого $a \in \Sigma$ и $(q, \varepsilon, Z) = \varnothing$, либо $\delta(q, a, Z) = \varnothing$ для всех $a \in \Sigma$ и $\delta(q, \varepsilon, Z)$ содержит не более одного элемента. В силу этих двух ограничений ДМП-автомат в любой конфигурации может выбрать не более одного такта.

В теореме 3.2.1 было показано, что класс языков, определяемых недетерминированными конечными автоматами, совпадает с классом языков, определяемых полностью определенными детерминированными конечными автоматами. Но, к сожалению, ДМП-автоматы не так мощны по своей распознавательной способности, как недетерминированные МП-автоматы, и существуют КС-языки, которые нельзя определить детерминированными МП-автоматами. При этом для более сильного вычислительного формализма (машин Тьюринга) снова справедлива эквивалентность детерминированной и недетерминированной версии.

ДМП-автоматные языки интересно соотносятся с уже известными классами языков. Укажем ещё два факта об этих отношениях, кроме упомянутого выше строгого включения в класс МП-автоматных языков.

1) Хотя исключить недетерминизм без уменьшения вычислительной мощности в случае МП-автоматов не удаётся, можно избавиться от ε-переходов, не меняя класса распознаваемых языков. Этот факт получается, если адаптировать доказательство теоремы 8.4.2 к КС-грамматике, которая находится в нормальной форме Грейбах. Подумайте, как реализовать эту идею.

2) ДМП-автоматные языки содержат в себе класс регулярных языков как собственное подмножество.

3) Все ДМП-автоматные языки не являются существенно неоднозначными, то есть для них существуют КС-грамматики без неоднозначности. В то же время, существуют языки без неоднозначности, которые не являются ДМП-автоматными. К последним относится, например, язык палиндромов четной длины L_{ww^r}.

8.6. Упражнения

Для каждого из следующих языков

1) $\{a^n b^n c^m d^m \mid n, m \in \mathbb{N}\}$,

2) $\{a^i b^j c^j d^i \mid i, j \in \mathbb{N}\}$,

3) $\{a^i b^j c^k \mid i, j, k \in \mathbb{N}$ и $i + j = k\}$,

4) $\{x \in \{a, b, c\}^* \mid |x|_a + |x|_b = |x|_c\}$

построить МП-автомат,

а) распознающий L,

б) распознающий L опустошением магазина.

Напомним, что запись вида $|w|_z$ (см. язык 4) означает количество вхождений символа z в строку w.

СПИСОК ЛИТЕРАТУРЫ

[1] *А. Ахо, Дж. Ульман.* Теория синтаксического анализа, перевода и компиляции. Т. 1. Синтаксический анализ. М.: Мир, 1978.

[2] *А. Ахо, М. Лам, Р. Сети, Дж. Ульман.* Компиляторы: принципы, технологии и инструментарий, 2-е изд.: Пер. с англ. М.: Вильямс, 2008.

[3] *Дж. Хопкрофт, Р. Мотвани, Дж. Ульман.* Введение в теорию автоматов, языков и вычислений. — 2-е изд. М.: Вильямс, 2008.

[4] *Ф. Л. Бауэр, Г. Гооз.* Информатика. М.: Мир, 1990.

[5] *А. Саломаа.* Жемчужины теории формальных языков. М.: Мир, 1986.

[6] *Ч. Хоар.* Взаимодействующие последовательные процессы. М.: Мир, 1989.

[7] *А. И. Белоусов, С. Б. Ткачёв* Дискретная математика. М.: МГТУ, 2010.

[8] *Ю. Громкович.* Теоретическая информатика. Введение в теорию автоматов, теорию вычислимости, теорию сложности, теорию алгоритмов, рандомизацию, теорию связи и криптографию. СПб: БХВ-Петербург, 2010.

ПРИЛОЖЕНИЕ А

АЛГОРИТМЫ ДЛЯ КОНТЕКСТНО-СВОБОДНЫХ ГРАММАТИК

В этом приложении алгоритмы, рассмотренные в главах 6 и 7, приведены в виде псевдокода, что может облегчить программирование, а иногда и понимание этих алгоритмов.

Некоторые обозначения, используемые при описании алгоритмов.

- $A \leftarrow B$ — операция присваивания («A присвоить B»);
- $A \leftarrow a$ — операция добавления элемента в множество («добавить в множество A элемент a»);
- $A \leftarrow B$ — операция добавления множества элементов во множество («добавить в множество A все элементы из B»);
- $\triangleright$ — начало комментария, продолжающегося до конца строки.

Алгоритм А.1 Нахождение порождающих символов

Вход КС-грамматика $G = (\Sigma, N, \mathcal{P}, S \in N)$.

Выход $\mathrm{Gen}_G(\Sigma)$ — множество порождающих в G символов.

---

1 $\mathrm{Gen}_G(\Sigma) \leftarrow \Sigma$ $\triangleright$ Терминалы являются порождающими символами
$\triangleright$ Ищем продукции, в правых частях которых только порождающие символы…

2 **while** $\exists A \to X_1 \dots X_n \in \mathcal{P}$, $n \geqslant 0$, такое что $\forall i\ X_i \in \mathrm{Gen}_G(\Sigma)$
do $\mathrm{Gen}_G(\Sigma) \leftarrow A$ $\triangleright$ левые части таких продукций добавляются в $\mathrm{Gen}_G(\Sigma)$

Замечание (о многократном просмотре продукций). В цикле на шаге 2 алгоритма А.1 одна и та же продукция $A \to X_1 \dots X_n$ может быть просмотрена несколько раз, причём если на ранних итерациях она не удовлетворяла условию $\forall i\ X_i \in \mathrm{Gen}_G(\Sigma)$, то на поздних, когда множество $\mathrm{Gen}_G(\Sigma)$ станет доста-

точно большим, положение дел может измениться (условие $\forall i\ X_i \in \mathrm{Gen}_G(\Sigma)$ выполнится и A надо будет добавить в $\mathrm{Gen}_G(\Sigma)$).

Аналогичное справедливо для просмотра продукций в алгоритме А.2. □

Алгоритм А.2 Нахождение достижимых символов

Вход КС-грамматика $G = (\Sigma, N, \mathcal{P}, S \in N)$.

Выход Reach_G — множество достижимых в G символов.

1 $\mathrm{Reach}_G \leftarrow \{S\}$ ▷ S достижим в G

▷ Ищем продукции, в левых частях которых стоит достижимый символ...

2 **while** $\exists A \to X_1 \dots X_n \in \mathcal{P}$, такое что $A \in \mathrm{Reach}_G$

▷ ...символы из правых частей таких продукций добавляются в Reach_G:

do $\mathrm{Reach}_G \leftarrow \{X_i\}_{i=1}^{n}$

Алгоритм А.3 Удаление бесполезных символов

Вход КС-грамматика $G = (\Sigma, N, \mathcal{P}, S \in N)$.

Выход КС-грамматика $G' = (\Sigma', N', \mathcal{P}', S \in N')$, не содержащая бесполезных символов, такая что $L(G') = L(G)$, либо сигнал о том, что язык исходной грамматики пуст и не существует эквивалентной G грамматики без бесполезных символов.

Шаг 1. Построить множество $\mathrm{Gen}_G(\Sigma)$, используя алгоритм А.1. Если $S \notin \mathrm{Gen}_G(\Sigma)$ — завершение алгоритма, сообщение о пустоте языка исходной грамматики. Иначе удалить из G все символы, не вошедшие в $\mathrm{Gen}_G(\Sigma)$.

Шаг 2. Построить множество Reach_G, используя алгоритм А.2. Удалить из G все символы, не вошедшие в Reach_G. Получившуюся грамматику обозначить G' и подать её на выход алгоритма.

Замечание (об операции удаления символа из грамматики). Когда в алгоритме требуется удалить символ X из грамматики G, необходимо не только исключить его из множества N или Σ, но и *удалить из $\mathcal{P}$ все продукции, в которых он участвует*. □

Замечание (о методе «устранения перегородок» в алгоритме А.4, с. 146). На шаге 2 алгоритма А.4 каждая продукция $\mathcal{P}$ просматривается ровно один раз.

Алгоритм А.4 Удаление ε-правил

Вход КС-грамматика $G = (\Sigma, N, \mathcal{P}, S \in N)$.

Выход КС-грамматика $G' = (\Sigma, N, \mathcal{P}', S \in N)$, без ε-правил (продукций вида $A \to \varepsilon$), такая что $L(G') = L(G) \setminus \{\varepsilon\}$.

Метод «Устранение перегородок».

Шаг 1. Построить множество $\mathrm{Gen}_G(\varepsilon) \subset N$ всех порождающих ε нетерминалов, используя следующую процедуру:

1 **for** $A \to \varepsilon \in \mathcal{P}$
  do $\mathrm{Gen}_G(\varepsilon) \leftarrow A$
2 **while** $\exists A \to X_1 \dots X_n \in \mathcal{P}$, где $\{X_i\}_{i=0}^n \subset \mathrm{Gen}_G(\varepsilon)$
  do $\mathrm{Gen}_G(\varepsilon) \leftarrow A$

Шаг 2. Выполнить следующие действия:

for $A \to \alpha_0 B_1 \alpha_1 \dots B_n \alpha_n \in \mathcal{P}$, где $\forall i\ B_i \in \mathrm{Gen}_G(\varepsilon) \wedge\ \alpha_i \in ((N \cup \Sigma) \setminus \mathrm{Gen}_G(\varepsilon))^*$
  do $\mathcal{P} \leftarrow \{\alpha_0 X_1 \alpha_1 \dots X_n \alpha_n \mid \forall i\ X_i = \varepsilon \vee X_i = B_i\}$

Шаг 3. Удалить из $\mathcal{P}$ все ε-правила, обозначить получившееся множество правил $\mathcal{P}'$ и подать на выход алгоритма грамматику $G' = (N, \Sigma, \mathcal{P}', S)$.

Понятно, что для подходящего n любая продукция может быть представлена в виде $A \to \alpha_0 B_1 \alpha_1 \dots B_n \alpha_n$ с указанными условиями для B_i и α_i. Например, если в продукции нет ε-порождающих символов, то это продукция вида $A \to \alpha_0$ и для неё нет необходимости добавлять новые продукции.

Пример. Продукция $C \to aDbD$, где $D \in \mathrm{Gen}_G(\varepsilon)$, $a, b \in \Sigma$, это продукция вида $A \to \alpha_0 B_1 \alpha_1 B_2 \alpha_2$, где $A = C$, $\alpha_0 = a$, $B_1 = B_2 = D$, $\alpha_2 = \varepsilon$, и для неё нужно добавить три новых продукции. Укажем их, пояснив название метода «удаления перегородок». Можно считать, что ε-порождающие символы B_i являются «перегородками» в продукции $A \to \alpha_0 B_1 \alpha_1 \dots B_n \alpha_n$ и новые продукции получаются из данной при помощи устранения этих перегородок *всеми возможными способами*. Для продукции $C \to aDbD$ это: $C \to abD$, $A \to aDb$ и $A \to ab$. Исходная продукция остаётся в множестве $\mathcal{P}$. □

Замечание (о введении новых нетерминалов в алгоритме А.6, с. 148). Отметим, что на каждой итерации цикла шага 4, если есть необходимость ввести новые нетерминалы, то они должны отличаться не только от тех, которые присутствовали в грамматике до начала этого цикла, но и от тех, ко-

Алгоритм А.5 Удаление цепных продукций

Вход КС-грамматика $G = (\Sigma, N, \mathcal{P}, S \in N)$, не содержащая ε-правил.

Выход КС-грамматика $G' = (\Sigma, N, \mathcal{P}', S \in N)$, без цепных правил (продукций вида $A \to B$), такая что $L(G') = L(G)$.

Шаг 1. Для каждого нетерминала $A \in N$ построить множество циклически достижимых из A нетерминалов $C(A)$, используя процедуру:

1 $C(A) \leftarrow \{A\}$
2 **while** $\exists D \to E \in \mathcal{P}$, такая что $D \in C(A)$
  do $C(A) \leftarrow E$

Шаг 2. Выполнить следующую процедуру:

for $A \in N$
  do for $B \to \alpha \in \mathcal{P}$
    do if $B \in C(A)$
      then $\mathcal{P} \leftarrow A \to \alpha$

Шаг 3. Удалить из $\mathcal{P}$ все цепные правила, обозначить получившееся множество правил $\mathcal{P}'$ и подать на выход алгоритма грамматику $G' = (N, \Sigma, \mathcal{P}', S)$.

торые были введены на предыдущих итерациях этого цикла. Таким образом, одного комплекта букв $\{C_i\}$ для выполнения цикла может не хватить. Для борьбы с нехваткой букв можно вводить нетерминалы, помеченные частями исходного слова $B_1 \dots B_n$, которое «разбивается на слоги». Например, вместо набора $\{C_i\}_{i=1}^{n-2}$ можно использовать набор $\{\langle B_{i+1} \dots B_n \rangle\}_{i=1}^{n-2}$, где для каждого i выражение $\langle B_{i+1} \dots B_n \rangle$ понимается как *один новый* нетерминал. Аналогично можно поступать на шаге 3, добавляя для рассматриваемого терминала a новый нетерминал $\langle a \rangle$, а не A'. □

Замечание (о табличной форме алгоритма А.7, с. 149). Алгоритм удобно выполнять, заполняя таблицу с N_{ij} в ячейках. Ячейки таблицы расположены в системе координат (i, s), в позиции (i, s) находится множество $N_{i,i+s-1}$. Подробно этот процесс описан в разделе 7.2. □

Алгоритм А.6 Приведение к нормальной форме Хомского

Вход КС-грамматика $G = (\Sigma, N, \mathcal{P}, S \in N)$, не содержащая ε-правил.

Выход КС-грамматика $G' = (\Sigma', N', \mathcal{P}', S \in N)$ в НФХ, такая что $L(G') = L(G) \setminus \{\varepsilon\}$ или сообщение о пустоте языка грамматики.

Метод «Разбиение слов на слоги».

Шаг 1. Последовательно воспользоваться алгоритмами А.4 и А.5, полученную грамматику обозначить $G'' = (N, \Sigma, \mathcal{P}'', S)$.

Шаг 2. Применить к G'' алгоритм А.3. Если получен ответ $L(G'') = \varnothing$, остановить алгоритм и подать на выход сигнал о пустоте языка исходной грамматики. Иначе получена грамматика $G' = (N', \Sigma', \mathcal{P}', S)$.

Шаг 3. К G' применить следующую процедуру:

for $a \in \Sigma$

do if $\exists A \to X_1 \dots X_n \in \mathcal{P}''$, такая что $n > 1 \wedge \exists i\ X_i = a$

then

1 добавить в N *новый* нетерминал A'

2 заменить a на A' в продукциях с правой частью длиннее 1

3 $\mathcal{P}'' \leftarrow A' \to a$

Шаг 4. К G' применить следующую процедуру:

for $A \to B_1 B_2 \dots B_n \in \mathcal{P}'$, где $n > 2$, $B_i \in N$

do

1 добавить в N *новые* нетерминалы $C_1, \dots C_{n-2}$

2 $\mathcal{P}' \leftarrow \{A \to B_1 C_1\} \cup \{C_i \to B_{i+1} C_{i+1}\}_{i=1}^{n-3} \cup \{C_{n-2} \to B_{n-1} B_n\}$

3 удалить $A \to B_1 B_2 \dots B_n$ из $\mathcal{P}'$

Подать G' на выход алгоритма.

Алгоритм А.7 Решение проблемы принадлежности для КС-языков, CYK-алгоритм

Вход Грамматика $G = (\Sigma, N, \mathcal{P}, S \in N)$ в НФХ, слово $w = w_1 \dots w_n \in \Sigma^*$.

Выход Истина, если $w \in L(G)$, ложь в противном случае.

Метод Последовательное определение нетерминалов, выводящих всевозможные подстроки w всё большей длины.

Шаг 1. Построить множества N_{ij}, используя процедуру:

for $i \leftarrow 1$ **to** n
  do $N_{ii} \leftarrow \{A \in N \mid A \to w_i \in \mathcal{P}\}$ ▷ Подстроки w длины 1
for $s \leftarrow 2$ **to** n ▷ Цикл по длине подстроки
  do for $i \leftarrow 1$ **to** $n - s + 1$ ▷ Цикл по месту начала подстроки
    $j \leftarrow i + s - 1$ ▷ Позиция конца подстроки с началом в w_i длины s
    $N_{ij} \leftarrow \{A \in N \mid A \to BC \in \mathcal{P};\ \exists k \in [i, j-1]_{\mathbb{Z}} : B \in N_{ik},\ C \in N_{k+1j}\}$

Шаг 2. Если $S \in N_{1n}$, то подать на выход алгоритма «да», иначе — «нет».

ПРИЛОЖЕНИЕ В

ЗАДАНИЕ К КУРСОВОЙ РАБОТЕ

1. Для языка L, выбранного в соответствии с вариантом, выполнить:

   (i) Построить ПЛ-грамматику G, порождающую L;

   (ii) Доказать вложения $L \subseteq L(G)$, $L(G) \subseteq L$;

   (iii) Путем решения системы линейных уравнений с регулярными коэффициентами построить регулярное выражение, описывающее L;

   (iv) Построить НКА (ε-НКА) M^{ND}, распознающий язык L, предъявить его граф;

   (v) Построить ДКА M^D путем детерминизации M^{ND}, предъявить его граф;

   (vi) Доказать, что $L(M^{ND}) = L(M^D) = L$;

   (vii) Минимизировать полученный конечный автомат, распознающий язык L, или доказать его минимальность;

   (viii) Методом последовательного исключения состояний выписать регулярные выражения для $L(M^{ND})$, $L(M^D)$;

   (ix) Построить ДКА $\overline{M^D}$, распознающий дополнение $\overline{L}$ к языку L, записать $\overline{L}$ в виде регулярного выражения;

   (x) На произвольно выбранном языке программирования написать программу, позволяющую распознать принадлежность вводимой строки языку L.

2. Для множества слов A над алфавитом Σ, заданных в соответствии с вариантом, выполнить:

   (i) Для каждого слова $w_i \in A$ построить НКА M_i^{ND}, распознающий наличие в произвольной строке $s \in \Sigma^*$ подстроки w_i;

(ii) Для каждого НКА M_i^{ND} построить соответствующий ДКА M_i^D;

(iii) На произвольно выбранном языке программирования написать программу, позволяющую подсчитать количество вхождений каждого слова из множества A во входную строку $s \in \Sigma^*$ (при этом программно реализовать хотя бы один автомат M_i^{ND} и хотя бы один автомат M_j^D).

3. Для языков L_1, L_2, выбранных в соответствии с вариантом, выполнить:

(i) Вычислить регулярное выражение, определяющее язык $L_1 \cap L_2$;

(ii) Вычислить регулярное выражение, определяющее язык $L_1 \Delta L_2$;

(iii) Определить: совпадают ли языки L_1 и L_2, является ли L_1 дополнением L_2;

(iv) Построить ε-НКА, распознающий один из языков: L_1^R или L_2^R;

(v) Вычислить регулярное выражение по построенному ε-НКА;

(vi) Построить ε-НКА, распознающий один из языков: $L_1 L_2$, $L_1 \cup L_2$ или L_1^*;

(vii) Детерминизировать один из построенных ε-НКА;

(viii) На произвольно выбранном языке программирования написать программу, реализующую какой-либо ε-НКА и какой-либо ДКА из этого пункта.

ПРИЛОЖЕНИЕ С

ВАРИАНТЫ ЗАДАНИЙ

Задание 1

1) Язык над алфавитом $\Sigma = \{0, 1\}$, состоящий из всех слов, в которых единицы встречаются только четное число раз, а первая сдвоенная пара нулей может появиться только после появления двух единиц.

2) Язык над алфавитом $\Sigma = \{0, 1\}$, состоящий из всех слов, в которых единицы встречаются только нечетное число раз, а после последней сдвоенной пары нулей может появиться не более трех единиц.

3) Язык над алфавитом $\Sigma = \{0, 1\}$, состоящий из всех слов, в которых после второй сдвоенной пары нулей количество единиц четно.

4) Язык над алфавитом $\Sigma = \{0, 1\}$, состоящий из всех слов, в которых до второй сдвоенной пары нулей количество единиц четно.

5) Язык над алфавитом $\Sigma = \{0, 1\}$, состоящий из всех слов, в которых, если в слове содержится три последовательные буквы, то оно содержит хотя бы два нуля.

6) Язык над алфавитом $\Sigma = \{0, 1\}$, состоящий из всех слов, в которых число единиц делится на 2, а число нулей – на 3.

7) Язык над алфавитом $\Sigma = \{0, 1\}$, состоящий из всех слов, которые начинаются или оканчиваются (или и то, и другое) последовательностью 01.

8) Язык над алфавитом $\Sigma = \{0, 1\}$, состоящий из всех слов, в которых хотя бы на одной из последних трёх позиций стоит 1, и не встречается более двух 0 подряд.

9) Язык над алфавитом $\Sigma = \{0, 1\}$, состоящий либо из повторяющихся один или несколько раз фрагментов 01, либо повторяющихся один или несколь-

ко раз фрагментов 010, либо всех слов четной длины и содержащих хотя бы одно подслово 00.

10) Язык над алфавитом $\Sigma = \{0, 1\}$, состоящий из всех слов, в которых если встречается три подряд идущих нуля, то слово имеет чётную длину, а если встречается две подряд идущие единицы, то длина слова нечётная.

11) Язык над алфавитом $\Sigma = \{0, 1\}$, состоящий из всех слов, в которых если встречается хотя бы одна пара 01, то пара 10 должна встретиться не менее двух раз.

12) Язык над алфавитом $\Sigma = \{0, 1\}$, состоящий из всех слов, в которых если слово содержит пару 10, то оно имеет четную длину и должно обязательно заканчиваться двумя подряд идущими единицами.

13) Язык над алфавитом $\Sigma = \{0, 1\}$, состоящий из всех слов, в которых не встречаются подслова 101 и 110 одновременно.

14) Язык над алфавитом $\Sigma = \{0, 1\}$, состоящий из всех слов, в которых четное число единиц может быть тогда и только тогда, когда слово содержит послово 101.

15) Язык над алфавитом $\Sigma = \{0, 1\}$, состоящий из всех слов, которые имеют четную длину тогда и только тогда, когда они содержат подслово 011.

16) Язык над алфавитом $\Sigma = \{0, 1\}$, состоящий из всех слов, в которых количество единиц четно, а пара сдвоенных нулей встречается не более трех раз.

17) Язык над алфавитом $\Sigma = \{0, 1\}$, состоящий из всех слов, содержащих не более трёх подряд идущих нулей, и количество единиц в которых четно.

18) Язык над алфавитом $\Sigma = \{0, 1\}$, состоящий из всех слов, в которых после третьей слева единицы стоит четное число групп вида 01.

19) Язык над алфавитом $\Sigma = \{0, 1\}$, состоящий из всех слов, в которых количество нулей четно, а первая сдвоенная пара единиц появляется не раньше группы строенных нулей.

20) Язык над алфавитом $\Sigma = \{0, 1\}$, состоящий из всех слов, содержащих нечетное количество нулей и единиц, начинаться и заканчиваться слова могут не более чем тремя подряд идущими нулями.

21) Язык над алфавитом $\Sigma = \{0, 1\}$, состоящий из всех слов, не содержащих двух смежных нулей, и в которых группа цифр 010 повторяется не более одного раза.
22) Язык над алфавитом $\Sigma = \{0, 1\}$, состоящий из всех слов, содержащих три смежные единицы, и в которых группа вида 101 содержится не более одного раза.
23) Язык над алфавитом $\Sigma = \{0, 1\}$, состоящий из всех слов, в которых между двумя парами единиц может находиться не более чем три подряд идущих нуля.
24) Язык над алфавитом $\Sigma = \{0, 1\}$, состоящий из всех слов, в которых после группы 010 следует чётное количество подряд идущих единиц.
25) Язык над алфавитом $\Sigma = \{0, 1\}$, состоящий из всех слов, в которых перед тремя подряд идущими нулями находится четное количество единиц.
26) Язык над алфавитом $\Sigma = \{0, 1\}$, состоящий из всех слов, таких что если слово начинается с нуля, то количество единиц в слове четно, а если с единицы, то нечетно.
27) Язык над алфавитом $\Sigma = \{0, 1\}$, состоящий из всех слов, в которых если встречается хотя бы одна группа из трех подряд идущих нулей, то пара сдвоенных единиц должна появиться не менее двух раз.
28) Язык над алфавитом $\Sigma = \{0, 1\}$, состоящий из всех слов, в которых если слово содержит группу цифр 010, то оно имеет нечетную длину.
29) Язык над алфавитом $\Sigma = \{0, 1\}$, состоящий из всех слов, в которых группа строенных нулей и строенных единиц не встречаются одновременно.
30) Язык над алфавитом $\Sigma = \{0, 1\}$, состоящий из всех слов, имеющих четную длину тогда и только тогда, когда они начинаются тремя нулями, а заканчиваются тремя единицами.

Задание 2

1) $\Sigma = \{0, 1, 2\}$, $A = \{01110, 1011, 11100, 010101\}$.
2) $\Sigma = \{0, 1, 2\}$, $A = \{110, 1110, 0000, 0111\}$.
3) $\Sigma = \{0, 1, 2\}$, $A = \{1011, 111, 0110, 10011\}$.
4) $\Sigma = \{a, b, c\}$, $A = \{bbab, ccac, bacb, baabc\}$.
5) $\Sigma = \{a, b, f\}$, $A = \{faafb, ffba, bfab, affa\}$.

6) $\Sigma = \{1, 2, 3\}$, $A = \{2112, 1233, 1322, 131313\}$.

7) $\Sigma = \{4, 5, 6\}$, $A = \{5555, 4665, 666, 4455, 665\}$.

8) $\Sigma = \{9, 10, 11, 12\}$, $A = \{9101112, 101111, 121110, 121211\}$.

9) $\Sigma = \{0, 1, 2, 3, 4, 5\}$, $A = \{22231, 1213, 0001, 11101\}$.

10) $\Sigma = \{0, 1, 2, 3, 4, 5\}$, $A = \{000, 010, 221, 3334, 0110\}$.

11) $\Sigma = \{a, b, c, d, e\}$, $A = \{aaac, bbd, bbe, abbc\}$.

12) $\Sigma = \{a, b, c, d, e\}$, $A = \{aaaa, bbbba, bbaa, cccac\}$.

13) $\Sigma = \{x, y, v, w, z\}$, $A = \{xyz, xyzz, zzzw, xyyw\}$.

14) $\Sigma = \{0, 1, 2, 3, 4, 5\}$, $A = \{01112, 32544, 12124, 5551\}$.

15) $\Sigma = \{a, b, w, z\}$, $A = \{wwwz, bbbww, zzbz, wzwzw\}$.

16) $\Sigma = \{0, 1, 2\}$, $A = \{010100, 111001, 110, 1110\}$.

17) $\Sigma = \{0, 1, 2\}$, $A = \{001100, 111, 0101, 1000\}$.

18) $\Sigma = \{0, 1, 2\}$, $A = \{1011, 0000, 1111, 1011101\}$.

19) $\Sigma = \{0, 1, 2\}$, $A = \{011100, 11101, 1010, 111\}$.

20) $\Sigma = \{x, y, z\}$, $A = \{xxyy, xxxy, xyxyyy, xyxyyx\}$.

21) $\Sigma = \{x, y, z\}$, $A = \{xy, yxxy, yxyxxy, xyxxx\}$.

22) $\Sigma = \{x, y, z\}$, $A = \{xyx, xxy, yyyxxy, xyxyx\}$.

23) $\Sigma = \{3, 5, 0\}$, $A = \{03530, 0033, 30335, 0350\}$.

24) $\Sigma = \{9, 8, 7\}$, $A = \{987, 99787, 87799, 9777\}$.

25) $\Sigma = \{3, 5, 0\}$, $A = \{3330, 05033, 30330, 535\}$.

26) $\Sigma = \{x, a, 35\}$, $A = \{35aaxx, xx35ax, 35xa, 3535ax\}$.

27) $\Sigma = \{10, 23, 45, 96\}$, $A = \{101045, 45109623, 23102345, 1010\}$.

28) $\Sigma = \{0, 1, 2, 3, 4, 5\}$, $A = \{0112, 0033, 30121, 331\}$.

29) $\Sigma = \{0, 1, 2, 3, 4, 5\}$, $A = \{53150, 53555, 5510, 0001\}$.

30) $\Sigma = \{a, b, c, d, e\}$, $A = \{abaac, ccbac, ccc, abac\}$.

Задание 3

1) $L_1 = (10 + 1)^*11(0 + 1)^*$, $L_2 = (101)^*111(0 + 1)^*$.

2) $L_1 = (0 + 1)^*1^*01(11 + 0)^*$, $L_2 = 1^*(0 + 1 + 11)^*0$.

3) $L_1 = 11(01 + 10)^*001^*$, $L_2 = 11((10)^*(01)^*)^*0011^*$.

4) $L_1 = (00 + 11)^*10(10 + 01)^*$, $L_2 = (0 + 1)^*1$.

5) $L_1 = (1 + 01)^*(00 + 1)^*$, $L_2 = (0 + 10)^*(11 + 0)^*$.

6) $L_1 = (111 + 01)^*1(00 + 11)^*$, $L_2 = 1^*0^*$.

7) $L_1 = (01 + 11)^*(10 + 11)^*$, $L_2 = (10 + 11)^*(01 + 11)^*$.

8) $L_1 = (a + bb)^*bba^*$, $L_2 = a^*(bb)^*a^*$.

9) $L_1 = b^*ab(ab + a)^*$, $L_2 = b^*a^*ab$.

10) $L_1 = a^*b^*(a + b)(bb)^*$, $L_2 = a^*(aa + b)^*$.

11) $L_1 = (b + ab)^*(a + ab)b^*$, $L_2 = (ab)^*b^*$.

12) $L_1 = (aba + bab)^*$, $L_2 = (ab + ba)^*$.

13) $L_1 = b(a + ba)^*(ab)^*$, $L_2 = b(a + ba + ab)^*$.

14) $L_1 = (aa + bb)^*ba(a + b)^*$, $L_2 = (aa)^*(bb)^*a^*b^*$.

15) $L_1 = ((a + b)^* + (ab + ba)^*)^*$, $L_2 = (a + b + ab + ba)^*$.

16) $L_1 = (1 + 01)^*010(10 + 01)^*$, $L_2 = (10)^*(0 + 10)10(1 + 0)^*$.

17) $L_1 = (0 + 1)^*010^*1(111 + 01)^*$, $L_2 = 0^*(10 + 11)^*0^*(1 + 0)^*$.

18) $L_1 = 01(0 + 1 + 10)^*01^*0$, $L_2 = 011((11)^*(10)^*)^*1^*001^*$.

19) $L_1 = (00 + 11)^*01(101 + 01)^*$, $L_2 = (0 + 1)^*1^*(0 + 1)^*0^*10^*$.

20) $L_1 = (11 + 00)^*(101 + 0)^*$, $L_2 = (11 + 000)^*(1 + 01)^*$.

21) $L_1 = (101 + 111)^*0(01 + 10)^*1$, $L_2 = 1^*0^*$.

22) $L_1 = (1 + 011)^*(01 + 11)^*$, $L_2 = (11 + 00)^*(01 + 10)^*$.

23) $L_1 = (aa + bbb)^*b^*ab^*$, $L_2 = a^*(bab)^*b^*$.

24) $L_1 = b^*bba^*(ab + b)^*$, $L_2 = a^*ba^*ab$.

25) $L_1 = a^*b^*(a + b + ab + ba)ab^*$, $L_2 = a^*(bb + aa)^*$.

26) $L_1 = (a + ab)^*b^*(b + ba)a^*$, $L_2 = (a + b)b^*(ab + bb)^*$.

27) $L_1 = (aba + b)^*(ab + a)^*$, $L_2 = (abb + baa)^*$.

28) $L_1 = b(a + b)^*(bb + aab)^*$, $L_2 = a(a + ab)(b + ab)^*$.

29) $L_1 = (aa + bb)^*a^*ab(a + ba)^*$, $L_2 = (aa)^*a(a + b)^*(bb)^*$.

30) $L_1 = (ab)^*(a + b)(b + a)(a + bab)^*$, $L_2 = ((a + b)^* + (aa + bb)^*)^*$.

ПРИЛОЖЕНИЕ D

ПРИМЕР ВЫПОЛНЕНИЯ ЗАДАНИЙ КУРСОВОЙ РАБОТЫ

Задание 1

Язык над алфавитом $\Sigma = \{0, 1\}$, состоящий из всех слов, в которых хотя бы на одной из последних трех позиций стоит 1

(i) Построим ПЛ-грамматику G, порождающую L. Вначале будем выводить (возможно пустой) префикс слова, а затем последние три символа, контролируя момент первого написания единицы.
$G = (\{S, P, T_{0F}, T_{1F}, T_{1T}, T_{2F}, T_{2T}\}, \{0, 1\}, P, S)$

$$
\begin{aligned}
S &\longrightarrow P \\
P &\longrightarrow 0P|1P|T_{0F} \\
T_{0F} &\longrightarrow 0T_{1F}|1T_{1T} \\
T_{1F} &\longrightarrow 0T_{2F}|1T_{2T} \\
T_{1T} &\longrightarrow 0T_{2T}|1T_{2T} \\
T_{2F} &\longrightarrow 1 \\
T_{2T} &\longrightarrow 0|1
\end{aligned}
$$

(ii) Покажем $L \subset L(G)$:

Предоставим алгоритм вывода произвольного слова $w \in L$ в нашей грамматике. Разобьем слово на префикс и последние три символа $w = pt, |t| = 3$. Префикс выведем продукциями $P \longrightarrow 0P|1P$. Пусть k - номер первой единицы в подслове t. Тогда первые k-1 символов слова t выводятся продукциями вида $T_{i-1F} \longrightarrow 0T_{iF} (i \in \{1, .., k-1\})$, единицу выведем $T_{k-1F} \longrightarrow 1T_{kT}$, конец слова допишем продукциями $T_{i-1T} \longrightarrow 0T_{iT}|1T_{iT} (i \in \{k+1, .., 3\})$. Таким образом мы можем вывести любое слово из L в грамматике $G \Rightarrow L \subset L(G)$.

Покажем $L(G) \subset L$:

Все слова, которые выводятся в грамматике G, можно разбить на префикс и 3х-символьный суффикс.

$\forall w \in L(G), w = pt, |t| = 3, p \in \{0,1\}^*$ Суффикс же содержит по крайней мере одну единицу, так как вывод завершается, либо продукцией $T_{9F} \longrightarrow 1$ и тогда единственная единица в слове стоит в самом конце слова, либо $T_{9T} \longrightarrow 0|1$, а нетерминалы вида T_{iT} выводятся только после написания первой единицы суффикса. Следовательно, все выводимые слова это слова написанные буквами из $\{0,1\}$ такие, что в последних трех символах стоит по меньшей мере одна единица.

(iii) Решим систему линейных уравнений с регулярными коэффициентами

$$S = P \tag{D.0.1}$$

$$P = 0P + 1P + T_{0F} \tag{D.0.2}$$

$$T_{0F} = 0T_{1F} + 1T_{1T} \tag{D.0.3}$$

$$T_{1F} = 0T_{2F} + 1T_{2T} \tag{D.0.4}$$

$$T_{1T} = 0T_{2T} + 1T_{2T} \tag{D.0.5}$$

$$T_{2F} = 1 \tag{D.0.6}$$

$$T_{2T} = 0 + 1 \tag{D.0.7}$$

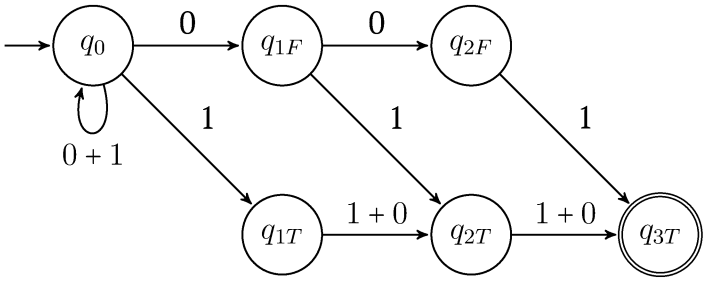
Для уравнений с (2) по (7) совершим последовательную подстановку снизу вверх и упрощая. Получим:

$$S = P \tag{D.0.8}$$
$$P = (0+1)P + 1(0+1)^2 + (0+1)1(0+1) + (0+1)^2 1$$

Затем для (9) по формуле $A = \alpha A + \beta \Rightarrow A = \alpha^* \beta$ получим подставив сразу в (8):

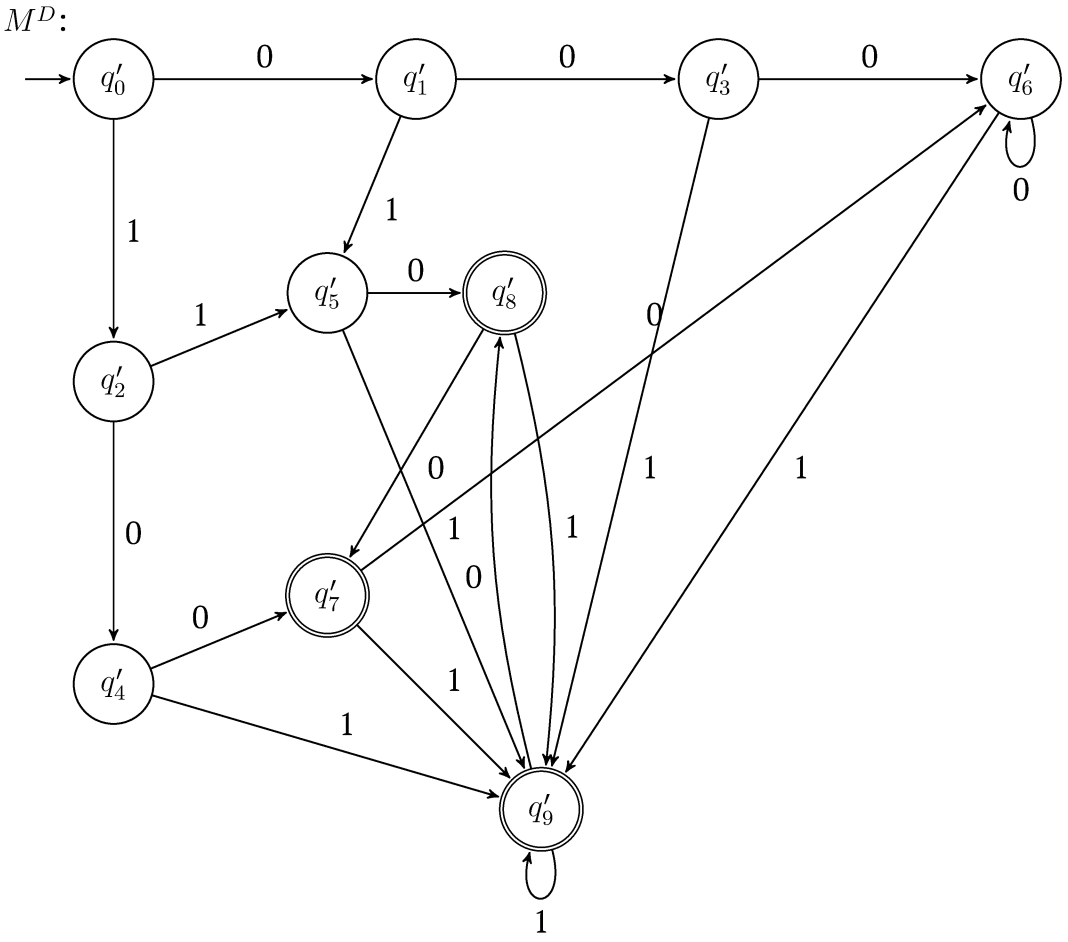
$$S = (0+1)^*(1(0+1)^2 + (0+1)1(0+1) + (0+1)^2 1)$$

(iv) Построим M^{ND}:



(v) Детерминируем M^{ND} воспользовавшись алгоритмом детерминизации конечных автоматов.

q'	q	0	1
q'_0	$\to \{q_0\}$	$\{q_0, q_{1F}\}$	$\{q_0, q_{1T}\}$
q'_1	$\{q_0, q_{1F}\}$	$\{q_0, q_{1F}, q_{2F}\}$	$\{q_0, q_{1T}, q_{2T}\}$
q'_2	$\{q_0, q_{1T}\}$	$\{q_0, q_{1F}, q_{2T}\}$	$\{q_0, q_{1T}, q_{2T}\}$
q'_3	$\{q_0, q_{1F}, q_{2F}\}$	$\{q_0, q_{1F}, q_{2F}, q_{3F}\}$	$\{q_0, q_{1T}, q_{2T}, q_{3T}\}$
q'_4	$\{q_0, q_{1F}, q_{2T}\}$	$\{q_0, q_{1F}, q_{2F}, q_{3T}\}$	$\{q_0, q_{1T}, q_{2T}, q_{3T}\}$
q'_5	$\{q_0, q_{1T}, q_{2T}\}$	$\{q_0, q_{1F}, q_{2T}, q_{3T}\}$	$\{q_0, q_{1T}, q_{2T}, q_{3T}\}$
q'_6	$\{q_0, q_{1F}, q_{2F}, q_{3F}\}$	$\{q_0, q_{1F}, q_{2F}, q_{3F}\}$	$\{q_0, q_{1T}q_{2T}, q_{3T}\}$
q'_7	$\{q_0, q_{1F}, q_{2F}, q_{3T}\}$	$\{q_0, q_{1F}, q_{2F}, q_{3F}\}$	$\{q_0, q_{1T}, q_{2T}, q_{3T}\}$
q'_8	$\{q_0, q_{1F}, q_{2T}, q_{3T}\}$	$\{q_0, q_{1F}, q_{2F}, q_{3T}\}$	$\{q_0, q_{1T}, q_{2T}, q_{3T}\}$
q'_9	$\{q_0, q_{1T}, q_{2T}, q_{3T}\}$	$\{q_0, q_{1F}, q_{2T}, q_{3T}\}$	$\{q_0, q_{1T}, q_{2T}, q_{3T}\}$



(vi) Покажем, что $L(M^{ND}) \subset L$: Рассмотрим произвольное слово $w \in L, |w| = k$. Рассмотрим работу автомата с этим словом на входе. Первые $k - 3$ символа прочтутся переходами по петле $0 + 1$ (конечно, будут клоны, которые будут переходить в состояния q_{1F}, q_{1T} и дальше, но они будут безусловно умирать, так как им на прочтение будет оставаться больше трех символов), затем, оставшиеся три символа будут прочтены автоматом, а так как это слово из L, т.е. содержит по меньшей мере одну единицу в суффиксе длины 3, то мы непременно прочтем в этих трех символах единицу и остановимся в q_{3T}.

Покажем, что $L \subset L(M^{ND})$: Рассмотрим пути на графе автомата, которые заканчиваются в финальном состоянии q_{3T}. Они все состоят из некоторого числа проходов по петле $0 + 1$, после чего следует последовательность из трех переходов, безусловно содержащих по меньшей мере одну единицу.

Таким образом доказано, что $L(M^{ND}) = L$.

Из того, что мы строили M^D по M^{ND} с помощью алгоритма детерминиза-

ции конечных автоматов для которого доказана эквивалентность в смысле равенства распознаваемых языков следует, что $L(M^{ND}) = L(M^D)$.

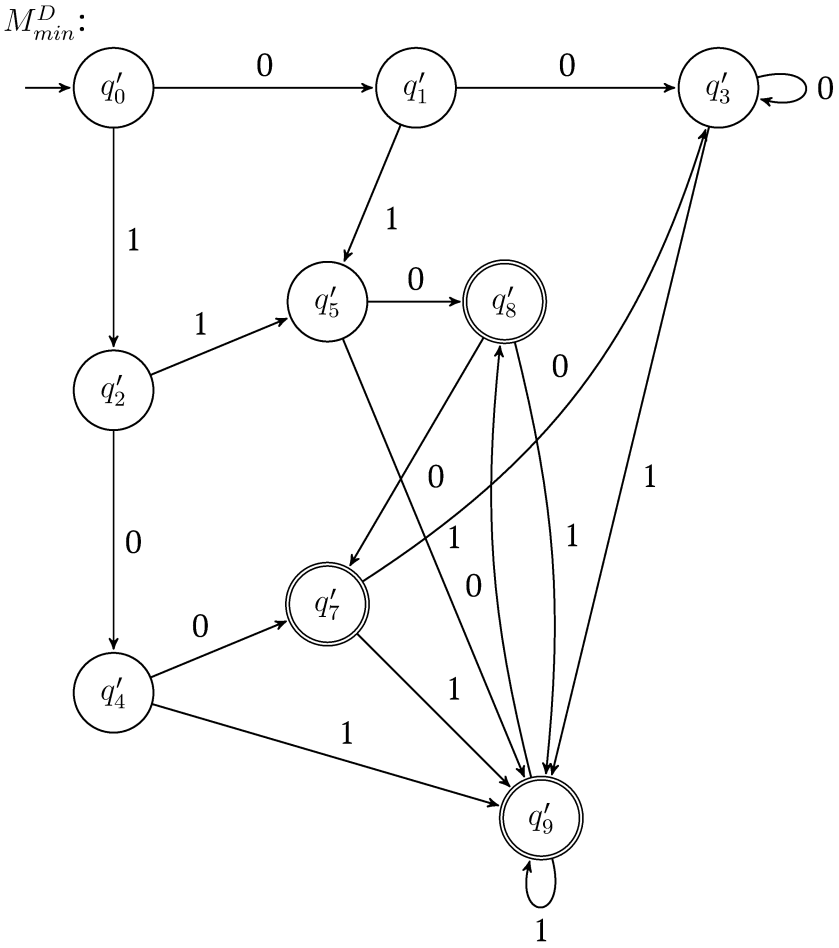
(vii) Для минимизации M^D вначале построим множества неразличимых состояний. Для этого на первой итерации образуем два множества неразличимых пустым словом состояний: финальные и не финальные. После чего на каждой итерации будем проверять, что два состояния из одного и того же множества неразличимы по каждой букве, т.е. переходят в одно и тоже множество из прошлой итерации. Если на некоторой итерации i они становятся различимы, то разделяем множество на подмножества неразличимых состояний словами длины i. Повторяем этот процесс пока множества не перестанут разделятся. Для минимизации автомата отождествим все неразличимые вершины графа:

Итерация №1: $\{q'_0, q'_1, q'_2, q'_3, q'_4, q'_5, q'_6\}; \{q'_7, q'_8, q'_9\};$

Итерация №2: $\{q'_0, q'_1, q'_2\}; \{q'_3, q'_6\}; \{q'_4, q'_5\}; \{q'_7\}; \{q'_8, q'_9\};$

Итерация №3: $\{q'_0\}; \{q'_1\}; \{q'_2\}; \{q'_3, q'_6\}; \{q'_4\}; \{q'_5\}; \{q'_7\}; \{q'_8\}; \{q'_9\};$

Итерация №4: $\{q'_0\}; \{q'_1\}; \{q'_2\}; \{q'_3, q'_6\}; \{q'_4\}; \{q'_5\}; \{q'_7\}; \{q'_8\}; \{q'_9\};$

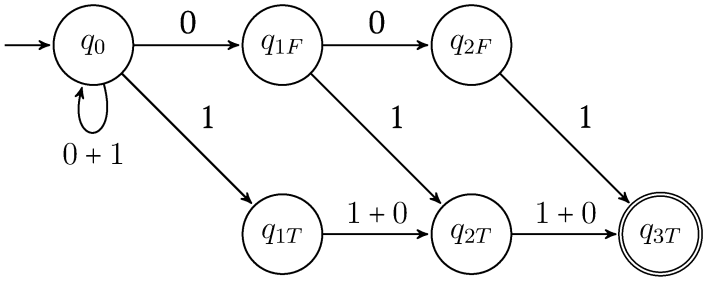


Для доказательства минимальности M^{ND} воспользуемся следующими рассуждениями: для распознавания слов из L и не распознавания слов из $\overline{L}$ нам необходимо читать префикс слова, что выполняет петля из q_0 по $0+1$ и читать суффикс убеждаясь, что он имеет длину равную 3 и содержит как минимум одну единицу. Для подсчета числа прочитанных символов из суффикса достаточно трех вершин, но, так как необходимо контролировать еще и факт прочтения единицы в суффиксе, число необходимых состояний удваивается за счет того, что на каждом из трех этапов мы можем как уже прочесть единицу, так и нет. Но так как состояние обозначающее, что мы прочли три символа суффикса и не встретили единицу - тупиковая ветвь нашего автомата, то её можно выбросить допустив смерть клона читающего ноль в состоянии "прочтено 2 символа и не встречена единица". Таким образом у нас остается 6 состояний все из которых необходимы для успешного функционирования автомата и мы убедились, что M^{ND} минимален.

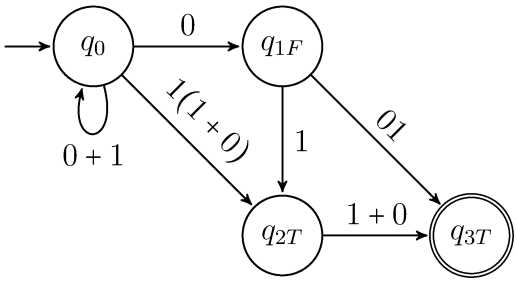
(viii) Воспользуемся методом последовательного исключения состояний для того, чтобы выписать регулярное выражение:

$L(M^{ND})$:

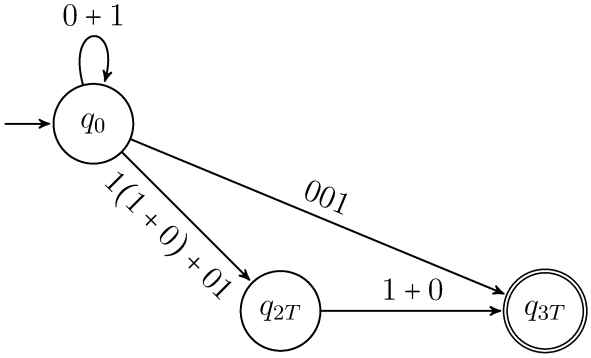
Шаг 1:



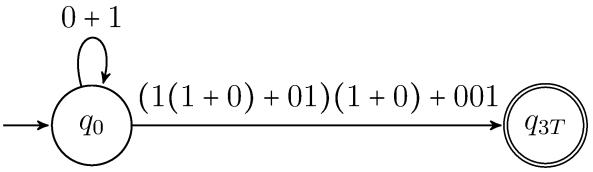
Шаг 2:



Шаг 3:



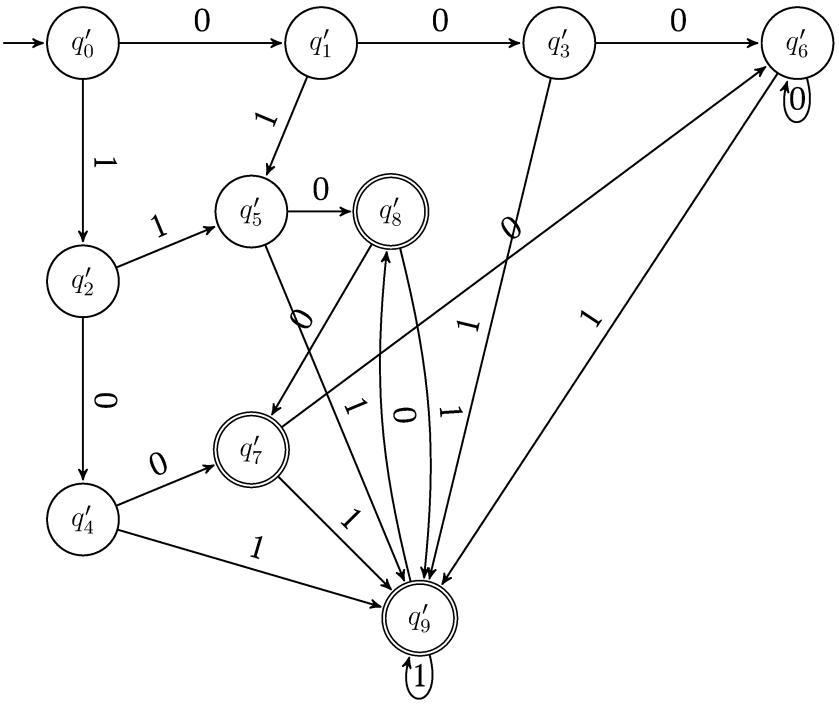
Шаг 4:



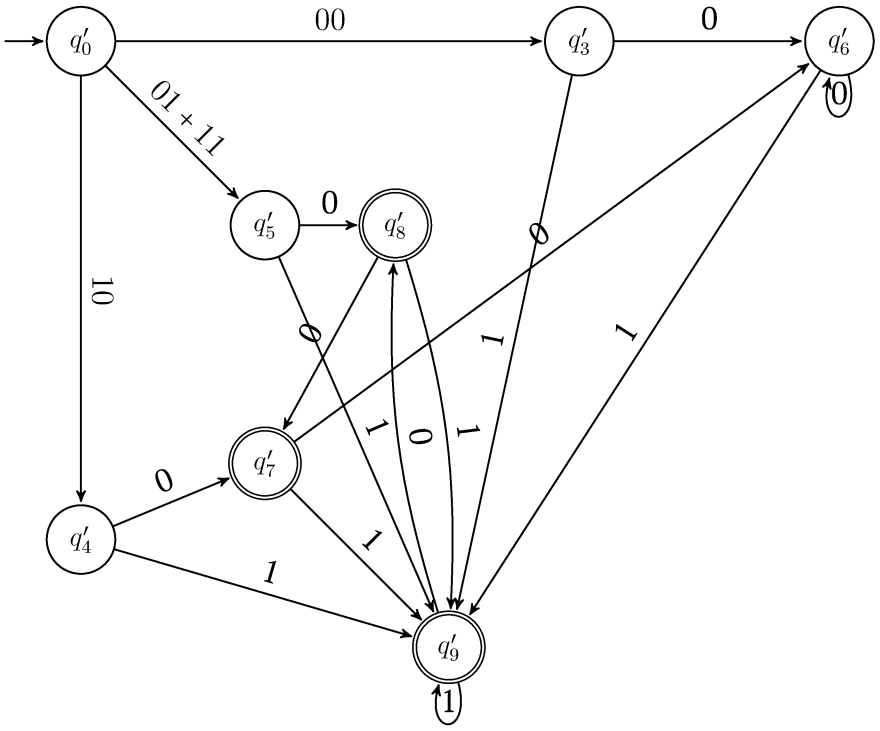
Таким образом мы получаем регулярное выражение $L(M^{ND}) =$
$(0+1)^*((1(1+0)+01)(1+0)+001) =$
$(0+1)^*((11+10+01)(1+0)+001) =$
$(0+1)^*(001+010+011+100+101+110+111).$

$L(M^D)$**:**

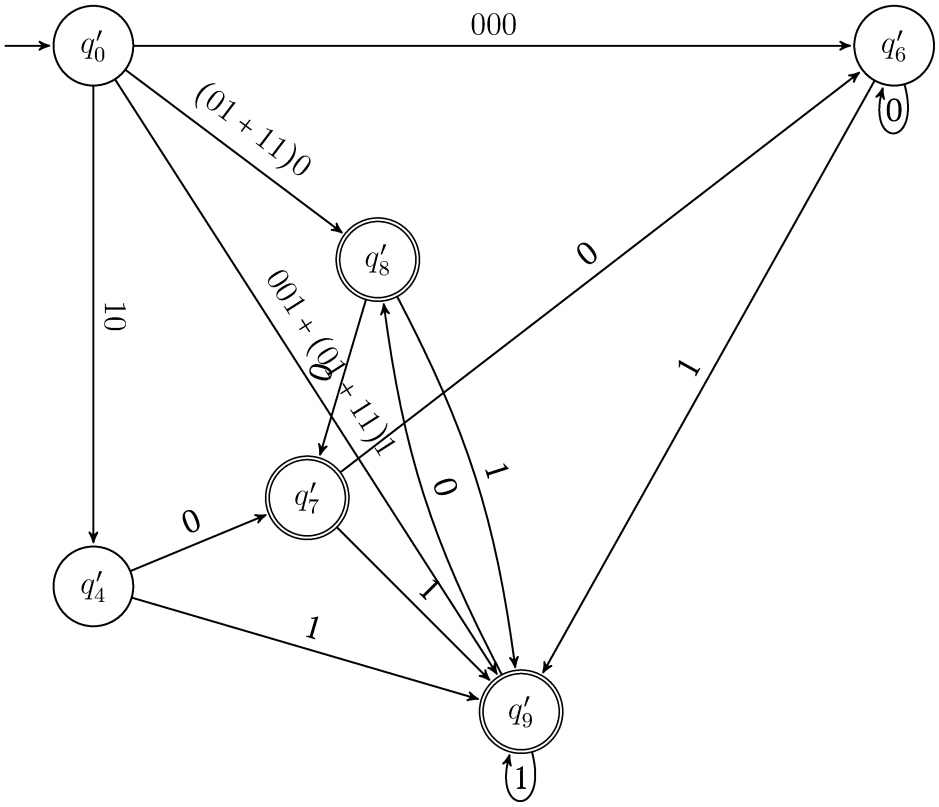
Шаг 1:



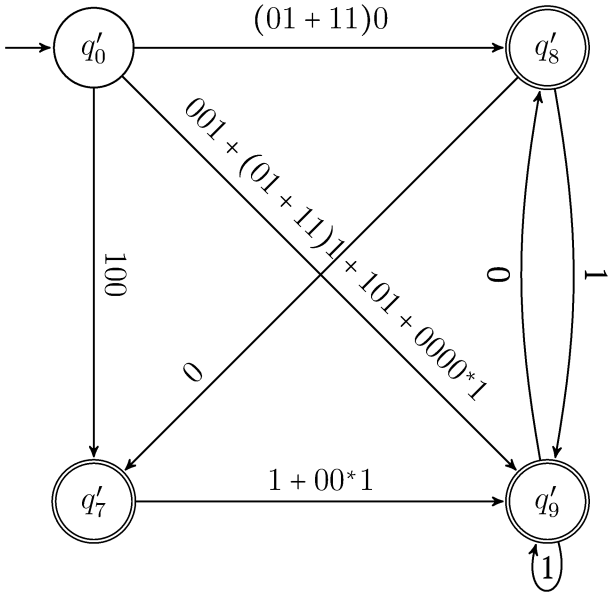
Шаг 2:


q'_0
q'_3
q'_6
q'_5
q'_8
q'_7
q'_4
q'_9
00
0
01 + 11
0
10
0
1
1
0
1
1
0
1
1
0
1
1

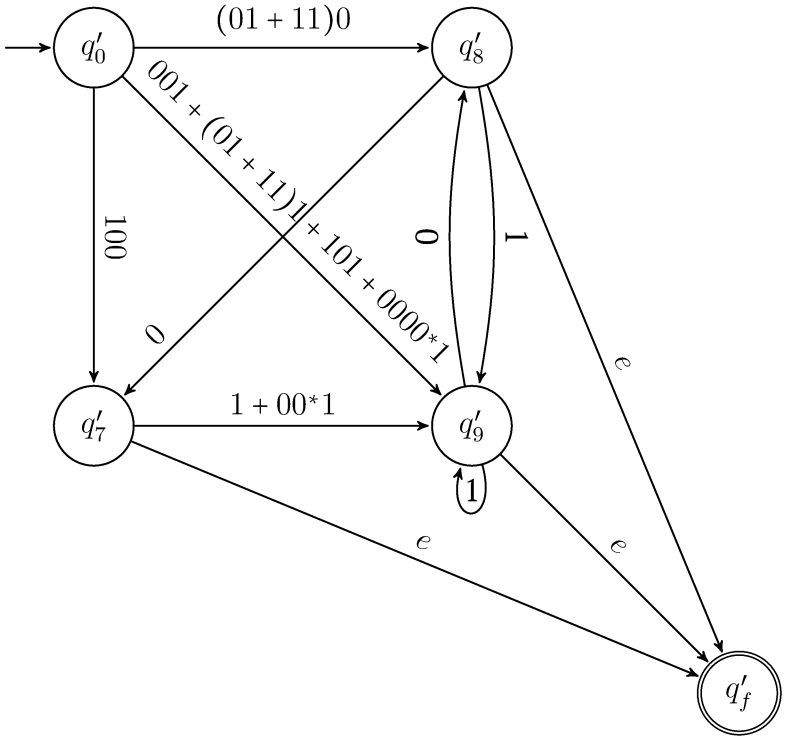
Шаг 3:


q'_0
q'_6
q'_8
q'_7
q'_4
q'_9
000
(01 + 11)0
10
001 + (01 + 11)1
0
1
0
1
0
1
1
0
1
1

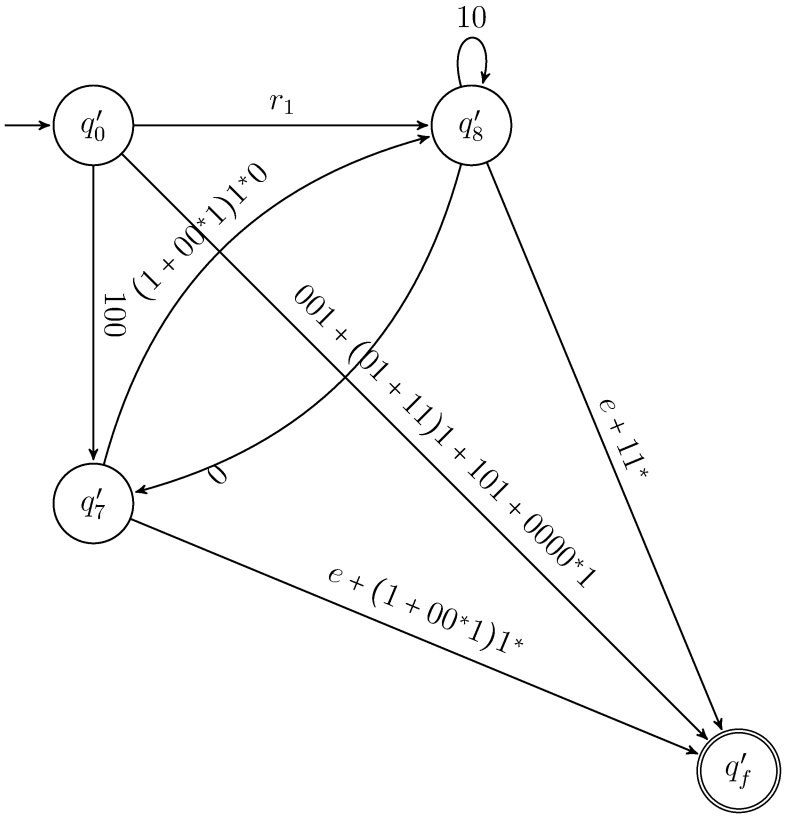
Шаг 4:



Шаг 5: Добавим новое состояние, которое сделаем финальным и в которое пустим спонтанные переходы из текущих финальных. Также текущие финальные сделаем нефинальными.



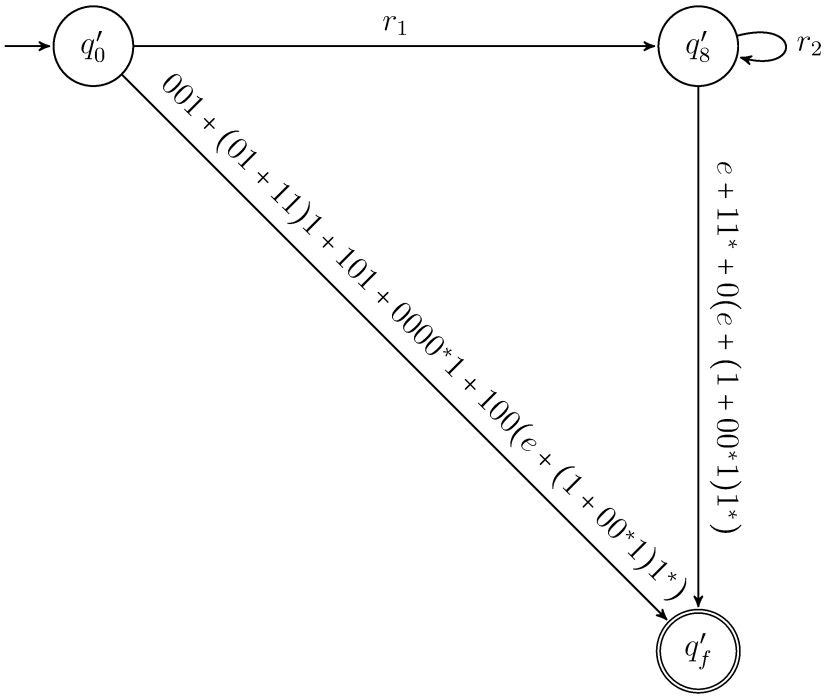
Шаг 6:



На автомате выше использовано обозначение:

$$r_1 = (01 + 11)0 + (001 + (01 + 11)1 + 101 + 0000^*1)1.$$

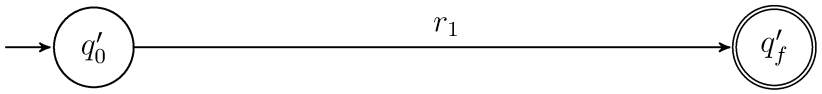
Шаг 7:



На автомате выше использованы обозначения:

$$r_1 = (01 + 11)0 + (001 + (01 + 11)1 + 101 + 0000^*1)1 + 100(1 + 00^*1)1^*0,$$
$$r_2 = 10 + 0(1 + 00^*1)1^*0.$$

Шаг 8:



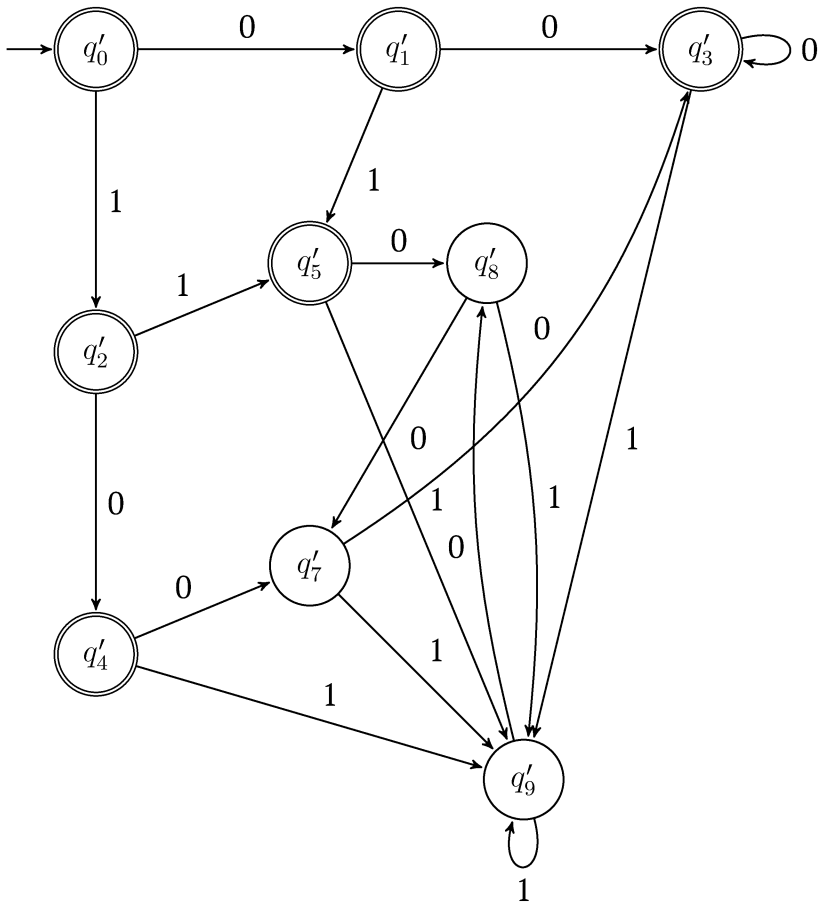
На автомате выше использовано обозначение:

$$L(M^D) =$$
$$001 + (01 + 11)1 + 101 + 0000^*1 + 100(e + (1 + 00^*1)1^*) +$$
$$((01 + 11)0 + (001 + (01 + 11)1 + 101 + 0000^*1)1 + 100(1 + 00^*1)1^*0)$$
$$(10 + 0(1 + 00^*1)1^*0)^*(e + 11^* + 0(e + (1 + 00^*1)1^*)).$$

Таким образом мы получаем регулярное выражение:

$$L(M^D) =$$
$$001 + (01 + 11)1 + 101 + 0000^*1 + 100(e + (1 + 00^*1)1^*) +$$
$$((01 + 11)0 + (001 + (01 + 11)1 + 101 + 0000^*1)1 + 100(1 + 00^*1)1^*0)$$
$$(10 + 0(1 + 00^*1)1^*0)^*(e + 11^* + 0(e + (1 + 00^*1)1^*)).$$

(ix) $\overline{M^D}$:



$\overline{L} = e + 0 + 1 + 00 + 01 + 10 + 11 + (0 + 1)^*000$

Задание 2

$\Sigma = \{a, b, c\}, A = \{bbab, ccac, bacb, baabc\}$.

(i) Для каждого слова $w_i \in A$ построить НКА M_i^{ND}, распознающий наличие в произвольной строке $s \in \Sigma^*$ подстроки w_i.

a) НКА M_1^{ND}, распознающий наличие в произвольной строке $s \in \Sigma^*$ подстроки w_1, где w_1 = bbab :

$M_1^{ND} = (\{q_0, q_1, q_2, q_3, q_4, q_f\}, \{a, b, c\}, \delta, q_0, \{q_f\})$

δ	Вход		
	a	b	c
$\{q_0\}$	$\{q_0\}$	$\{q_1\}$	$\{q_0\}$
$\{q_1\}$	$\{q_0\}$	$\{q_2\}$	$\{q_0\}$
$\{q_2\}$	$\{q_4\}$	$\{q_3\}$	$\{q_0\}$
$\{q_3\}$	$\{q_4\}$	$\{q_2, q_3\}$	$\{q_0\}$
$\{q_4\}$	$\{q_0\}$	$\{q_f\}$	$\{q_0\}$
$\{q_f\}$	$\{\varnothing\}$	$\{\varnothing\}$	$\{\varnothing\}$

b) НКА M_2^{ND}, распознающий наличие в произвольной строке $s \in \Sigma^*$ подстроки w_2, где w_2 = ccac :

$M_2^{ND} = (\{q_0, q_1, q_2, q_3, q_4, q_f\}, \{a, b, c\}, \delta, q_0, \{q_f\})$

δ	Вход		
	a	b	c
$\{q_0\}$	$\{q_0\}$	$\{q_0\}$	$\{q_1\}$
$\{q_1\}$	$\{q_0\}$	$\{q_0\}$	$\{q_2\}$
$\{q_2\}$	$\{q_4\}$	$\{q_0\}$	$\{q_3\}$
$\{q_3\}$	$\{q_4\}$	$\{q_0\}$	$\{q_2, q_3\}$
$\{q_4\}$	$\{q_0\}$	$\{q_0\}$	$\{q_f\}$
$\{q_f\}$	$\{\varnothing\}$	$\{\varnothing\}$	$\{\varnothing\}$

c) НКА M_3^{ND}, распознающий наличие в произвольной строке $s \in \Sigma^*$ подстроки w_3, где w_3 = bacb :

$M_3^{ND} = (\{q_0, q_1, q_2, q_3, q_4, q_f\}, \{a, b, c\}, \delta, q_0, \{q_f\})$

δ	Вход		
	a	b	c
$\{q_0\}$	$\{q_0\}$	$\{q_1\}$	$\{q_0\}$
$\{q_1\}$	$\{q_3\}$	$\{q_2\}$	$\{q_0\}$
$\{q_2\}$	$\{q_3\}$	$\{q_1, q_2\}$	$\{q_0\}$
$\{q_3\}$	$\{q_0\}$	$\{q_1, q_2\}$	$\{q_4\}$
$\{q_4\}$	$\{q_0\}$	$\{q_f\}$	$\{q_0\}$
$\{q_f\}$	$\{\varnothing\}$	$\{\varnothing\}$	$\{\varnothing\}$

d) НКА M_4^{ND}, распознающий наличие в произвольной строке $s \in \Sigma^*$ подстроки w_4, где w_4 = baabc :

$M_4^{ND} = (\{q_0, q_1, q_2, q_3, q_4, q_5, q_f\}, \{a, b, c\}, \delta, q_0, \{q_f\})$

δ	Вход		
	a	b	c
$\{q_0\}$	$\{q_0\}$	$\{q_1\}$	$\{q_0\}$
$\{q_1\}$	$\{q_3\}$	$\{q_2\}$	$\{q_0\}$
$\{q_2\}$	$\{q_3\}$	$\{q_2\}$	$\{q_0\}$
$\{q_3\}$	$\{q_4\}$	$\{q_1, q_2\}$	$\{q_0\}$
$\{q_4\}$	$\{q_0\}$	$\{q_5\}$	$\{q_0\}$
$\{q_5\}$	$\{q_3\}$	$\{q_1, q_2\}$	$\{q_f\}$
$\{q_f\}$	$\{\varnothing\}$	$\{\varnothing\}$	$\{\varnothing\}$

(ii) Для каждого НКА M_i^{ND} построить соответствующий ДКА M_i^D.

a) ДКА M_1^D, соответствующий НКА M_1^{ND} :

Переобозначим : $\{q_2, q_3\} = \{q_5\}$

$M_1^D = (\{q_0, q_1, q_2, q_3, q_4, q_5, q_f\}, \{a, b, c\}, \delta, q_0, \{q_f\})$

δ	Вход		
	a	b	c
$\{q_0\}$	$\{q_0\}$	$\{q_1\}$	$\{q_0\}$
$\{q_1\}$	$\{q_0\}$	$\{q_2\}$	$\{q_0\}$
$\{q_2\}$	$\{q_4\}$	$\{q_3\}$	$\{q_0\}$
$\{q_3\}$	$\{q_4\}$	$\{q_5\}$	$\{q_0\}$
$\{q_4\}$	$\{q_0\}$	$\{q_f\}$	$\{q_0\}$
$\{q_5\}$	$\{q_4\}$	$\{q_5\}$	$\{q_0\}$
$\{q_f\}$	$\{\varnothing\}$	$\{\varnothing\}$	$\{\varnothing\}$

b) ДКА M_2^D, соответствующий НКА M_2^{ND} :

Переобозначим : $\{q_2, q_3\} = \{q_5\}$

$M_2^D = (\{q_0, q_1, q_2, q_3, q_4, q_5, q_f\}, \{a, b, c\}, \delta, q_0, \{q_f\})$

δ	Вход		
	a	b	c
$\{q_0\}$	$\{q_0\}$	$\{q_0\}$	$\{q_1\}$
$\{q_1\}$	$\{q_0\}$	$\{q_0\}$	$\{q_2\}$
$\{q_2\}$	$\{q_4\}$	$\{q_0\}$	$\{q_3\}$
$\{q_3\}$	$\{q_4\}$	$\{q_0\}$	$\{q_5\}$
$\{q_4\}$	$\{q_0\}$	$\{q_0\}$	$\{q_f\}$
$\{q_5\}$	$\{q_4\}$	$\{q_0\}$	$\{q_5\}$
$\{q_f\}$	$\{\varnothing\}$	$\{\varnothing\}$	$\{\varnothing\}$

c) ДКА M_3^D, соответствующий НКА M_3^{ND} :

Переобозначим : $\{q_1, q_2\} = \{q_5\}$

$M_3^D = (\{q_0, q_1, q_2, q_3, q_4, q_5, q_f\}, \{a, b, c\}, \delta, q_0, \{q_f\})$

δ	Вход		
	a	b	c
$\{q_0\}$	$\{q_0\}$	$\{q_1\}$	$\{q_0\}$
$\{q_1\}$	$\{q_3\}$	$\{q_2\}$	$\{q_0\}$
$\{q_2\}$	$\{q_3\}$	$\{q_5\}$	$\{q_0\}$
$\{q_3\}$	$\{q_0\}$	$\{q_5\}$	$\{q_4\}$
$\{q_4\}$	$\{q_0\}$	$\{q_f\}$	$\{q_0\}$
$\{q_5\}$	$\{q_3\}$	$\{q_5\}$	$\{q_0\}$
$\{q_f\}$	$\{\varnothing\}$	$\{\varnothing\}$	$\{\varnothing\}$

d) ДКА M_4^D, соответствующий НКА M_4^{ND} :

Переобозначим : $\{q_1, q_2\} = \{q_6\}$

$M_4^D = (\{q_0, q_1, q_2, q_3, q_4, q_5, q_6, q_f\}, \{a, b, c\}, \delta, q_0, \{q_f\})$

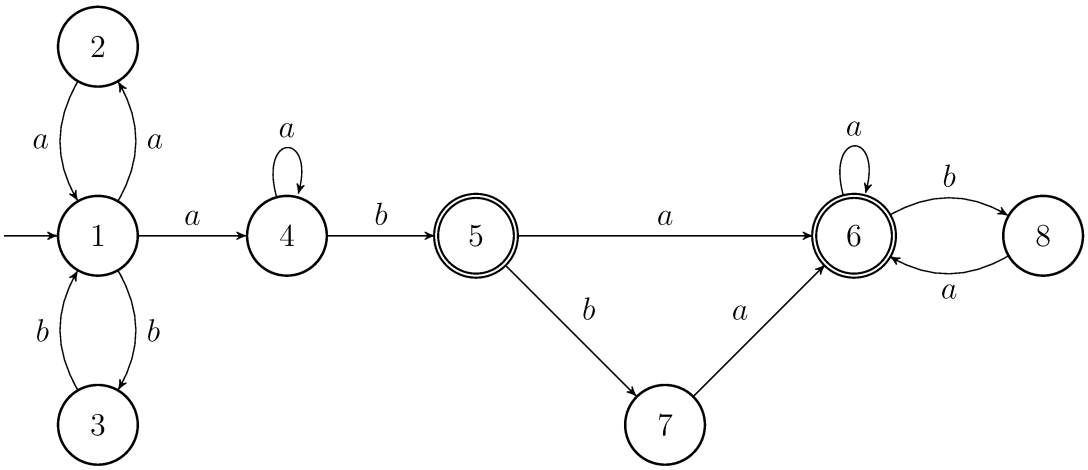
δ	Вход		
	a	b	c
$\{q_0\}$	$\{q_0\}$	$\{q_1\}$	$\{q_0\}$
$\{q_1\}$	$\{q_3\}$	$\{q_2\}$	$\{q_0\}$
$\{q_2\}$	$\{q_3\}$	$\{q_2\}$	$\{q_0\}$
$\{q_3\}$	$\{q_4\}$	$\{q_6\}$	$\{q_0\}$
$\{q_4\}$	$\{q_0\}$	$\{q_5\}$	$\{q_0\}$
$\{q_5\}$	$\{q_3\}$	$\{q_6\}$	$\{q_f\}$
$\{q_6\}$	$\{q_3\}$	$\{q_2\}$	$\{q_0\}$
$\{q_f\}$	$\{\varnothing\}$	$\{\varnothing\}$	$\{\varnothing\}$

Задание 3

$L_1 = (aa + bb)^* a^* ab(a + ba)^*$, $L_2 = (aa)^* a(a + b)^* (bb)^*$

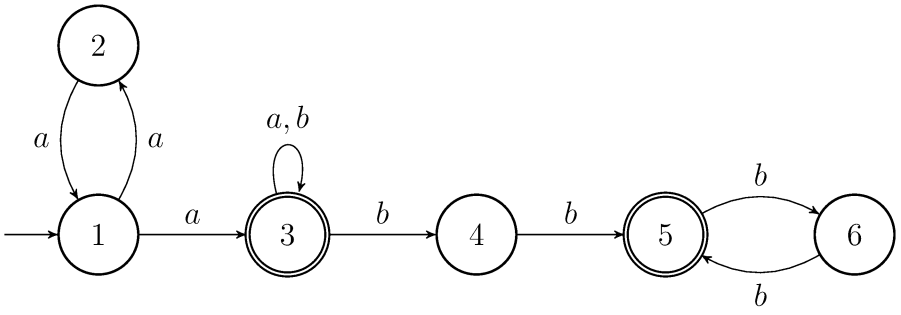
Вспомогательные построения автоматов для решения задания 3:

НКА для L_1:



δ	Вход	
	a	b
$\to \{1\}$	$\{2\}, \{4\}$	$\{3\}$
$\{2\}$	$\{1\}$	$\{\varnothing\}$
$\{3\}$	$\{\varnothing\}$	$\{1\}$
$\{4\}$	$\{4\}$	$\{5\}$
$\{\mathbf{5}\}$	$\{6\}$	$\{7\}$
$\{\mathbf{6}\}$	$\{6\}$	$\{8\}$
$\{7\}$	$\{6\}$	$\{\varnothing\}$
$\{8\}$	$\{6\}$	$\{\varnothing\}$

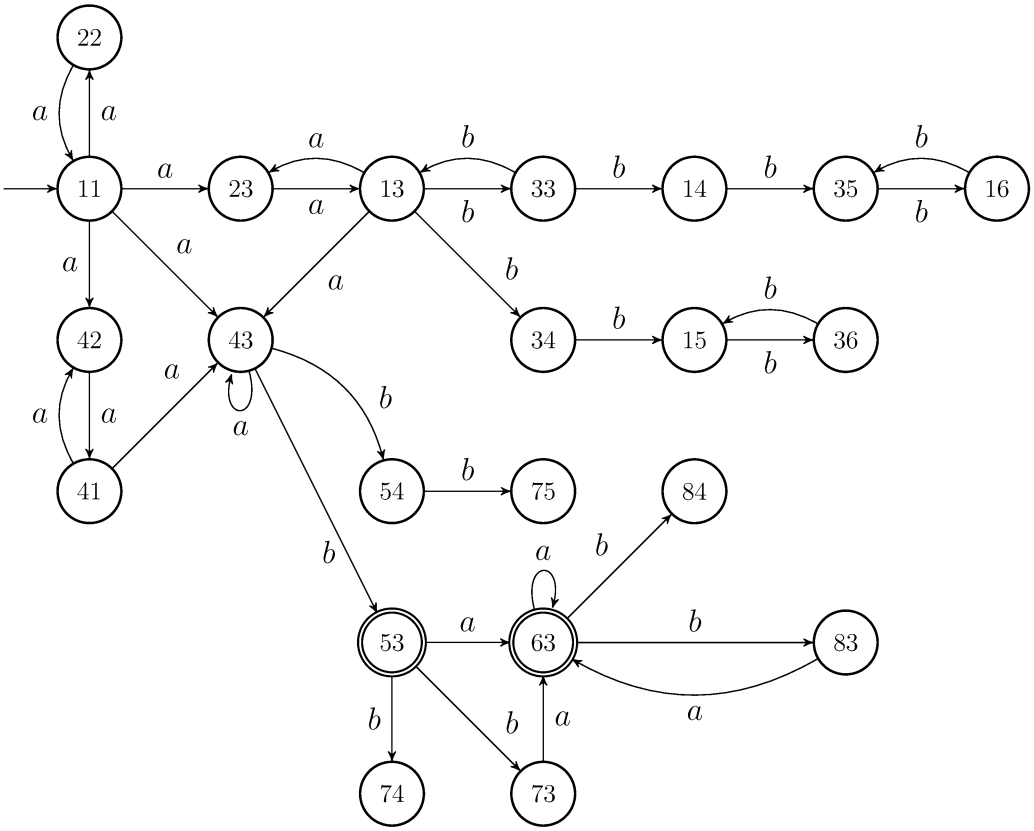
НКА для L_2:



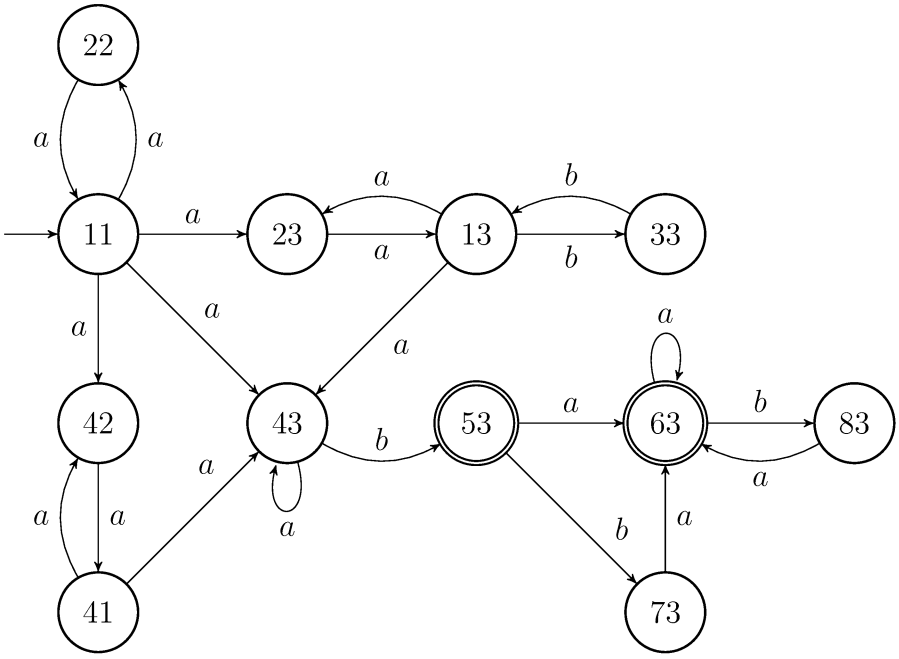
δ	Вход	
	a	b
$\to \{1\}$	$\{2\}, \{3\}$	$\{\varnothing\}$
$\{2\}$	$\{1\}$	$\{\varnothing\}$
$\{\mathbf{3}\}$	$\{3\}$	$\{3\}, \{4\}$
$\{4\}$	$\{\varnothing\}$	$\{5\}$
$\{\mathbf{5}\}$	$\{\varnothing\}$	$\{6\}$
$\{6\}$	$\{\varnothing\}$	$\{5\}$

(i) Вычислить регулярное выражение, определяющее язык $L_1 \cap L_2$.

δ	Вход	
	a	b
$\to \{1,1\}$	$\{2,2\}, \{2,3\}, \{4,2\}, \{4,3\}$	$\{\varnothing\}$
$\{2,2\}$	$\{1,1\}$	$\{\varnothing\}$
$\{2,3\}$	$\{1,3\}$	$\{\varnothing\}$
$\{1,3\}$	$\{2,3\}, \{4,3\}$	$\{3,3\}, \{3,4\}$
$\{3,3\}$	$\{\varnothing\}$	$\{1,3\}, \{1,4\}$
$\{3,4\}$	$\{\varnothing\}$	$\{1,5\}$
$\{1,4\}$	$\{\varnothing\}$	$\{3,5\}$
$\{1,5\}$	$\{\varnothing\}$	$\{3,6\}$
$\{3,5\}$	$\{\varnothing\}$	$\{1,6\}$
$\{3,6\}$	$\{\varnothing\}$	$\{1,5\}$
$\{1,6\}$	$\{\varnothing\}$	$\{3,5\}$
$\{4,2\}$	$\{4,1\}$	$\{\varnothing\}$
$\{4,3\}$	$\{4,3\}$	$\{5,3\}, \{5,4\}$
$\{4,1\}$	$\{4,2\}, \{4,3\}$	$\{\varnothing\}$
{5, 3}	$\{6,3\}$	$\{7,3\}, \{7,4\}$
$\{5,4\}$	$\{\varnothing\}$	$\{7,5\}$
{6, 3}	$\{6,3\}$	$\{8,3\}, \{8,4\}$
$\{7,3\}$	$\{6,3\}$	$\{\varnothing\}$
$\{7,4\}$	$\{\varnothing\}$	$\{\varnothing\}$
$\{7,5\}$	$\{\varnothing\}$	$\{\varnothing\}$
$\{8,3\}$	$\{6,3\}$	$\{\varnothing\}$
$\{8,4\}$	$\{\varnothing\}$	$\{\varnothing\}$

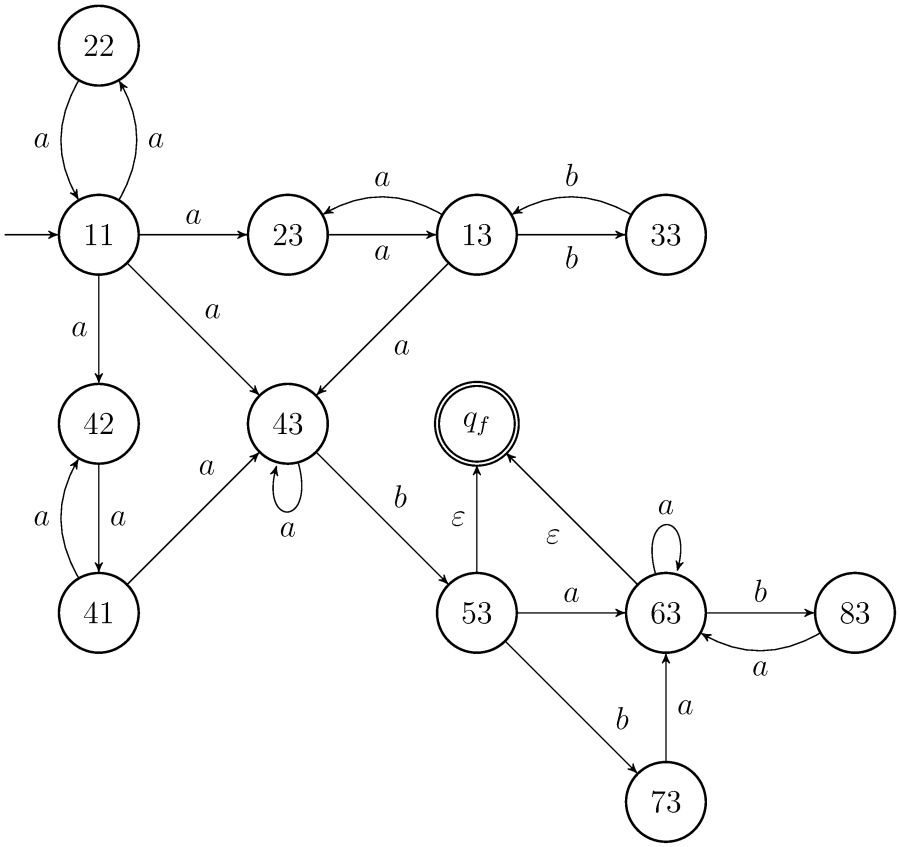


Состояния 84, 74, 54, 75, 14, 35, 16, 34, 15, 36 являются непродуктивными, поэтому их можно удалить.

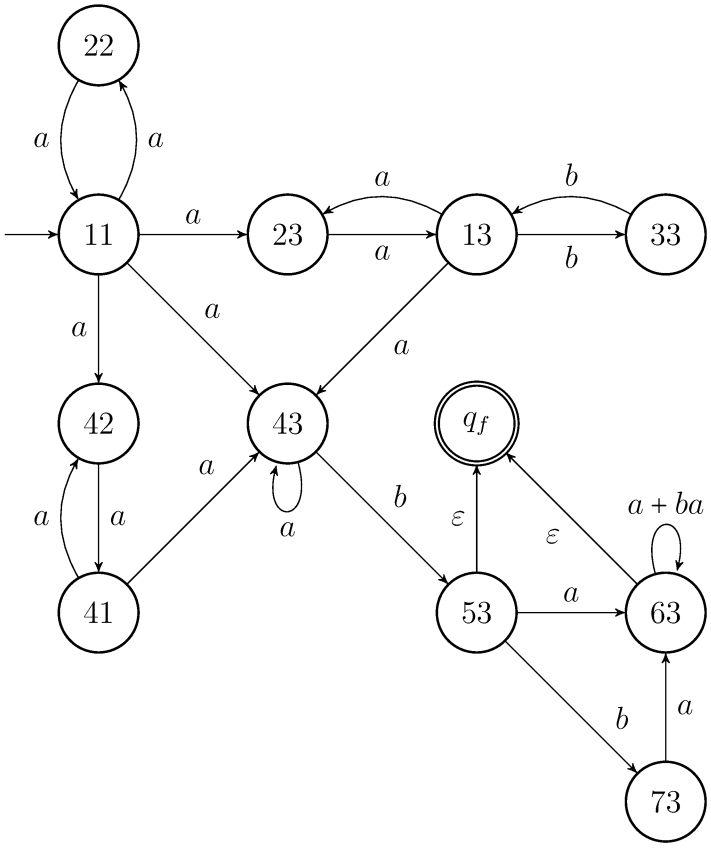


Воспользуемся методом последовательного исключения состояний. Пустим ε-переходы из всех допускающих состояний в новое состояние q_f. Все

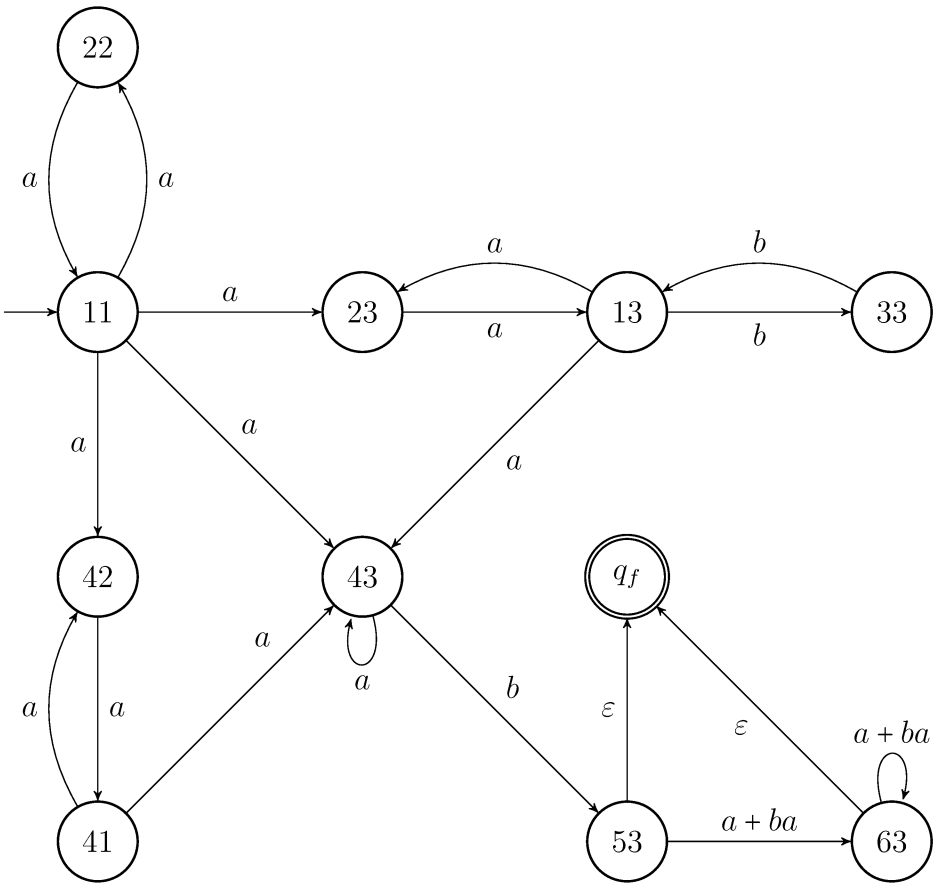
допускающие состояния сделаем недопускающими, а новое состояние q_f - допускающим.



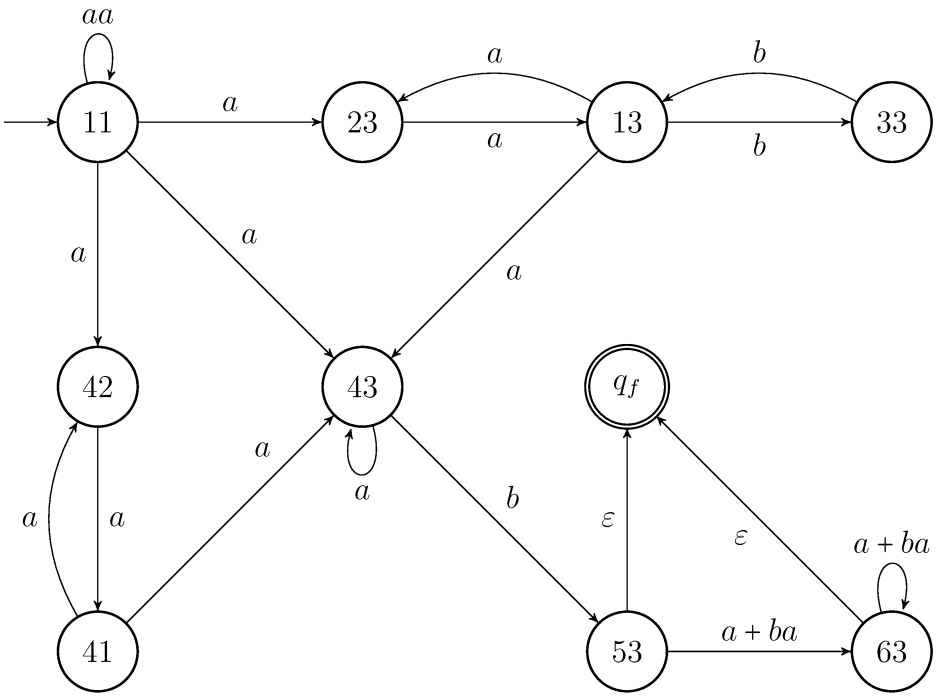
1) Исключим состояние 83:



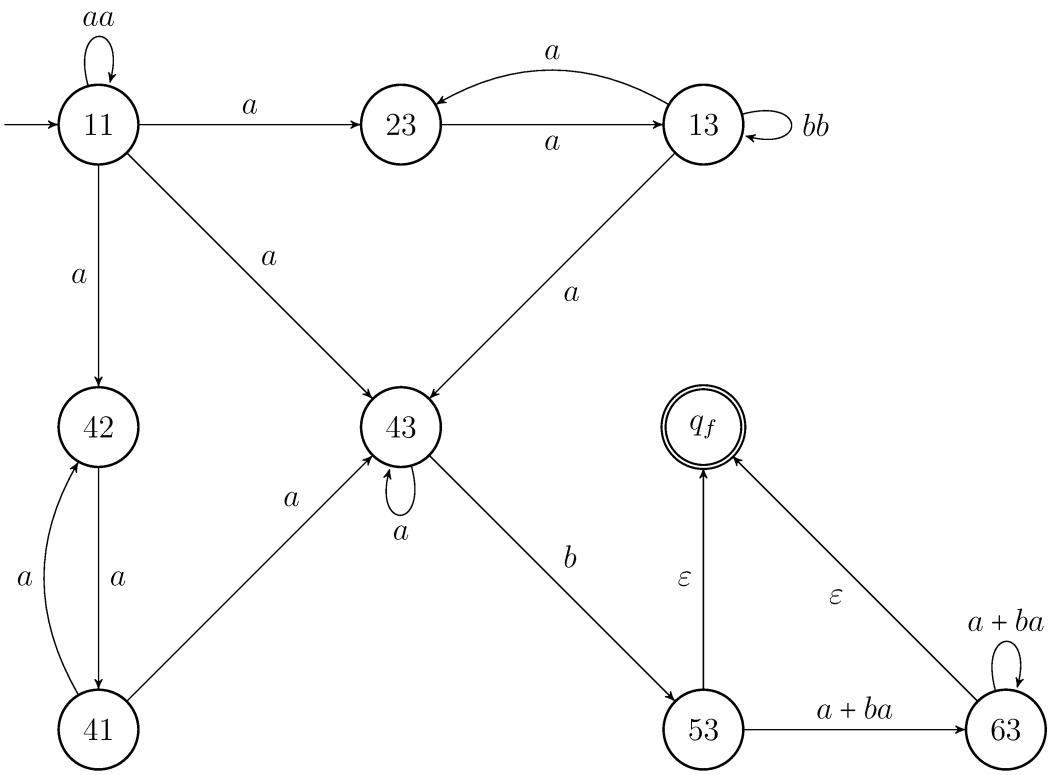
2) Исключим состояние 73:



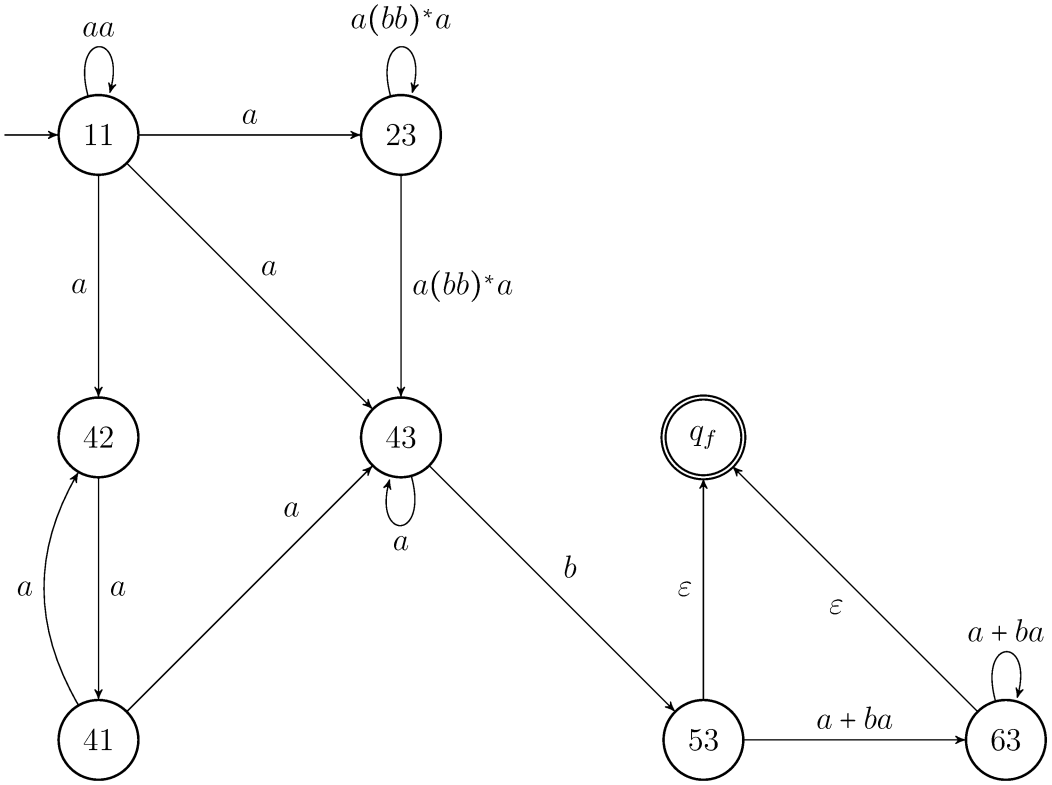
3) Исключим состояние 22:



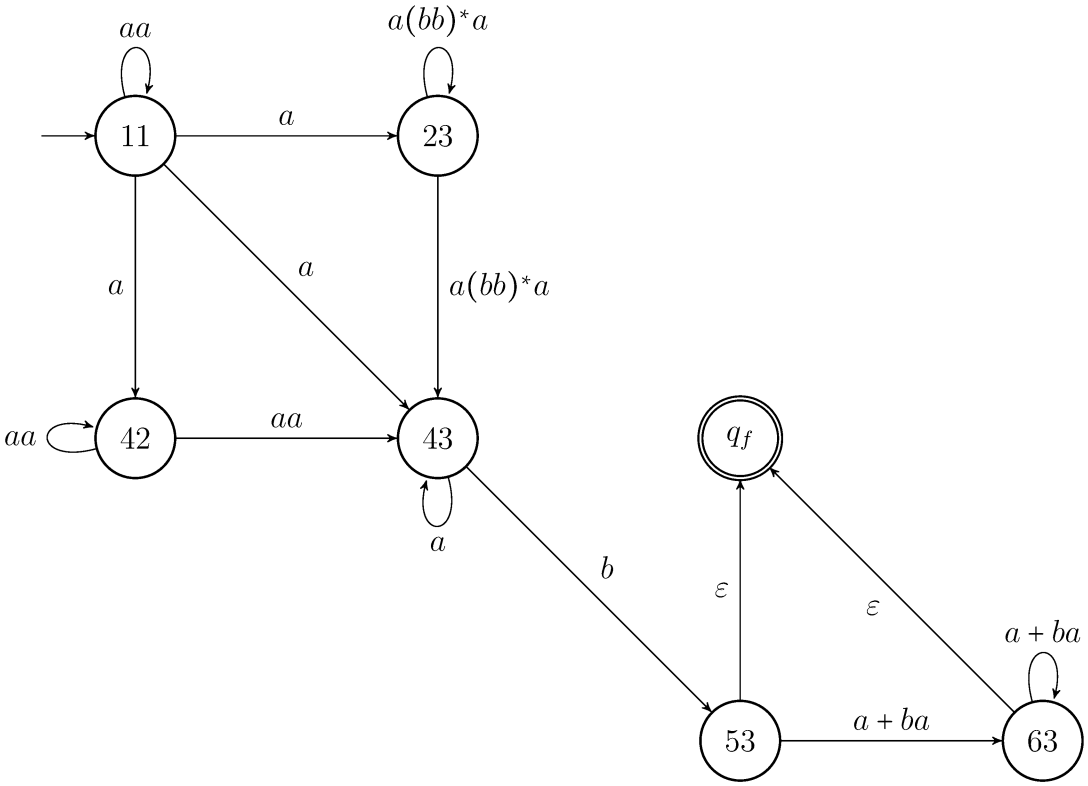
4) Исключим состояние 33:



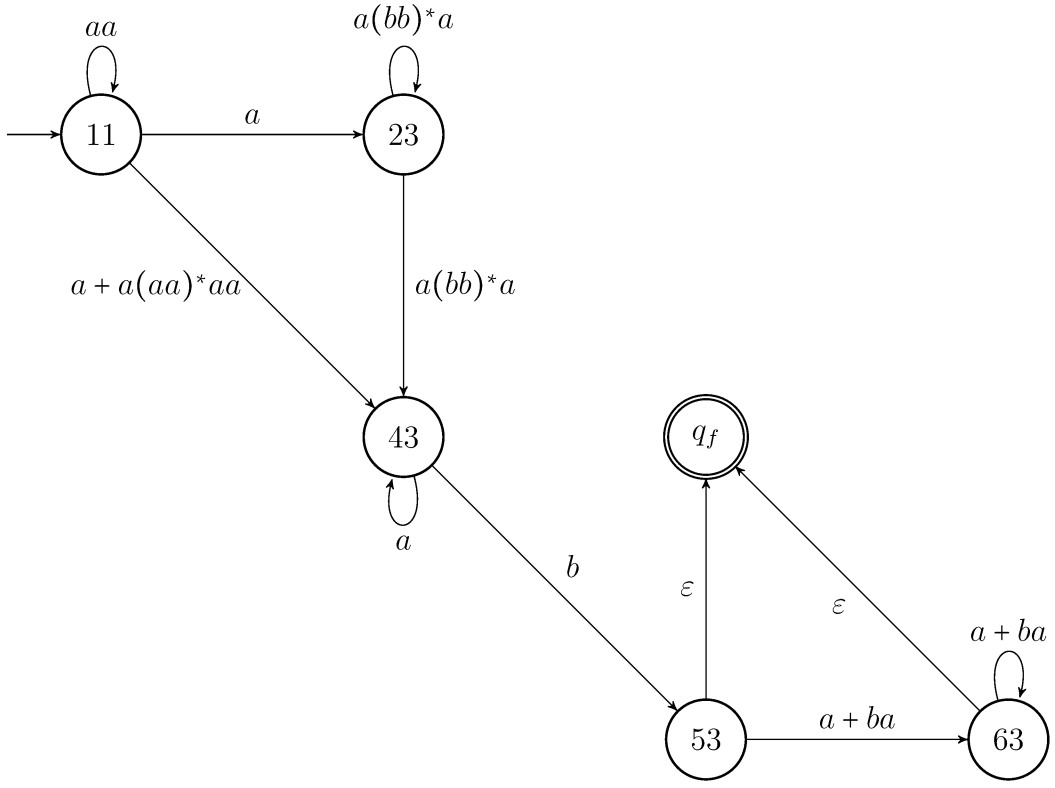
5) Исключим состояние 13:



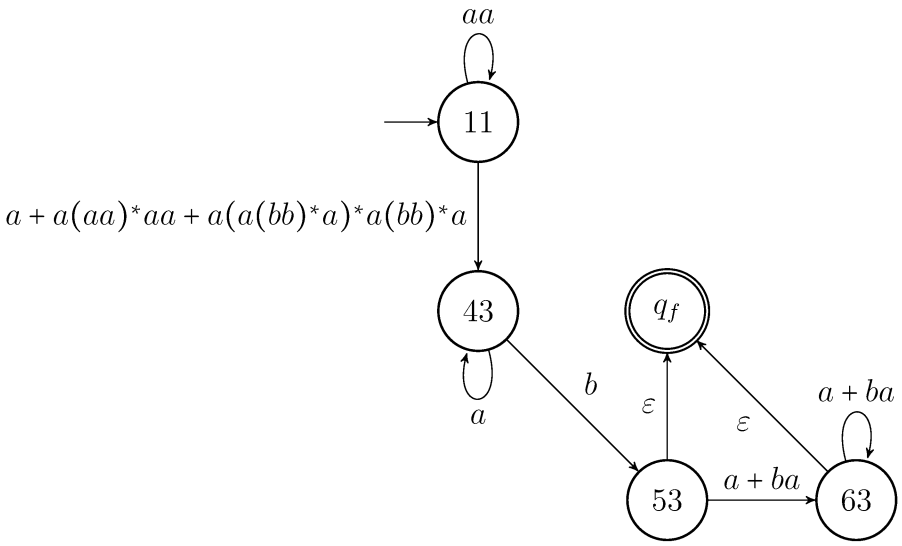
6) Исключим состояние 41:



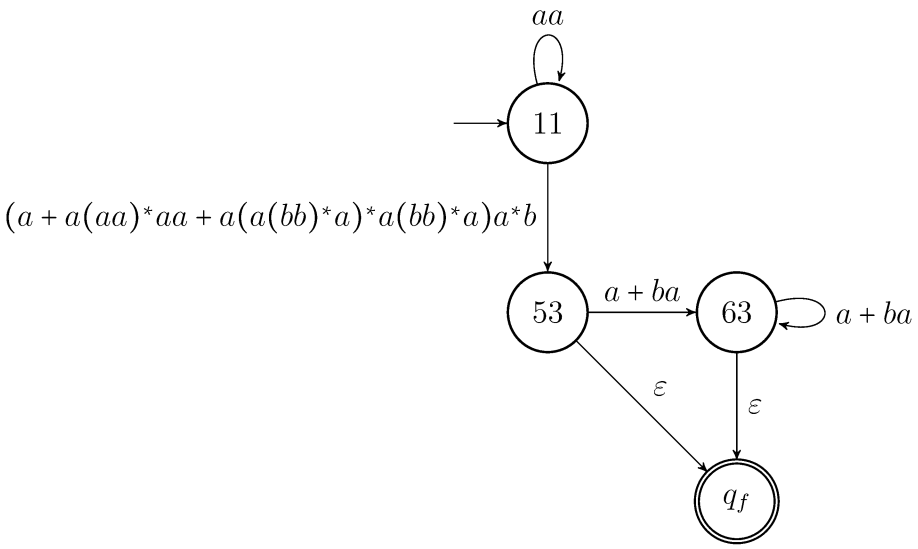
7) Исключим состояние 42:



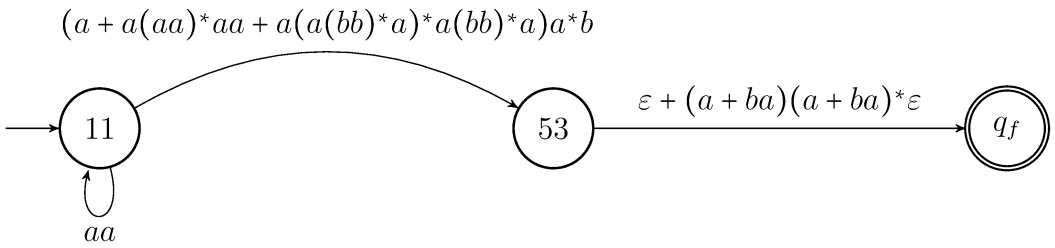
8) Исключим состояние 23:



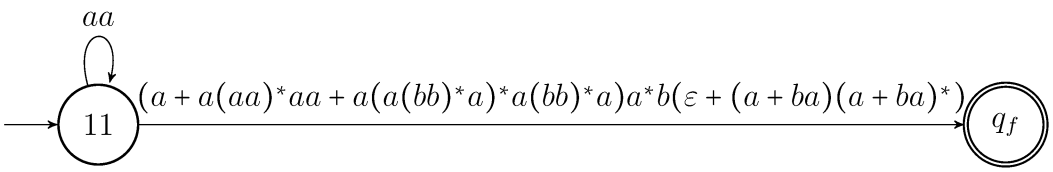
9) Исключим состояние 43:



10) Исключим состояние 63:



11) Исключим состояние 53:



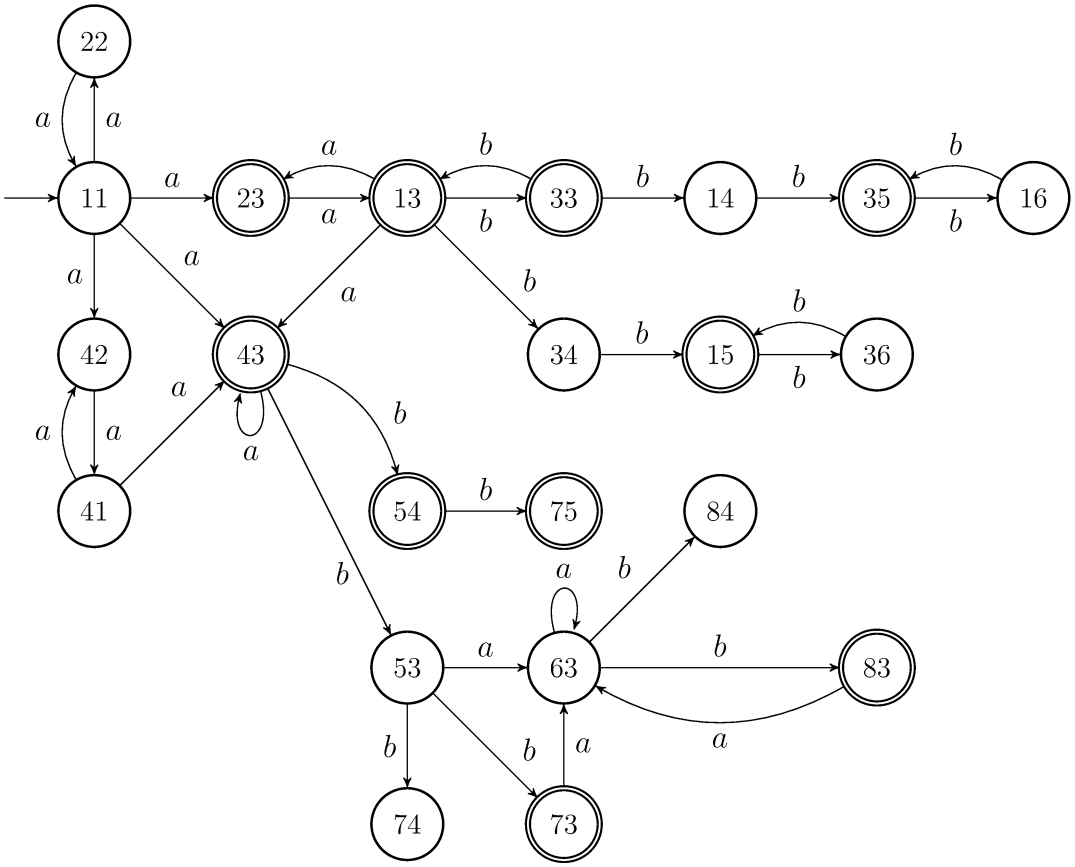
Получаем : $Reg = (aa)^*(a+a(aa)^*aa+a(a(bb)^*a)^*a(bb)^*a)a^*b(\varepsilon+(a+ba)(a+ba)^*)$

(ii) Вычислить регулярное выражение, определяющее язык $L_1 \triangle L_2$.

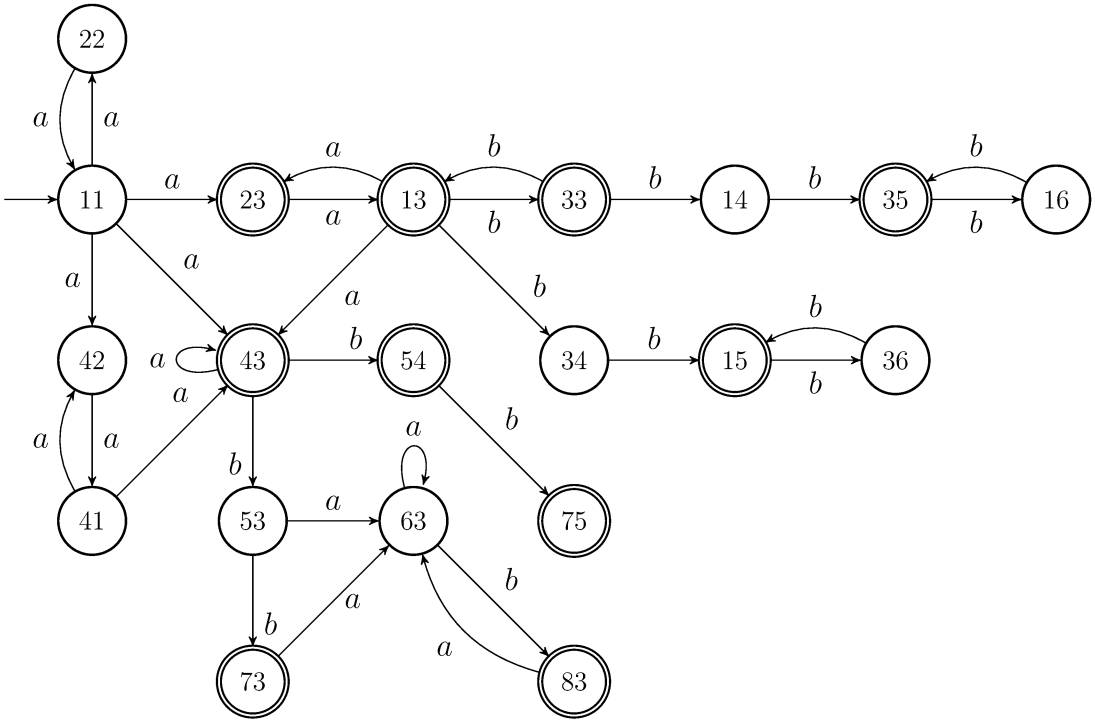
Напомним, что

$$L_1 \triangle L_2 = (L_1 \backslash L_2) \cup (L_2 \backslash L_1) = (L_1 \cap \overline{L_2}) \cup (L_2 \cap \overline{L_1}).$$

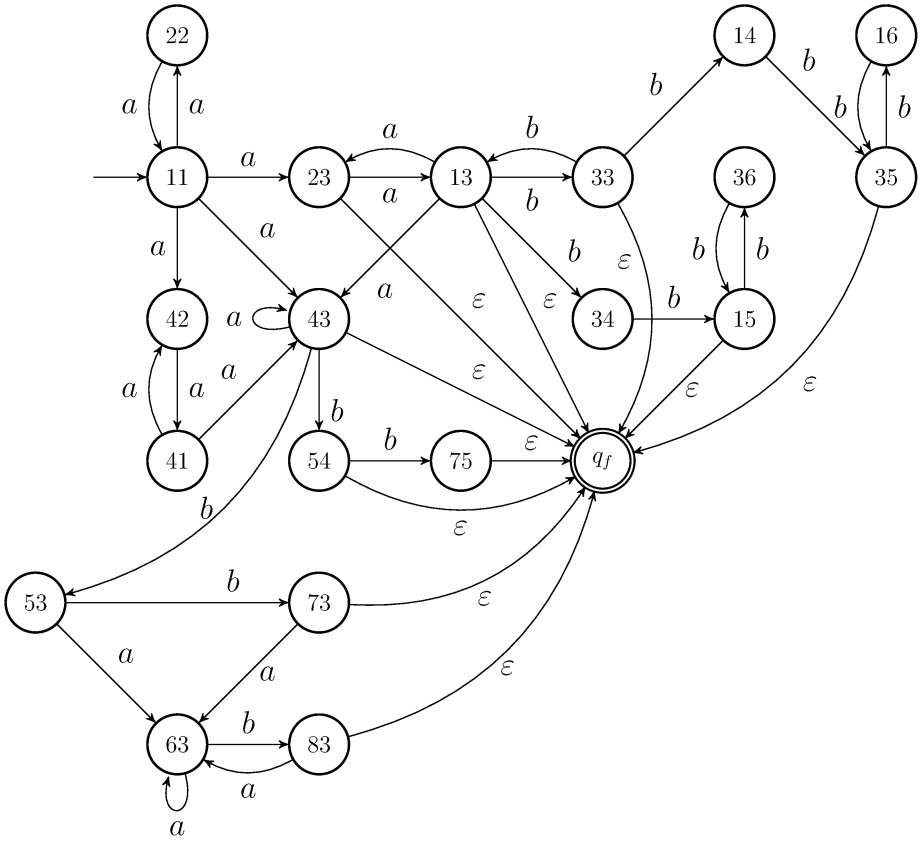
δ	Вход	
	a	b
$\to \{1,1\}$	$\{2,2\},\{2,3\},\{4,2\},\{4,3\}$	$\{\varnothing\}$
$\{2,2\}$	$\{1,1\}$	$\{\varnothing\}$
{2, 3}	$\{1,3\}$	$\{\varnothing\}$
{1, 3}	$\{2,3\},\{4,3\}$	$\{3,3\},\{3,4\}$
{3, 3}	$\{\varnothing\}$	$\{1,3\},\{1,4\}$
$\{3,4\}$	$\{\varnothing\}$	$\{1,5\}$
$\{1,4\}$	$\{\varnothing\}$	$\{3,5\}$
{1, 5}	$\{\varnothing\}$	$\{3,6\}$
{3, 5}	$\{\varnothing\}$	$\{1,6\}$
$\{3,6\}$	$\{\varnothing\}$	$\{1,5\}$
$\{1,6\}$	$\{\varnothing\}$	$\{3,5\}$
$\{4,2\}$	$\{4,1\}$	$\{\varnothing\}$
{4, 3}	$\{4,3\}$	$\{5,3\},\{5,4\}$
$\{4,1\}$	$\{4,2\},\{4,3\}$	$\{\varnothing\}$
$\{5,3\}$	$\{6,3\}$	$\{7,3\},\{7,4\}$
{5, 4}	$\{\varnothing\}$	$\{7,5\}$
$\{6,3\}$	$\{6,3\}$	$\{8,3\},\{8,4\}$
{7, 3}	$\{6,3\}$	$\{\varnothing\}$
$\{7,4\}$	$\{\varnothing\}$	$\{\varnothing\}$
{7, 5}	$\{\varnothing\}$	$\{\varnothing\}$
{8, 3}	$\{6,3\}$	$\{\varnothing\}$
$\{8,4\}$	$\{\varnothing\}$	$\{\varnothing\}$



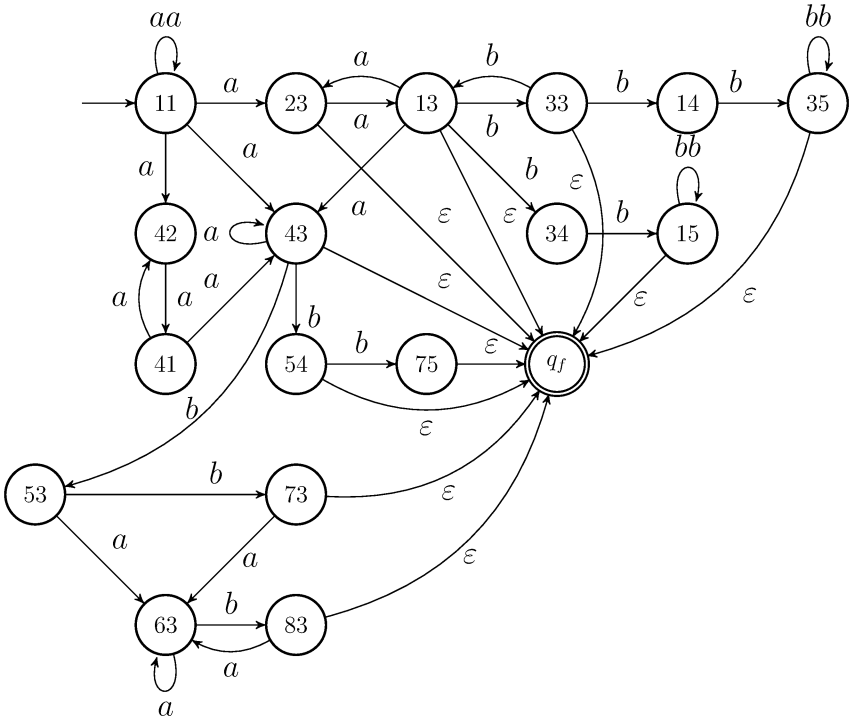
Состояния 74, 84 являются непродуктивными, поэтому их можно удалить.



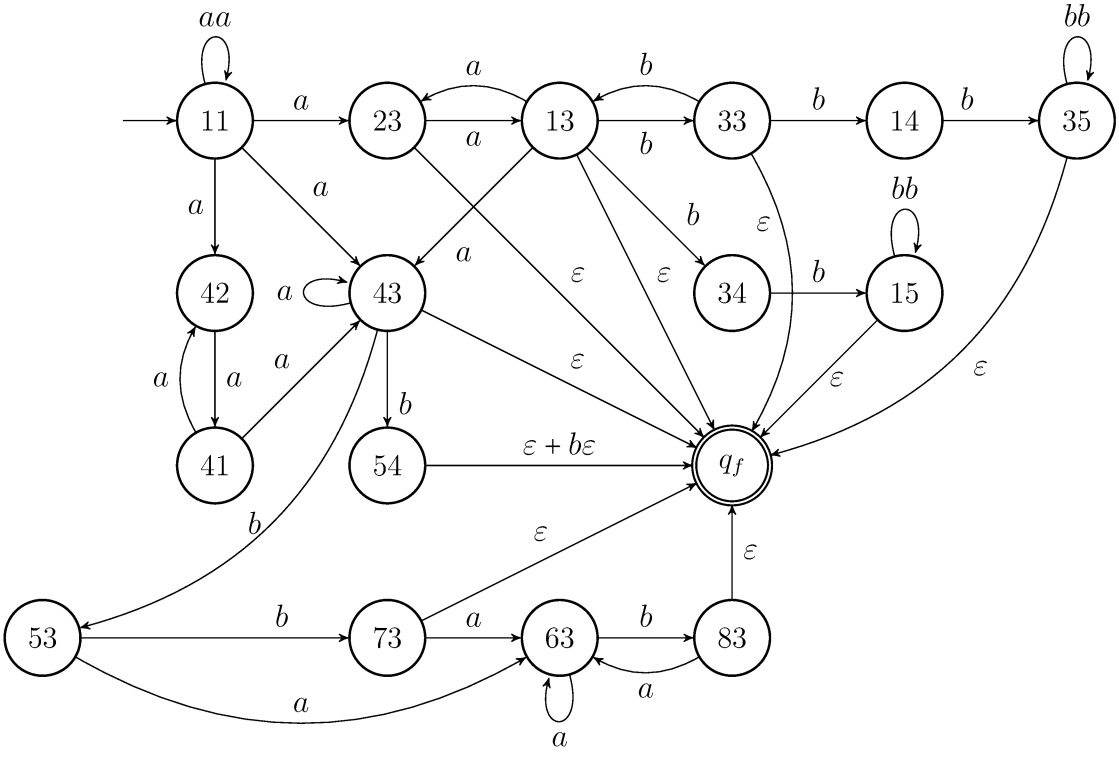
Воспользуемся методом последовательного исключения состояний. Пустим ε-переходы из всех допускающих состояний в новое состояние q_f. Все допускающие состояния сделаем недопускающими, а новое состояние q_f — допускающим.



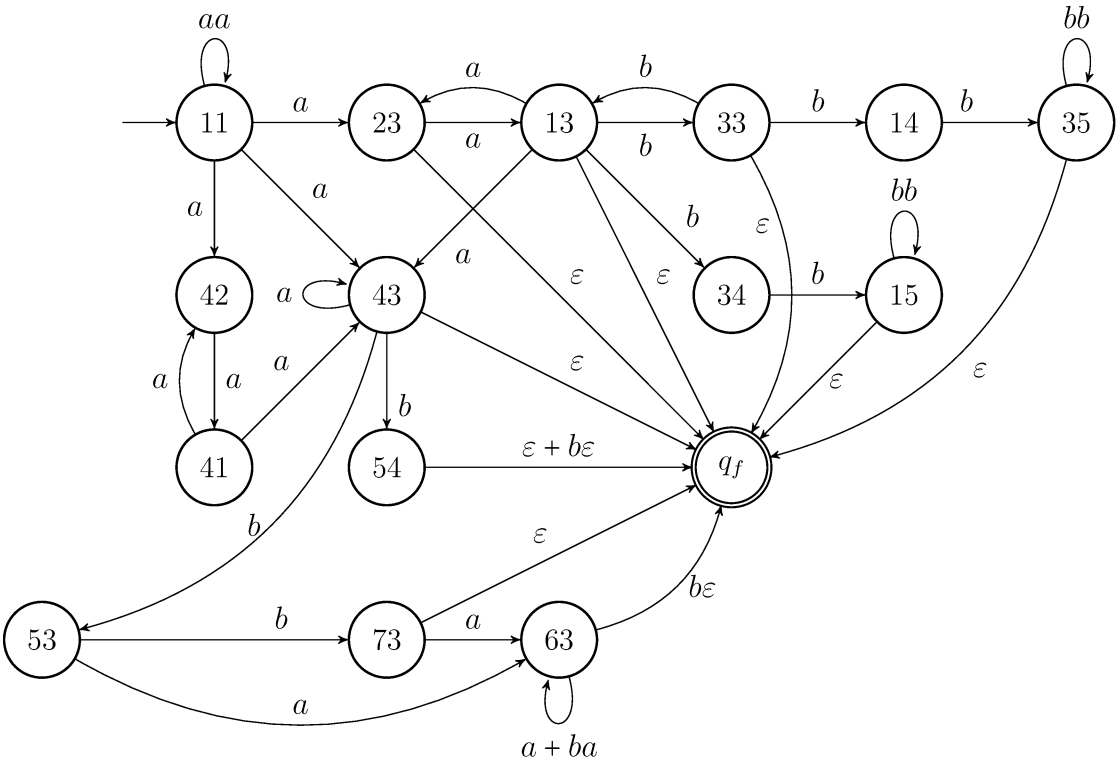
1) Исключим состояния 22, 36 и 16:



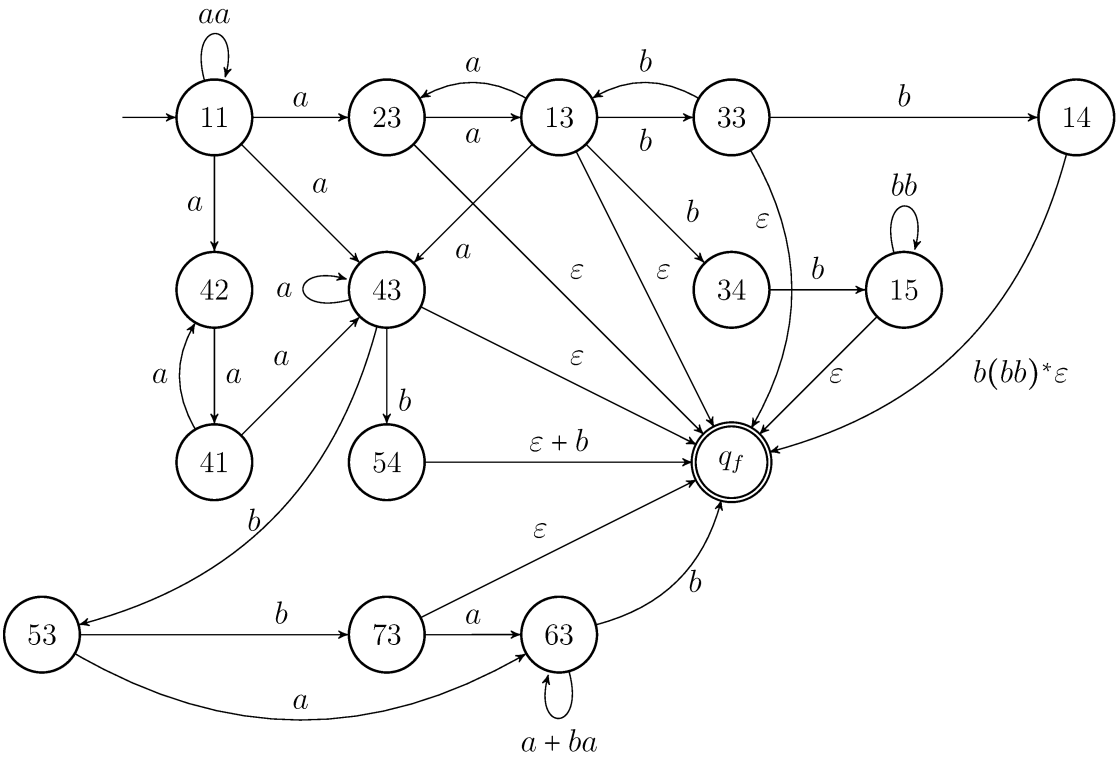
2) Исключим состояние 75:



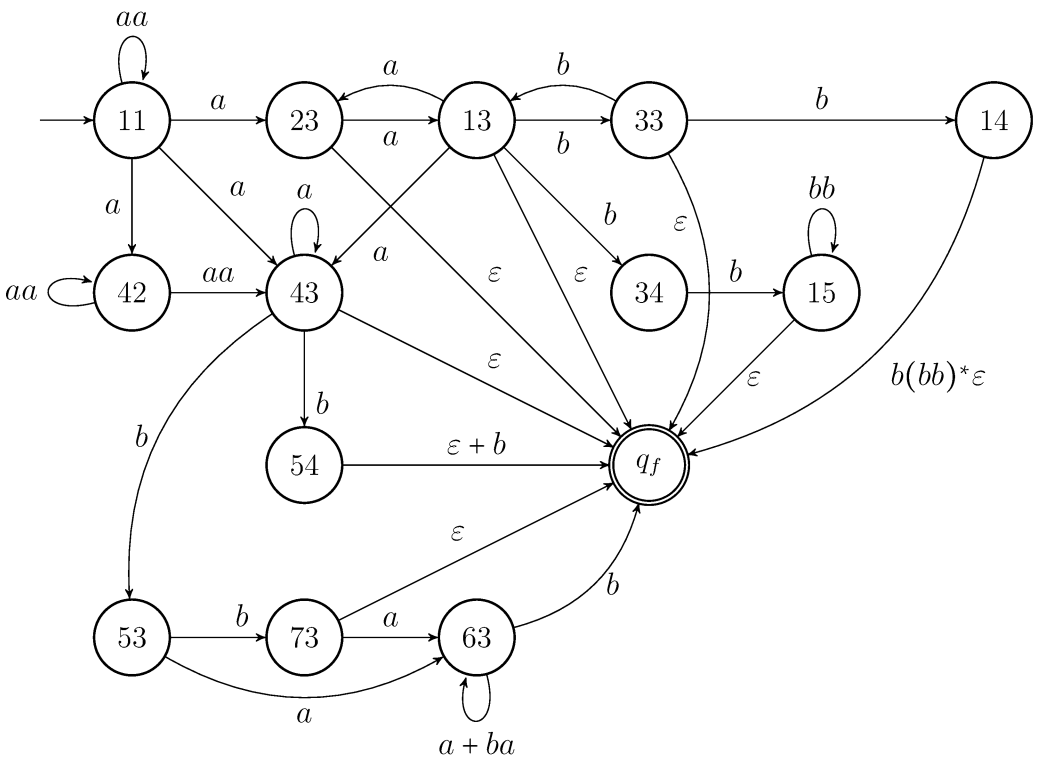
3) Исключим состояние 83:



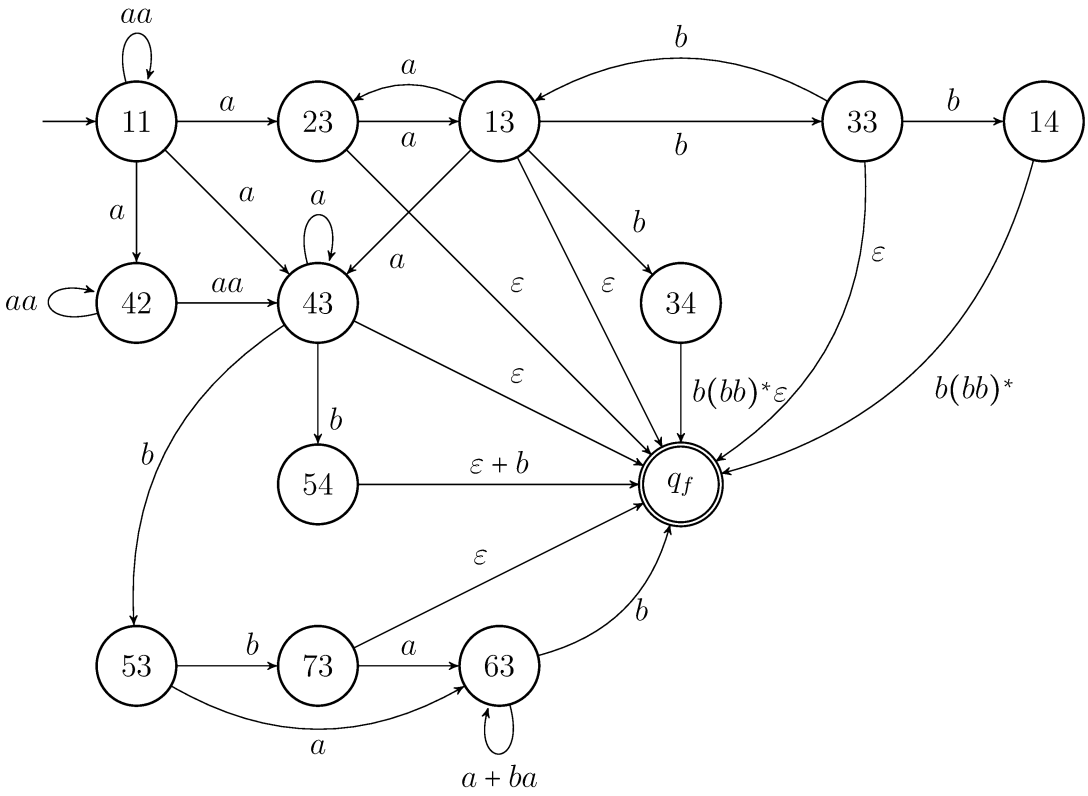
4) Исключим состояние 35:



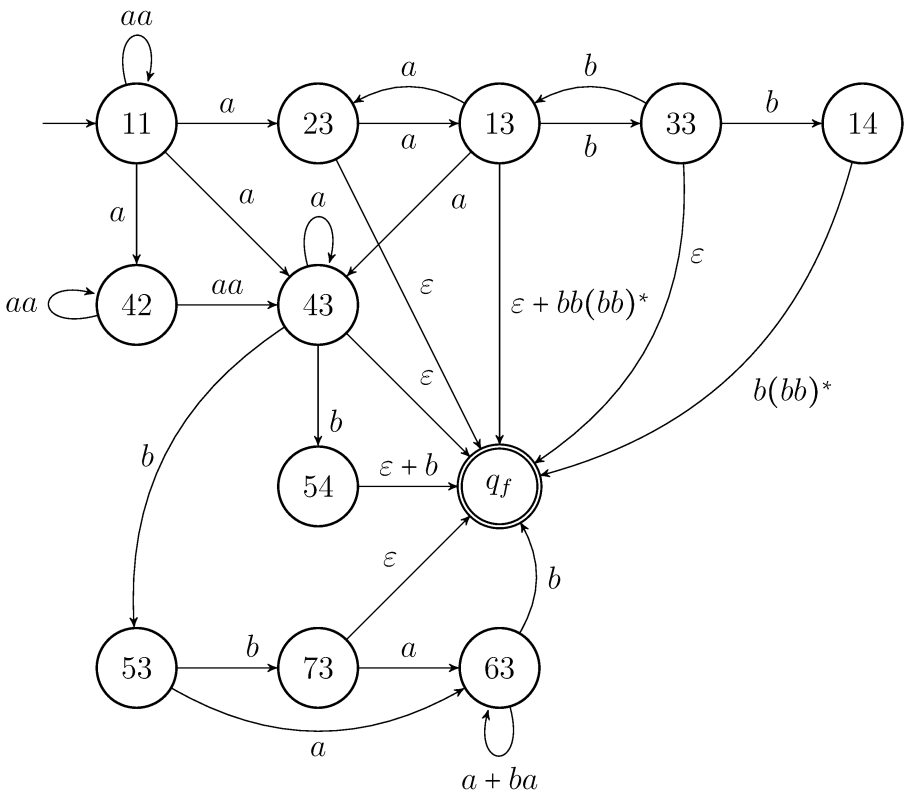
5) Исключим состояние 41:



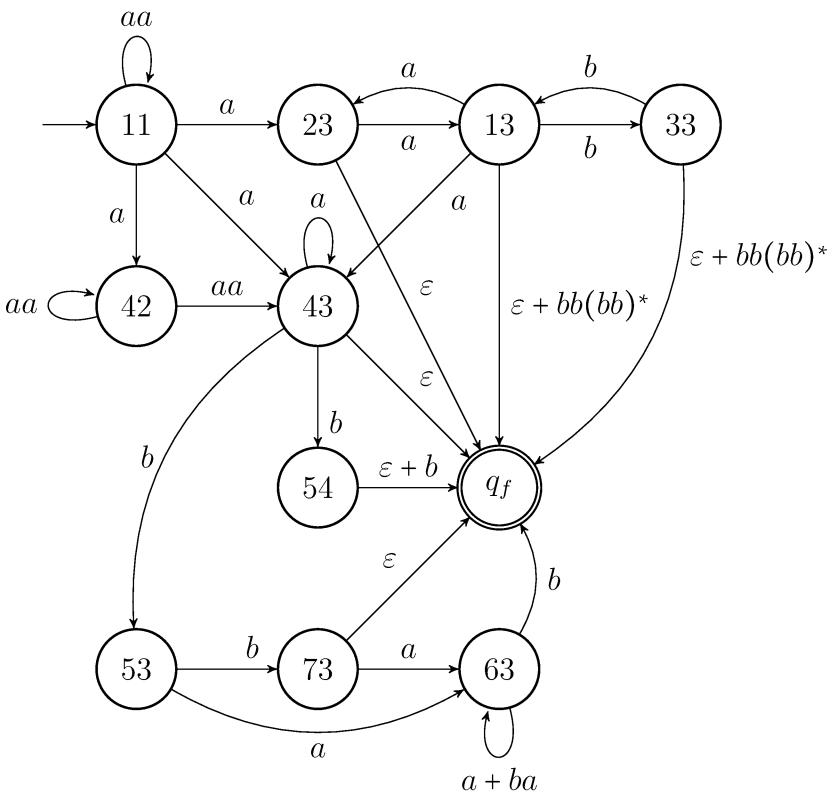
6) Исключим состояние 15:



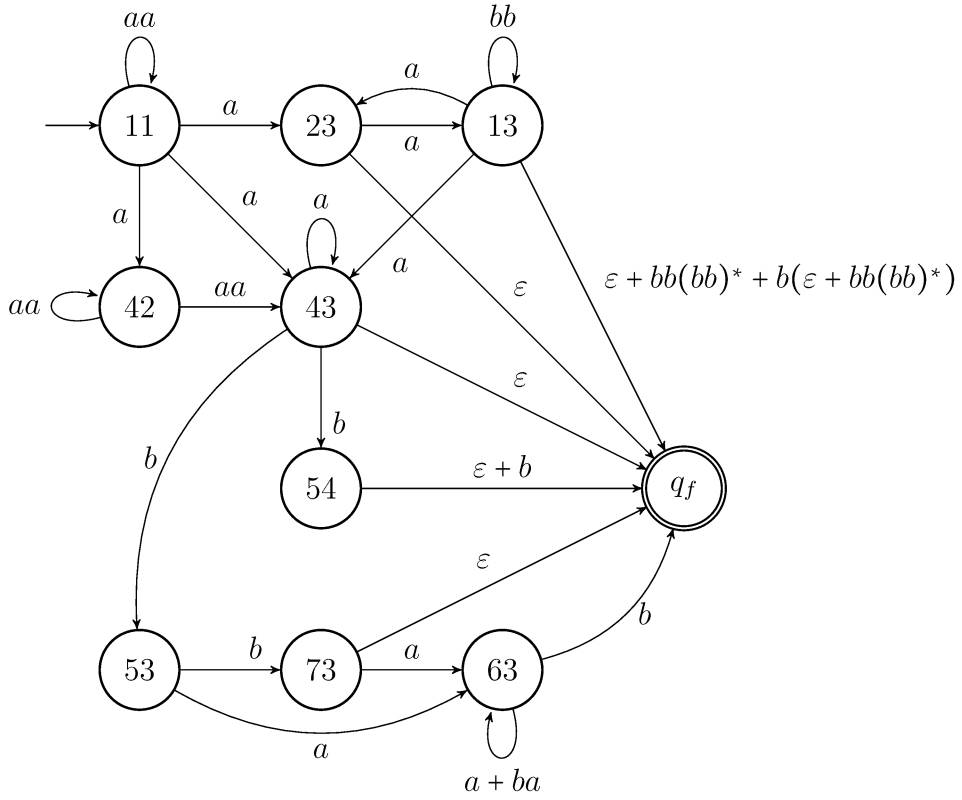
7) Исключим состояние 34:



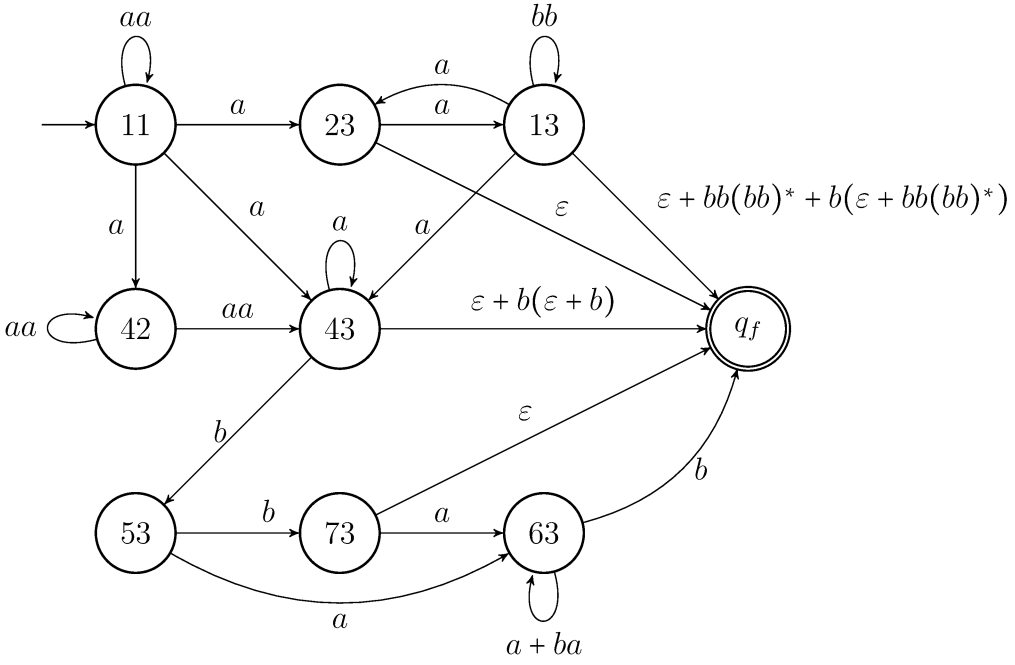
8) Исключим состояние 14:



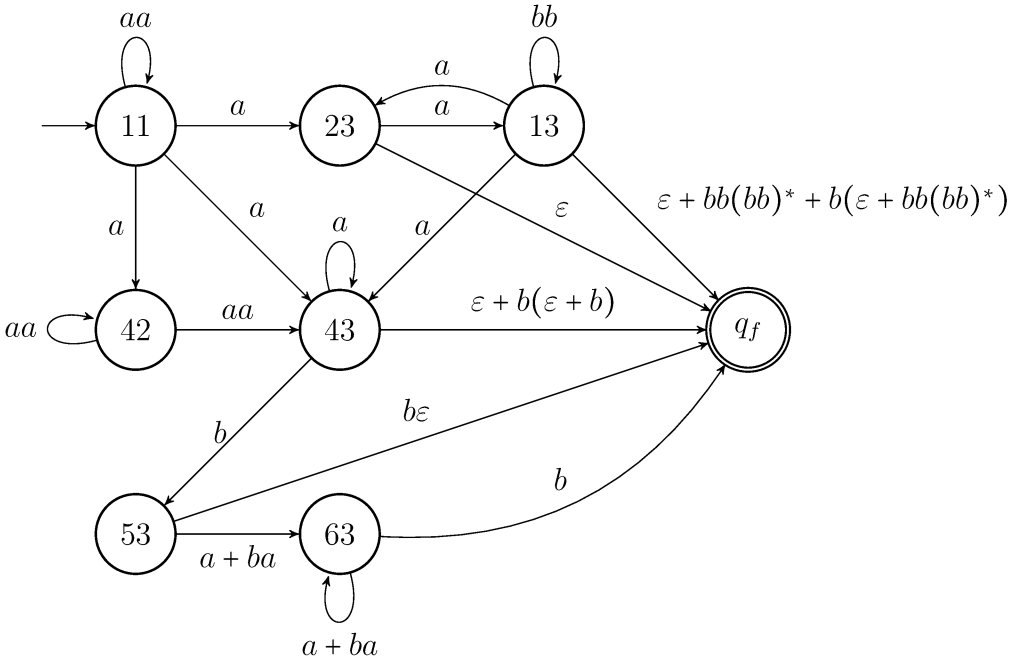
9) Исключим состояние 33:



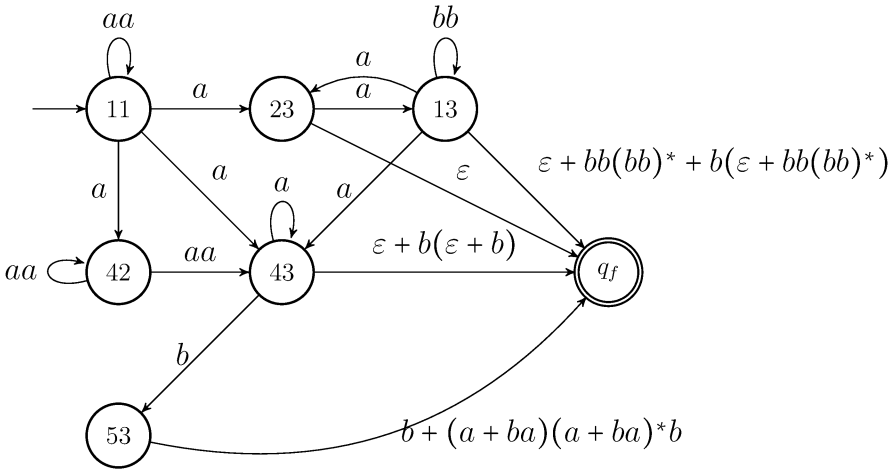
10) Исключим состояние 54:



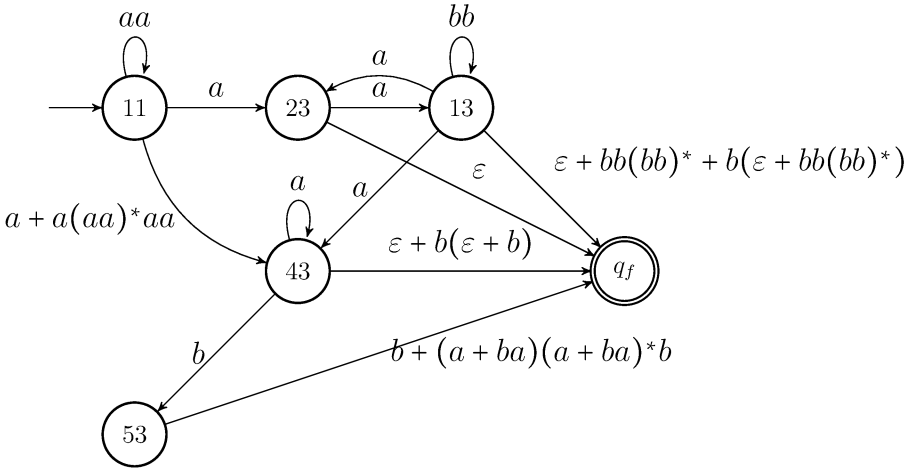
11) Исключим состояние 73:



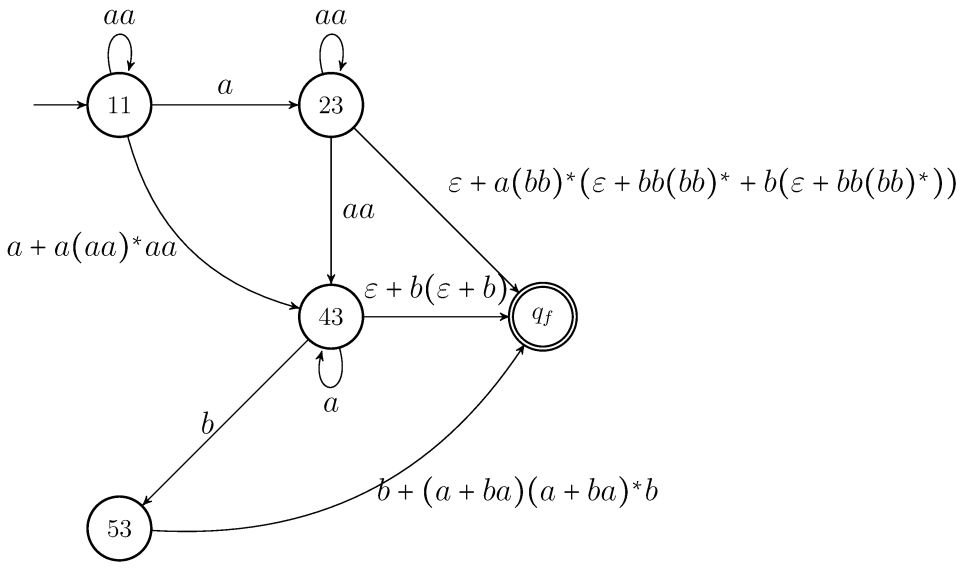
12) Исключим состояние 63:



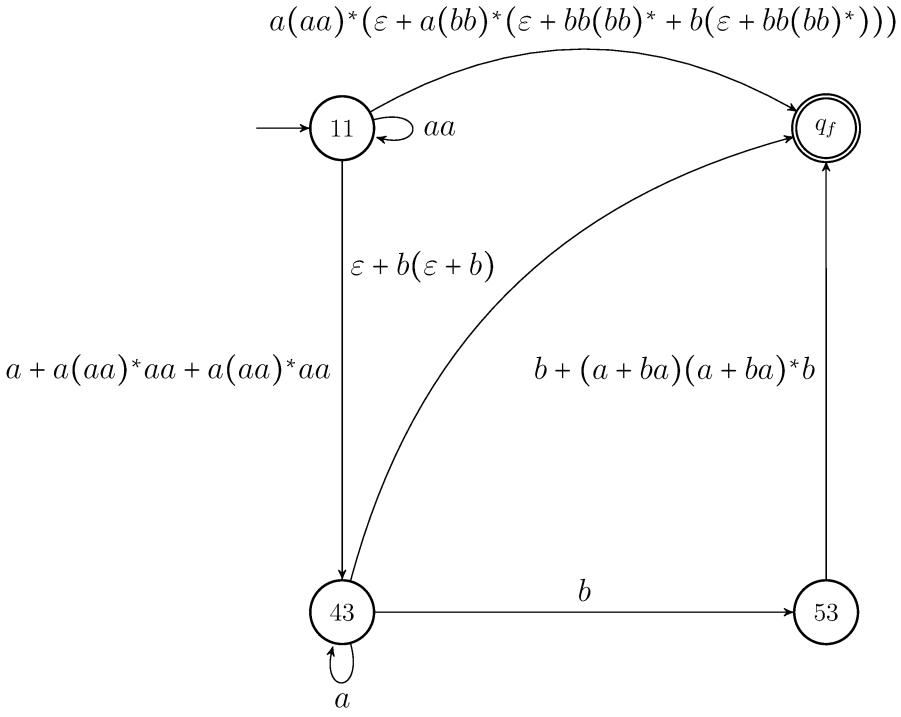
13) Исключим состояние 42:



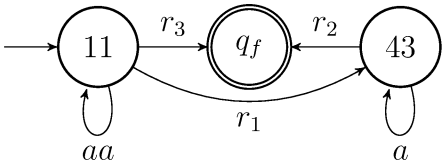
14) Исключим состояние 13:



15) Исключим состояние 23:



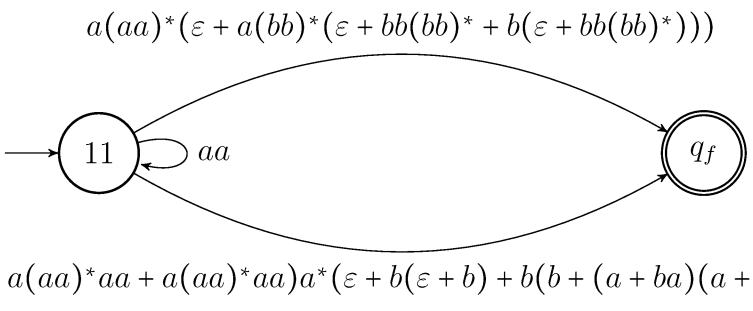
16) Исключим состояние 53:



На автомате выше использованы обозначения:

$$r_1 = a + a(aa)^*aa + a(aa)^*aa,$$
$$r_2 = \varepsilon + b(\varepsilon + b) + b(b + (a + ba)(a + ba)^*b),$$
$$r_3 = a(aa)^*(\varepsilon + a(bb)^*(\varepsilon + bb(bb)^* + b(\varepsilon + bb(bb)^*))).$$

17) Исключим состояние 43:



Получаем

$$Reg = (aa)^*(a(aa)^*(\varepsilon + a(bb)^*(\varepsilon + bb(bb)^* + b(\varepsilon + bb(bb)^*))) + (a + a(aa)^*aa + a(aa)^*aa)a^*(\varepsilon + b(\varepsilon + b) + b(b + (a + ba)(a + ba)^*b))).$$

(iii) Определить: совпадают ли языки L_1 и L_2, является ли L_1 дополнением L_2.

1) Допустим, L_1 совпадает с L_2. Приведем контрпример. Слово $bbaba \in L_1$,

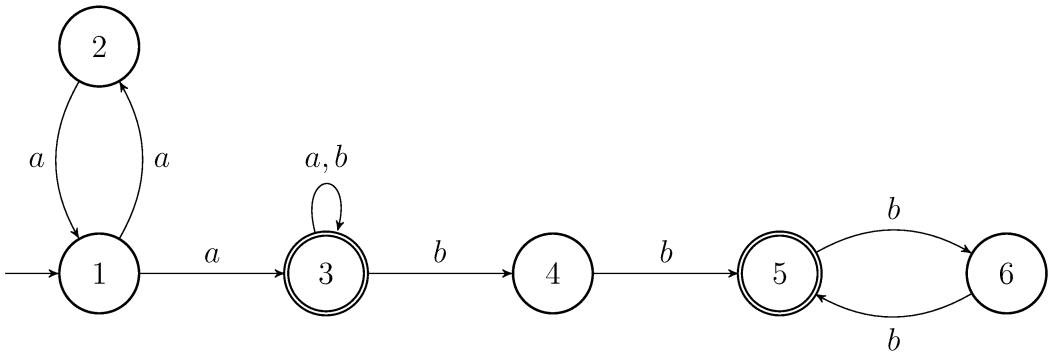
но $bbaba \notin L_2$. Делаем вывод, что $L_1 \neq L_2$.

2) Допустим, $L_1 = \overline{L_2}$. Приведем контрпример. Слово $aaaab \in L_1$,

но при этом $aaaab \in L_2$. $L_1 \cup L_2 \neq \varnothing$, что противоречит определению дополнения языка.

(iv) Построить ε-НКА, распознающий один из языков L_1^R или L_2^R.

НКА, распознающий язык L_2 имеет вид :



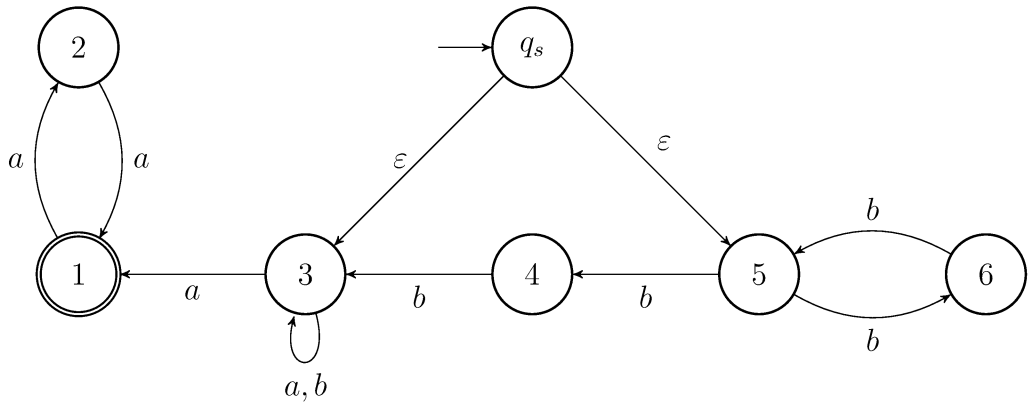
Следуя алгоритму, преобразуем его к ε-НКА, распознающему язык L_2^R:

1) Меняем направления дуг.

2) Начальное состояние автомата делаем единственным конечным.

3) Создаём новое начальное состояние q_s с ε-переходами во все исходные допускающие состояния.

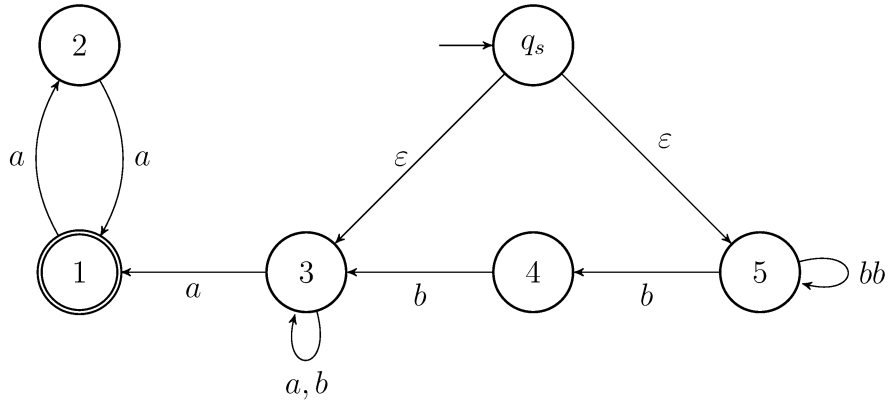
$M = (\{q_s, 1, 2, 3, 4, 5, 6\}, \{a, b\}, \delta, q_s, \{1\})$



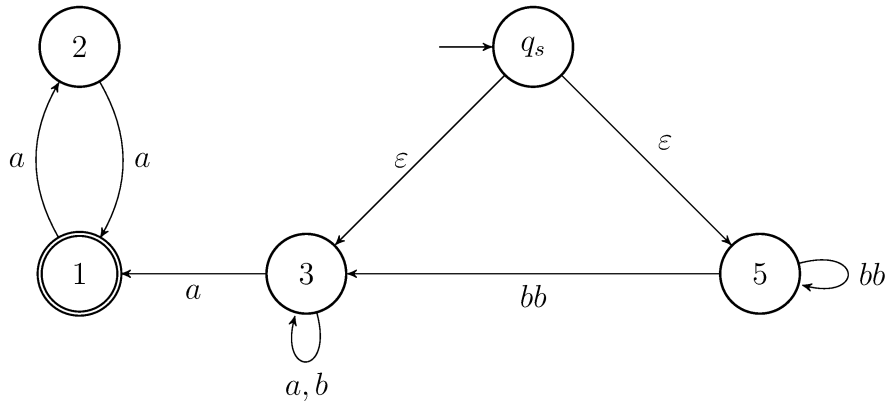
(v) Вычислить регулярное выражение по построенному ε-НКА.

Воспользуемся методом исключения состояний:

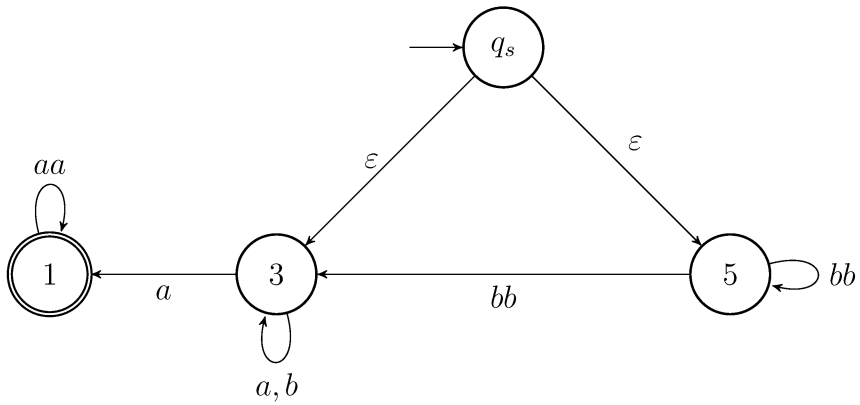
1) Исключим состояние 6:



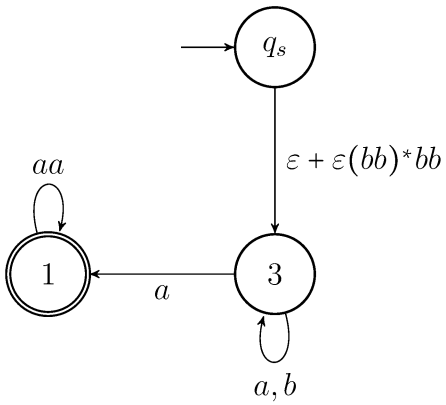
2) Исключим состояние 4:



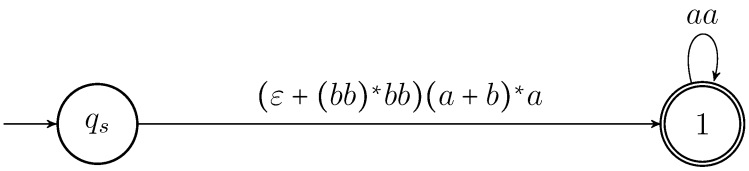
3) Исключим состояние 2:



4) Исключим состояние 5:



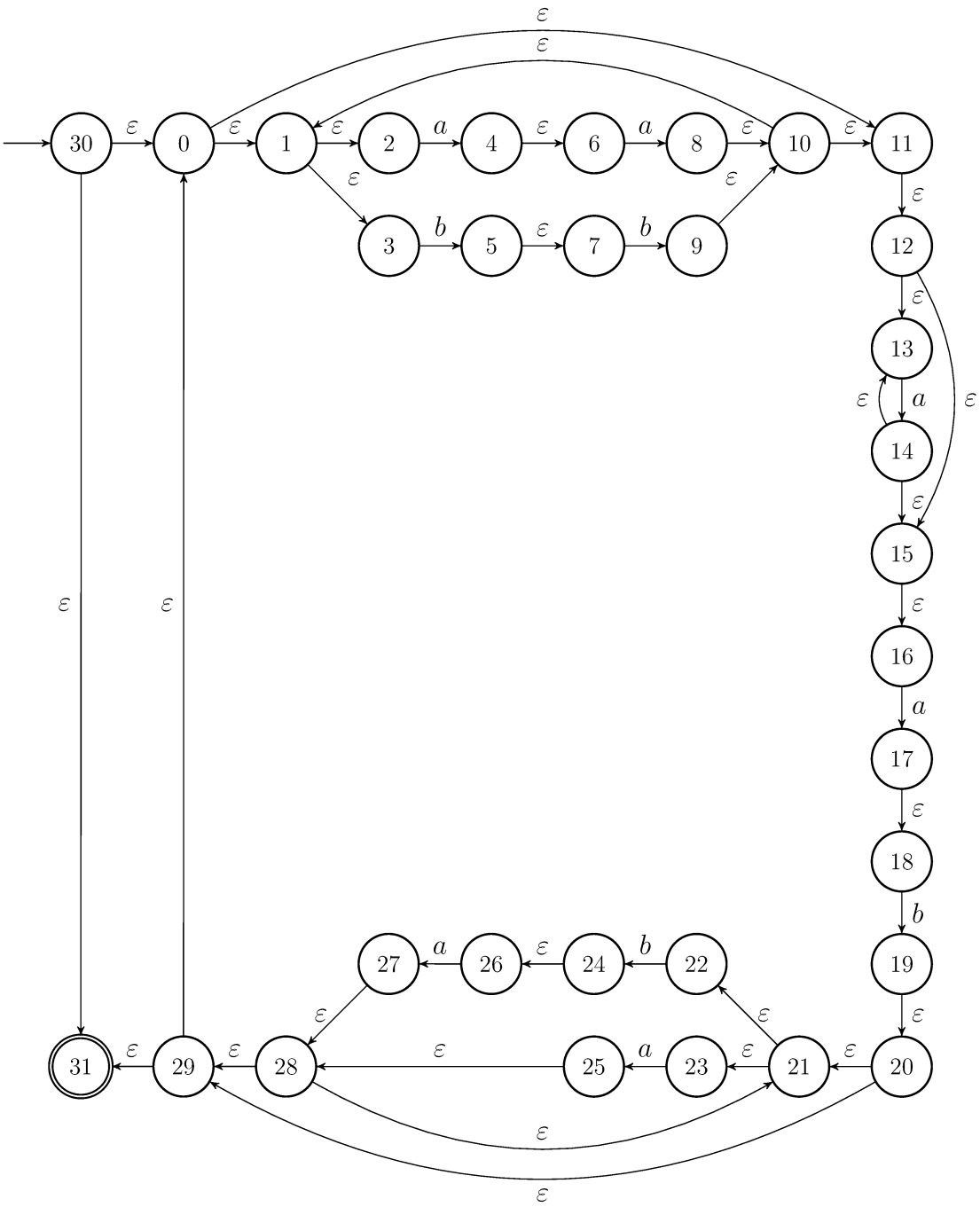
5) Исключим состояние 3:



Получаем : $Reg = (\varepsilon + (bb)^*bb)(a + b)^*a(aa)^*$

(vi) Построить ε-НКА, распознающий один из языков L_1L_2, $L_1 \cup L_2$ или L_1^*.

Будем строить ε-НКА, распознающий язык L_1^*:



(vii) Детерминизировать один из построенных ε-НКА.

ε-НКА, соответствующий языку L_2^R:

δ	Вход		
	a	b	ε
$\{q_s\}$	$\{\varnothing\}$	$\{\varnothing\}$	$\{3\},\{5\}$
$\{1\}$	$\{2\}$	$\{\varnothing\}$	$\{\varnothing\}$
$\{2\}$	$\{1\}$	$\{\varnothing\}$	$\{\varnothing\}$
$\{3\}$	$\{1\},\{3\}$	$\{3\}$	$\{\varnothing\}$
$\{4\}$	$\{\varnothing\}$	$\{3\}$	$\{\varnothing\}$
$\{5\}$	$\{\varnothing\}$	$\{4\},\{6\}$	$\{\varnothing\}$
$\{6\}$	$\{\varnothing\}$	$\{5\}$	$\{\varnothing\}$

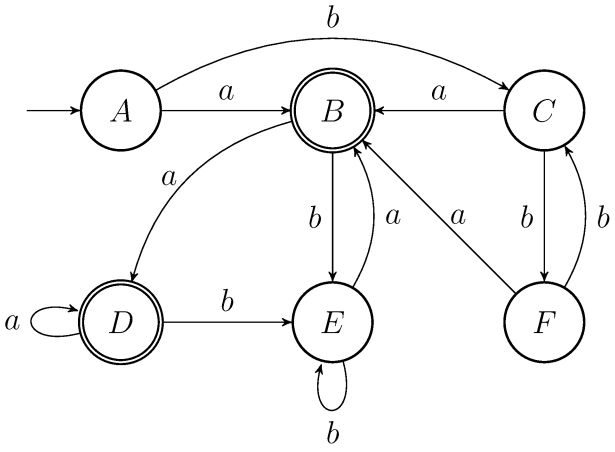
Детерминизируем его :

δ	Вход	
	a	b
$\{q_s,3,5\}$	$\{1,3\}$	$\{3,4,6\}$
$\{1,3\}$	$\{1,2,3\}$	$\{3\}$
$\{3,4,6\}$	$\{1,3\}$	$\{3,5\}$
$\{1,2,3\}$	$\{1,2,3\}$	$\{3\}$
$\{3\}$	$\{1,3\}$	$\{3\}$
$\{3,5\}$	$\{1,3\}$	$\{3,4,6\}$

Введем обозначения :

$$\{q_s,3,5\} = A$$
$$\{1,3\} = B$$
$$\{3,4,6\} = C$$
$$\{1,2,3\} = D$$
$$\{3\} = E$$
$$\{3,5\} = F$$

Представим его граф:


b
A
a
B
a
C
a
b
a
a
b
b
a
D
b
E
F
b